Security of Smart Environments - Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Gemini to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

CHAPTER 1: IoT

Introduction: The Physical Turn in Computer Security

A **smart environment** is a physical space rather than a purely digital domain. It is a home, a hospital, a factory, an office building, a city, or a vehicle that has been instrumented with digital devices capable of bridging the physical and the digital worlds. What distinguishes a smart environment from an ordinary computer network is that the computation is embedded in, and acts upon, the physical space in which people actually live and move.

For this bridge between the physical and digital to exist, two categories of components are indispensable: **sensors** and **actuators**. Sensors collect information from the physical world and translate it into the digital space; actuators perform the reverse operation, transforming digital commands into physical actions. Working together, they allow a smart environment to perceive what is happening around it, reason about that information, take decisions, and then intervene in the physical world.

The behaviour of a smart environment is governed by a small set of defining properties. It exhibits **context awareness**, meaning it understands the situation in which it is operating. It supports **automation**, carrying out actions without continuous human intervention. It provides **adaptation**, adjusting its behaviour to changing conditions. And it permits **remote access**, so that the space can be observed and controlled from outside its physical boundaries. A frequently cited early definition captures the spirit of the concept: a smart environment is a small world in which many kinds of smart devices work continuously to make the lives of its inhabitants more comfortable (Cook and Das, 2004).

Why Security Failures Become Physical and Personal

The single most important conceptual shift when moving from conventional IT security to smart-environment security is that **the users live inside the system**. This changes the nature of the harm that a security breach can cause.

Consider a compromise of a conventional online service such as a social network. If it is breached, people are upset: they may lose access to their content, or their private information may be exposed. The damage is real but remains within the informational domain. In a smart environment the situation is qualitatively different. Compromising it does not merely threaten the *security* of information; it threatens the *safety* of people. If a nuclear power plant's control system is compromised, the consequences propagate into the physical world, causing damage to cities and to the citizens living in them. The lesson is compact and central to the entire subject: in a smart environment, **security failures become physical, and privacy failures become personal**.

The Ubiquity of Smart Environments

Smart environments are no longer exotic; they are everywhere, and each category carries its own devices and its own risk profile.

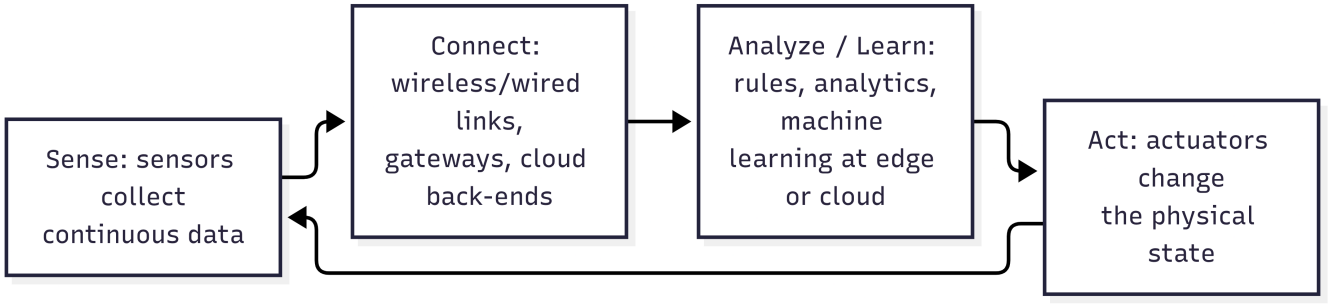
The **smart home** contains voice assistants, cameras, locks, thermostats, and connected appliances. **Smart buildings** manage heating, ventilation and air-conditioning (HVAC), lighting, access control, and elevators, typically through building-management systems and protocols such as BACnet and KNX. At the urban scale, **smart cities and mobility** encompass traffic lights, public transport systems, and connected and autonomous vehicles. The **smart grid and utilities** category includes smart meters, electrical substations, and water-treatment plants. **Smart health** spans wearables, implantable medical devices, and fully connected hospitals. **Smart industry**, the domain of Industry 4.0, is built on programmable logic controllers, industrial robots, and digital twins. Finally, **immersive and mobile spaces** introduce extended-reality (XR) headsets and drones. The breadth of this list is itself a security observation: the attack surface of modern society is now co-extensive with its physical infrastructure.

The Common Operational Pattern: Sense, Connect, Analyze, Act

Despite their diversity, virtually all smart environments operate through the same four-stage cycle. Understanding this loop is essential because each stage is also a distinct opportunity for attack and for defence.

In the **sense** stage, sensors collect continuous data about people and the physical world. This collection is often continuous, though in some designs sensors are triggered only by specific conditions. In the **connect** stage, the heterogeneous devices are linked together through a mixture of wireless and wired connections, passing through gateways and up to cloud back-ends; connection is what makes the environment's information available beyond the physical space itself. In the **analyze** stage, the collected data is interpreted using rules, analytics, and increasingly machine learning, performed either at the edge or in the cloud. In the **act** stage, actuators change the physical state of the world: opening a window, heating a room, moving a mechanism, or applying a brake.

The following diagram expresses this cyclical relationship.



Three things are ultimately at stake in this loop, and together they explain why the security problem is uniquely hard. First, **the data is personal and continuous**: everything sensed can be linked to confidential information about individuals. Second, **the actuation is physical**, which means the concern is safety and not only security. Third, **the scale is enormous**: billions of devices that are cheap, long-lived, and rarely updated. A concrete illustration of the sense-and-act loop is the artificial-pancreas system for diabetes management, in which a continuous glucose sensor feeds readings into a control algorithm that in turn commands an insulin pump; here the intelligent decision is being made about a human body in real time, and an error is not an inconvenience but a medical emergency.

Smart Environments Are Built on the Internet of Things

The building blocks of every smart environment are **IoT devices**, which are characteristically constrained, heterogeneous, and networked. This foundational fact has a direct security consequence: the smart environment **inherits the weaknesses** of its component devices. Default credentials that are never changed, the absence of update mechanisms, and unnecessarily exposed network services all flow upward from the individual device into the environment as a whole.

Worse, the environment does not merely inherit device weaknesses; it **adds entirely new attack surfaces** on top of them. Gateways, cloud back-ends, mobile applications, artificial-intelligence pipelines, and even other people's devices all become potential points of entry. This is why the study of smart-environment security begins from the IoT device itself, understanding what a "thing" is, how it is built and connected, and why it is so difficult to secure, before climbing back up to the architectures, the threat landscape, and the case studies of whole environments.

Defining the Internet of Things

Several complementary definitions of the IoT exist, and each highlights a different facet.

The first, from the European Union Agency for Network and Information Security (ENISA), describes the IoT as a *cyber-physical ecosystem of interconnected sensors and actuators which enable intelligent decision making.* This definition is valuable for foregrounding the cyber-physical nature and the decision-making purpose, though it is notable that it does not itself address the security dimension at all.

A second, more expansive definition comes from the Telecommunication Standardization Sector of the International Telecommunication Union (ITU-T). It characterises the IoT as a *global infrastructure for the information society, enabling advanced services by interconnecting physical and virtual things based on existing and evolving interoperable information and communication technologies.* This definition adds two important clarifications. It notes that, through the exploitation of identification, data-capture, processing, and communication capabilities, the IoT makes full use of things to offer services to all kinds of applications while ensuring that security and privacy requirements are fulfilled. It further observes that, from a broader perspective, the IoT can be understood as a vision with both technological and societal implications.

The Sensor

A **sensor** is an element that allows an IoT system to monitor the environment and the context in which it operates. Sensors measure defined physical, chemical, or biological indicators, and at the digital level they also collect information about the network and the applications running on it. The range of possible measurements is vast: temperature, pressure, body position, airflow, glucose concentration, blood pressure, electrical activity of the heart or muscles, and many others. In a connected-health platform, for instance, a single hub may aggregate readings from a temperature sensor, an electrocardiogram sensor, an airflow sensor, a galvanic-skin-response sensor, a pulse oximeter, a glucometer, a spirometer, and a blood-pressure sensor simultaneously, painting a continuous physiological portrait of a person.

The Actuator

An **actuator** is the entity responsible for moving or controlling a system or mechanism. It operates in the reverse direction of a sensor: whereas a sensor turns a physical quantity into a digital reading, an actuator takes an electrical input and turns it into physical action. Motors, valves, locks, and pumps are all actuators. This complementary relationship between sensing and actuating is what closes the loop between perception and intervention.

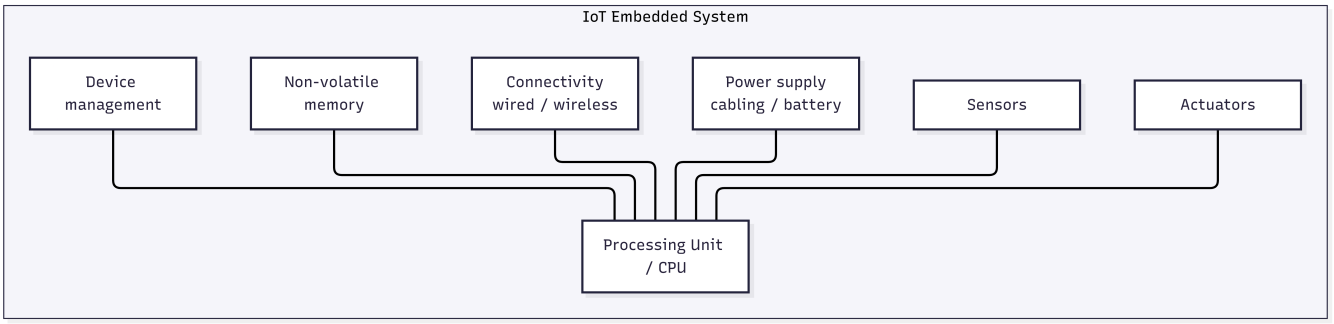
The "Thing"

The ITU-T also defines the **thing** at the heart of the IoT. With regard to the Internet of Things, a thing is an object of the physical world (a physical thing) or of the information world (a virtual thing) that is capable of being identified and integrated into communication networks. In practice, a "thing" can be almost anything. It may be a fully capable device such as a server or a smartphone, which is not resource-constrained. More often, however, the thing of interest in security is an **embedded system**.

The Embedded Device

An embedded device is best understood as a small computer dedicated to a specific function. It possesses a central processing unit (CPU) and non-volatile memory in which its program and data persist. It requires connectivity, which may be either wired or wireless, in order to communicate. And it requires a power supply, which may be an energy plug or a battery. Because embedded devices must communicate, and because operators often wish to control the smart environment from outside the physical space, **gateway devices** are introduced. Gateways forward information to the cloud, from which the state of the smart environment can be accessed remotely.

The internal structure of an IoT embedded system can be visualised as a processing unit at the centre, surrounded by the supporting elements it needs to function.



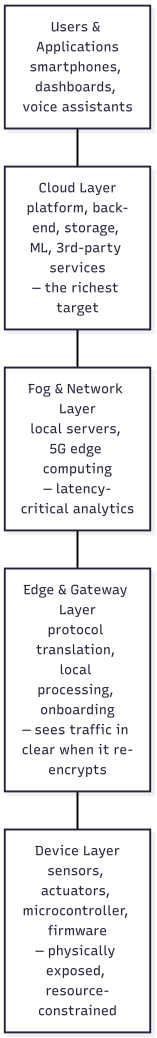
The IoT Ecosystem and Its Layered Architecture

An individual device rarely acts alone. It participates in a broader **IoT ecosystem** in which devices at the bottom, such as connected cars, smartwatches, and washing machines, communicate through gateways, which in turn connect upward to a cloud platform hosting the back-end and services. Communications flow through this hierarchy in both directions, and end users reach the system through smartphones and laptops.

This ecosystem is conventionally organised into a **layered architecture**, and it is essential to grasp that every boundary between layers is simultaneously a **point of attack** and a **place for defence**, where authentication, encryption, and monitoring can be applied.

At the bottom sits the **device layer**, comprising the sensors, actuators, microcontrollers, and firmware. This layer is physically exposed and resource-constrained, which is precisely what makes it vulnerable. Above it is the **edge and gateway layer**, responsible for protocol translation, local processing, and device onboarding. This layer occupies a particularly sensitive position: because it re-encrypts traffic as it passes between the device world and the wider network, it necessarily sees that traffic in clear text at the moment of translation. Next is the **fog and network layer**, consisting of local servers and 5G edge-computing resources that handle latency-critical analytics close to where data is generated. Above that is the **cloud layer**, which hosts the platform, the back-end, storage, machine learning, and third-party services; because it concentrates so much data and capability, it is the richest target of all. Finally, the

users and applications layer contains the smartphones, web dashboards, and voice assistants through which people manage the environment.



Because the layers are heterogeneous, so too are the devices within them. Different devices operate under different constraints of energy and computational power, which is the underlying reason the IoT relies on such a wide variety of protocols and standards rather than a single unifying one.

The Scale of the Problem

The number of connected IoT devices has already **surpassed the number of human beings on the planet**, and it continues to climb across every industry, including personal and consumer products, healthcare, automotive, and manufacturing. The growth trajectory is steep: from roughly 7.74 billion devices in 2019, the count rises year by year toward a projected 25.44 billion by 2030 in one widely reproduced estimate.

It is instructive, however, to compare forecasts with reality, because the history of IoT prediction is a history of over-optimism. In 2011, Cisco forecast 50 billion connected devices by 2020; the actual figure turned out to be only about a quarter of that. More recent and more sober estimates from IoT Analytics' *State of IoT 2025* report place the count at 18.5 billion connected IoT devices at the end of 2024, an increase of about twelve percent, with a forecast of 21.1 billion for the end of 2025, an increase of about fourteen percent. Looking further ahead, roughly 39 billion devices are expected by 2030, rising above 50 billion by 2035. Whatever the precise number, the security implication is the same and it is sobering: **each of these devices is a potential entry point, and most of them will never be patched.**

The Protocol and Asset Landscape

The heterogeneity of the IoT manifests concretely in an exceptionally crowded **protocol landscape**. Spanning from the physical link up to business applications, the ecosystem draws on connectivity standards such as Ethernet, RS-232, RS-485, Modbus, USB, and SPI at the lowest levels; short- and medium-range wireless technologies including Bluetooth and Bluetooth Low Energy, Zigbee, Z-Wave, 6LoWPAN, NFC, RFID, and Wi-Fi; wide-area and cellular technologies from 2G through LTE-Advanced and specialised low-power options; and application- and messaging-layer protocols such as MQTT, CoAP, AMQP, XMPP, HTTP, DDS, and REST APIs. These are organised geographically into personal-area networks (PAN, 802.15), local-area networks (LAN, 802.11), metropolitan-area networks (MAN, 802.16), and wide-area networks (WAN, 802.20). Above the communication stack sit data-aggregation, processing, storage, analytics, and business-process functions. The essential takeaway is that this sprawling diversity multiplies the number of implementations, and therefore the number of possible flaws.

The **assets** of an IoT system are correspondingly numerous, and each is a potential target. They include the IoT devices themselves along with their hardware, sensors, actuators, and embedded systems; the infrastructure of routers, gateways, firewalls, and power supply; the security components such as intrusion-detection and intrusion-prevention systems and authentication systems; the platform and back-end comprising cloud infrastructure and web-based services; the decision-making machinery of data mining, big data, algorithms, and machine learning; the information itself in all its states, whether at rest in storage and databases, in transit across the network, or in use during processing; the communications and their protocols; and the networks spanning LAN, WAN, VPN, and PAN. Any one of these can become the object of an attack.

The State of IoT Security

There is a well-worn joke in the security community that captures the field's central anxiety: someone asks whether the "S" in "IoT" stands for security, and the punch line is that there is no "S" in IoT at all, which is exactly the point. Security was, for a long time, simply absent from the design of connected things.

The history of the security problem is rooted in how these systems evolved. The **initial design** of many industrial and embedded systems assumed a private communication network. Over time these systems were migrated onto IP networks, and later still exposed to the open Internet, a context for which they had never been engineered. The earliest security measures were rudimentary, dealing with authentication through embedded passwords, and it was therefore still simple to find and exploit security faults. Crucially, systems **started with basic security** and only later, once flaws were discovered, had more complex security requirements bolted on. Because of this reactive posture, low-security devices from the early design era are **still deployed today**, and they continue to be used in compatible fall-back modes, meaning a modern system will often downgrade to an insecure legacy protocol simply to remain interoperable. Compounding all of this, **firmware updates are hard or nearly impossible** once devices have been installed, so vulnerabilities discovered after deployment tend to persist for the entire operational life of the device.

The Three Ages of IoT Threats

The evolution of the IoT threat landscape can be divided into three successive ages, each defined by the prevailing attitude toward security and the technologies used to provide it.

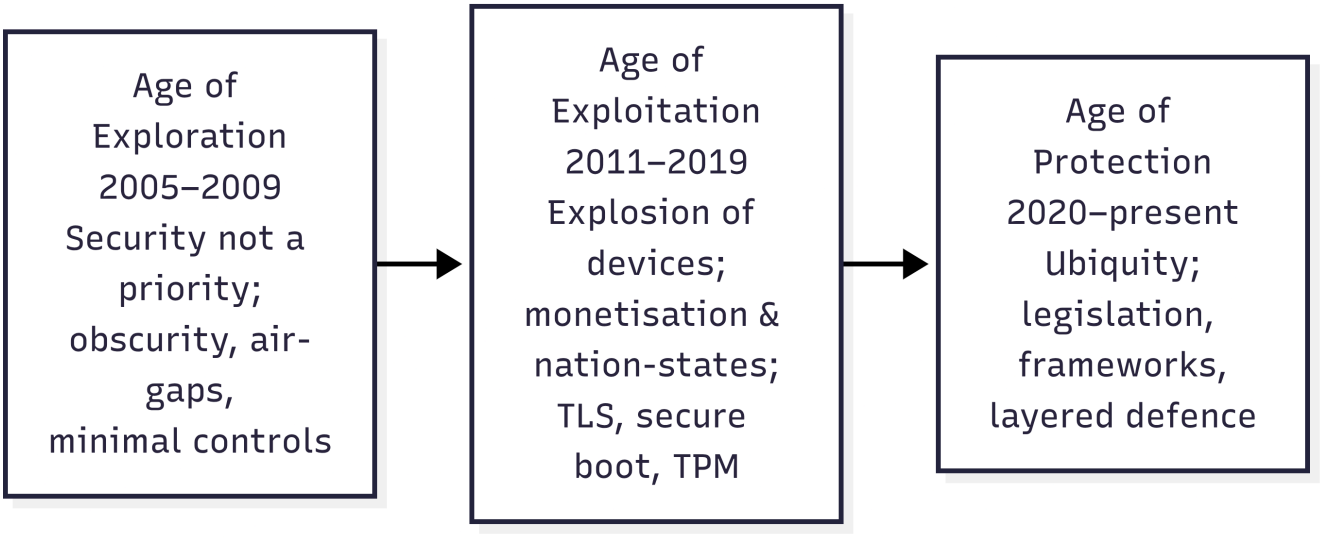
The first is the **Age of Exploration**, spanning roughly 2005 to 2009. In this period, security was not a priority for early IoT and embedded devices. Most cyberattacks were limited to malware and viruses impacting Windows-based embedded control systems. Rather than actively mounting a defence, organisations assumed that no one would bother to attack devices running in isolated networks. The

security methods of the era reflected this complacency. Chief among them was **security by obscurity**, the belief that keeping the operating system, application, or protocol secret would keep it safe; this approach never works, because obscurity is not a substitute for genuine protection. Where security existed at all it was minimal and often easily bypassed. Secure protocols such as SSH or SSL were used in only a handful of systems, and even then usually with no other security controls alongside them. Many networks were **air-gapped**, physically isolated from other networks, and this isolation was itself treated as the primary defence.

The second is the **Age of Exploitation**, spanning roughly 2011 to 2019. During this period the number of connected devices exploded and cloud connectivity became commonplace. Criminals improved their ability to **monetise** attacks on IoT devices through crypto-mining, ad-click fraud, and spam-email campaigns. At the same time, nation-state actors began to use IoT devices for politically motivated attacks. Many new security technologies were being adopted in this age, but their use was inconsistent, incomplete, and sometimes flawed, so a great many devices remained vulnerable in practice. The technologies introduced in this period included security protocols such as TLS and SSH, **secure boot** to verify the integrity of software at start-up, **Trusted Platform Modules (TPM) or Secure Elements** for secure key storage, hardened operating systems, and embedded firewalls.

The third is the **Age of Protection**, beginning in 2020 and continuing to the present. Connected devices are now ubiquitous in every area of life. In response to the growing number and severity of attacks, governments and industrial groups began to enact legislation requiring higher levels of security for IoT devices. Because attackers will always continue to find "soft targets" among both legacy devices and newly deployed devices that lack strong protection, companies worldwide are now building robust security controls directly into IoT devices, using security frameworks and unified solutions in which several key technologies work together to provide multiple, layered defences.

This progression is worth internalising as a single arc, because it explains why the same root causes recur across decades of incidents.



Across this timeline sits a long roster of landmark incidents that mark the maturation of the threat: from the Stuxnet virus and early attacks on smart meters and traffic-light systems, through the compromise of medical devices such as insulin pumps and heart monitors, connected vehicles such as the Jeep Cherokee and the Tesla Model S, and industrial targets such as steel mills and power grids, to the great botnets, Mirai, Hajime, Reaper, Persirai, Amnesia, and their successors, and on to attacks on consumer devices such as the Amazon Ring and the Philips Hue lightbulb. The pattern that emerges is that novelty of the target changes over time, but the underlying weaknesses do not.

Recent Incidents: The Same Root Causes

The persistence of these root causes is starkly visible in the incidents of the current decade. In **2021, the Verkada breach** exposed roughly 150,000 security cameras installed in hospitals, prisons, and factories, all through a single leaked super-admin credential. In **2023, the CyberAv3ngers** group hijacked Unitronics programmable logic controllers in United States water utilities by exploiting default passwords. In **2024, a flaw in Kia's dealer web portal** allowed remote control of millions of cars starting from nothing more than a licence plate. In **2025, BadBox 2.0** saw roughly 10 million Android TV boxes shipped already pre-infected from the factory. Also spanning **2025 and into 2026, the Aisuru and Kimwolf botnets**, both descendants of Mirai, set distributed-denial-of-service records above 30 terabits per second before being disrupted by the United States Department of Justice in March 2026. The common thread through every one of these episodes is identical to that of the previous decade: weak or default **credentials**, unnecessary **exposure** of services, absent **updates**, and vulnerable **ecosystem interfaces**.

Case Study I: Stuxnet

Overview and Strategic Intent

Stuxnet is the pivotal case study in smart-environment security because it was the moment a piece of software crossed the line from the digital into the destructively physical. Its **goal was to sabotage the Iranian nuclear program**, and it succeeded, causing genuine physical damage to the enrichment infrastructure and slowing the nuclear program down by an estimated four years.

The *how* of Stuxnet is what makes it a masterclass in cyber-physical attack. It **interfered with the Industrial Control Systems (ICS)** that were managed by Programmable Logic Controllers, specifically the Siemens Step7 platform. Having gained control, it made the enrichment **centrifuges operate at the wrong speeds**, stressing them toward failure. Most insidiously, while it was doing this, it **presented acceptable, normal-looking values to the monitoring software**, so that the human operators watching their screens saw nothing amiss.

A subtle but important point about the attackers' intent emerges from the impact. The attackers were in a position where they could have simply broken the plant outright; instead, they **preferred to slow the program down**, quietly and over an extended period, which was strategically far more valuable than an obvious act of destruction that would have been immediately detected and repaired. For this reason Stuxnet is widely regarded as the **opening act of cyber warfare**, the first time a state-level cyber weapon produced strategic physical effects.

The Anatomy of the Operation

The overall operation can be pictured as a set of converging elements. The suspected creators, associated with the United States and Israel, developed the weapon. It employed **stolen digital certificates** to sign its components, which helped it avoid suspicion by making its files appear legitimately signed. The malware reached its ultimate destination, the **Natanz enrichment facility**, not over the Internet directly but by being carried across the air gap: plant personnel, themselves manipulated as unwitting carriers, introduced it via **USB removable drives** into the control computers, which in turn were connected to the PLCs governing the centrifuges. Command-and-control communication, where possible, was conducted through innocuous football-themed (futbol-themed) websites to blend in with ordinary traffic.

Statistics and the Signature of a Targeted Weapon

The statistical profile of Stuxnet betrays its targeted nature. The first known sample dates to **2007**, though the malware was not discovered until **2010**, and it was rendered inactive from 2012 onward. It was classified as a **worm**, its targeted platform was **industrial SCADA systems**, and its infections numbered somewhere between ten thousand and three hundred thousand hosts. The overwhelming concentration of those infected hosts was in **Iran**, dwarfing every other country, and this geographic concentration is itself the fingerprint that reveals Iran to have been the intended initial target.

Stuxnet is often described as a **"fire and forget" weapon**: once released, it operated autonomously without needing further direction. Although it was fundamentally a targeted threat, the propagation techniques it used to reach an air-gapped facility inevitably let it **spread beyond its intended target**, producing what is aptly termed **"collateral damage"** as it appeared on machines around the world that had nothing to do with Natanz. The scale of this spread relative to its origin is striking: by February 2011, some 12,000 infections could be traced back to only **10 initial infections in 5 organisations in Iran** between 2009 and 2010, illustrating how a tiny, precisely aimed seed could blossom into a global presence.

The Installation Routine: Export 15

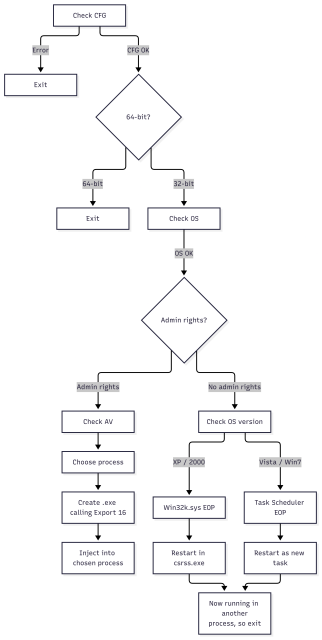
Stuxnet's main library exposed its functionality through numbered exports, two of which govern its life on a newly reached machine. The first, **Export 15**, is the installation routine, and it performs a careful sequence of environmental checks before committing to infection. This defensiveness is deliberate: a state-level weapon must avoid detection and avoid wasting itself on unsuitable or suspicious hosts.

The routine begins by **checking the machine's configuration (Check CFG)**. It inspects the victim's PC to see whether anything about the machine looks strange or unexpected, and if it detects an anomaly it exits immediately rather than risk exposure. If the configuration is acceptable, it next **checks whether the machine is 64-bit**. If the machine is 64-bit, Stuxnet stops. The reason is a precise piece of targeting intelligence: while by 2010 most of the world had migrated to 64-bit systems, the machines in the Iranian facility were still running 32-bit Windows. By refusing to run on 64-bit systems, Stuxnet effectively restricted itself to infecting only the 32-bit Windows PCs that matched its intended environment.

Having confirmed a 32-bit machine, the routine then **checks the operating system**, and subsequently **checks whether the current user has administrative rights**. The two branches that follow depend on the answer to this privilege question.

If the user **does have administrative rights**, Stuxnet proceeds to **check the antivirus (Check AV)** installed on the machine. It uses knowledge of the specific antivirus product to decide how to inject itself safely, then **chooses a process** to inhabit, **creates an executable that calls Export 16**, and **injects itself into the chosen process**, thereby beginning the true infection.

If the user **does not have administrative rights**, Stuxnet must escalate its privileges, and it does so by exploiting one of two elevation-of-privilege vulnerabilities selected according to the operating system version. On **Windows XP or Windows 2000**, it exploits a vulnerability in **Win32k.sys**, after which the threat restarts within the `csrss.exe` process. On **Windows Vista or Windows 7**, it exploits a vulnerability in the **Task Scheduler**, after which it restarts as a new task. In either case, the elevation gives it the administrative rights it lacked, the threat is now running in another process, and the original instance exits.



The Infection Routine: Export 16

The second export, **Export 16**, carries out the infection proper, and like the installation routine it opens with a set of guard conditions before doing anything destructive.

It first **checks the configuration** again, exiting on error. It then inspects a specific **registry key**: if the value `NTVDM Trace` is set to the value **19790509**, the malware exits. This is one of the most famous details of the entire case. The value is a date, 9 May 1979, and its presence functions both as an infection marker and, in the eyes of analysts, as a possible signature; it has been read as an allusion that supports attribution of the attack's origins. Next, the routine performs a **date check**, verifying that the current date is before **24 June 2012**; this establishes a hard **deadline**, a kill date after which the malware would cease to operate, and if the date is past that deadline it exits.

Assuming these checks pass, the routine **checks the operating system** and, depending on the version, sets access-control structures on its files, using a **DACL** on Windows XP or earlier and a **SACL** on Windows Vista or higher. It then **creates a global mutex** to ensure only one instance runs, and **creates its `.pnf` and `.cfg` files** on disk. It performs a further **date check** against the same 24 June 2012 deadline, then **decrypts and loads itself from disk, calling Export 6 to retrieve its version number**.

At this point a version-management step occurs, which is central to how Stuxnet kept itself current across an isolated network. It **compares the running version number against the version present on disk**; if the version is acceptable, it **decrypts resources 201 and 242 and writes them to disk**, then **sets the file times** to blend in, and **creates the rootkit-service registry keys** whose purpose is to **hide the malicious files**. It then **creates global mutexes** and proceeds to three parallel infection activities: it **infects removable drives** and **injects into services, calling Export 32**; and, along a second path, it **injects into Step7 and calls Export 32** in order to **infect Step7 projects**. Two of the rootkit files it deploys in this process are named **Mrxcls.sys** and **Mrxnet.sys**. When these steps are complete, that branch exits.

The essential conceptual takeaway from Export 16 is that Stuxnet was not merely a piece of malware that spread; it was a self-managing, self-hiding, version-aware system with a built-in expiry date, engineered to remain quietly resident and current inside a network it could not reach from the outside.

Stuxnet Propagation: Three Avenues of Spread

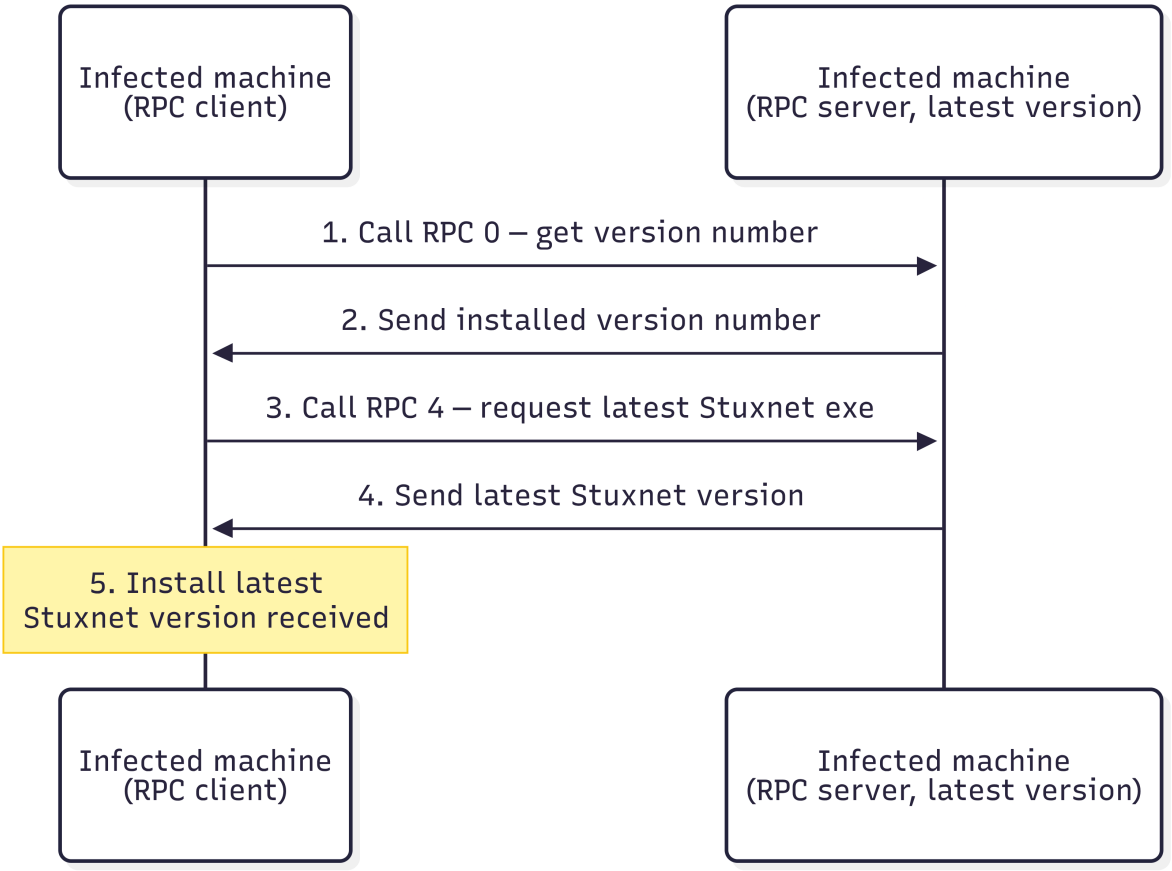
Reaching an air-gapped facility required Stuxnet to be exceptionally versatile in how it moved from machine to machine. It employed three distinct propagation strategies, each suited to a different part of the journey from the outside world to the isolated PLCs at Natanz: **network propagation**, **removable-drive propagation**, and **Step7 project-file infection**.

Peer-to-Peer Network Propagation and Self-Updating

The most sophisticated aspect of Stuxnet's network behaviour was its ability to **update itself across a network of compromised machines**, without any central server, using a peer-to-peer mechanism. This was indispensable for an operation targeting an isolated network: once a newer version of the malware managed to reach even one machine, that version could propagate itself to all the older instances already present, keeping the entire infected population synchronised on the latest code.

The mechanism worked by installing both an **RPC server and an RPC client** on each infected machine. The server was started and listened for incoming connections, while the client searched the network for other Stuxnet servers and initiated communication with them. When two infected machines established contact, they compared versions, and any **older instance was updated to the newer one**. As always with Stuxnet, the update was not executed naively; it was **injected into another process** to remain concealed.

The update exchange followed a defined request-response sequence between an infected machine acting as the RPC client and another acting as the RPC server:



The client first asks the server for its version number; the server replies with its installed version; if the server's version is newer, the client requests the latest executable; the server transmits it; and the client installs the received version. Through this quiet, decentralised gossip, updates diffused throughout the network of compromised machines.

Propagation Through Network Shares

A second network avenue exploited Windows file sharing. Stuxnet **enumerated all user accounts** of both the local computer and the domain. It then attempted to reach **all available network resources**, using either the user's own credential token or **Windows Management Instrumentation (WMI)**, in order to copy itself onto the remote share and execute there. By walking the list of accessible shares with legitimate credentials, it could move laterally through an organisation's internal network from any foothold it had already established.

The Microsoft Vulnerabilities Exploited

To copy itself onto remote machines that were not otherwise reachable, Stuxnet weaponised two specific Microsoft vulnerabilities.

The first was **MS10-061**, a zero-day vulnerability in the **Print Spooler** service. Exploiting it, Stuxnet copied itself to remote computers as `%System%\winsta.exe` by way of the Printer Spooler, and then executed itself there. Because this was a zero-day at the time, no patch existed and no defender was prepared for it.

The second was **MS08-067**, a vulnerability in the **Windows Server Service**. Here Stuxnet connected over the **Server Message Block (SMB)** protocol and sent a **malformed path string** that permitted arbitrary code execution, allowing it to copy itself to unpatched remote computers. Between the peer-to-peer updating, the network-share enumeration, and these two remote-execution exploits, Stuxnet possessed a full toolkit for silent lateral movement through a Windows network.

Removable-Drive Infection

Because the target network was air-gapped, the truly decisive propagation vector was the **removable drive**, since a USB stick carried by a human being was the only realistic way to cross the physical isolation into the plant.

The LNK Vulnerability (CVE-2010-2568)

Three of Stuxnet's exports, numbered **1, 19, and 32**, were responsible for adding or removing the infection on a removable drive. The malware waited for a removable drive to be inserted, verified that it contained a logical volume and was genuinely a removable drive, and if the drive was deemed a suitable candidate, infected it. A notable piece of operational discipline governed this behaviour: **the infection was removed from a drive after it had already infected three PCs**, limiting uncontrolled spread and reducing the chance of discovery.

When infecting a drive, Stuxnet created a specific set of files. It placed a large temporary file, `~WTR4132.tmp` of roughly 500 kilobytes, which **contained Stuxnet's main DLL in its stub section**, alongside a smaller loader file, `~WTR4141.tmp` of roughly 25 kilobytes, whose job was to **load `~WTR4132.tmp`**. It then created a series of shortcut files with escalating names: `Copy of Shortcut to.lnk`, `Copy of Copy of Shortcut to.lnk`, `Copy of Copy of Copy of Shortcut to.lnk`, and `Copy of Copy of Copy of Copy of Shortcut to.lnk`. There were several of these because each one contained an exploit crafted for a different version of Windows, ensuring that whichever operating system the target machine ran, one of the shortcuts would trigger. These shortcuts exploited the **LNK vulnerability, CVE-2010-2568**, which caused the malicious code to execute merely when the drive's contents were displayed, without the user having to knowingly run anything.

The AutoRun.Inf Vector

An alternative activation method relied on the `autorun.inf` mechanism. A key observation here is that **`autorun.inf` parsing in Windows is quite forgiving**: any characters that are not understood as legitimate autorun directives are simply ignored. Stuxnet exploited this leniency by constructing a file that was simultaneously a valid executable and a valid autorun configuration; the executable content of the malware could be padded into the same file that Windows parsed as autorun instructions, because Windows discarded the parts it did not recognise. One visible symptom of this technique was that the right-click context menu on the infected removable disk showed **two "Open" commands** instead of one, both of which led to executing the autorun payload. Through either the LNK exploit or the autorun mechanism, simply plugging in and viewing the drive was enough to trigger infection.

Infecting WinCC Computers

Once inside the plant's network, Stuxnet sought out a specific piece of industrial software: **SIMATIC WinCC**. WinCC is a supervisory control and data acquisition (SCADA) system with a human-machine interface (HMI), produced by Siemens. It is used to monitor and control the physical processes involved in industry and infrastructure on a large scale and over long distances, and it holds the historical distinction of being among the **first known SCADA systems to be specifically targeted by malware**.

When Stuxnet detected a WinCC installation, it carried out a precise sequence of database attacks. It first **connected to the database server using a password hardcoded within the WinCC software itself**, a credential that was fixed and known, and which no operator could realistically change. It then **sent malicious SQL code to the database**, infecting the computer running the WinCC database. It **modified an existing database view**, adding code that would be executed every time that view was subsequently accessed, guaranteeing repeated re-execution of its payload. Finally, it **sent an SQL statement that created a new table and inserted into it a hexadecimal-string representation of the main Stuxnet DLL as an executable file**, identified as resource 210. The hardcoded database password here is a textbook example of exactly the kind of credential weakness that recurs throughout the history of IoT and industrial security.

Step7 Project Infection

The third propagation route targeted the engineering software used to program the controllers, **Step7**. Stuxnet **infected the APIs used to manage a WinCC or Step7 project**, and these infected APIs were the ones typically invoked to **open `.s7p` project files and `.mcp` project components**. In this way, merely opening an engineering project could trigger the infection.

Operationally, Stuxnet took a path to a project, or a compressed archive of a project, as its input. It **listed all the files inside that path**, **validated which of them were candidates for infection**, and then **infected the qualifying files by injecting three things into them**: an encrypted copy of the main Stuxnet DLL, a copy of a Stuxnet data file, and an encoded, updated version of the Stuxnet configuration data block. This meant that infected engineering projects became carriers, propagating the malware to any other engineering workstation on which they were later opened.

The Final Target: The PLC

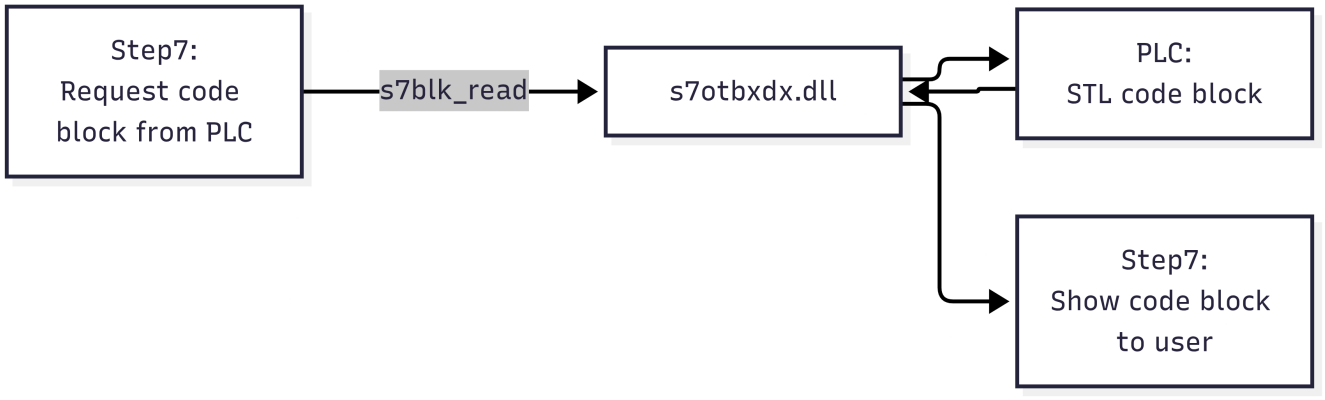
Every propagation mechanism described so far served a single ultimate purpose: reaching and subverting the **Programmable Logic Controller (PLC)**. A PLC is an industrial digital computer adapted specifically for the control of manufacturing processes. In the Natanz scenario it was the targeted device, and PLCs in general are notable for being **highly fault-tolerant** and for being **programmed through dedicated software**, in this case Step7.

The normal relationship between the engineering workstation and the controller is straightforward. The **Step7 control software** runs on a control PC, and it issues **read and write requests** to the **PLC**, which stores and executes its control logic in the form of **STL code** (Statement List, a low-level PLC programming language).



How Step7 Communicates With the PLC

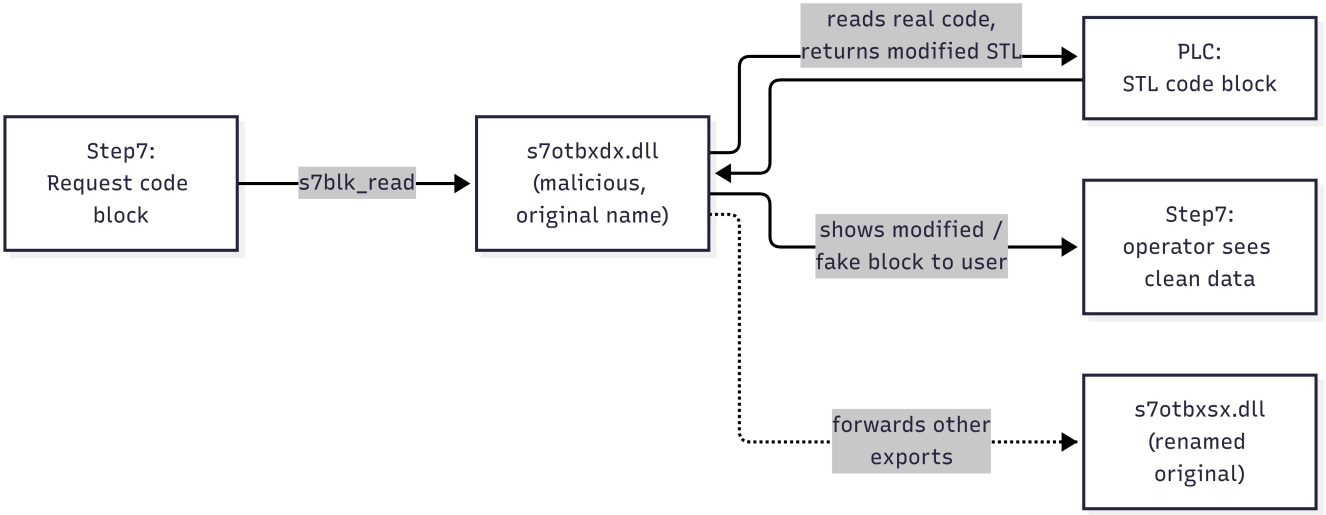
When a PLC is connected to the workstation by cable, the Step7 software is permitted to perform a range of privileged operations: it can **access the memory content** of the PLC, **reconfigure it**, **upload code into it**, and **debug the code running on it**. All of this communication is mediated by a single, critical library file named **`s7otbxdx.dll`**, which **contains all the necessary communication routines** between Step7 and the PLC. In normal operation, when Step7 requests a code block from the PLC, the request passes through this DLL's `s7blk_read` function to the PLC, the PLC returns its STL code block, and the DLL passes that block back up to Step7 for display to the user.



Modifying the PLC: The Man-in-the-Middle DLL

Stuxnet's method for seizing control of the PLC was an elegant and devastating library-substitution attack. It **renamed the legitimate `s7otbxdx.dll` to `s7otbxsx.dll`** and **replaced the original with its own malicious version** bearing the original name. The malicious DLL was carefully constructed to **contain all the exports of the original**, so that Step7 would continue to function normally and suspect nothing. The **majority of these exports were simply forwarded to the renamed legitimate library** (`s7otbxsx.dll`), preserving genuine behaviour, while **sixteen exports were replaced** with Stuxnet's own implementations.

Those sixteen replaced exports gave Stuxnet **control of the read, write, and enumerate functions**, which is precisely the set of capabilities needed to dominate the controller. As a result, Stuxnet could **write its own code into the PLC**, **change operating parameters**, **hide its malicious code from anyone inspecting the controller**, and, critically, **provide fake information to the operator**. This last capability is what allowed the attack to proceed undetected: when an engineer used Step7 to read back the code or the state of the PLC, the malicious DLL intercepted the request and returned a clean, expected result, concealing the modifications underneath.



This man-in-the-middle position between the engineering software and the physical controller is the heart of the entire Stuxnet operation. Having achieved it, the malware could rewrite the logic governing the centrifuges while showing the operators a reassuring fiction.

The PLC Infection Process

With control of the communication library established, Stuxnet turned to the actual infection of the controller. The threat contained **three distinct infection sequences**, conventionally labelled A, B, and C. **Sequences A and B had similar functionality but were aimed at different targets**, while **Sequence C appeared to be disabled or only partially completed**, suggesting either an abandoned line of attack or a component never fully deployed.

The mechanics of infection were driven by the malicious library, which **started two threads**. The **first thread ran an infection routine, either sequence A or sequence B, every fifteen minutes**, repeatedly attempting to compromise the controller. The **second thread queried the PLC for a specific block that would have been injected by the first thread**, but only if the infection process had already succeeded. This injected block was **customised**, and its presence and content **impacted the way sequences A or B ran on the infected PLC**, allowing the attack to tailor its behaviour to the particular controller it had compromised.

The Infection Sequence in Detail

Each infection sequence unfolded through a series of stages that together transformed a legitimate controller into a covert saboteur.

The sequence began with a **System Data Block check**. The PLC's blocks were **enumerated and parsed**, and the malware **searched for PLCs that used specific frequency-converter drives**. This targeting detail is crucial: Stuxnet was not interested in any PLC, but only in those controlling the particular type of variable-frequency drives used to spin the enrichment centrifuges. This selectivity is what confined the destructive payload to its intended victims.

Next came the **replacement of communication protocols**. From this point on, **all communications received from or sent to the PLC were processed by Stuxnet before delivery**, cementing the man-in-the-middle position at the level of the controller's own traffic and enabling both the manipulation of commands and the falsification of readings.

Then came the **modification of Organization Blocks**. Organization Blocks are in charge of coordinating the activity of the operational blocks within the PLC's program. Stuxnet arranged for its **malicious code sequence to be executed at the start of a cycle**, hijacking the controller's regular

scan cycle, and it took measures to **prevent the blocking of its malicious code execution**, ensuring its payload could not be easily stopped. Finally, the **malicious blocks were written into the PLC**, completing the subversion of the controller's logic.

What Happened: The Physical Sabotage

The purpose of all this machinery was a specific, patient act of physical sabotage against the centrifuges, executed through two attack routines that each followed a similar five-step rhythm of watch, wait, strike, and reset.

In the first routine, the **frequency-converter slaves sent records about their operating frequencies**, and Stuxnet **examined these frames and recorded the relevant fields**, quietly building a picture of normal operation. After **approximately thirteen days**, enough events had been recorded to establish that the system had been **operating correctly between 807 Hz and 1210 Hz**. Only then did the malware strike: the infected PLC **commanded its frequency-converter drives to set the frequency to 1410 Hz**, well above the normal operating range, before shortly **dropping the frequency to 1064 Hz**, after which the **PLC reset its variables** and the cycle began again.

The second routine followed the same logic with different timing and values. The **PLC collected information**, and after **approximately twenty-seven days**, once enough events had been recorded, the infected PLC **commanded its frequency-converter drives to operate at 2 Hz**, a near-halt, before commanding them **back to 1064 Hz**, and again the **PLC reset its variables**.

The physical significance of these frequency manipulations is what turned software into a weapon. Centrifuges are precision devices spinning at enormous speeds; driving them far above their normal frequency and then abruptly far below it subjects them to mechanical stresses that damage or destroy them over time. Crucially, because the changes were interspersed with long periods of normal operation and separated by many days, they resembled sporadic equipment faults rather than a coordinated attack.

What the Engineers Observed

From the perspective of the plant's engineers, the situation was maddeningly opaque, and this opacity was itself the masterstroke of the attack. The engineers were **coming to understand that the turbines were operating over-speed and under-speed**, sensing that something was physically wrong with the centrifuges. Yet **the PLC, under Stuxnet's control, reported that everything was within the limits**, because the malicious library was feeding the monitoring software fabricated, acceptable values. Compounding their helplessness, any **actual manipulation of the legitimate control logic that the engineers attempted had no effect**, since Stuxnet sat between their commands and the hardware, intercepting and neutralising their interventions. The engineers were, in effect, locked out of their own plant while being shown a screen that insisted all was well, the precise embodiment of what it means for a security failure in a smart environment to become a physical failure.

Stuxnet's Legacy: A Modern Echo

The lessons of Stuxnet are thrown into sharp relief by a much more recent and far cruder incident that struck the same class of target. On **25 November 2023**, a booster station of the **Municipal Water Authority of Aliquippa**, in Pennsylvania, was hijacked, and its pumping had to be switched to manual operation. The target was a **Unitronics Vision PLC** whose human-machine interface was **exposed directly on the Internet**, on **TCP port 20256**, protected by nothing more than the **default password 1111**. The attacker was a group called **CyberAv3ngers**, affiliated with the Iranian Revolutionary Guard Corps; their motive was that the equipment was Israeli-made, and they defaced the

device's screen. This was part of a broader campaign of **four waves of attacks against Unitronics devices** in the water, energy, food, and healthcare sectors, documented in **CISA advisory AA23-335A**.

The contrast with Stuxnet is the entire lesson. **No exploit was needed at all**: the device simply did exactly what it had been configured to do for anyone who supplied the default password. Where Stuxnet represented the pinnacle of sophistication, requiring stolen certificates, multiple zero-days, and a bespoke man-in-the-middle library, Aliquippa required only a guess. It stands as a demonstration of **OWASP IoT risk number one, weak, guessable, and hard-coded passwords, appearing in critical infrastructure five years after that list was published**. The sophistication of the attacker had collapsed to nearly nothing, and yet the impact on a physical utility was immediate, precisely because the underlying weaknesses of industrial smart environments had not changed.

The Kia Portal Case: When the Weakest Link Is the Ecosystem

A second modern case demonstrates a different but equally important lesson: the most vulnerable part of a smart environment is frequently not the device at all, but the **ecosystem interface** built around it. In **June 2024**, a group of researchers, Curry, Rivera, Rhinehart, and Carroll, discovered that they could **control any Kia vehicle manufactured after 2013 starting from nothing more than its licence plate**.

The attack was a chain of steps, none of which involved compromising the car itself. It exploited the **dealer web portal**. The researchers were able to **create a fake dealer account** on that portal, which granted them access to an **API that mapped a licence plate to the vehicle identification number (VIN), and the VIN to the owner**. Using this, the attacker could simply **add themselves as a second authorised user of the vehicle**. The effects were immediate and severe. In **under thirty seconds**, an attacker could **locate, unlock, start, and honk** the target car, and could also retrieve the **owner's name, phone number, e-mail address, and home address**. Notably, **no active subscription to any connected-car service was required** for this to work. Kia fixed the flaw in **August 2024**, and there is no evidence it was ever exploited maliciously in the wild.

The lesson is the essential complement to Stuxnet's. The car was fine; the portal was not. **The weakest link of a smart environment is often its ecosystem interface, corresponding to OWASP IoT risk number three, insecure ecosystem interfaces, rather than the device itself.** A perfectly secure vehicle can be wholly compromised through a badly designed web application that surrounds it, because in a smart environment the boundary of the attack surface extends far beyond the physical object into every supporting service, API, and back-end that touches it.

Case Study II: Mirai and the Botnet Threat

The Mirai Malware

Where Stuxnet was a precision instrument of state sabotage, **Mirai** represents the opposite pole of the threat landscape: mass, indiscriminate compromise of cheap consumer devices for the purpose of raw disruptive power. Mirai **turns networked devices running Linux into remotely controlled bots that can be marshalled as part of a botnet operating at large scale**. It **primarily targets online consumer devices such as IP cameras and home routers**, the very sort of inexpensive, rarely-updated equipment that populates smart homes by the billion. Its speed was extraordinary: in its

early operation it **infected nearly 65,000 devices in just 20 hours**, a rate of growth that illustrates how quickly a self-propagating worm can weaponise the consumer IoT.

Understanding Bots and Botnets

To understand Mirai it is necessary to be precise about the vocabulary of botnets. A **bot** is a program that secretly takes over another Internet-attached computer and then uses that computer to launch attacks that are difficult to trace back to the bot's true creator; a compromised machine of this kind is also known as a **zombie** or a **drone**. Bots are typically **planted on hundreds or thousands of devices belonging to unsuspecting third parties**, who have no idea their equipment is participating in an attack. A **collection of bots acting in a coordinated manner is a botnet**.

A botnet, to function as such, exhibits **three defining characteristics**: the **bot functionality** itself, meaning the malicious capability installed on each device; a **remote-control facility**, meaning the command-and-control channel through which the operator directs the bots; and a **spreading mechanism** that propagates the bots and thereby constructs and grows the botnet.

The Many Uses of Bots

Once an attacker commands a botnet, the aggregated resources of many devices can be turned to a wide range of malicious ends. The most notorious use is the **distributed denial-of-service (DDoS) attack**, which causes a loss of service to legitimate users by overwhelming a target. Botnets are also used for **spamming**, sending massive volumes of bulk e-mail, and for **sniffing traffic**, in which a packet sniffer running on the compromised device retrieves sensitive information such as usernames and passwords. They perform **keylogging**, capturing the keystrokes of the device's legitimate user, and they are used for **spreading new malware**, since an existing botnet is an ideal delivery network for new bots.

Beyond these, botnets support **installing advertisement add-ons and browser helper objects (BHOs)**, in which the attacker sets up a fake website and negotiates a deal with hosting companies that pay for advertising clicks, then uses the bots to generate fraudulent clicks. They are used for **attacking Internet Relay Chat (IRC) chat networks**, flooding a victim with requests to bring down the IRC network in a manner similar to a DDoS attack. And they are used for **manipulating online polls and games**, exploiting the fact that every bot has a distinct IP address so that each fraudulent vote or action appears to come from a distinct real person.

The Mirai Event Timeline

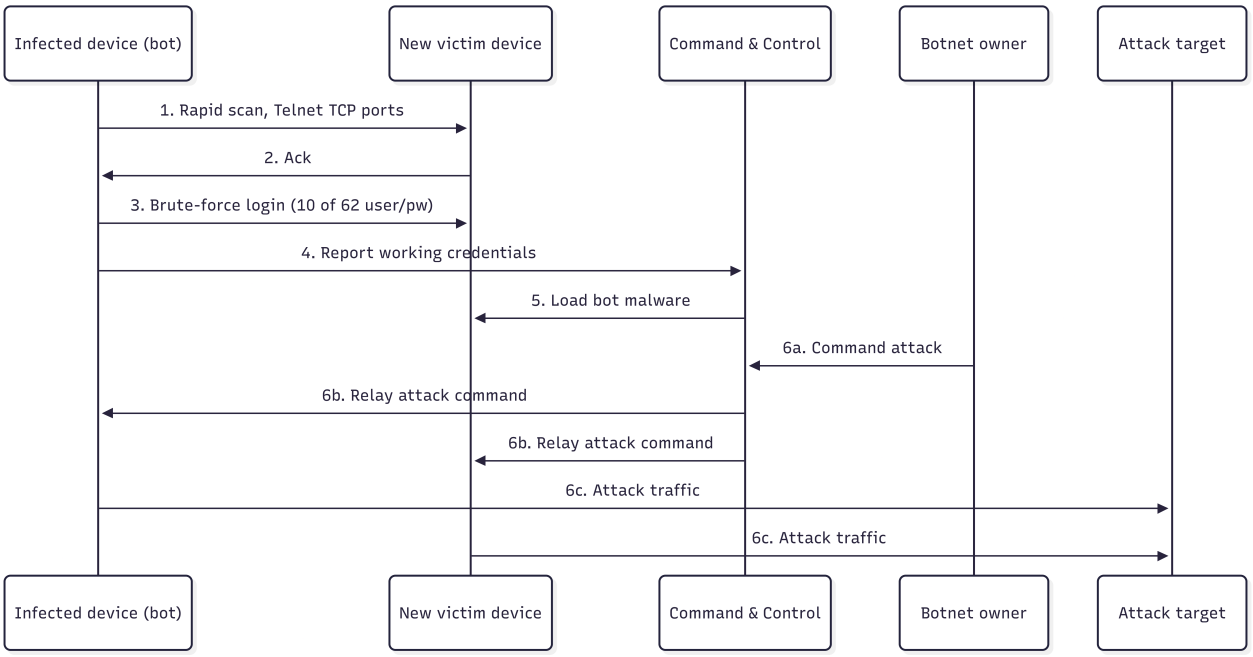
Mirai's public history unfolded rapidly across late 2016 and into 2017. The malware **surfaced on 1 August 2016**. Its first large attacks came against the hosting provider **OVH on 18 September 2016**, followed by a **peak attack on the security journalist Brian Krebs's site (Krebs on Security) on 21 September 2016**. On **30 September 2016 the Mirai source code was released** publicly, a pivotal moment whose consequences are discussed below. The attacks against the DNS provider **Dyn occurred on 21 October 2016**, and an attack against **Liberia's Lonestar network began on 31 October 2016**. A new exploit, the **Deutsche Telekom CWMP exploit, appeared on 26 November 2016**. The **author of Mirai was identified on 18 January 2017**, and a **Deutsche Telekom attacker was arrested on 23 February 2017**.

How Mirai Operates: Scanning, Brute-Forcing, and Recruiting

The operational cycle by which Mirai grows and then attacks can be reconstructed step by step. Imagine an already-infected IP camera. That compromised device begins by performing a **rapid scan of the**

network, probing Telnet on TCP ports, searching for additional devices to infect. When it finds a candidate that responds, it attempts to log in through a **brute-force process**, trying a small dictionary of credentials. A key efficiency choice governs this step: Mirai typically tried only about **ten passwords out of a list of sixty-two possible username-and-password combinations per device**, deliberately limiting the effort so as not to waste time on any single target and instead infect a larger set of devices overall.

When a valid password is found, the compromised device **sends a report to the command-and-control (C2) infrastructure**, communicating the credentials and address of the newly discovered vulnerable device. The **C2 then arranges for the Mirai bot to be loaded onto the new device**, infecting it and enrolling it in the botnet. Finally, when the operator wishes to strike, the **owner of the botnet communicates through the C2 to the assembled devices, commanding them to launch the attack** against a chosen victim.



This self-amplifying loop, in which each newly infected device becomes a scanner recruiting still more devices, is what allowed Mirai to grow to tens of thousands of bots within a single day.

The Dyn Botnet DDoS Attack

The most consequential deployment of Mirai was the **distributed denial-of-service attack of October 2016 against the DNS provider Dyn**. Because Dyn provided domain-name resolution for a large portion of the Internet, taking it down had cascading effects, and this attack was **staged and launched from IoT devices running Mirai**. It became the defining demonstration of how the compromised consumer IoT could be turned into a weapon capable of disrupting major online services.

Mirai was engineered for **two main purposes**. The first was to **find and infect IoT devices in order to grow the botnet**, the self-propagating behaviour described earlier. The second was to **participate in DDoS attacks based on commands received from the remote command-and-control (C2) infrastructure**. To serve these ends, Mirai operates in **three stages**: it infects the device, it protects itself, and it launches the attack.

Stage 1: Infect the Device

In the first stage, Mirai **scans for IoT devices that are accessible over the Internet**. It **primarily scans for open ports such as 22, 23, and 5747**, among others, though it **can be configured to**

scan for additional ports. Port 23 is Telnet and port 22 is SSH, the remote-access services through which the malware seeks entry. Once it connects to a device, it **brute-forces usernames and passwords to log in**, and having gained access it **uses that device to scan networks looking for still more IoT devices**, perpetuating the recruitment cycle.

Stage 2: Protect Itself

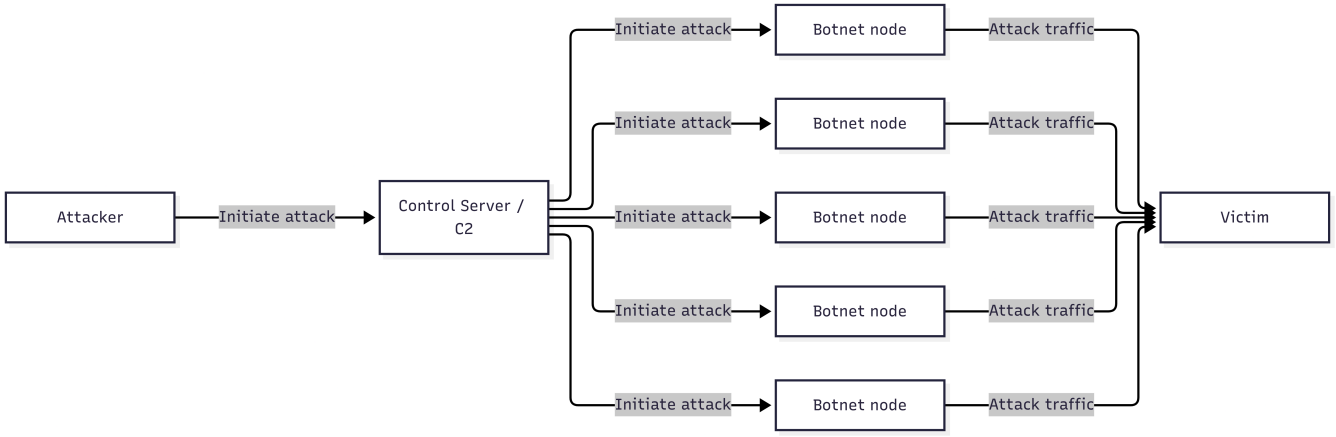
In the second stage, Mirai acts to secure its hold on each newly captured device. It **kills other processes running on the infected device, specifically SSH, Telnet, and HTTP**, in order to **prevent the legitimate owner from gaining remote access to the device while it is infected**. This both denies the owner the ability to detect and remove the malware and prevents rival malware from claiming the same device. An important practical note accompanies this stage: **rebooting the device can remove the malware**, because Mirai resides in volatile memory and does not achieve persistence, **but the device can simply become infected again** almost immediately, since the underlying vulnerability, the weak default credential, remains unchanged after the reboot.

A further notable design detail is that **Mirai contained a hardcoded list of known networks in the United States that it was instructed to avoid attacking**, among them the **United States Postal Service and the Department of Defense**. This deliberate avoidance of certain high-profile government networks is a recurring observation about the malware's behaviour.

Stage 3: Launch the Attack

In the third stage, upon command, the **infected device launches different types of attacks**. These included **HTTP floods and SYN floods**, among other **DDoS-based techniques**. In the Dyn campaign the assault took the form of **twenty-one short-lived attacks of about twenty-five seconds each, followed by two long attacks**, and it targeted **the Dyn infrastructure and the PlayStation infrastructure**.

The overall architecture of such an attack can be visualised as a hierarchy flowing from a single human attacker down through control infrastructure to a mass of geographically dispersed bots, all converging their traffic on one victim.



The key structural insight of this diagram is the separation between the attacker and the attack. The traffic that strikes the victim originates not from the attacker's own machine but from thousands of innocent third-party devices, which is precisely what makes a DDoS attack both powerful, because of its aggregate volume, and difficult to trace, because the true originator is hidden behind the botnet.

The Creators of Mirai

The origin story of Mirai carries its own ironic lesson. The botnet was created by **Paras Jha and Josiah White**, who were **both twenty years old** at the time. Tellingly, they had **co-founded Protraf Solutions, a company that offered mitigation services for DDoS attacks**. In other words, the creators of one of history's most disruptive DDoS tools were simultaneously in the business of selling protection against DDoS attacks, an arrangement that blurred the line between the arsonist and the fire brigade.

Why Mirai Remains Dangerous

Mirai did not end with the arrest of its creators, and understanding why is essential. The malware **remains dangerous because it is mutating**, and above all because **its source code lives on**: it was published publicly and remains available in repositories online, notably a well-known GitHub mirror. Once the source code was released, any attacker could take it, modify it, and deploy a new variant, which is exactly what happened.

Consequently, Mirai **has given birth to a family of variants**, including **Okiru, Satori, Masuta, PureMasuta, and IoTrooper and Reaper**. Each of these descendants inherited Mirai's fundamental approach while adding new capabilities or targeting new devices. The public release of the source code transformed Mirai from a single botnet into an enduring template, a "playbook" that would be reused for years to come.

Mirai a Decade Later: Aisuru and Kimwolf

The durability of that playbook is confirmed by the most recent botnets, which show that a decade of progress in the industry has not closed the underlying gaps. **Aisuru** is a Mirai-derived botnet first observed in late 2024, growing to encompass up to millions of routers, digital video recorders, and IP cameras. **Kimwolf**, which appeared in December 2025, infected more than two million Android TVs and set-top boxes, recruited through residential proxy networks and through an **exposed Android Debug Bridge** interface.

The scale of the disruption these botnets could inflict reached unprecedented levels. Record distributed-denial-of-service events included a **15.7-terabit-per-second attack against Microsoft Azure in October 2025**, followed by a **31.4-terabit-per-second attack, generating 14 billion packets per second, in November 2025**. Their **business model remained fundamentally unchanged since 2016: DDoS-for-hire and residential proxies**. In **March 2026, the United States Department of Justice disrupted Aisuru, Kimwolf, and the related botnets JackSkid and Mossad, together comprising more than three million devices**.

The lesson is stated plainly and it is the culmination of the entire Mirai narrative: **the Mirai playbook still works; the devices changed, but the vulnerabilities did not.** Ten years of new hardware, from IP cameras to Android TV boxes, has done nothing to change the fundamental weaknesses, default credentials, exposed services, absent updates, that the original Mirai exploited.

BadBox 2.0: Compromised Before the First Boot

A final recent case introduces a threat that even a perfectly diligent user cannot prevent: the **supply-chain compromise**. In the **BadBox 2.0** campaign of 2025, a **backdoor was pre-installed at the factory** in low-cost Android TV boxes, tablets, and projectors. The scale was enormous, affecting **about ten million devices**; the **FBI issued a public alert in June 2025**, and Germany's federal cybersecurity agency, the **BSI, had already sinkholed 30,000 of the devices in December 2024**.

The infected devices were **used for ad fraud, residential proxying, and as a springboard for delivering further malware**.

What makes BadBox 2.0 conceptually distinct is that **no user action can prevent it**: the device ships already compromised, and the update channel itself belongs to the attacker, so there is no clean state to return to and no trusted mechanism through which to remediate. The lesson expands the threat model in a fundamental way: **supply-chain trust is part of the security picture, and OWASP IoT risk number five, the use of insecure or outdated components, begins before the device is even purchased.** Security can no longer be assumed to start at the moment of installation; it must be traced all the way back through the manufacturing and distribution chain.

Securing the Internet of Things

The Main Challenges of IoT Security

Before examining defensive measures, it is important to consolidate the reasons why securing the IoT is intrinsically difficult. These challenges are not incidental; they are structural features of the domain that any solution must contend with.

The first challenge is the **very large attack surface combined with widespread deployment**: with tens of billions of devices distributed across every corner of physical and digital life, the sheer number of entry points is unmanageable by conventional means. The second is the **limited resources of the devices themselves**: constrained energy budgets and modest computational power make it hard to run the cryptographic and monitoring machinery that security demands. The third is the **lack of standards and regulations**, which leaves the field fragmented and inconsistent. The fourth is the difficulty of **integrating safety and security processes**, since in cyber-physical systems the two concerns are entangled and must be reconciled rather than treated separately.

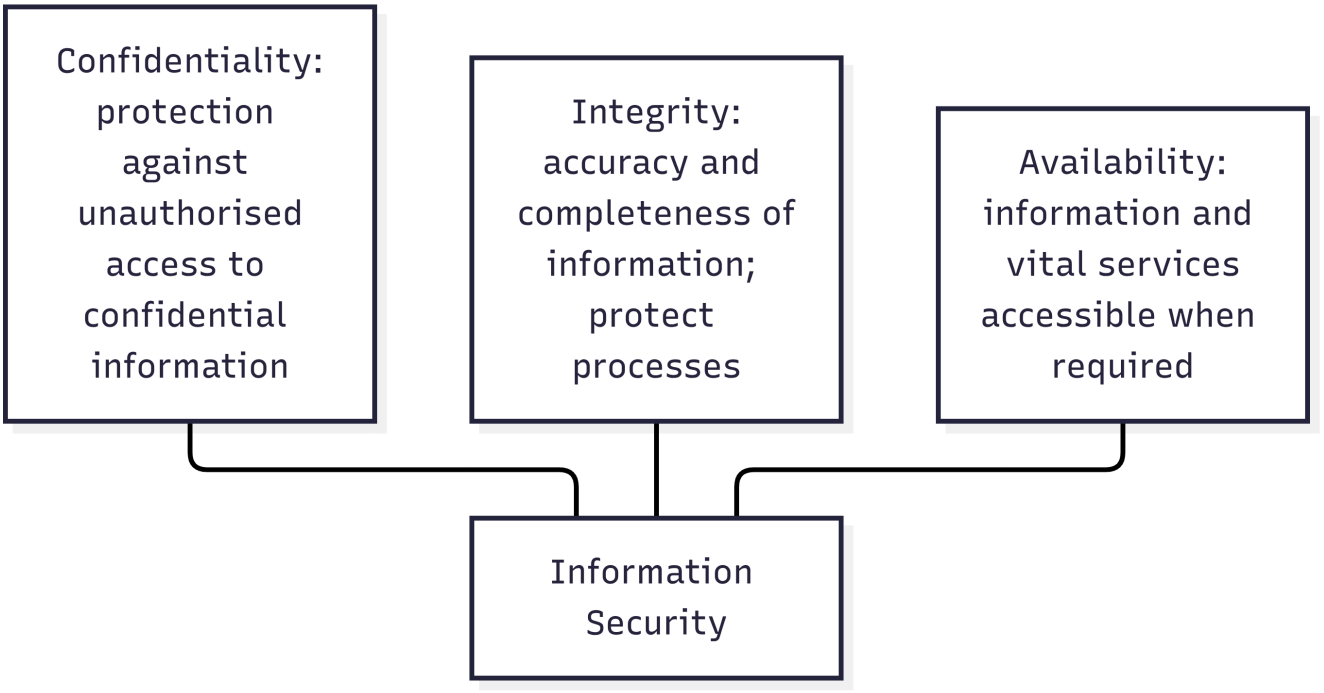
Further challenges compound these. **Security by design is not a top priority** for many manufacturers, who compete on price and features rather than protection. There is a pervasive **lack of expertise**, as the developers of cheap connected devices are often not security specialists. **Applying security updates** is difficult or impossible once devices are fielded, as noted throughout the history of the domain. **Insecure development** practices introduce flaws at the source. And **unclear liabilities** mean that responsibility for a breach is often diffuse, so that no single party is fully accountable for securing the device across its life. Together these challenges explain why the same vulnerabilities recur decade after decade.

The CIA Triad

The foundational framework for reasoning about any security problem, including the IoT, is the **CIA triad**, which names the three core properties of information security: **Confidentiality, Integrity, and Availability**.

Confidentiality exists to ensure protection against unauthorised access to, or use of, confidential information. **Integrity** exists to ensure the accuracy and completeness of information, so as to protect the business processes that depend on it. **Availability** exists to ensure that information and vital services are accessible for use whenever they are required. These three properties form the vertices of a triangle at whose centre sits the overall goal of information security; a defence that satisfies all three is

complete, while a weakness in any one of them compromises the whole.



The CIA Triad Applied to the IoT

When the triad is mapped onto the specific realities of the IoT, each property acquires a concrete and somewhat expanded meaning.

Confidentiality and privacy in the IoT require that the data collected from a device should not reveal secure information to any unauthorised device, and that data should remain controlled by the corresponding user to whom it belongs. Because IoT data is so often personal, privacy is inseparable from confidentiality here. Three families of technique are used to protect it. **Anonymity-based** approaches strip or obscure the identity associated with data so that even if it is seen, it cannot be linked to an individual. **Encryption-based** approaches render the data unreadable to anyone lacking the key. **Perturbation-based** approaches deliberately add noise or distortion to the data so that its aggregate statistical value is preserved while individual records are protected.

Integrity in the IoT means, straightforwardly, that the **data is not tampered with**, so that the readings a system acts upon and the commands it issues are exactly as intended and have not been altered by an attacker, which, as Stuxnet showed, is the difference between a plant that is safe and one that is being quietly destroyed.

Availability in the IoT means that the **services can always access the data** they need, and, critically, that they can do so in **real time**. The real-time requirement is a distinctive and demanding feature of the IoT: a delayed reading in a control loop for a vehicle brake or an insulin pump is not a minor degradation but a safety failure, which is why availability in this domain carries a weight it does not always carry in conventional IT.

Other Security Features

Beyond the three core CIA properties, two additional security features are essential in the IoT context.

The first is **identification and authentication**, which must be established for three distinct classes of entity: the **devices** themselves, the **applications** that interact with them, and the **users** who operate the system. Each must be reliably identified and its identity verified before it is granted access, because

in an environment of billions of interacting components, the question of "who or what is this" is foundational to every other control.

The second is **trust**, which must be established along several dimensions: between the **layers** of the architecture, between the **devices**, and between the **applications**. Because a smart environment is assembled from heterogeneous components made by different manufacturers and operated by different parties, the relationships between these components cannot be assumed to be trustworthy by default; trust must be deliberately established and maintained, at each layer boundary and between each pair of interacting entities.

The OWASP Top 10 IoT Risks and Vulnerabilities

The single most authoritative catalogue of the ways in which IoT devices go wrong is the list of the **ten most important IoT risks and vulnerabilities** published by OWASP (the Open Worldwide Application Security Project). This list is worth committing to memory, because nearly every real-world incident examined in this chapter, from Stuxnet's hardcoded WinCC password to the Aliquippa water authority's default `1111` to the Kia dealer portal, maps directly onto one of its ten entries. The list functions as the connective tissue of the whole subject: it is the vocabulary in which the recurring root causes are named.

The **first risk is weak, guessable, or hardcoded passwords**. This is the use of credentials that are easily brute-forced, publicly available, or simply unchangeable because they are baked into the device. It is the most common and most consequential IoT weakness, and it is the one that enabled both Mirai's brute-forcing of consumer devices and the CyberAv3ngers' takeover of water-utility PLCs.

The **second risk is insecure network services**. These are unneeded or insecure network services running on the device itself, especially those exposed to the Internet, which can compromise the confidentiality, integrity, or availability of information or allow unauthorised remote control. Telnet, the service Mirai scanned for on port 23, is the archetypal example of an insecure network service that should never have been exposed.

The **third risk is insecure ecosystem interfaces**. These are insecure web, back-end API, cloud, or mobile interfaces existing in the ecosystem *outside* of the device, which nonetheless allow the compromise of the device or its related components. The Kia case is the perfect illustration: the vulnerability lived entirely in the dealer web portal and its API, not in the car.

The **fourth risk is the lack of a secure update mechanism**. This is the absence of the ability to securely update the device, and its symptoms include the lack of firmware validation on the device, the lack of secure delivery such as transmitting updates in plaintext, and the lack of anti-rollback mechanisms that would prevent an attacker from forcing a device back to a known-vulnerable version.

The **fifth risk is the use of insecure or outdated components**. This is the use of deprecated or insecure software components and libraries that could allow the device to be compromised, and it includes the insecure customisation of operating-system platforms and the use of vulnerable third-party software. The BadBox 2.0 supply-chain compromise is a manifestation of this risk operating even before purchase.

The **sixth risk is insufficient privacy protection**. This is the situation in which a user's personal information is stored on the device and is then used insecurely or without permission, a direct affront to the confidentiality-and-privacy property discussed earlier.

The **seventh risk is insecure data transfer and storage**. This is the lack of encryption or access control for sensitive data anywhere within the ecosystem, whether that data is at rest in storage, in transit across the network, or in use during processing. All three states of data must be protected, not merely one.

The **eighth risk is the lack of device management**. This is the absence of security support for devices deployed in production, encompassing asset management, update management, secure decommissioning, systems monitoring, and response capabilities. Without device management, an operator cannot even know what devices they have, let alone secure or retire them.

The **ninth risk is insecure default settings**. This describes devices or systems shipped with insecure default settings, or those that lack the ability to be made more secure because they restrict operators from modifying their configurations. A device that arrives insecure and cannot be hardened is insecure for its entire life.

The **tenth risk is the lack of physical hardening**. This is the absence of physical hardening measures, which allows potential attackers to gain sensitive information that can aid future remote attacks, or to take local control of the device. Because IoT devices live in the physical world, often in places accessible to attackers, physical access is a genuine and often underestimated threat.

The following table consolidates the list for study and reference.

#	Vulnerability / Risk	Description
1	Weak, Guessable, Hardcoded Passwords	Using easily brute-forced, publicly available, or unchangeable credentials
2	Insecure Network Services	Unneeded or insecure network services on the device, especially those exposed to the Internet, compromising the CIA of information or allowing unauthorised remote control
3	Insecure Ecosystem Interfaces	Insecure web, back-end API, cloud, or mobile interfaces outside the device that allow compromise of the device or its related components
4	Lack of Secure Update Mechanism	Inability to securely update the device: no firmware validation, plaintext delivery, no anti-rollback mechanisms
5	Use of Insecure or Outdated Components	Deprecated or insecure software components/libraries, insecure OS customisation, vulnerable third-party software
6	Insufficient Privacy Protection	User's personal information stored on the device and used insecurely or without permission
7	Insecure Data Transfer and Storage	Lack of encryption or access control for sensitive data at rest, in transit, or during processing
8	Lack of Device Management	No security support in production: asset management, update management, secure decommissioning, monitoring, response
9	Insecure Default Settings	Insecure default settings, or inability to harden the system because operators cannot modify configurations
10	Lack of Physical Hardening	No physical hardening, allowing attackers to extract sensitive information for future remote attacks or to take local control

Two Categories of Flaw: Design and Library

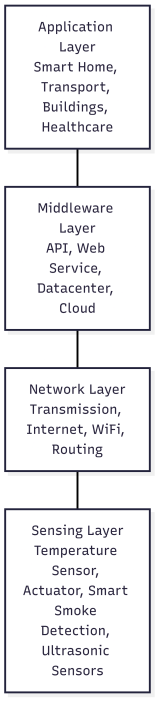
It is illuminating to observe that the failures catalogued by OWASP tend to originate in one of two broad categories. The first is the **design flaw**, a weakness rooted in the fundamental architecture of a product,

such as an unpatchable flaw in a modern car that allows an attacker to disable safety features; because the flaw is architectural, it often cannot be remediated after the fact. The second is the **library flaw**, a weakness introduced by a defective shared software component, such as the "Devil's Ivy" vulnerability, in which a single bad code library triggered a vulnerability across millions of IoT devices at once. The library category is particularly dangerous because a single defect propagates automatically into every product that incorporates the affected component, echoing directly the OWASP risk of insecure or outdated components.

Sources of Security Threats by Architectural Layer

Because an IoT application is built from several architectural layers, and because **each of these layers uses diverse technologies that bring their own issues and security threats**, it is analytically useful to catalogue the attacks that afflict each layer in turn. This layered view of the threat landscape complements the OWASP list: where OWASP names the categories of weakness, the per-layer catalogue locates the specific attacks at the point in the architecture where they occur.

For this purpose the architecture is conventionally divided into four layers stacked from the physical world upward: the **sensing layer** at the bottom, containing temperature sensors, actuators, smart-smoke detectors, and ultrasonic sensors; the **network layer** above it, handling transmission, Internet connectivity, Wi-Fi, and routing; the **middleware layer**, containing APIs, web services, the datacenter, and the cloud; and the **application layer** at the top, encompassing smart-home, smart-transport, smart-buildings, and smart-healthcare applications.



Security Issues at the Sensing Layer

The sensing layer, being physically exposed and resource-constrained, is subject to a broad range of attacks. **Node capturing** is the physical seizure of a sensor node by an attacker. **Malicious code injection** introduces attacker-controlled code into a node. **False data injection** feeds the system fabricated sensor readings, corrupting the information on which decisions are based. **Replay attacks** capture legitimate transmissions and retransmit them later to deceive the system. **Cryptanalysis and side-channel attacks** attempt to recover secret keys, either by breaking the cryptography directly or by observing physical characteristics such as timing or power consumption. **Eavesdropping and interference** intercept or disrupt the communications of the layer. **Sleep-deprivation attacks** exploit

the energy constraints of battery-powered nodes by keeping them awake until their batteries are exhausted, a form of denial of service unique to constrained devices. And **booting vulnerabilities** exploit the moment of start-up, when a device may be in an insecure state before its protections are fully active.

Security Issues at the Network Layer

The network layer, responsible for moving data, is subject to attacks against communication and routing. **Phishing-site attacks** attempt to lure users or systems to fraudulent destinations. **Access attacks** seek unauthorised entry into the network. **Denial-of-service and distributed-denial-of-service, (D)DoS, attacks** overwhelm the network to deny service. **Data-transit attacks** target information while it is moving across the network. **Routing attacks** subvert the paths that traffic follows, and two important sub-types are named explicitly: the **sinkhole attack**, in which a malicious node advertises itself as an attractive route in order to draw traffic through itself, and the **wormhole attack**, in which two colluding nodes tunnel traffic between distant points to distort the network's topology. Finally, **spoofing attacks** forge identities or addresses to impersonate legitimate participants.

Security Issues at the Middleware Layer

The middleware layer, which hosts the APIs, web services, and cloud back-end, is subject to attacks characteristic of web and cloud infrastructure. The **man-in-the-middle attack** interposes the attacker between two communicating parties. **SQL injection** exploits improperly handled input to manipulate a back-end database, exactly the class of attack Stuxnet used against the WinCC database. The **signature-wrapping attack** manipulates the digital signatures used in web-service messages to alter their content while preserving apparent validity. **Cloud malware injection** introduces malicious code or services into the cloud environment. And the **flooding attack in the cloud** overwhelms cloud resources to deny service or inflate costs.

Security Issues at the Gateways

The gateway, occupying the sensitive boundary between the device world and the wider network, presents its own distinct set of security concerns. **Secure onboarding** is the challenge of safely admitting a new device into the environment through the gateway without allowing an impostor to join. **Extra interfaces** are a concern because gateways often expose additional management or debugging interfaces that enlarge the attack surface. **End-to-end encryption** is a concern because, as noted earlier, the gateway may terminate and re-encrypt traffic, seeing it in clear at the point of translation, so ensuring genuinely end-to-end protection is difficult. And **firmware updates** are a concern because the gateway, like the devices it serves, must itself be kept current, yet is subject to the same update difficulties.

Security Issues at the Application Layer

The application layer, closest to the user and the business logic, is subject to attacks aimed at data and functionality. **Data thefts** target the sensitive information the application handles. **Access-control attacks** attempt to bypass the permissions that govern who may do what. **Service-interruption attacks** deny the application's availability to legitimate users. **Malicious code injection** introduces attacker code into the application. **Sniffing attacks** capture sensitive application data in transit. And **reprogram attacks** attempt to alter the application's behaviour by rewriting its logic, echoing at the application level the kind of logic subversion that Stuxnet performed at the controller level.

Making the IoT Reliable, Secure, and Robust

Having surveyed the threats layer by layer, the final concern is the constructive one: the practices and technologies through which an IoT system can be made genuinely reliable, secure, and robust. These measures span the full life of a device, from its development through its deployment to its eventual retirement, and they address the challenges enumerated earlier at their source.

Penetration testing subjects a system to simulated attacks in order to discover its weaknesses before a real attacker does. The **use of encryption at different layers** protects data throughout the architecture rather than at a single point, so that confidentiality and integrity are maintained end to end. **Authentication** ensures that only legitimate devices, applications, and users gain access. **Device test and validation** verifies that a device behaves correctly and securely before it is trusted. **Reducing security costs** matters because, given the extreme price sensitivity of IoT hardware, security measures must be affordable enough to be adopted at all. **Data-flow validation** checks that information moves through the system only along expected and legitimate paths.

Further measures address the operational and end-of-life phases. **Machine-learning-based security** brings adaptive detection of anomalies and attacks, particularly valuable given the scale and variety of IoT traffic. **Automatic firmware patching and updating** confronts head-on the perennial difficulty of keeping fielded devices current, removing the reliance on manual intervention that so often fails. **Notification** ensures that operators and users are informed of security-relevant events. **Facilitating device identification** supports the foundational requirement that every entity in the environment be reliably known. And **end-of-life** management ensures that devices are securely decommissioned when they are retired, so that they do not linger as unmonitored, unpatched liabilities, the very "soft targets" that the Age of Protection was meant to eliminate.

Defensive Security Measures Catalogue

The defensive posture can be organised as a mapping from each category of weakness to the concrete security considerations that address it. This catalogue is the practical counterpart to the OWASP list of risks: for every way a device can be weak, it names what a responsible designer or operator should do. The following two tables consolidate the full set of measures.

Category	IoT Security Consideration
Insecure Web Interface	Disallow weak user passwords; provide an account-lockout mechanism; test the interface for SQL-injection, XSS, and CSRF vulnerabilities
Insufficient Authentication / Authorization	Require strong passwords for authentication; implement two-factor authentication; force password expiration after a certain date
Insecure Network Services	Ensure all devices operate with minimal ports active; ensure devices do not expose network ports or services to the Internet via UPnP; review required network services for vulnerabilities
Lack of Transport Encryption	Ensure traffic is encrypted between all system components; ensure SSL/TLS implementations are updated and configured properly
Privacy Concerns	Ensure only the minimal amount of personally identifiable information (PII) is collected from consumers; ensure only non-sensitive data is analysed; ensure a data-retention policy is in place

The measures continue with the interfaces and physical dimensions of the system.

Category	IoT Security Consideration
Insecure Cloud Interface	Ensure all cloud interfaces are reviewed for vulnerabilities; ensure any cloud-based web interface disallows weak passwords; ensure all cloud interfaces use transport encryption
Insecure Mobile Interface	Ensure any mobile application disallows weak passwords; ensure any mobile application has an account-lockout mechanism; implement two-factor authentication for mobile applications
Insufficient Security Configurability	Ensure password-security options are made available (for example, enabling 20-character passwords or two-factor authentication); ensure encryption options are made available (for example, enabling AES-256); ensure secure logging is available for security events
Insecure Software / Firmware	Ensure all system devices have update capability and can be updated quickly when vulnerabilities are discovered; ensure update files are encrypted and are also transmitted using encryption
Poor Physical Security	Ensure the device is produced with a minimal number of physical external ports (for example, USB ports); ensure the firmware or operating system cannot be accessed through unintended methods such as an unnecessary USB port; ensure the product is tamper-resistant

Read together, these tables express the core philosophy of IoT defence. The recurring themes, disallowing weak passwords, providing lockout and two-factor mechanisms, minimising exposed ports and interfaces, encrypting data everywhere and in every state, enabling updates, minimising collected data, and hardening the device physically, are precisely the countermeasures to the OWASP Top 10 risks and to the layer-by-layer attacks catalogued above. The defensive story therefore closes the loop that the whole chapter has traced: the same small set of root causes appears again and again across every incident from Stuxnet to BadBox, and the same small set of disciplined measures, applied consistently across the device's entire life and across every layer and interface of the ecosystem, is what stands between a smart environment and the physical, personal harm that its compromise would cause.

The ultimate synthesis is the one with which the subject began. Because users live inside these systems, security failures become physical and privacy failures become personal; and because the devices are cheap, numerous, long-lived, and rarely updated, the discipline of securing them cannot be an afterthought bolted on after deployment but must be designed in from the start, sustained through operation, and honoured all the way to end-of-life.

CHAPTER 2: IoT part 2

Introduction to the Attestation Problem

The security of the Internet of Things rests on a problem that is deceptively simple to state yet remarkably difficult to solve: how can one party become confident that a remote device is running the software it is supposed to be running, and nothing else? In a smart environment populated by cameras, sensors, actuators, medical monitors, and industrial controllers, each device is a potential foothold for an attacker. A compromised device may continue to behave outwardly as expected while silently exfiltrating data, manipulating readings, or serving as a launching point for further intrusion. The family of techniques designed to detect such compromise is known collectively as **attestation**.

Attestation occupies a specific niche within the broader landscape of IoT security. A useful way to situate it is to recognize that IoT security concerns span three stacked layers, each with its own distinctive

threats and countermeasures. At the top sits the **application layer**, which encompasses concrete IoT applications such as intelligent logistics, smart home systems, remote medical care, smart grids, intelligent traffic control, and environmental monitoring, together with the application-support infrastructure beneath them: middleware, service-support platforms, cloud computing platforms, and information-development platforms, each requiring its own security treatment. In the middle lies the **network layer**, which carries data across local area networks (where local area network security and technologies such as 3G security operate), across the core network (dominated by Internet security concerns), and across the access network (where ad hoc, GPRS, and WiFi security mechanisms apply). At the foundation rests the **perception layer**, the stratum of physical data acquisition. It is here, among the perception networks and perception nodes, that attestation finds its home. The perception layer must secure RFID technology, wireless sensor networks (WSN), and radio-sensor networks (RSN), addressing protocol security, base-station and reader security, tag counterfeit and tag-encoding security, routing-protocol security, cryptographic algorithms, key management, node-trust management, and the fusion of sensor, RFID, and WSN security. The physical, resource-constrained devices that populate this layer are precisely the devices whose internal state attestation seeks to verify.

This chapter develops the subject in three movements. It begins with **remote attestation** as a general concept and formal object, proceeds to **device attestation** in its software-based, hardware-based, and hybrid variants, and culminates in **collective remote attestation**, which scales the problem up to entire networks and swarms of devices.

Remote Attestation

Remote attestation is, at its core, a protocol between two parties. The first is a **challenger**, conventionally denoted `Chal`, whose role is to verify the internal state of a second party. That second party is a device called the **prover**, denoted `Prov`, whose internal state is the object under scrutiny. The entire interaction is structured so that the challenger can reach a justified conclusion about whether the prover is in an acceptable condition.

The goal of the protocol is to allow an honest prover—one that has not been compromised—to create an **authentication token** that convinces the challenger that the prover is in some well-defined, expected state. The expected state is typically a specific configuration of memory contents: the firmware, application code, and critical data that the device is supposed to be running. When the prover is genuinely in that state, it must be able to produce a token that the challenger will accept.

The protocol's security property is the mirror image of its functional goal. If the prover has been compromised and its internal state has been modified—for instance, if malware has been injected into its memory—then the authentication token must reflect this fact. In other words, a compromised prover must be unable to produce a token that falsely convinces the challenger that everything is normal. The entire value of attestation collapses if a compromised device can forge an acceptable token, so the design of the token-generation mechanism is where the intellectual difficulty concentrates.

The Adversary Model

To reason rigorously about attestation, one must first be precise about what the attacker is assumed to be capable of. The adversaries relevant to IoT attestation divide naturally into two broad families: **software-based** adversaries and **hardware-based** (physical) adversaries. Cutting across both families are two orthogonal capabilities that any serious adversary may possess.

The first such cross-cutting capability concerns **communication**. A communication adversary is assumed to have full control of the communication channel. This control is expansive: it includes eavesdropping on

all traffic, injecting packets of the attacker's own construction, modifying packets in transit, dropping packets entirely, and delaying their delivery. An adversary with this degree of channel control can manipulate the timing and content of every message exchanged during an attestation protocol, which has profound implications for protocols that rely on timing guarantees.

The second cross-cutting capability is **mobility**. A mobile adversary is one that is able to erase any trace of its presence. This capability matters because it undermines forensic approaches that would otherwise detect compromise after the fact; if the attacker can clean up after itself, then detection must happen during the attestation itself rather than through later inspection of logs or residual artifacts.

Within the **software-based** family, three distinct attack flavors appear, each with its own purpose. The **remote** software attack aims to infect one or more devices with malware, operating entirely from a distance without any physical access. The **local** software attack has the purpose of impersonating devices, standing in for a legitimate device in protocol exchanges. The **physical non-intrusive** attack, despite involving physical proximity, is grouped with software attacks because it does not alter the device's internals; its purpose is to learn a device's secret, to impersonate or clone the device, and potentially to launch side-channel attacks that extract information by observing physical emanations such as timing, power consumption, or electromagnetic radiation.

Within the **hardware (physical)** family, two more aggressive attack flavors appear, distinguished by how deeply they violate the device's physical integrity. The **stealthy physical intrusive** attack has the purpose of capturing a device and physically extracting its secrets, then cloning the device—all while attempting to remain undetected. The **physical intrusive** attack goes further still: its purpose is to capture a device and actively modify its contents or components, rewiring or reprogramming the hardware itself. The progression from software-only to stealthy-physical to fully-physical-intrusive represents an escalating ladder of attacker power, and attestation schemes are generally characterized by how far up this ladder they can defend. (This taxonomy of adversaries follows the framework articulated in the work on building trust in IoT systems by Abera and colleagues.)

Device Attestation: The Software-Based Approach (SWATT)

The earliest and conceptually simplest form of device attestation is purely software-based, requiring no specialized hardware on the device. The canonical example is **SWATT** (Software-based ATTestation), introduced by Seshadri, Perrig, Van Doorn, and Khosla in 2004. The scenario it addresses is one in which a **prover**—imagine a networked surveillance camera running some untrusted software—must convince a **verifier**, a more capable machine such as a server, that its memory contents are exactly what they should be.

The main idea of the software-based approach rests on three pillars. First, the adversary is assumed to be a **software-only attacker**, meaning the defenses need not withstand physical tampering. Second, the scheme makes **no use of specialized hardware**, which is precisely what makes it attractive for cheap, low-end devices. Third, the scheme typically relies on a **challenge-response protocol**: the verifier sends a challenge, the prover computes a response that depends on both the challenge and its memory contents, and the verifier checks the response.

The approach rests on two explicit assumptions about the setup. The device must contain a **memory-content-verification procedure** that the verifier can activate remotely—that is, there must be code on the device whose job is to read through memory and compute a summary of what it finds. And the verifier must know the **exact hardware architecture and the expected memory contents** of the device. This second assumption is what gives the verifier the ability to predict, in advance, what the correct response to any given challenge should be. Because the verifier possesses an exact replica of the

expected device state, it can compute the expected response independently and compare it against what the prover actually returns.

Formalizing Remote Attestation

Beyond the intuitive description, remote attestation admits a precise formal definition, which clarifies exactly what guarantees are being sought. Following the minimalist formulation of Francillon, Nguyen, Rasmussen, and Tsudik, an attestation protocol **P** is composed of three algorithms.

The first is **Setup(1^n)**, a *probabilistic* algorithm that takes as input a security parameter 1^n (the superscript n expressed in unary, which is the standard cryptographic convention for parameterizing security) and outputs a **long-term key k**. This key is the shared secret or key material that underpins the trust relationship between the parties. Because it is probabilistic, each invocation can produce fresh key material.

The second algorithm is **Attest(k; s)**, a *deterministic* algorithm. Given the key **k** and a device state **s**—where the state is structured as the pair s = (s_chal, s_prov), capturing the relevant state of both the challenger and the prover—it outputs an **attestation token α**. Determinism here is essential: for a fixed key and a fixed state, the token is always the same, which is what allows the verifier to predict it.

The third algorithm is **Verify(k; s; α)**, also *deterministic*. Given the key **k**, a device state **s**, and an attestation token **α**, it outputs **1 if and only if α corresponds to s**—that is, if and only if Attest(k; s) = α—and outputs **0 otherwise**. Verification thus reduces to recomputing the expected token and checking for an exact match.

The security of this construction is captured by a single inequality concerning forgery. The probability that an adversary wins the attestation-forgery game, written Pr[Att-Forgery_{chal,prov}(n) = 1], must be less than or equal to **negl(n)**, a *negligible* function of the security parameter. A negligible function is one that shrinks faster than the inverse of any polynomial, meaning that as the security parameter grows, the attacker's chance of producing a token that is accepted despite a compromised state becomes vanishingly small. This inequality is the formal expression of the earlier intuitive requirement that a compromised prover must not be able to forge an acceptable token.

The Properties a Secure Scheme Must Enforce

For the formal guarantees to hold in practice, the attestation mechanism must enforce a set of concrete properties on the device. These properties are what prevent a clever attacker from subverting the computation of the token.

The first property is **exclusive access to k**: only the attestation routine may read or use the long-term key, and no other code on the device may touch it. If arbitrary code could read the key, then malware could compute valid tokens on behalf of a compromised state. The second property is that there must be **no leaks of k other than α**: the only thing the attestation process reveals to the outside world is the token itself, and the token must not inadvertently disclose the key. The third property is **code immutability**: the attestation code itself cannot be modified by an attacker, because if the verification routine could be rewritten, it could simply be made to lie.

Two further properties govern the execution environment rather than the key. **Uninterruptible execution** demands that once the attestation routine begins, it runs to completion without being interrupted—an interrupt handler controlled by malware could otherwise seize control mid-computation, inspect intermediate values, or substitute results. **Controlled invocation** requires that the attestation

code can only be entered at its proper entry point and executed as a coherent whole, rather than being called into at arbitrary offsets where an attacker might skip critical checks.

How an Attacker Tries to Evade Software Attestation

The central intuition behind software attestation's security—in the absence of any trusted hardware—is a clever argument about *overhead*. The reasoning runs as follows. Suppose an attacker has replaced the legitimate firmware with malicious code but has kept a copy of the original, correct code elsewhere in memory so that it can still produce correct answers when challenged. To pass the attestation, this **malicious prover needs to redirect the attestation challenge to the trusted (original) software** to obtain the proper result. The key claim is that **the overhead incurred by this memory redirection will be noticed**—the extra indirection costs time, and if the verifier measures how long the response takes, the delay betrays the attack.

This argument becomes concrete when one compares the two memory layouts. In the **expected memory layout** of an honest device, memory is organized into three contiguous regions: a block of **verification code**, followed by the **firmware**, followed by an **empty** region. The verifier knows this layout precisely and expects the attestation to sweep across exactly these contents.

In the **attacker's memory layout**, by contrast, the organization is subtly different. The region that should hold the verification code instead holds **malicious code**. The firmware region remains, but crucially, the previously empty region now holds a copy of the **old verification code**—the attacker has stashed the legitimate code there so it can be consulted to generate correct responses. For the attacker to produce the right answer, its malicious code must detect when memory is being read for attestation purposes and redirect those reads to the stashed copy of the legitimate code, so that the checksum comes out correct even though the live verification region has been corrupted. It is this redirection, performed on potentially every memory access during the attestation sweep, that imposes measurable overhead and, in principle, reveals the deception.

The Verification Procedure

Having established *why* memory redirection is detectable in principle, the question becomes *how* to design the verification procedure so that the overhead is both unavoidable and conspicuous. The design hinges on forcing the attestation routine to read memory in an unpredictable, exhaustive manner that leaves no room for shortcuts.

The mechanism begins with a **nonce**—a fresh random value supplied by the verifier with each challenge. This nonce is used as the **seed to a pseudo-random generator (PRG)**, and the PRG in turn generates the sequence of memory addresses that the verification routine will access. Because the addresses are derived pseudo-randomly from a nonce the attacker cannot predict in advance, the attacker cannot know ahead of time which locations will be touched or in what order. The access pattern, visualized, resembles a chaotic scribble of arrows crisscrossing the entire memory space, jumping unpredictably from region to region rather than marching through memory in any orderly sweep.

Two consequences follow from using a pseudo-random traversal. First, the procedure will **touch every memory location with high probability**. The relevant bound is **O(n log n)**: this is the classical coupon-collector result, which says that to cover all n locations with high probability when sampling them at random, roughly $n \log n$ samples are required. By running the traversal for this many iterations, the designer ensures that no region of memory escapes inspection, so an attacker cannot hide malicious code in some corner that the checksum happens to skip. Second, the verification code itself must have a

small code size, both so that it fits on constrained devices and so that there is little room for an attacker to hide alterations within it.

The design also builds in a specific response to tampering. If memory has been altered by the attacker, then whenever the traversal reads one of the altered locations, the attacker's code must substitute the **original value** that *should* be there, because returning the modified value would produce a wrong checksum. This substitution is exactly where the overhead is incurred. Concretely, the attacker's code must insert a conditional check—an **if instruction**—before each memory read, to decide whether the address being read is one of the tampered locations (in which case the stashed original must be returned) or an untouched location (which can be read directly). Every such conditional adds instructions to the inner loop, producing a cumulative **slowdown** across the millions of iterations in a full attestation.

Three further properties reinforce the scheme's resistance to forgery. There is **no replay attack**, because the fresh nonce guarantees that the challenge—and therefore the expected response—differs every time; a response recorded from a previous session is useless for a new one. The checksum computation is deliberately engineered to be **not parallelizable**, meaning the attacker cannot distribute the work across multiple cooperating devices to recover the time lost to redirection—each step of the computation depends on the previous one, forcing strictly sequential execution. And there is **no precomputation**: because the response depends on the unpredictable nonce, the attacker cannot compute acceptable responses in advance during idle time.

Checksum Design and Implementation

The concrete realization of these principles is a carefully constructed **checksum** routine. The checksum is chosen to have a size of **n = 64 bits**, a length selected to make the probability of a **collision**—two different memory states producing the same checksum—extremely low. If the checksum were too short, a compromised state might accidentally or deliberately hash to the same value as the legitimate state, allowing the attacker to slip through; sixty-four bits makes such a coincidence negligibly unlikely. The checksum is maintained as a structured value composed of eight-bit cells, which the implementation updates one at a time as the traversal proceeds.

The structure of the computation can be read directly from its pseudocode. The checksum index **j** is initialized to zero. The pseudo-random generator is then "warmed up" by discarding its first 256 outputs (the values PRG_0 through PRG_{255}), a standard precaution with stream-cipher-style generators whose earliest outputs may exhibit statistical weaknesses. The checksum **C** is then initialized from the next eight generator outputs, PRG_{256} through PRG_{263}, giving it a pseudo-random starting value rather than a fixed constant.

The heart of the routine is a loop running over the index **i** from 265 to m + 265, where *m* governs the number of memory accesses. Each iteration performs several tightly coupled operations. First, it **constructs the address for the memory read** by combining a fresh generator output with feedback from the checksum itself: the address is formed as PRG_i shifted left by eight bits and added to a byte of the current checksum, C_{j−1 mod 8}. This dependence of the address on the current checksum state is crucial, because it chains the accesses together—the location read next depends on everything read so far, which is one of the mechanisms enforcing non-parallelizability.

The routine then **updates the checksum byte**. The relevant cell is recomputed as $Cj = C_j + (mem[addr] \text{ xor } C\{j-2 \bmod 8\} + \mathrm{PRG}_{i-1})$: the value actually found in memory at the computed address is mixed, via exclusive-or and addition, with another checksum byte and a generator output. This is the step at which the *content* of memory enters the checksum, so any discrepancy between the real memory and the expected memory propagates into the final value. Immediately afterward, the updated byte is

passed through a **rotate_left_one_bit** operation, a cheap diffusion step that spreads the influence of each input across bit positions and prevents simple cancellations. Finally the index is advanced, $j = j + 1 \bmod 8$, cycling through the eight checksum cells, and once the loop completes, the finished checksum **C** is returned.

The significance of this construction is that the address generation, the memory read, and the checksum update are **interlocked**: the next address depends on the checksum, and the checksum depends on the memory read at the current address. An attacker who wishes to feed false memory values into this pipeline must intercept each read and substitute the correct value, and must do so without disturbing the delicate chain of dependencies—an intervention that, as established, costs measurable time.

Assembly-Level Cost of Tampering

The abstract claim that an attacker's redirection "adds an if and slows things down" can be made quantitative by descending to the assembly level and examining a single cycle of the checksum loop. On a representative microcontroller architecture, one iteration of the legitimate routine decomposes into a sequence of primitive operations, each a single machine instruction.

The cycle begins by **generating the i-th member of the random sequence using an RC4-style generator**. This stage loads a constant into a register, reads a byte from one pointer with post-increment, performs a series of additions and loads that implement the stream generator's internal state update, and stores results back—a handful of load, add, and store instructions that together produce the next pseudo-random byte. The routine then **generates the sixteen-bit memory address** from that output with a register move. Next it **loads a byte from memory and computes a transformation**: it loads the program byte at the computed address, exclusive-ors it against a register holding checksum state, and adds in another register. Finally it **incorporates the output of this transformation into the checksum**: it adds the transformed value into the checksum register, shifts the checksum register left by one bit, adds back the carry bit (the rotate operation realized as a shift plus carry-add), and moves a register to prepare for the next iteration.

The economy of this loop is the whole point. Because the legitimate cycle is already minimal, there is no "slack" the attacker can exploit. When the attacker inserts the conditional needed to redirect tampered addresses—the **if**—it costs **three additional instructions** per iteration. Measured against the tight baseline loop, those three instructions translate into roughly a **13% overhead**. Over a full attestation of hundreds of thousands of memory accesses, this percentage compounds into a time difference the verifier can reliably detect.

The empirical behavior confirms the analysis. When one plots execution time against the number of memory accesses, three lines emerge. The **legitimate verification code** traces a straight line rising steadily with the number of accesses. The **attacker's verification code**, burdened by the extra conditional, traces a parallel but visibly steeper line lying above it. And the **time difference** between them—plotted as a third line near the bottom—grows roughly linearly with the number of accesses, starting near zero and widening continuously. This growing gap is the verifier's signal: by setting a sufficiently large number of iterations, the honest-versus-malicious time difference can be pushed well beyond the noise floor of measurement and network jitter, making the attack detectable.

Avoiding Device Attestation: The Full Challenge-Response Protocol

The pieces assembled so far can now be composed into the complete software-based attestation protocol, which makes vivid both how legitimate attestation proceeds and how a physically relocated attacker

attempts to subvert it. The central tension remains the same: a malicious prover must redirect the attestation challenge to trusted software to obtain the proper result, and the overhead of that redirection is what the scheme aims to expose.

Consider a scenario with a verifier and two provers. **Prover A** is the device under test, but it has been compromised—its live software is untrusted, and it is in collusion with **Prover B**, a separate device that still holds a copy of the correct, trusted software. The attack strategy is for Prover A to secretly forward the attestation work to Prover B, which can compute the honest answer, and then relay that answer back to the verifier as if it had computed it locally.

The protocol unfolds in four numbered steps. In **step 1**, the verifier generates a random nonce **N** and sends it to the prover. In **step 2**, the honest computation that is *supposed* to happen is the hashing of the device's own software: h = **HASH**(Software). On the trusted device (Prover B), this produces the correct hash **h** of the correct software. In **step 3**, the prover computes an authentication token δ = **MAC_k**(N ‖ h)—a message authentication code, keyed with the shared secret **k**, taken over the concatenation of the nonce and the software hash—and sends δ back to the verifier. In **step 4**, the verifier independently computes what the token should be, δ' = **MAC_k**(N ‖ h′), using the hash **h′** of the configuration it *expects* the device to be running, and then checks whether **δ' = δ**. If they match, the device is deemed to be in the expected state.

The use of a keyed MAC rather than a bare hash is what binds the response to both the fresh nonce (defeating replay) and the secret key (ensuring the attacker cannot simply compute the MAC without possessing **k**). The collusion attack by Prover A succeeds in producing a numerically correct δ only if the redirection to Prover B can be hidden—and in the software-only model the redirection's latency is exactly what the timing analysis of the preceding sections is designed to unmask.

The Limitations of Software-Based Attestation

For all its elegance, software-based attestation purchases its hardware-free simplicity at the cost of several serious limitations, each of which motivates the move toward hardware assistance in later sections.

The first limitation is that the entire security argument holds only against **software-only attackers**; it offers **no protection against a physical attacker**. The whole edifice rests on timing, and timing guarantees evaporate if an adversary can physically augment the device—for example, by attaching a faster processor or additional memory—so that the redirection overhead is absorbed by superior hardware rather than paid in observable time.

The second limitation is the reliance on a **strict time-bound response**. The verifier must know, with fine precision, how long an honest attestation should take, so that it can flag a response that arrives too late. Establishing this bound requires detailed knowledge of the device's exact clock speed and architecture, and leaves little tolerance for variability.

The third limitation follows directly from the second: the approach is **not feasible for multi-hop distances**. When the prover is several network hops away from the verifier, the round-trip time becomes dominated by unpredictable network latency and queuing delays, which swamp the small computational overhead the scheme is trying to measure. The timing signal that distinguishes an honest prover from a redirecting attacker is simply lost in the noise of the network. This fragility over realistic network topologies is one of the strongest arguments for grounding attestation in something more robust than execution timing—namely, trusted hardware, the subject toward which the discussion now turns.

SCUBA: Secure Code Update By Attestation

Detecting that a device has been compromised is only half of a useful security story; the other half is doing something about it. **SCUBA**—Secure Code Update By Attestation—addresses this second half. Introduced by Seshadri, Luk, Perrig, Van Doorn, and Khosla, its goal is to **securely recover a device in the presence of malicious code**. That is, SCUBA assumes that a device may already be infected, and it provides a procedure by which a trusted base station can restore the device to a known-good state—patching out the malware—even though the malware is actively present and may try to resist.

The system model rests on a set of assumptions about the devices and the infrastructure. **Every node is identified by a unique ID embedded in a ROM**, so that identity is rooted in immutable read-only memory rather than in software that an attacker could rewrite. The **base station has complete knowledge of the devices**, meaning it holds an exact picture of each node's expected hardware and memory contents—the same prerequisite that made verification possible in the software-based setting. **Nodes know the public key of the base station**, which allows them to verify messages genuinely originating from the base station; here it is noted that public-key cryptography on such constrained hardware is costly, taking on the order of tens of seconds on low-power devices, which is why the design uses it sparingly. Finally, the **base station and the sensor nodes share a cryptographic key**, providing an efficient symmetric channel for the frequent operations where public-key operations would be too slow.

The adversarial assumptions complete the model. The attacker **can compromise sensor nodes by injecting malicious code**, and once a node is compromised, the **base-device shared key can be accessed** by the attacker—a sobering admission that the symmetric key alone cannot be trusted to prove a node's integrity, since the malware may have stolen it. The **base station itself is not compromised**; it is the anchor of trust. **Corrupted nodes can collude**, pooling their resources and coordinating their deceptions. There is, however, one essential restriction on the attacker's power: the **attacker's own hardware devices are not present in the sensor network for the duration of the repair process**. This assumption is what makes recovery tractable—during the critical window of repair, the attacker cannot wheel in a faster machine to help a compromised node fake its responses, so the timing-based guarantees can hold.

The TOCTTOU Attack

Before SCUBA's recovery mechanism can be trusted, a subtle but devastating gap in naive attestation must be confronted. This is the **TOCTTOU** attack, an acronym for **Time-Of-Check-To-Time-Of-Use**. The attack exploits the interval between two distinct moments: the moment when code is verified, and the later moment when that same code is actually invoked for execution.

The problem is this: attestation, as described so far, checks what is in memory *at the time of the check.* But an attacker can **modify the code between the time the code is verified and the time the code is invoked for execution**. A device might pass its attestation honestly—its memory genuinely contains the correct code at the instant of measurement—and then, in the brief window before that code runs, the malware swaps in a malicious version. The verifier would have been shown a truthful snapshot of a clean state, yet the device would execute corrupted code moments later.

The consequence is stated starkly: **attestation cannot guarantee untampered code execution**. Verifying the contents of memory is simply not the same as guaranteeing that those verified contents are what actually run. This realization is pivotal, because secure recovery depends on being able to run recovery code that the attacker cannot subvert. If the very act of running the recovery routine opens a TOCTTOU window, then the malware could allow the recovery code to be verified and then hijack

execution before the repair takes effect. SCUBA therefore needs a stronger primitive—one that binds verification and execution together so tightly that no window exists between them.

ICE: Indisputable Code Execution

That stronger primitive is **ICE**—Indisputable Code Execution—the engine at the heart of SCUBA. ICE is designed to deliver what plain attestation cannot: a guarantee not merely that the right code is present, but that the right code actually *executes*, in an environment the attacker cannot tamper with. Achieving **untampered code execution** proceeds in three stages.

The first stage is to **check the integrity of the executable**. This ensures that the executable has not been modified before it is invoked for execution—the familiar attestation check that confirms the code in memory matches the expectation. On its own this would still be vulnerable to TOCTTOU, which is why the remaining stages are essential.

The second stage is to **set up an untampered execution environment**, and it is this stage that closes the TOCTTOU gap. Two guarantees define this environment. First, the **execution of the executable is guaranteed to be atomic**: once begun, it completes as an indivisible unit. Second, **no other code on the sensor node is allowed to execute until the executable exits**. Together these guarantees eliminate the window the TOCTTOU attack relied upon—because nothing else can run between the integrity check and the execution, and because the execution cannot be interrupted and resumed, there is no moment at which malware could slip in a substitution. Verification and execution become welded into a single uninterruptible transaction.

The third stage is to **execute ICE and any other needed executable**. Crucially, **ICE is self-checksumming**: the ICE routine computes a checksum over itself (and over the code it is about to protect), so that any modification to the ICE code or its protected payload disturbs the checksum. This self-referential property is what makes ICE "indisputable"—the routine vouches for its own integrity as part of the very computation it performs, so an attacker cannot modify the routine without the modification showing up in the result. Once ICE has established this trustworthy bubble of execution, the SCUBA recovery code can run inside it with confidence that it will execute faithfully to completion.

Attacks Against Self-Checksumming

Because ICE's guarantee rests entirely on the integrity of its self-checksumming routine, the security analysis must anticipate every way an attacker might try to produce a correct checksum while actually running malicious code. The attacker's overarching goal in all of these is the same: to **fake the correct checksum within the expected time**. The defenses fall into two broad categories—**checksum forgery attacks**, which try to make a wrong state look right, and **speed-based attacks**, which try to buy back the time that cheating costs. Each attack is paired with the specific design countermeasure that neutralizes it.

Checksum Forgery Attacks

The **precomputation and replay attack** attempts to compute the checksum over the memory region containing ICE and the target executable *before* making any changes to that memory region. The attacker would compute the right answer while the memory is still clean, store it, then corrupt the memory and simply replay the stored answer. The countermeasure is that the **verifier sends a random challenge with every request**, and this challenge is fed into the checksum function. Because the challenge is unpredictable, no checksum computed in advance can be valid for a future request—precomputation is rendered useless.

The **data substitution attack** attempts to change some locations of the memory region containing the ICE verification function and the executable while keeping the original values at a different location in memory, so that reads can be redirected to the pristine copy. The countermeasure is that the **ICE verification function accesses memory in a pseudorandom pattern**, the same technique encountered with SWATT: unpredictable access ordering forces the attacker to insert per-access conditionals to redirect the reads, incurring detectable overhead.

The **memory copy attack (1)** attempts to compute the checksum over a copy of the correct code: the correct code is copied to another location in memory while the malicious code executes at the location where the correct code is supposed to reside. The defense is to **ensure that the contents over which the checksum is computed are fetched from the correct address locations in memory**. By weaving the actual addresses into the checksum computation, a copy stored at the wrong address cannot produce the right result.

The **memory copy attack (2)** is the mirror image: the correct code resides at the correct memory location, but the *malicious* code executes at another location of memory, so that the checksum sweeps the genuine code while control actually rests with the malware elsewhere. The countermeasure is to **ensure that the program counter is pointing to the correct memory addresses**, achieved by **including the program counter in the checksum**. If the PC is folded into the computation, then code running from the wrong address produces a different checksum, exposing the displacement.

The **forging CPU state inputs** attack attempts to forge the creation of the untampered execution environment by incorrectly setting up the CPU state and then forging the correct CPU state during checksum computation—faking the atomicity and isolation guarantees rather than genuinely providing them. The countermeasure **guarantees atomic execution by setting up the corresponding CPU state and incorporating that state into the checksum**. Specifically, by including the **interrupt-disable bit** and the **CPU's interrupt vector table** in the checksum, the scheme ensures that the checksum is correct *only if* these inputs are correct—meaning only if interrupts are genuinely disabled and the interrupt handlers have not been tampered with. The very conditions that make execution untamperable are thereby made prerequisites for a passing checksum.

Speed-Based Attacks

The second category of attacks does not try to fake the checksum directly but rather to recover the time that any cheating would cost, so that the forgery can hide within the expected time window.

The **optimized implementation attack** attempts to decrease the execution time of the ICE verification function by optimizing the code, which would let the attacker use the time gained to forge the checksum without being detected. The countermeasure is to **keep the assembly code minimal**, using automated tools to either exhaustively find the most efficient implementation or to apply theorem-proving techniques to show that a given code fragment is provably optimal. If the legitimate implementation is already the fastest possible, the attacker has no faster version to run, so there is no time to steal.

The **multiple colluding devices attack** seeks to speed up execution by leveraging multiple devices to compute the checksum in parallel—several devices might collude to each compute a different range of the ICE verification loop and then combine their partial results into the final checksum. The countermeasure is to make the **verification function non-parallelizable**, forcing sequential execution. By chaining each step's input to the previous step's output, the computation cannot be split across cooperating devices, so additional hardware yields no speedup.

The **proxy attack** attempts to forward the ICE challenge to a proxy node with greater computing resources; the proxy holds a copy of the correct memory contents of the sensor node and computes the answer faster. Three defenses converge here. The scheme assumes that **all sensor nodes in the network have identical computing resources**, so no node can serve as a faster proxy; it assumes **the attacker is not physically present during verification**, so no external fast machine is available; and it relies on a **firewall in the base station that blocks packets during the execution**, so the challenge cannot be relayed out and an answer relayed back in during the critical window.

The **impersonation attack** has a legitimate sensor node impersonate the target in the ICE protocol by forwarding protocol messages from the base station to that legitimate node. The countermeasure is to **include the node ID in the computation**, so that each node's checksum is bound to its own identity and cannot stand in for another; it is also noted that proxy attacks of this kind require additional time, which the timing bound already penalizes.

Experimental Validation of ICE

The credibility of the entire timing argument rests on whether, in practice, the honest computation is reliably faster than the best possible cheating computation. This is tested by running ICE against the *fastest known* memory-copy attack over 40,000 iterations and recording the times across repeated samples.

The resulting measurements separate into clearly ordered bands. At the bottom lies the **legitimate code's runtime**—the pure computation time of the honest routine—forming the fastest and most stable trace. Slightly above it sits the **legitimate code's runtime plus network round-trip time (RTT)**, which adds the communication delay to the honest computation; this remains low, oscillating gently around a value a little below 2.9 seconds. Above these honest bands lies a dotted horizontal line marking the **expected time to compute the checksum**, the threshold the verifier uses to decide acceptance—positioned just above the honest traces to accommodate legitimate variation. Critically, both adversary traces lie *above* this threshold: the **adversary's runtime** sits at a flat band near 2.95 seconds, and the **adversary's runtime plus RTT** oscillates higher still, around 2.97–2.99 seconds. Because even the adversary's bare runtime exceeds the expected-time threshold, the cheating attacker cannot fit within the acceptance window, and the verifier distinguishes honest from malicious execution. The experiment thus empirically confirms the central claim: the overhead of the best available attack cannot be hidden within the time an honest device requires.

The Complete SCUBA Protocol

With ICE established as a trustworthy execution primitive and its resistance to forgery validated, the full SCUBA recovery protocol can be assembled. It is a timed challenge-response exchange between the **base station** and a **sensor node**, structured so that the base station first confirms the node is executing honest code and then delivers the precise patches needed to heal it.

The protocol runs as an ordered sequence of messages and local actions. The base station begins by recording **T1 = current time** and sending an **ICE challenge** to the sensor node. Upon receiving the challenge, the node **computes the ICE checksum over the memory region containing the ROM, the ICE verification function, and the SCUBA executable**—that is, it checksums exactly the code that must be intact for recovery to proceed, including its own identity-bearing ROM and the recovery routine itself. The node returns the **ICE checksum** to the base station.

The base station then records **T2 = current time** and performs two verifications. It checks that **(T2 − T1) < Time allowed**, confirming that the response arrived within the strict timing bound that rules

out redirection and proxying, and it verifies the **ICE checksum** itself against the expected value. Only if both checks pass does the base station trust that the node has genuinely executed the honest ICE-protected code.

Trust established, the recovery phase proceeds. The node sends a **hash of its code memory** to the base station. The base station uses this **hash from the node to determine whether the node's code memory is modified**—comparing it against the known-good configuration to identify exactly what has been corrupted—and then **prepares code patches for the sensor node**, computing the minimal set of changes needed to restore it. These **code patches** are sent to the node, which finally **applies the patches**, overwriting the malicious modifications and returning to a known-good state. Because the patch application happens within the trustworthy window that ICE guarantees, the malware cannot interfere with its own removal.

Hardware-Based Attestation

Software-based attestation, however ingenious, is built on a foundation of sand: its guarantees rest on timing, and timing crumbles against physical attackers, multi-hop networks, and any adversary with faster hardware. **Hardware-based attestation** replaces this fragile foundation with a concrete one. Rather than inferring integrity from how *long* a computation takes, it anchors integrity in a dedicated, tamper-resistant hardware component that holds secrets the untrusted software can never reach and performs cryptographic operations that software cannot forge. The canonical embodiment of this approach is the architecture proposed by IBM, built around a **Trusted Platform Module (TPM)**.

The overall protocol follows the familiar challenge-response shape, but with the trusted hardware doing the cryptographic heavy lifting. Consider a **prover**—again, imagine a networked camera—facing a **verifier**. Inside the prover sits an **Attestation Service** running in cooperation with a **TPM**, and within the TPM reside three protected elements: **secure code**, one or more **PCRs** (Platform Configuration Registers), and cryptographic **keys**.

The exchange proceeds in three numbered steps. In **step 1**, the verifier generates a nonce **N** and sends it to the prover. In **step 2**, the nonce is passed into the TPM, where the attestation service draws on the PCR contents—which record the measured state of the platform—together with the nonce. In **step 3**, the TPM produces the attestation token $\delta = \mathbf{SIG_AIK(PCR} \parallel \mathbf{N)}$: a *digital signature*, computed with the **Attestation Identity Key (AIK)**, over the concatenation of the PCR values and the nonce. This signed token is sent to the verifier. The verifier then checks it by computing $\delta' == \mathbf{(SIG}_{\boldsymbol{AIK(PCR} \parallel \boldsymbol{N))}}\mathbf{\{AIK_PUB\}}$—that is, it verifies the signature using the **public AIK**—and tests whether $\delta' = \delta$.

The shift from a MAC to a *digital signature* is the quiet but important change here. In the software and hybrid schemes a symmetric MAC sufficed, which required the verifier to share the secret key. With a signature produced by the TPM's private AIK and verifiable by anyone holding the public AIK, attestation results become **publicly verifiable**: any party can check the token without possessing a secret, a property that becomes decisive when attestation scales to networks.

Beyond this core attestation flow, the hardware-based paradigm encompasses a broader family of trusted-computing schemes. These include mechanisms for **platform integrity**, realized through **secure boot** and **authenticated boot**; mechanisms for **secure storage**, which protect data at rest using hardware-held keys; and mechanisms for **isolated execution**, chief among them the **Trusted Execution Environment (TEE)**, which carves out a protected enclave in which sensitive code can run shielded from the rest of the system. These primitives recur throughout the remainder of the chapter as building blocks for more advanced architectures.

The Static Root of Trust

For any attestation to be meaningful, the chain of trust must terminate somewhere in something that is trusted *a priori*—a **root of trust** that is not itself verified by anything else but is instead assumed trustworthy by construction. The classical approach establishes this root at boot time, yielding a **static root of trust**.

The first mechanism is **secure boot**. Secure boot **verifies system integrity at boot time**, ensuring that only authorized code is permitted to run as the system starts. It relies on an **immutable bootloader stored in ROM along with a public key**: because the bootloader lives in read-only memory, an attacker cannot modify it, and because a trusted public key accompanies it, the bootloader can verify signatures on the code it is about to load. The rule it enforces is simple and strict—it **verifies the code signature and executes the code (that is, boots) only if the signature is valid**. Any unsigned or tampered code is refused, so the very first software to run is guaranteed authentic, and the chain of trust extends upward from there.

The second mechanism is the **Trusted Platform Module** itself, considered now as a root of trust rather than merely a signing engine. The TPM is **present in many commercial systems**, making it a widely available foundation. It contains **Platform Configuration Registers (PCRs)** that are **accessible only via a fixed API**—software cannot write arbitrary values directly into them but may only interact with them through the controlled interface, which is what makes their contents trustworthy. The **PCRs are reset only on boot**, so their values reflect everything measured since the machine started, and they cannot be surreptitiously cleared mid-session. Critically, **each new measurement added to a PCR is "extended" rather than simply stored**: a new measurement is combined with the existing register value by computing a **cryptographic hash of the previous PCR value concatenated with the new measurement**. This "extend" operation gives the PCRs a powerful property. Because each value depends on the entire ordered history of what came before, the final PCR value is a tamper-evident summary of the complete boot sequence—no component can be added, removed, or reordered without producing a different final value, and no later measurement can erase the record of an earlier one.

The Dynamic Root of Trust

The static root of trust measures the system *as it boots*, which means its picture of integrity is fixed at startup and may grow stale as the system runs. Sometimes one needs to establish a trusted measurement of the system's *current* state, long after boot, without rebooting. This is the role of a **dynamic root of trust**.

A dynamic root of trust **can be executed dynamically on the current state of the software**, establishing a fresh, trustworthy measurement at an arbitrary moment during operation rather than only at power-on. This flexibility is valuable precisely because it captures the live state, including any changes that have accumulated since boot. Achieving it, however, **requires specific modules inside the CPU**—special processor instructions and hardware support that can forcibly create a clean, measured execution environment on demand. The representative example is **Intel TXT** (Trusted Execution Technology), which provides CPU-level instructions to launch a measured environment dynamically, resetting the relevant registers and establishing a new root of trust in the middle of a running system.

Limitations of Hardware-Based Attestation

Hardware-based attestation buys stronger guarantees than its software-only counterpart, but it is not free, and its costs define the boundary of where it can realistically be deployed.

The strength is clear: **hardware-based attestation schemes have stronger security properties** than software-based ones, because they do not depend on fragile timing assumptions. But these properties **rely on** specific hardware being present—either a **per-device TPM**, which must be fitted to every single device in the network, or a **trusted BIOS that begins attestation at boot time**, establishing the static root of trust described above. Both prerequisites presuppose that each device carries a nontrivial piece of trusted hardware.

Herein lies the practical limitation. **Employing tamper-resistant hardware on low-end embedded devices is not yet feasible from a cost perspective.** The IoT is populated overwhelmingly by cheap, mass-produced, severely constrained devices—the very perception-layer nodes with which this chapter began—for which adding a full TPM would multiply the unit cost beyond what the application can bear. A full-featured trusted module makes economic sense on a laptop or a server, but not on a one-dollar sensor deployed by the thousands. This tension—strong security requiring expensive hardware that constrained devices cannot afford—is the central dilemma that the next architectural idea sets out to resolve.

Necessary Hardware Components: The SMART Minimal Architecture

The resolution comes from asking a sharper question: rather than demanding a full TPM, what is the *minimum* hardware necessary to make remote attestation secure? This is the question answered by **SMART** (Secure and Minimal Architecture for Establishing a Dynamic Root of Trust), proposed by El Defrawy, Francillon, Perito, and Tsudik. SMART identifies a small set of lightweight hardware features that, together, suffice for secure attestation on even very cheap microcontrollers—far less than a TPM, yet enough to do the job.

Four hardware components are necessary. The first is **Attestation Read-Only Memory**. The attestation code resides inside the microcontroller unit (MCU) in ROM, and crucially, the **key k can only be accessed from this region**. Placing the key behind a hardware rule that ties its accessibility to the ROM-resident attestation code ensures that only the genuine, immutable attestation routine—and no injected malware—can ever read the secret.

The second component is **Secure Key Storage**. This is a dedicated memory region inside the CPU that **can be accessed only from the attestation code in ROM**. It is the physical home of the key, and its access restriction is enforced in hardware, providing the "exclusive access to k" property that the formal model demanded.

The third component is the **MCU Access Controls**, the hardware logic that actually enforces these rules. It **controls access to K and prevents non-attestation code from accessing it**—the circuitry that stands guard, permitting the ROM attestation routine to reach the key while blocking every other instruction stream.

The fourth component is **Reset and Memory Erasure**. This is a fail-safe: **if any error is reported by the above components**—for instance, an attempt by unauthorized code to access the key—**a hardware reset of the MCU is performed, and upon reset, the hardware enforces a memory cleanup**. Any secrets or sensitive intermediate state are wiped, so that a detected violation cannot leave exploitable residue behind.

The memory organization that realizes this can be read off the architecture directly. The MCU core communicates with a memory controller, which in turn governs the device's address space. Within that address space, the **SMART ROM** holds the attestation code, and adjacent to it—inside the protected

region—sits the **Key**. Separately lie the general-purpose **SRAM** and **Flash** memories, which hold ordinary, untrusted code and data. The decisive design point is the placement of the key immediately alongside the SMART ROM and behind the same access-control barrier: the hardware permits the key to be read only when execution is flowing through the SMART ROM code, so untrusted software running from SRAM or Flash is physically incapable of reaching it. With only these few additions to a commodity microcontroller—no full TPM required—SMART provides a hardware-anchored, dynamic root of trust at a cost that even low-end embedded devices can bear.

The SMART Protocol

The behavior of SMART is captured by the interaction between the **verifier (VRF)** and the **prover (PRV)**, together with the algorithm that the prover's SMART ROM executes. At the protocol level the exchange is compact. The verifier **generates a nonce n** and sends to the prover the tuple **(n, a, b, x, x_flag)**. The prover then runs **C = SMART(n, a, b, x, x_flag, −, −)**, and if **x_flag == True**, it additionally performs **Exec(x)**. It returns the result **C** to the verifier, who checks: **if C is correct then Accept, else Reject**.

The parameters carry specific meanings that the underlying algorithm makes clear. The pair **a, b** are the start and end addresses of the memory region to be attested. The value **x** is the address to jump to after attestation, and **x_flag** signals whether such a jump should occur at all—whether the attested code should be executed once verified. The nonce **n** is sent by the verifier to guarantee freshness, and an output address **out** specifies where to store the resulting checksum, with an optional input parameter **in** available for passing data. The output of the whole routine is an **HMAC**.

The ROM algorithm itself weaves together every hardware property discussed above. It **begins by disabling interrupts** during SMART code execution—the hardware realization of the "uninterruptible execution" property, closing the TOCTTOU window by ensuring nothing can interleave with the attestation. The **attestation key K is unlocked automatically by the MCU** precisely because execution has entered the ROM region, after which the routine initializes the HMAC with the key. It then **attests all the parameters** by processing them through the HMAC, binding the token to the exact request (the addresses, the flag, the nonce, and any input). Next it **attests the memory region [a, b]** by iterating over every address i in that range and feeding each memory word into the HMAC, so that the final tag depends on the complete contents of the region under test. The HMAC is then finalized to produce **C**, which is copied to the designated output address.

Before returning control, the routine performs its hygiene and dispatch. It **resets memory to erase temporary variables**, clearing any sensitive residue left by the key-bearing computation. Then it consults the execution flag: **if x_flag is True, it calls the function at address x** (passing the optional input), executing the just-verified code within the still-protected context—so that verification and execution are bound together and no TOCTTOU substitution can intervene. Otherwise, it **restores the interrupt status to what it was before** the SMART execution and returns. In this single, uninterruptible, key-protected routine, SMART delivers the complete chain: exclusive key access, measurement of the exact requested region, freshness via the nonce, and optional trustworthy execution of the attested code—all on hardware barely more complex than a bare microcontroller.

Hybrid Attestation

Between the two poles of pure software attestation—cheap but fragile—and full hardware attestation—robust but expensive—lies a middle path that has become the dominant paradigm for constrained IoT devices: **hybrid attestation**. The hybrid philosophy is to combine a minimal, carefully chosen set of trusted hardware features with a software routine that performs the actual attestation work. The

hardware provides exactly the properties that software alone cannot guarantee—exclusive key access, immutability, uninterruptible execution—while the measurement itself remains a software computation. SMART, encountered in the preceding section, is itself an early exemplar of this hybrid approach; the construction described here generalizes the pattern.

The protocol again follows the challenge-response structure, now explicitly partitioning the prover into an **untrusted software** portion and a protected **trusted component**. The prover and verifier **share a key k**. The exchange proceeds in four numbered steps. In **step 1**, the verifier sends a random nonce **N** to the prover. In **step 2**, the trusted component computes **h = HASH(Software)**, a hash over the device's software, reaching across the boundary to measure the untrusted region. In **step 3**, the trusted component computes the attestation token **δ = MAC_k(N ‖ h)**—a message authentication code, keyed with the shared **k**, over the nonce concatenated with the software hash—and transmits δ to the verifier. In **step 4**, the verifier, which expects a particular reference configuration **h′**, computes **δ′ == MAC_k(N ‖ h′)** and tests whether **δ′ = δ**. A match confirms that the device's software hashes to the expected value.

What distinguishes this from pure software attestation is that the hashing and MAC computation occur inside the trusted component, which the hardware guarantees cannot be bypassed, interrupted, or made to leak its key. The security no longer rests on fragile timing; it rests on the hardware's enforcement of the protected region's integrity. The feasibility of this approach is grounded in real hardware that provides exactly such protected regions—among them **TyTAN** and **TrustLite**, as well as commodity technologies such as **ARM TrustZone** and **Intel SGX**—each of which offers a means of isolating trusted code from untrusted code on the same device.

TyTAN: A Tiny Trust Anchor for Tiny Devices

The hybrid model becomes especially interesting when a single embedded device must run **more than one application** at once. A low-end device is no longer imagined as running one monolithic firmware image, but as a small multi-tasking platform hosting several applications of differing sensitivity—some critical, some ordinary, possibly supplied by mutually distrusting parties. **TyTAN** (Tiny Trust Anchor for Tiny Devices), proposed by Brasser, El Mahjoub, Sadeghi, Wachsmann, and Koeberl, is a hybrid architecture designed precisely for this richer setting.

The motivation is economic and architectural at once. Fully **trusted CPUs are expensive**, so TyTAN sets out to design a cheap yet secure architecture for low-end embedded systems that nonetheless provides four substantial guarantees: a **hardware-assisted dynamic root of trust** that allows secure task loading at runtime; **secure inter-process communication (IPC)**; both **local and remote attestation**; and **real-time guarantees**. It is explicitly built for **multi-stakeholder scenarios**, enabling the secure execution of mutually distrusting tasks on the same chip—so that an application from one vendor cannot spy on or corrupt an application from another, even while they share hardware.

The architecture layers a set of services over a modest hardware base. At the hardware level sit the **CPU**, a **platform key (K_P)**, and—the central innovation—an **Execution-Aware Memory Protection Unit (EA-MPU)**. An ordinary MPU enforces which memory regions a given piece of code may access; an *execution-aware* MPU additionally takes into account *which code is currently executing* when deciding whether an access is permitted. This execution-awareness is what allows the hardware to grant a secret, say the platform key, to one specific trusted routine while denying it to all other code, even code that would normally share the same privilege level.

Above this hardware foundation, TyTAN hosts a spectrum of software. It supports ordinary **normal tasks** and isolated **secure tasks**, the latter walled off from all other software including the operating

system. Secure tasks provide the security primitives—access control, digital signatures, secure storage—upon which other applications depend, while normal tasks handle non-sensitive work such as a vehicle's collision-prevention logic. A **real-time operating system** schedules these tasks. Notably, this OS is *not* itself a trusted component in the strongest sense; rather, it is an operating system selected to meet the system's requirements and treated as "almost trusted" by virtue of having been deliberately chosen and vetted. The crucial asymmetry is that **secure tasks are isolated from one another, whereas normal tasks are permitted to communicate** freely.

TyTAN's guarantees flow from several interacting mechanisms. Every **task carries a unique identifier**, computed for example as a hash of its own code—so that a task's identity is cryptographically bound to its contents, and any modification changes its identity. The **EA-MPU provides memory access control enforcement**: normal tasks may access only the memory allotted to them, while secure tasks may be granted access to more. To prevent an attacker from jumping into the middle of a task to skip its safety checks—the "controlled invocation" property made concrete—**each task can be invoked only at a dedicated entry point**, so that its first executed instruction is always its true beginning and malware cannot leap back and forth into arbitrary offsets. Interrupts are not forbidden, since a real-time system needs them, but they are carefully constrained: **interrupts are handled such that a malicious interrupt handler cannot gain any information about the state of an interrupted task**, closing a channel through which one task might otherwise spy on another. The **platform key is access-controlled by the EA-MPU**, reachable only by the legitimate trusted code. The real-time OS provides **real-time scheduling**, and **secure boot** ensures that TyTAN's trusted software components are loaded in their correct, authentic versions and isolated from the rest of the system by the EA-MPU from the very first instant—so that an attacker cannot compromise the OS or trusted software before the protections take effect.

The operation of several TyTAN subsystems deserves closer attention. The **EA-MPU driver** sets the memory-access-control rules in the EA-MPU when a secure task is loaded or unloaded, dynamically reconfiguring which regions each running task may touch. The rules for the *static* components—the fixed trusted software present from boot—are established during secure boot and are not altered at runtime; it is only the rules governing dynamically loaded tasks that change as tasks come and go.

Attestation in TyTAN is performed by having the root of trust compute a cryptographic hash over the binary code of each created task. Because the architecture must honor real-time deadlines, this **attestation must be interruptible during the hash calculation**—a significant departure from the uninterruptible model of SMART. A long hash computation that could not be preempted would risk missing a real-time deadline, so TyTAN permits the hash to be interrupted; it preserves security by ensuring that while a task is being measured, that **task's memory is isolated and its execution is prevented**, so no change can occur between successive fragments of the measurement. The measurement itself is now simple and linear rather than pseudo-random: there is no longer any need to traverse memory in an unpredictable order, because the hardware—not timing—guarantees integrity, so the routine computes a straightforward linear hash. **Local attestation** then compares this hash directly with the task's stored ID, while **remote attestation** additionally involves computing a **MAC** so that the result can be validated by a remote verifier.

Secure inter-process communication addresses the consequence of isolation: because a secure task cannot access another task's memory, two such tasks cannot cooperate except through an explicitly secured channel. TyTAN's IPC therefore includes a **mechanism to authenticate the task sending messages and to limit access to shared memory**. Whoever opens a communication channel—a FIFO or pipe—must be authenticated, so that the receiving task knows exactly which task it is talking

to. This authentication is what defeats replay attacks and man-in-the-middle attacks on the inter-task channel, since an impostor cannot masquerade as a trusted sender.

Secure storage builds on these primitives. For each task, a dedicated key is generated as **K_t = HMAC(id_t ‖ K_p)**—derived by applying an HMAC, keyed with the platform key, to the task's identifier—so that each task's storage key is both unique to that task and unforgeable without the platform key. Tasks interact with a dedicated secure-storage task over secure IPC, and **all data sent to the secure-storage task is encrypted with K_t**, ensuring that data at rest is protected and that one task cannot read another's stored secrets. Overseeing the whole system is a component responsible for **real-time monitoring**, charged with verifying that every real-time application continues to perform its functions without disruption—so that the security machinery does not inadvertently cause a safety-critical task to miss its deadline.

A Vulnerability of Attestation: Denial of Service

Every attestation scheme examined so far shares an implicit structure: the verifier initiates, and the prover must respond. This interactivity harbors a vulnerability. Because the device is obliged to perform a (sometimes costly) attestation computation whenever a verifier asks, an attacker can abuse the mechanism to mount a **denial-of-service (DoS)** attack. The attacker simply **requests an attestation, with a valid-looking nonce, over and over again**, flooding the device with challenges. Each challenge compels the device to drop its real work and grind through an attestation, so that a device under such a barrage is prevented from carrying out its actual function. The security mechanism itself becomes the lever by which the device is disabled.

Non-Interactive Attestation: SeED

The remedy for this DoS vulnerability is to invert the initiative. Instead of the verifier starting the attestation, the **device itself starts the attestation at a moment of its own choosing**—a design known as **non-interactive attestation**. If the device, not an external party, decides when to attest, then flooding it with external requests accomplishes nothing, because those requests no longer trigger any work. This is the approach of **SeED** (Secure non-interactive attestation for Embedded Devices), proposed by Ahmad, Sadeghi, and Zeitouni.

Inverting the initiative, however, creates a new difficulty. If the device self-triggers attestation, a *compromised* device might simply arrange never to trigger, or might detect an impending attestation and clean up its malware just beforehand, then re-infect itself afterward. To foreclose these evasions, SeED augments the minimal SMART hardware with two additional components. The first is a **secure real-time clock (RTC)**: a time source whose value cannot be modified by the attacker. This matters because, if an attacker could learn that an attestation is about to occur and could manipulate the device's sense of time, it could rewind or stall the clock to prevent the attestation from firing; a tamper-proof clock removes this option. The second is an **attestation trigger**: a hardware mechanism that interrupts normal execution and launches the attestation process at the appointed time, together with a **timeout** that governs the scheduling.

The memory and hardware organization reflects these additions. The device retains the SMART hardware features—its ROM-resident attestation code and protected key—and layers on the RTC and the attestation trigger (AT), both fed by the system clock. In RAM reside the operating system, ordinary software, and a protected block of attestation-related state: the current seed-derived value and the attestation timing variables. In ROM reside the platform key and the secure-boot routine, together with the persistent attestation state. The protected attestation region is placed behind memory-protection

rules so that not all code can reach it, and the attacker cannot read the values that determine *when* the next attestation will occur.

The protocol itself is a one-way transmission—the device speaks, the verifier listens—organized around a pseudo-random schedule that both parties can reproduce. The prover maintains a chain of seeds. Given the previous seed value **σ_{i−1}**, it generates a new pair **{σ*i*, ϑ_*i*} ← *gen*(σ{i−1})**: a fresh seed σ_i to carry the chain forward, and a value ϑ_i that determines the next attestation interval. From ϑ_i it computes the sleep interval as **t_att ← ϑ_i mod t_att^Max**, bounding the wait by a maximum, and arms the hardware with **timeout(t_att)**. It advances its running record of the attestation schedule, **T_att ← T_att + t_att**. When the timeout fires, the device takes a software measurement, **h ← getSoftMeas()**—the hash of its own software state—and computes the authentication tag **μ ← mac(k_att; h ∥ T_att)**, binding the measurement to the scheduled attestation time. This single message μ is sent to the verifier.

The reproducibility of the schedule is the linchpin. Because both prover and verifier start from the same initial seed **σ_0**, both generate the identical sequence of seeds, intervals, and attestation times—the verifier can regenerate everything the prover computed and thereby know, independently, both *when* an honest device should have attested and *what* tag it should have produced. On receiving μ, the verifier **regenerates the expected values and checks the tag**. It recomputes the seed chain to obtain ϑ_i and hence t_att, advances its own T_att, and verifies the MAC with **vermac(k_att; b ∥ T_att, μ)**. If the MAC verifies, it reads the current time **T_rec ← time()** and checks that the report arrived within an acceptable window, **T_att − δ_t < T_rec < T_att + δ_t + t_tr**, where δ_t tolerates clock skew and **t_tr** absorbs transmission delay. Only if the report is both authentic and punctual does the verifier set its result **r ← 1**, accepting the device. The seed σ lives within the protected memory region, inaccessible to ordinary code, so the attacker cannot predict the schedule.

SeED's resistance to the full adversary hierarchy can now be stated precisely. Against a **communication adversary**, it is secure because the prover and verifier maintain synchronized clocks and reproduce the same schedule; anything out of the ordinary—a missing report, or one arriving at the wrong time—is read as evidence of a compromised device. Against a **software adversary**, it is secure because the hardware guarantees the integrity of the measurement process and of the clock, and the confidentiality of the key, so malware can neither forge the measurement nor steal the signing key. And against a **mobile adversary**—one able to erase its traces—it is secure because the adversary knows the *previous* attestation time but not the *next* one: the next attestation time is randomized by the pseudo-random generator seeded in protected memory, so the attacker cannot clean up just before a measurement and re-infect just after. For this to hold, neither the triggering code nor any other observable component may reveal the attestation time in advance; the trigger is realized either by polling or, more robustly, as a hardware non-maskable interrupt that the attacker cannot suppress.

The performance profile of SeED is favorable on exactly the axes that matter for constrained devices. It uses **one-way communication**, eliminating the round trip that interactive schemes require. It exhibits **lower runtime** and demands **lower computational power** than its interactive counterparts. When attestation runtime is plotted against software size, three regimes appear: a traditional interactive scheme traces a middle line; a DoS-protected variant of the traditional scheme traces the highest line, paying extra for its protection; and **SeED traces the lowest line**, achieving both DoS resistance and the least runtime of the three. SeED thus delivers DoS protection essentially for free relative to the traditional baseline, because shifting the initiative to the device removes the very channel through which the flooding attack operated.

The Limits of the Single-Prover Setting

Every scheme examined up to this point—software-based, hardware-based, hybrid, interactive, and non-interactive—shares a common horizon: it attests a *single* device. This single-prover framing, adequate for verifying one camera or one sensor, strains badly when confronted with the reality of modern IoT deployments, which routinely involve hundreds, thousands, or millions of devices operating as a collective.

Two limitations make the single-prover setting unsuitable at scale. First, it is simply **not realistic for large-scale implementations**: a verifier that must contact every device individually, one after another, faces a workload that grows linearly with the size of the network, and the aggregate attestation of a large swarm would take prohibitively long. If each device requires its own round-trip exchange and its own verification, a network of a million devices imposes a million such interactions on the verifier. Second, the approach is **time-consuming and relies on costly hardware**—tamper-resistant modules or TPMs—on the devices, which **makes it unrealistic for large-scale employment**. Fitting expensive trusted hardware to every one of a million cheap nodes, and then attesting each one serially, is economically and temporally infeasible. These twin obstacles motivate a fundamentally different architecture: one that attests an entire network efficiently, rather than one device at a time.

From Device Attestation to Swarm Attestation

To appreciate the shift, it is worth recalling the shape of ordinary **device attestation** in its most distilled form. A single **verifier** issues a **challenge** to a single **prover**; the prover performs its **attestation** computation and returns a **response**; the verifier **verifies the response**. The entire transaction concerns one device and one verifier.

Now imagine this picture multiplied across a dense field of devices—dozens or hundreds of identical embedded boards spread across an environment. The verifier, confronted with this swarm, is overwhelmed: attesting each board individually, through its own challenge-response exchange, does not scale. This is the problem of **swarm attestation**, and it demands that the work of attestation be distributed across the network itself rather than concentrated at a single verifier.

Collective Remote Attestation

The answer is **collective remote attestation (CRA)**. The problem it solves is precisely stated: **to verify the integrity of a network of devices, more efficiently than attesting them individually**. Rather than having a single verifier laboriously poll every prover, CRA enlists the devices to attest one another, so that the verification effort is spread across the whole network and the verifier receives a single, aggregate result.

The architecture involves a population of **provers**—the many devices whose integrity is in question—and a **verifier** that wishes to learn the overall health of the network. The central idea, which recurs across all the CRA protocols that follow, is that each prover does not merely answer the verifier; it also participates in attesting its neighbors, so that **each prover becomes, in effect, the verifier of another device**. This changes the trust landscape in an important way: unlike the single-device setting, one can no longer assume that every verifier is a trusted party, because the "verifiers" are now ordinary devices that may themselves be compromised. A further and decisive benefit of this device-to-device structure is that **prover-verifier interaction takes place at one-hop distance**—each device attests only its immediate neighbors—which directly solves the multi-hop timing problem that crippled software-based attestation. Because every attestation exchange is local, transmission times remain short and predictable, and the unreliable latency of long multi-hop paths never enters the picture. (The study of CRA at IoT scale, including its state of the art and open challenges, is surveyed by Ambrosin, Conti, Lazzeretti, Rabbani, and Ranise.)

SEDA: Secure Embedded Device Attestation

The first and foundational CRA protocol is **SEDA** (Secure Embedded Device Attestation), proposed by Asokan, Brasser, Ibrahim, Sadeghi, Schunter, Tsudik, and Wachsmann. SEDA organizes the network's attestation around a **spanning tree**: the devices are arranged into a tree rooted at the device nearest the verifier, and the attestation flows through this structure. **Each node attests its children and reports the aggregated result to its parent**, so that results flow upward from the leaves to the root, and the root ultimately delivers a single summary to the verifier.

This design yields two notable advantages. First, SEDA **scales "logarithmically" in the number of provers**: because the tree's depth grows only logarithmically with the number of nodes (for a balanced tree), the time for results to propagate from leaves to root grows far more slowly than the linear cost of attesting each device separately. Second, SEDA **uses symmetric cryptography**, which is cheap enough to run on constrained devices—avoiding the costly public-key operations that would burden low-end hardware.

SEDA's System Model and Assumptions

SEDA rests on two system assumptions. First, **all devices are equipped with a trusted component**—a minimal hybrid-style trust anchor of the kind developed in earlier sections, present on every node so that each can perform a trustworthy local measurement. Second, **devices talk only to their neighbors**: all communication is one-hop, consistent with the locality principle that underpins CRA's resistance to multi-hop timing problems. The verifier sits at the edge of this network, communicating with the root of the spanning tree.

SEDA's Attacker Model

SEDA defends against two classes of adversary. The first is an **eavesdropper**, who can observe the traffic flowing across the network's links. The second is a **software-only attacker**—for instance, malware—resident inside a device. Each prover is understood as comprising an **untrusted software** portion, which the attacker may corrupt, and a **trusted component**, which it cannot. Notably, SEDA's threat model does *not* include physical attackers; an adversary who can physically tamper with a device's hardware falls outside the protection SEDA offers, a gap that later protocols set out to close.

The SEDA Protocol

SEDA's operation divides into an **offline** phase, performed once at setup, and an **online** phase, performed at each attestation.

The **offline** phase prepares the devices and their relationships. During **initialization (Init)**, each device is provisioned with an **asymmetric key** and with **certificates attesting its code configuration and public key**. During **registration**, neighboring devices **exchange their certified code configurations** with one another and **establish a shared cryptographic key for communication with each neighbor**. The asymmetric key and certificates serve to guarantee, to the verifier, the authenticity of signatures, while the per-neighbor symmetric keys provide the cheap channels used during the online phase.

The **online** phase is described from the perspective of a single device. A device **receives the global nonce from a neighbor and accepts that neighbor as its parent**—unless it has already received the nonce from someone else, in which case it recognizes that it already has a parent and **returns NULL** to the later requester, so that a device with two potential parents attaches to only one and tells the other to disregard it (this is how a single tree is carved out of a general network). Having accepted a

parent, the device **shares the nonce with its own neighbors**, propagating the attestation outward, and **attests itself**. It then **waits for the responses from its children**. Finally, it combines its own result with its children's results and **returns to its parent a signed tuple** [**b**, **β**, **τ**], together with its configuration. The components of this tuple carry the aggregated verdict: **b** is a boolean indicating whether everything in the subtree is fine; **β** is the count of correctly attested nodes in the subtree; and **τ** is the total number of nodes in the subtree. Optionally, the response may also carry the **IDs of any compromised devices**, so that the verifier learns not merely *how many* nodes failed but *which* ones. As these tuples flow upward and are combined at each level, the root eventually holds a complete tally for the whole network, which it reports to the verifier.

SEDA's Limitations

For all its pioneering value, SEDA carries three significant limitations, each of which drives the development of its successors.

The first is a **lack of flexibility**: in SEDA, *all* devices must participate in the attestation. There is no provision for attesting only a subset of the network, nor for gracefully handling devices that are temporarily unavailable; the protocol presumes a fixed, fully participating population.

The second is that the **final attestation result is not publicly verifiable**. Because SEDA relies on symmetric cryptography and shared keys, the aggregated verdict can be checked only by a party holding the appropriate secrets. A third party cannot independently confirm the result, which limits the protocol's usefulness in settings where the outcome must be auditable by others.

The third, and most serious, is that SEDA **fails against physical attacks**. This weakness is intrinsic to the tree structure. Consider an attacker who physically compromises an *internal* node of the spanning tree—a node with children of its own. Because that node is responsible for collecting, combining, and forwarding the attestation results of its entire subtree, a physically compromised node **can arbitrarily modify the attestation results of its subtree**. It might, for instance, receive honest reports from its children indicating that several of them are compromised, and simply rewrite the aggregate to claim that all is well before passing it upward. Since the parent trusts the tuple it receives, the deception propagates all the way to the verifier. The tree's efficiency—having each node summarize its subtree—is thus also its Achilles' heel, because a single corrupted intermediary can falsify the verdict for everything beneath it.

SEDA's Performance

SEDA's experimental evaluation confirms its favorable scaling and characterizes its costs along three dimensions.

Regarding overall **performance for tree topologies**, the run-time is plotted against the number of devices in the swarm for trees of varying branching factor—binary, 4-ary, 8-ary, and 12-ary. All of these curves rise steeply at first and then flatten into a plateau, so that run-time grows only very slowly as the swarm expands from tens of thousands to a million devices. This flattening is the visible signature of logarithmic scaling: beyond a certain point, adding more devices barely increases the total attestation time. Among the branching factors, the trade-off is that higher branching reduces tree depth (fewer levels to traverse) but increases the work at each node (more children to attest), with the 4-ary tree offering a favorable balance.

Regarding **run-time per device**, the cost is plotted against the number of neighbors a device has, separately for an initiating device and for other devices, under two different trusted-hardware bases—SMART and TrustLite. The run-time of the device that *initiates* the attestation, using the heavier

SMART base, rises most steeply with the number of neighbors, reaching several seconds; the other devices under SMART rise less steeply; and both the initiator and other devices under **TrustLite remain very low and nearly flat**, demonstrating that the choice of trust anchor strongly influences per-device cost and that a well-chosen anchor keeps it modest even as connectivity grows.

Regarding **energy consumption**, the cost is plotted against the number of neighbors for two sensor-node platforms, MICAz and TelosB, again distinguishing the initiator from other devices. The initiator on the MICAz platform consumes the most energy, rising from roughly 50 millijoules upward as neighbors increase; the initiator on TelosB consumes substantially less; and the *other* (non-initiating) devices on both platforms consume very little, their curves hugging the bottom of the chart. The pattern confirms that the bulk of the energy burden falls on the initiating node, while ordinary participants pay only a small energy price—an encouraging property for battery-powered swarms, provided the initiator role is not too onerous.

SANA: Secure Aggregate Network Attestation

SEDA established that a network could be attested collectively through a spanning tree, but it left three wounds open: a compromised internal node could falsify its subtree's verdict, the final result could not be publicly verified, and the scheme buckled against physical attackers. **SANA** (Secure Aggregate Network Attestation), proposed by Ambrosin, Conti, Ibrahim, Neven, Sadeghi, and Schunter, is engineered to heal exactly these wounds. Its central technical device is an **aggregate signature scheme** that allows many devices' signed proofs to be combined into one compact, publicly verifiable signature, with security that survives even the physical compromise of the intermediaries doing the combining.

SANA works over a network of **heterogeneous devices**, each of which **talks only to its neighbors** and which together can **form an overlay aggregation tree**. What distinguishes SANA's architecture is that it recognizes four distinct roles: **provers**, **aggregators**, an **owner**, and a **verifier**—and, crucially, a single device may play more than one role, so that **a device may be both a prover and an aggregator** simultaneously. In the topology, leaf and interior devices act as provers (labeled P_1 through P_5), certain interior devices additionally serve as aggregators (labeled A_1 and A_2) that combine the proofs flowing through them, the **owner** holds the reference information defining what a healthy device looks like, and the **verifier** ultimately receives and checks the aggregated result.

SANA's Attacker Model

SANA's threat model is strictly stronger than SEDA's. It defends against the familiar **eavesdropper** and **software-only attacker**, but it now adds a **physical attacker** to the roster. This is the decisive advance. The way SANA accommodates physical compromise is subtle and rests on separating the trust requirements of the two roles. A physically compromised **aggregator** is tolerated, because—as will be seen—it cannot forge a valid attestation result even if it is fully in the attacker's hands. The remaining limitation is that a physically compromised **prover** can evade detection, since an attacker who has physically extracted a prover's secrets can produce a valid-looking proof for that one device; but it cannot thereby corrupt the verdicts of *other* devices. The attacker is modeled as able to sit on the links (eavesdropping), inside devices' untrusted software, and—for aggregators and provers alike—physically present, while the owner and verifier remain trustworthy anchors.

SANA's Core Idea

The elegance of SANA lies in how it distributes trust requirements asymmetrically between provers and aggregators, so that the expensive, highly-trusted hardware is needed only where it is unavoidable.

Only the **provers (P_j)** require a **Trusted Execution Environment (TEE)**. Each prover, from within its TEE, **builds an attestation proof**. This proof **contains a digital signature σ_j over the measured configuration**, and—because SANA aims at public verifiability—**σ_j is designed to be publicly verifiable**: anyone holding the appropriate public key can check it, without needing any shared secret. The use of digital signatures rather than MACs is precisely what delivers the public-verifiability property that SEDA lacked.

The **aggregators (A_i)** have a different job: they **"combine" the proofs of the provers**. For this combination to be efficient, the signatures must be **aggregatable**—meaning that many individual signatures can be merged into a single short signature whose size does not grow with the number of signers. This is the technical heart of SANA: an aggregate-signature construction lets an aggregator fold together the proofs of its entire subtree into one compact object, so that the final result transmitted to the verifier remains small regardless of how many devices contributed.

The security payoff of this division is profound and directly addresses SEDA's fatal flaw. Because the individual signatures are generated inside the provers' TEEs and merely combined (not fabricated) by the aggregators, a **physically compromised aggregator cannot forge the attestation result**. An aggregator can combine the signatures it receives, but it cannot invent a valid signature for a configuration that no honest prover actually signed—so it **cannot mount a man-in-the-middle attack** by rewriting its subtree's verdict. This is exactly the attack that broke SEDA, and SANA neutralizes it by ensuring that the authority to vouch for a device's state rests solely with that device's TEE-protected signature, not with the intermediary that forwards it. The residual weakness is narrow and acknowledged: a **physically compromised prover can evade detection** for itself, but the damage is confined to that single device and cannot spread to falsify the network's overall result.

Network Software Integrity Verification: The SANA Flow

The full SANA procedure can be traced as a flow of information down the tree from the verifier and back up through the aggregators, culminating in a single verification at the verifier.

The challenge descends. The verifier constructs a challenge of the form $\mathbf{Ch = \{N, T = \{h_g, H\}\}}$ and disseminates it down through the aggregation tree—from the verifier to A_2, from A_2 to A_1 and to the other provers it parents, from A_1 to its own children, and so on until every device has received **Ch**. Within this challenge, **N** is the familiar random nonce ensuring freshness. The set **H** is the set of **"known good" configurations**—the reference list, supplied by the owner, of every software state a device is permitted to be in. And $\mathbf{h}g = HASH(h_1 \parallel \ldots \parallel h\{|\mathbf{H}|\})$ is a single hash computed over the concatenation of all the known-good configuration hashes—a compact digest of the entire set of acceptable states.

Each prover attests and produces a proof. Every device measures its own configuration and compares it against the set of known-good configurations encoded in the challenge. The outcome determines the content of the proof it generates. If the device's **configuration is "good"—that is, present in H**—then the resulting proof contains an **aggregatable (OAS) signature over h_g ‖ N ‖ ...**: by signing the digest of the good set together with the nonce, the device certifies "I am in one of the approved states, freshly, for this challenge," without needing to reveal which specific good state it is in. If, on the other hand, the configuration is **not good**, the proof instead contains an **OAS signature over the "bad" configuration** actually measured, together with the **device's public key pk_i**—so that the verifier will learn both that this device is compromised and precisely which device and which bad configuration are at fault.

Aggregators combine proofs as they ascend. As the proofs flow back up the tree, each aggregator **aggregates the signatures it receives and propagates the public keys of any provers reporting a "bad" configuration**. The crucial efficiency is that the many "good" signatures—all over the same h_g ‖ N—collapse into a single aggregate signature, while only the comparatively few "bad" reports need to be carried individually along with their identifying public keys. At aggregator A_1, the good proofs from its subtree are merged into one; at A_2, A_1's aggregate is further merged with A_2's own subtree, and the equality of the combined good-signatures confirms they all signed the identical h_g ‖ N. Step by step, the whole network's good-device attestations condense into one signature, accompanied by a short list of the bad devices' identities.

The verifier checks once. Ultimately a single aggregated proof reaches the verifier, who performs **Verify(·)** on it. Because the good signatures have all been aggregated into one, the verifier confirms in a single check that every device whose signature was folded in was indeed in a good state, and it reads off, from the propagated public keys, exactly which devices were bad. The computational cost of this final verification is **linear in the number of bad provers**, which in a healthy network is far smaller than N, the total number of devices. This is a remarkable property: the verifier's work scales not with the size of the network but with the (hopefully tiny) number of compromised devices, so that attesting a vast but mostly-healthy network costs almost nothing at the verifier.

SANA's Improvements

Gathering these threads, SANA's contribution over the prior state of the art can be stated crisply. It **allows end-to-end secure and scalable attestation**, and it improves upon earlier protocols in three specific ways. It **guarantees public verifiability**, so that the attestation result can be checked by any party rather than only by a holder of shared secrets. It **considers less strict requirements for protocol entities**, demanding a full TEE only of provers and tolerating weaker assumptions elsewhere. And it **provides better resilience to both physical attacks**—since compromised aggregators cannot forge results—**and denial-of-service attacks**.

SANA's Performance

SANA's run-time is evaluated against the number of devices in the network, holding fixed a certain number of bad provers, considering fifty different bad configurations, and assuming that aggregators are more powerful devices than ordinary provers. The comparison pits SANA against SEDA and against an identity-augmented variant called SEDA-ID (SEDA extended to report the identities of compromised nodes). Plotted across networks scaling up to a million devices, the curves for 4-ary trees reveal that SANA's run-time rises to a modest plateau and then stays essentially flat as the network grows, remaining close to that of SEDA-ID for large networks. The headline result is that SANA achieves **comparable run-time to SEDA-ID for large networks** while delivering its far stronger security guarantees—public verifiability and resilience to physical attacks—essentially without a scalability penalty. SEDA itself, lacking identity reporting, runs fastest but offers the weakest guarantees; SANA buys dramatically better security for a cost that, at scale, is negligible relative to the identity-aware baseline.

DARPA: Device Attestation Resilient to Physical Attacks

SANA tolerated physical attackers by ensuring that a compromised aggregator could not *forge* a result, but it did not actively *detect* when a device had been physically tampered with. **DARPA** (Device Attestation Resilient to Physical Attacks), proposed by Ibrahim, Sadeghi, Tsudik, and Zeitouni, attacks the problem from a complementary angle. Rather than only verifying software integrity at attestation

time, DARPA adds a mechanism for **continuously monitoring the physical presence of devices in the intervals between attestations**, so that the act of physically removing a device—which any hardware tampering requires—can itself be caught.

The insight behind DARPA concerns the physical mechanics of a hardware attack. To physically compromise a device, an attacker must take it offline: a device has to be physically removed from the network, have a component swapped or modified, and then be reinserted. This removal necessarily creates a window during which the device is absent from the network. DARPA is designed to notice that absence. By doing so, it closes a variant of the time-of-check-to-time-of-use gap at the hardware level—an attacker cannot quietly extract a device, tamper with it, and slip it back without the gap in its participation being observed by its neighbors.

The mechanism works through a protocol of periodic liveness announcements. In the time between two successive attestations, **each device in the network periodically monitors the other devices**. Every device **broadcasts its heartbeat—an announcement of its continued presence, or "attendance"—to its neighbors**. Each device **collects the heartbeats it receives from its neighbors and broadcasts them onward to its own neighbors, and so on**, so that knowledge of each device's liveness propagates across the network. These **heartbeats are stored into logs by every device**, building up a distributed record of which devices have been continuously present and which have gone missing. When the time comes to attest, the network **performs SEDA to collect the heartbeat logs**, aggregating this presence information up the spanning tree in the same manner that SEDA aggregates attestation results. A device that disappeared from the heartbeat record during the inter-attestation interval—because it was physically removed for tampering—is thereby flagged, even if, after reinsertion, its software appears to attest correctly.

The trade-offs of this design are clear. On the positive side, DARPA is **able to address physical adversaries along with the software and hybrid adversaries** handled by earlier schemes: the heartbeat mechanism catches the physical-removal signature that pure attestation misses. On the negative side, DARPA **inherits the same flaws as SEDA**, upon which it is built. It is **not suitable for dynamic networks**, because the spanning-tree structure and the heartbeat neighborhoods assume a relatively stable topology; and the **details of compromised devices are not present in the attestation report**, since the SEDA aggregation it relies upon does not natively carry per-device identity information. There is, moreover, an implicit cost worth naming: the continuous broadcasting and relaying of heartbeat messages consumes power, which makes such a solution poorly suited to low-battery devices for which every transmitted message is an expenditure of a scarce energy budget.

A Comparative View: Other Spanning-Tree Protocols

DARPA, SEDA, and SANA are the most prominent members of a larger family of CRA protocols all built upon the spanning-tree idea. Three further protocols round out this landscape, each improving on its predecessors along a particular axis while sharing the family's characteristic weakness in the face of highly dynamic networks.

Lightweight Swarm Attestation (LISA), presented by Carpent and colleagues, improves on SEDA in terms of **scalability and resiliency**. Its distinctive contribution is conceptual as much as technical: it introduces a metric called **Quality of Swarm Attestation (QoSA)**, a yardstick for comparing the quality of different attestation schemes along dimensions such as how much information the final report conveys. Its limitations are that it **requires full connectivity during attestation**, that it **does not support device mobility during attestation**, and that, like its relatives, it is **unsuitable for dynamic networks**.

SCAPI (A Scalable Attestation Protocol to Detect Software and Physical Attacks), by Kohnhäuser and colleagues, improves on DARPA in terms of **resiliency**, and it likewise **mitigates denial-of-service attacks and addresses physical adversaries**. Its limitation is that it is **not suitable over highly dynamic networks**.

SeED (Secure Non-Interactive Attestation for Embedded Devices), by Ibrahim and colleagues —encountered earlier as the non-interactive single-device scheme—also belongs to this comparative picture at the swarm level. Its contribution is the introduction of a **non-interactive attestation scheme**, and it too **mitigates denial-of-service attacks and addresses physical adversaries**. Its limitation, again, is that it is **not suitable over highly dynamic networks**.

The recurring refrain across this entire comparison is unmistakable: every spanning-tree protocol, however it improves on scalability, resiliency, DoS resistance, or physical-attack detection, stumbles when the network becomes **highly dynamic**—when devices move, join, leave, and change their neighbors during the attestation itself. The spanning tree is a static scaffold, and a static scaffold cannot be erected on shifting ground. This shared limitation is precisely the frontier that the final family of protocols, built for mobile and disruptive networks, sets out to conquer.

From Static Trees to Dynamic Swarms

The spanning-tree protocols share a hidden assumption that their recurring failure mode lays bare: they presume that the network's topology holds still long enough for a tree to be built and used. In a **tree-based collective remote attestation**, the devices are arranged into a fixed hierarchy—a verifier at the root, and beneath it a cascade of parent-child links through which challenges flow downward and aggregated results flow upward. Each link is a committed relationship: a parent attests specific children, and those children report to that specific parent. The whole edifice depends on these relationships persisting for the duration of the attestation.

But many of the most compelling IoT scenarios involve devices that will not hold still. Picture a swarm of drones in flight, or a fleet of mobile robots, or any collection of nodes whose physical positions—and therefore whose radio neighborhoods—change continuously. In such a setting, the parent-child links of a spanning tree dissolve as fast as they are formed: a node that was a child of one device a moment ago may have flown out of range and into the neighborhood of an entirely different device. A drone that is attesting a neighbor may, mid-protocol, lose contact with that neighbor and gain three others. This is **dynamic collective remote attestation**, and it demands a fundamentally different organizing principle. One cannot build a rigid tree on a swarm whose edges are in constant flux; the attestation must instead be structured so that it tolerates—indeed, embraces—continual change in who is talking to whom.

PADS: Practical Attestation for Highly Dynamic Swarm Topologies

The protocol that breaks decisively with the tree is **PADS** (Practical Attestation for Highly Dynamic Swarm Topologies), proposed by Ambrosin, Conti, Lazzeretti, Rabbani, and Ranise. PADS abandons the spanning tree altogether and replaces it with a principle borrowed from distributed computing: **consensus**, achieved through the continual, opportunistic exchange of information among whatever neighbors happen to be in range at any moment.

PADS operates over **heterogeneous devices**, each of which **talks only to its neighbors**, but with a defining new characteristic: the **devices are mobile and do not form an overlay spanning tree for attestation**. Several design commitments follow from this. Attestation is based on **self-attestation**:

rather than one device attesting another, every device measures *itself*, and all devices begin their local self-attestation together, synchronized by a real-time clock and an attestation trigger (the same non-interactive machinery developed for SeED). Having each attested itself, the devices then use the concept of **"consensus" among themselves to corroborate the attestation result**—they spread their individual findings across the swarm so that, through repeated exchange, knowledge of the whole network's state diffuses to every participant. A powerful consequence is that the **verifier can obtain a "snapshot" of the network at runtime**: at any moment, because the devices have been continually sharing, each device holds an approximate picture of the entire network's health, and the verifier can read off that picture by attesting any single device and querying its accumulated knowledge. This diffusion-by-repeated-local-exchange is the classic distributed-systems pattern known as **gossip**: information propagates the way a rumor spreads through a crowd, hop by hop, until everyone has heard it.

PADS Device Requirements

PADS asks relatively little of each device's hardware, keeping to the minimal hybrid foundation established earlier. Each device needs a **Read-Only Memory (ROM)**, a **Memory Protection Unit (MPU)**, and a **secure Real-Time Clock (RTC)**. The role of each is visible in the device's memory organization. The CPU is driven by a clock system that supplies both the RTC and an attestation trigger (AT). The MPU governs access to memory according to a table of rules, each rule naming an allowed code region, a memory region, and an access policy (read, or read-and-write). The protected regions of memory hold the attestation-critical state: in one region reside the attestation variable **v_att**, the set of known-good configurations **H = {h₁, ..., h_M}**, and the attestation key **k_att**; another region holds the **PADS routines** and the operating system. The ROM holds the **platform key k_platform** and the **secureBoot()** routine. The MPU's rules are arranged so that the PADS routines may read the sensitive attestation state while ordinary software cannot, enforcing—on cheap hardware—the confidentiality that the consensus mechanism depends upon.

PADS Entities and the State Encoding

As in SANA, **only the provers (P_j) require a Trusted Execution Environment (TEE)**, and each prover **builds an attestation proof** from within it. This proof **contains the hash value of the underlying software**, and it encodes a verdict drawn from **three possible states**, represented by two bits each: **Good (10)**, **Bad (00)**, and **Unknown (11)**. The three-state encoding is essential to a dynamic setting, because a device may genuinely not yet know the status of some far-off node it has not heard about—"unknown" is a first-class answer, distinct from both "healthy" and "compromised." Every prover **shares its knowledge with the other nodes in range**, which is the mechanism by which the gossip proceeds. The **verifier (vrf)** participates by first attesting an individual node before trusting and reading its accumulated knowledge about the network. As with SANA, the residual weakness is that a **physically compromised prover can evade detection** for itself, though it cannot corrupt the collective picture.

The representation that makes this scalable is a compact binary string. To attest a network of **N devices**, PADS uses a binary string of **2N bits**—two bits per device, matching the three-state encoding. Each device is assigned a device number **i**, and the two bits recording that device's state occupy positions **2i and 2i+1** of the string. In this way, the complete status of the entire network—good, bad, or unknown for every one of its members—is packed into a single bitmask of length 2N that every device carries, updates, and shares. When a device learns something new about node i, it writes the corresponding verdict into those two bit positions.

Maximum Consensus

The rule by which devices reconcile their differing bitmasks is called **maximum consensus**, and it is designed for efficient network-status communication among devices. When two devices meet and exchange their observations, each updates its own bitmask by taking, position by position, the "maximum" of its current value and the values reported by the neighbors currently in range. Formally, a prover's observation at the next step is computed as

$$x_i^{t+1} = \max_{j \in N\{i,t\} \cap R_t\}} \left(x_i^t, \{x_j^t\} \right),$$

where the update is applied componentwise: for each position l,

$$x_i^{t+1}[l] = \max_{j \in N\{i,t\} \cap R_t\}} \left(x_i^t[l], \{x_j^t[l]\} \right), \text{ for } l = 0, \ldots, m.$$

Here x_i^t is the network observation held by prover i at step t; m is the length of the Bloom filter (introduced in the next part of the course); $N_{i,t}$ is the set of neighbors of prover P_i at step t; and R_t is the set of active and responsive provers at step t. The device thus merges its own view with the views of exactly those neighbors that are both in range and responsive at that moment—which is precisely the set that shifts continually in a mobile swarm.

The semantics of taking the maximum encode a deliberate security policy. The three states are ordered by their bit patterns so that taking the maximum resolves disagreements conservatively. Concretely, if two devices hold **discordant opinions about a given node**—one asserting that, say, device 1 is compromised and another asserting it is not—then the consensus rule resolves the conflict by **declaring the device compromised**. A "bad" or suspicious verdict, once it appears anywhere in the swarm, propagates and dominates, so that a compromise detected by even a single honest observer cannot be erased by others' more optimistic reports. The operation is, in effect, **equivalent to a bitwise OR** across the devices' observations, which monotonically accumulates evidence of compromise and never discards it. This monotonicity is what allows the gossip to converge: evidence only ever grows, so the shared picture stabilizes as information spreads.

The PADS Protocol in Motion

The life of a PADS attestation can be traced through four phases, which together show how a global verdict crystallizes out of purely local exchanges.

Initialization. Every device begins with its bitmask set entirely to zeros—a blank slate in which it has recorded nothing about anyone, including itself. The verifier stands by, ready eventually to query the swarm. At this moment no device knows anything about the network's state.

Self-attestation. Triggered together by their synchronized clocks, every device measures its own software and writes its own verdict into its bitmask. An honest device records itself as "healthy" in its own two bit positions, while a device that detects corruption records "compromised." After this phase, each device knows its *own* status with certainty but still knows nothing about the others—the bitmasks differ from device to device, each populated only in the slot corresponding to itself.

Consensus. The devices now begin to gossip. Whenever two devices come within range—a relationship that changes constantly as the swarm moves—they exchange their MAC-protected observations for the current time step and each applies the maximum-consensus (bitwise-OR) update. Through repeated such encounters, the verdicts recorded by individual devices spread outward: a "compromised" mark placed by one device on itself, or by one observer on a neighbor, diffuses across the swarm until many devices carry it. Over successive rounds the bitmasks grow richer and begin to agree, converging toward a common picture of the whole network.

Verification. Eventually the verifier obtains the **network state** by reading the accumulated bitmask from a device it has attested. A device that has been in contact with much of the swarm will hold a nearly complete picture and can report the overall state. But the dynamic setting introduces two honest caveats that the verifier must respect. A device that has had limited contact may have **no complete knowledge**—its bitmask still contains "unknown" entries for nodes it never heard about—and a device that has drifted **out of the communication range** of the rest of the swarm simply cannot have gathered or contributed recent information. The three-state encoding is what makes these situations expressible: "unknown" honestly signals the gaps that mobility inevitably creates, rather than forcing a device to guess.

Interpreting the Result

Once the verifier holds a device's bitmask, it can extract two kinds of conclusion. It can **estimate the number of compromised devices** by counting the "bad" entries across the mask, obtaining an overall measure of the network's health. And it can **determine whether a specific device is compromised**, reading off that device's two bit positions—though, in the dynamic and gossip-based setting, this determination carries some **uncertainty**, since the information may be incomplete or stale depending on how thoroughly it has propagated. The verifier thus gains a probabilistic, network-wide snapshot rather than the crisp, exhaustive verdict that a static tree could in principle provide—a reasonable price for the ability to attest a swarm that never stops moving.

PADS Security

PADS is **secure against communication adversaries, software adversaries, and mobile software adversaries**—the three software-level threat classes developed at the outset of the chapter. Against these, the combination of self-attestation, MAC-protected exchanges, protected memory, and the monotonic consensus rule holds. The natural question is what an adversary *can* still do within this model, and the honest answer is narrow: the principal residual action available to an adversary is **not participating in the protocol**—simply staying silent, refusing to broadcast or relay. But silence is itself informative in a consensus scheme, because a device that never contributes its "good" status, and about which neighbors can learn nothing, will show up in others' bitmasks as "unknown" rather than "good," which already signals that something may be amiss.

PADS Performance

PADS is evaluated by its **run-time in seconds for different coverage levels**—specifically, the time for 85%, 90%, or 95% of the total devices in the network to acquire knowledge (coverage) of 95% of the network. Plotted against the number of devices, the run-time curves for all three coverage targets rise together: nearly flat and low for small networks, then climbing increasingly steeply as the network grows into the thousands, reaching on the order of 40–48 seconds for a swarm of roughly 8,000 devices. The three curves sit close to one another, with the most demanding target (95% of devices reaching 95% coverage) highest, as expected. The upward bend at scale reflects the cost of diffusing information across an ever-larger mobile population through purely local exchanges.

PADS Pros and Cons

PADS's improvements over its tree-based predecessors are exactly the capabilities the dynamic setting demanded. It is **suitable for dynamic networks**; it **considers device movement during attestation** rather than assuming a frozen topology; and it allows the **verifier to obtain a snapshot of the network at run-time** through the gossip-maintained shared state.

Its drawbacks, however, stem from the rigidity of its membership model. In PADS, **nodes cannot easily join the network**, because the fixed-length 2N-bit bitmask and the per-device bit assignments are baked in from the start. Consequently, **adding a new node requires reconfiguration of all the devices**: every device's bitmask must be resized and its indexing updated to accommodate the newcomer, a global reconfiguration that is awkward precisely in the open, fluid environments where new devices might naturally wish to appear. This tension—between embracing *mobility* while still struggling with *membership change*—is the gap that the final protocols of the course, built on Bloom filters, set out to close.

SALAD

Before arriving at the Bloom-filter construction that resolves PADS's membership rigidity, one intermediate protocol deserves mention. **SALAD** (Secure And Light-weight Attestation of highly Dynamic and disruptive networks), proposed by Kohnhäuser, Büscher, and Katzenbeisser, targets the same regime as PADS—highly dynamic and disruptive networks—but takes a different approach to disseminating attestation information.

SALAD's distinguishing features, and their trade-offs, can be summarized directly. On the positive side, it uses a **broadcast of challenges and responses**, spreading attestation information through the network by broadcasting rather than through structured point-to-point exchanges; it **provides a report for each device**, yielding per-device verdicts rather than only an aggregate count; and it rests on **strong cryptographic assumptions**, giving it robust security guarantees. The cost of these properties is a **high overhead**: the broadcasting of challenges and responses, combined with the strong cryptography, makes SALAD comparatively expensive to run.

Internally, SALAD employs **three different types of aggregation**, named **MACGreedy**, **MACSimple**, and **MACSmart**, which differ in how they combine the MAC-protected reports as they propagate. Their relative performance is visible when one plots, for a network of 3,000 devices under high dynamics (with device speeds of 10–20 meters per second), the fraction of the final report that an average device has stored as time passes. For a given MAC-tag length, **MACSmart** causes devices to acquire the complete report fastest, **MACGreedy** is slowest, and **MACSimple** lies between them; the average time until the first and last network devices store the final report is marked by vertical lines on such plots. Comparing across tag lengths, shorter tags (for instance 128-bit versus 256-bit) let the report propagate faster because less data must be moved, and increasing the number of MAC computations per message likewise shifts the completion times. SALAD thus offers a family of aggregation strategies trading speed against the robustness of its guarantees, but always at a notably higher overhead than the lighter consensus approach of PADS.

Bloom-Filter-Based Collective Remote Attestation

The final and most refined protocol in this progression is the **Bloom-filter-based collective remote attestation** scheme of Frontera and Lazzeretti. It inherits PADS's gossip-and-consensus philosophy for dynamic swarms but replaces the rigid, fixed-length 2N-bit state representation with a **Bloom filter**, a probabilistic data structure that dissolves exactly the membership-rigidity problem that burdened PADS. By doing so it retains PADS's advantages while permitting devices to join and leave fluidly.

The Bloom Filter

A **Bloom filter** is a **space-efficient probabilistic data structure** used to test whether an element belongs to a set. Its purpose here is **element verification in the set**—answering the question "is this item a member?"—while occupying far less space than storing the set explicitly. Its operation **requires**

multiple hash functions. To insert an element, each of the k hash functions is applied to it, and each produces an index into a bit array; the bits at all those indices are set to 1. To test membership, the same hash functions are applied to the query element, and if *all* the corresponding bits are set, the element is reported as present.

The structure's defining characteristic—and the source of the word "probabilistic"—is that it can produce **false positives but never false negatives**. This is illustrated by considering several elements against a filter. Elements **x, y, z** have genuinely been inserted, so each maps, through the several hash functions, to bit positions that have all been set to 1; a membership test for any of them succeeds correctly. An element **w** that is **not in the set** maps to at least one position that remains 0, so the filter correctly reports it absent. But an element **u** may be a **false-positive element**: although u was never inserted, every one of its hashed positions happens to have been set to 1 by the insertions of *other* elements, so the filter mistakenly reports u as present. False negatives, by contrast, are impossible—an element that was truly inserted can never test as absent, because its bits, once set, are never cleared.

This asymmetry is precisely what makes the Bloom filter suitable for attestation consensus. The filter can be used, for example, to determine whether a particular secret key has been used, or—in the swarm setting—to record which devices have been observed in a particular state. Because the filter only ever accumulates set bits and never clears them, it is naturally monotonic, matching the bitwise-OR consensus semantics that PADS relied upon.

The filter also supports an estimate of how much it contains. Given a filter of **m** bits using **k** hash functions, of which **X** bits are currently set, the approximate number of elements inserted is

$$\mathbf{z = -(m/k) \cdot \ln(1 - X/m)}.$$

This formula lets a device estimate how many items have been folded into a filter purely from the count of set bits, without enumerating the elements—a capability that, in the attestation setting, allows the verifier to estimate how many devices have contributed to the consensus.

Self-Attestation with a Trusted Challenge

As in the non-interactive and dynamic schemes before it, the Bloom-filter protocol avoids the DoS vulnerability of verifier-initiated attestation by having **the challenge generated by a trusted component inside the device itself**. The hardware requirements mirror those of PADS: a **TrustLite-style platform augmented with a real-time clock**, with the attestation **parameters stored in writable memory** but **read-write protected by the MPU**, and a **secure boot** that guarantees the integrity of the attestation code (here the BADS routines). The memory layout parallels PADS's: the CPU is clocked by a system that drives both the RTC and an attestation trigger (AT); the MPU enforces a rule table of allowed-code-region, memory-region, and access-policy triples; the protected RAM region holds the attestation variable v_att, the known-good set H = {h₁, …, h_M}, and the attestation key k_att, alongside the attestation routines and the OS; and the ROM holds the platform key and the secure-boot routine. The device therefore self-triggers its attestation on a tamper-proof schedule, closing the DoS channel while keeping the attestation state confidential.

Maximum Consensus over Bloom Filters

The consensus rule is identical in form to PADS's maximum consensus, now understood as operating over the Bloom-filter bit array. Each prover updates its filter by taking the componentwise maximum of its own filter and those of the in-range, responsive neighbors—**x_i^{t+1}[l] = max(x_i^t[l], {x_j^t[l]})** for each of the m bit positions—which, over binary values, is once again **equivalent to a**

bitwise OR. The parameters carry the same meaning as before: x*i^t is prover i's observation at step t, m is the Bloom filter length, N*{i,t} is prover i's neighbor set at step t, and R_t is the set of active and responsive provers. When two devices meet, their filters are OR-ed together, so that any device recorded in either filter becomes recorded in both—evidence accumulates and never vanishes, exactly as the monotonic Bloom filter requires.

The Protocol in Motion

The dynamics mirror PADS's four phases, now with each device carrying a Bloom filter rather than a fixed 2N-bit mask.

Initialization. Every device starts with its Bloom filter cleared to all zeros, having recorded nothing about the network.

Self-attestation. Triggered together by their synchronized real-time clocks, the devices each measure their own software and declare a verdict—"healthy" or "compromised." An honest device inserts the appropriate token into its own filter; a device detecting corruption records the compromised verdict. In the illustrative snapshot, most devices announce "healthy" and set the corresponding bits, while a few that detect tampering announce "compromised" and set bits encoding that fact—so that the individual filters begin to differ, each reflecting its owner's self-knowledge.

Consensus. The devices gossip. As mobility brings pairs of devices into range, they exchange their filters and apply the bitwise-OR update, and the recorded verdicts—healthy and compromised alike—diffuse across the swarm. A "compromised" mark, once inserted anywhere, propagates and can never be cleared, so the shared picture monotonically accumulates the evidence of which devices are in which state.

Verification. Eventually the verifier reads the accumulated filter from an attested device to recover the **network state**. From that filter it can, using the element-count formula, estimate how many devices have been recorded, and it can test individual devices for membership in the "compromised" set—subject to the Bloom filter's intrinsic false-positive possibility.

Security

The Bloom-filter scheme is **secure against communication adversaries, software adversaries, and mobile software adversaries**, the same three software-level threat classes that PADS withstands. The mechanisms are the same: self-attestation anchored in protected hardware, MAC-protected exchanges, confidential attestation state behind the MPU, and a monotonic consensus rule that evidence of compromise can only enter, never leave. As in PADS, the residual action available to an adversary is essentially to **decline to participate**, and silence manifests as an absence of "healthy" evidence rather than as a successful forgery of it.

Simulations and Results

The scheme was evaluated on the **OMNeT++ v5.6.2 simulation platform with the INET framework**, under a configuration chosen to stress its dynamic behavior. The minimum coverage times were measured at the 85%, 90%, and 95% levels; the simulation area was scaled in proportion to the number of devices—starting from a 125 × 125 m^2 field for 128 devices and growing to 500 × 500 m^2 for 2,048 devices—so that device density remained roughly constant as the swarm grew; the broadcast frequency was 500 ms; devices moved with random mobility at about 10–15 meters per second; and each reported figure is the average of 50 runs.

The Bloom-filter sizing, drawn from the simulation parameters, shows the structure's economy and its tunability against the false-positive rate. Assuming 10% of the devices are compromised and tolerating a **1% false-positive probability**, the number of compromised devices n_c scales with the network (13, 26, 52, 103, 205 for networks of 128, 256, 512, 1024, 2048 devices), and the number of bits required m grows roughly in proportion (125, 250, 499, 988, 1965), with 7 hash functions throughout and the filter needing between 1 and 3 fragments to transmit as the network grows. Relaxing the tolerance to a **5% false-positive probability** shrinks the filter substantially for the same networks: m drops to 70, 139, 309, 611, 1246 bits, with only 4 hash functions and 1 to 2 fragments required. The trade-off is explicit—accepting a higher false-positive rate buys a smaller, cheaper-to-transmit filter.

The run-time results confirm both the scheme's scalability and its advantage over PADS. Plotting run-time against the number of devices for the three coverage targets (85%, 90%, 95%), all curves rise together—low and flat for small networks, climbing more steeply toward roughly 5–6 seconds at 2,000 devices under the 1% false-positive setting, and toward a lower 2.5–3 seconds under the more permissive 5% setting—with the three coverage targets lying close together. Most tellingly, when the Bloom-filter scheme is compared directly against PADS at the 95% coverage level, PADS's run-time rises steeply with network size while the **Bloom-filter scheme's run-time rises far more gently**, so that at around a thousand devices PADS is markedly slower. The Bloom-filter representation thus not only restores flexible membership but also propagates faster than PADS's fixed bitmask, under both the 1% and 5% false-positive regimes.

Bloom-Filter CRA: Advantages and Limitations

The Bloom-filter-based scheme can finally be assessed against the whole lineage that produced it. On the positive side, it retains **all of PADS's advantages**—suitability for dynamic networks, tolerance of device movement during attestation, and a run-time network snapshot for the verifier—while being **faster than PADS**. Decisively, it is **flexible in the joining and leaving of devices**: because a Bloom filter is not bound to a fixed per-device index the way PADS's 2N-bit mask was, a new device can be folded into the filter without resizing a global structure or reconfiguring every node, dissolving the membership rigidity that was PADS's chief flaw. And, like its consensus-based predecessor, it **provides a global view of the network state**.

The limitations are the honest costs of these gains. The scheme relies on **shared symmetric keys**, a simplification adopted because, as in the dynamic setting generally, storing the individual public keys of all devices proved too expensive—so a single secret key shared by all the nodes is used instead, which concentrates trust in that shared key. The approach is **probabilistic**, inheriting the Bloom filter's false positives, which means **the compromised devices cannot be identified exactly**—a device may be flagged as compromised when it is merely a false positive of the filter. And because the consensus is reached through opaque, accumulating gossip, **it is not known how many nodes actually participated in the consensus**, leaving a residual uncertainty about how thoroughly any particular verdict has been corroborated. These are the characteristic trade-offs of trading exact, heavyweight accounting for lightweight, flexible, probabilistic scalability—the bargain that makes attestation practical for the fluid, open, resource-starved swarms that increasingly populate the Internet of Things.

CHAPTER 3: IoT Authentication

1. Why Authentication in the IoT Is a Special Problem

Authentication is the act of convincing a verifier that a communicating party is who it claims to be. In the classical, well-resourced world of servers and laptops this is a solved engineering problem: a device

proves that it knows a secret — a private key, a password, a pre-shared symmetric key — and the mathematics of cryptography guarantees that proving knowledge of the secret is infeasible without possessing it. The Internet of Things breaks the comfortable assumptions behind this picture in two ways at once, and understanding both is the motivation for everything that follows.

The first problem is **physical exposure**. Classic authentication proves knowledge of a secret *stored in the device*, but an IoT node is typically a small, cheap object left unattended in the field — a sensor on a lamppost, a meter in a basement, a tag on a shipping container. An attacker who physically captures such a device can open it, probe its memory, and extract the stored key. Once the key is out, the attacker can impersonate the device perfectly, because to the verifier the key *is* the identity. The secret, in other words, is only as safe as the tamper-resistance of the cheapest component the manufacturer was willing to pay for.

The second problem is **energy**. Cryptographic operations — modular exponentiation, elliptic-curve point multiplication, even repeated hashing — cost computation, and computation on a battery-powered node costs energy that directly shortens the device's operational lifetime. A protocol that is trivially affordable on a plugged-in server may be prohibitively expensive on a node expected to run for years on a coin cell.

These two pressures — keys can be stolen from captured hardware, and cryptography drains limited energy — are what open the door to a fundamentally different idea: building authentication (and even key generation) not on a secret stored *in* the device, but on physical properties of the radio link and the hardware *itself*, which an attacker cannot easily read out or reproduce.

2. Physical Layer Security: The Core Idea

Physical Layer Security (PLS) abandons the premise that security must live in the bits of a stored key. Instead it exploits properties of the wireless channel and the analog hardware — properties that are measurable by the legitimate parties but difficult for an attacker to observe or imitate. Three physical facts make this possible.

The channel is random but reciprocal. The wireless channel between two points A and B is the combined result of reflection, diffraction, and scattering off every object in the environment. Its exact behaviour — how it attenuates and phase-shifts a signal — is effectively random and depends on the full geometry of the surroundings. Crucially, over a short time window this channel is **reciprocal**: the channel seen when A transmits to B is (to a very good approximation) the same as the channel seen when B transmits to A. The link A→B *looks like* the link B→A. This shared, private randomness is the raw material PLS mines: two legitimate parties can measure the *same* thing, while nobody else can.

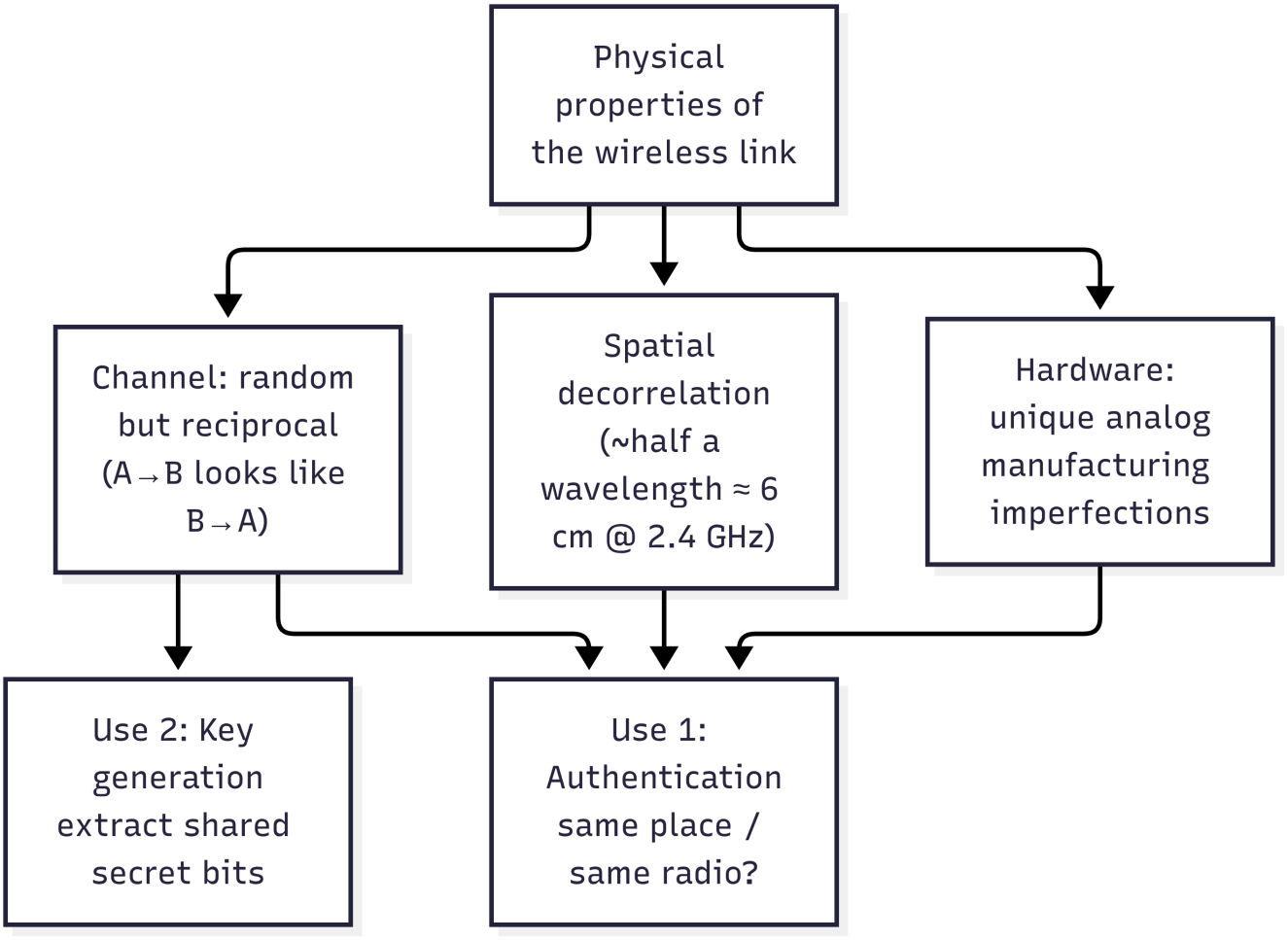
The channel decorrelates quickly in space. The randomness is not only shared between A and B — it is also *local*. Moving the receiving antenna by only about **half a wavelength** produces an essentially different channel. At the common 2.4 GHz band, half a wavelength is roughly **6 cm**. This means that an eavesdropper standing even a short distance away from a legitimate node does not measure the same channel; the physical secret is confined to the exact location of the legitimate parties.

Every transmitter is physically unique. Beyond the channel, the radio hardware itself carries a fingerprint. Manufacturing is never perfect: every transmitter leaves the factory with tiny, uncontrollable **analog imperfections** — slight deviations in oscillator frequency, mixer balance, amplifier behaviour. These imperfections are consistent for a given device and distinctive between devices, so the way a radio shapes its own signal acts like a hardware signature.

From these facts flow the **two uses of PLS**:

- **Authentication** — asking "is this frame coming from the same place, or the same radio, as the frames I trusted before?" The channel measurement or hardware fingerprint of a new frame is compared against a known reference; a close match confirms continuity of identity.
- **Key generation** — because A and B measure the *same* reciprocal channel, each can turn its measurement into bits, and those bits will largely agree. They thereby extract a **shared secret** from the environment without ever transmitting it.

All of this rests on a specific **threat model**: the attacker is assumed to be located **farther than half a wavelength** from the legitimate nodes (so it cannot observe their private channel) and is assumed to be **unable to copy the hardware** (so it cannot forge the analog fingerprint). PLS is secure exactly to the extent that these two assumptions hold — and, as we will see, its limitations are precisely the situations in which they fail.



3. PLS Authentication: Channel and Hardware Fingerprints

PLS authentication comes in two complementary flavours, distinguished by *what* physical quantity is used as the identity.

3.1 Channel-based authentication

Channel-based authentication answers the question of *location continuity*: it compares the channel observed in the **current frame** with the channel observed in the **previous frame** from the same claimed sender. The intuition is that a legitimate transmitter that has not moved will produce a near-identical channel from one frame to the next, whereas a spoofer transmitting from a different position will produce a measurably different channel.

The physical quantity measured is a **channel feature**, and several are used in practice: the **Channel State Information (CSI)**, which describes amplitude and phase across subcarriers; the **Received Signal Strength Indicator (RSSI)**, a coarse single-number summary of received power; and the **channel impulse response**, the time-domain description of how the channel spreads a transmitted pulse. Whatever the feature, the decision is framed as a **hypothesis test**: a distance metric is computed between the current and reference measurements, and if that distance lies **below a threshold** the frame is accepted as coming from the same sender, while a distance **above the threshold** is flagged as potential spoofing.

The method has an inherent structural requirement that must be stated plainly: it works only *between consecutive frames*. It can verify that frame n came from the same place as frame $n-1$, but it has no way, on its own, to establish the legitimacy of the very **first frame**. That initial anchor must be authenticated **by other means** (for example, a one-time cryptographic handshake), after which PLS cheaply maintains continuity for the rest of the session.

3.2 Hardware-based authentication (RF fingerprinting)

Hardware-based authentication, also called **RF fingerprinting**, identifies **the radio, not the position**. Instead of measuring the environment between sender and receiver, it measures the idiosyncrasies that the transmitter's own imperfect electronics impress on every signal it emits. The distinguishing **features** include the **carrier frequency offset (CFO)** — the small, device-specific error between the nominal and actual carrier frequency produced by an imperfect oscillator; **IQ imbalance** — asymmetries between the in-phase and quadrature components introduced by the analog front end; the **transient shape** — the characteristic waveform of the brief moment when the transmitter powers on; and **power amplifier (PA) non-linearity** — the distinctive distortion signature of the amplifier driving the antenna.

Historically these features were hand-engineered, but the dominant modern approach is **deep learning applied directly to raw IQ samples**: a neural network is trained on the complex-valued baseband samples themselves and learns to associate the subtle, entangled distortions with a particular physical device. Because the fingerprint tracks the hardware rather than the location, RF fingerprinting can identify a radio even if it moves — a property channel-based methods lack.

3.3 The central difficulty: the threshold

Both approaches ultimately reduce to a decision governed by a **threshold**, and that threshold is where the real engineering difficulty lies. Setting it too strictly produces **false alarms** — legitimate frames rejected because natural fluctuation pushed their distance over the line. Setting it too loosely produces **missed detections** — spoofed frames accepted because the attacker happened to fall inside the tolerance. The two error types trade off against each other directly, and the optimal operating point depends on an environment that is itself constantly changing. For this reason, the threshold is increasingly treated as something to be *learned and adapted* rather than fixed: **machine learning and reinforcement learning** are used to tune it continuously in response to observed conditions.

4. Keys from the Channel, and the Limits of PLS

4.1 Secret key generation from reciprocity

The second use of PLS turns the shared, private channel randomness into an actual cryptographic key. Because A and B observe a reciprocal channel, they can each measure it, convert their measurements to bits, and end up with *nearly* the same bit string — nearly, but not exactly, because of noise and

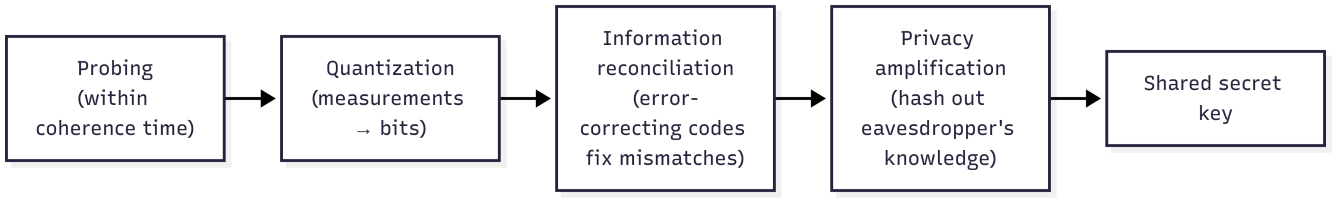
imperfect reciprocity. A standard four-stage pipeline converts these noisy, almost-matching measurements into an identical, secret key.

Probing. A and B exchange probe signals and each measures the channel. This must happen within the **coherence time** — the short interval over which the channel stays essentially constant. Measurements taken too far apart in time would describe different channels and would fail to agree.

Quantization. Each side independently maps its real-valued channel measurements into **bits**, for example by comparing samples against thresholds or quantization levels. After this step A and B each hold a bit string, and the two strings are similar but not yet identical.

Information reconciliation. The small number of **mismatching bits** — caused by noise and residual non-reciprocity — are corrected using **error-correcting codes**, so that both parties converge on exactly the same string. This step necessarily involves some public exchange, which is why the next step is needed.

Privacy amplification. Whatever partial information an eavesdropper might have gleaned (from the public reconciliation messages, or from a weakly correlated observation of its own) is scrubbed out by passing the reconciled string through a **hash function**, compressing it into a shorter key about which the eavesdropper knows essentially nothing.



4.2 Limits and attacks

PLS is powerful but not a panacea, and a serious analyst must hold its weaknesses in mind as firmly as its strengths.

A **static environment yields few random bits**. The entropy PLS harvests comes from *change* in the channel; in a room where nothing moves, the channel barely varies, the extractable randomness is small, and the resulting **key rate is low**. A nearby attacker is also dangerous: although the channel decorrelates over half a wavelength, an adversary positioned *close* to a legitimate node sees a **correlated channel**, eroding the assumption that only the legitimate parties share the secret. The channel can furthermore be **predictable or actively manipulated** — an attacker who **moves objects** in the environment or **injects signals** can shape the channel A and B measure, undermining both its randomness and its secrecy. Finally, the hardware-based line of defence is not immune either: **RF fingerprints can be imitated** by an adversary wielding a high-end **software-defined radio (SDR)**, or simply **replayed** from a previously captured signal.

The honest conclusion, and the one to carry forward, is that **PLS complements cryptography; it does not replace it**. It offers cheap, energy-light, continuous assurance that is especially valuable for resource-constrained nodes, but it is layered alongside conventional cryptographic mechanisms rather than substituted for them.

5. The Four Main PLS-Based Authentication Approaches

As PLS matured, several concrete authentication protocols emerged, and they cluster into **four main approaches**. Each makes a different trade-off between cost and protection, and the comparison below is

the compact map of that design space.

Dimension	1. Compare channel of two consecutive frames	2. Third-party authority (relay) using XOR and simple multiplications	3. Secret keys derived from the channel used for encryption	4. "Tags" generated using encryption or hashing
Advantage	No additional cost and overhead	Non-repudiation	Utilizes the notion of a secret key	Utilizes the notion of a secret key
Limitation	Ineffective when the attacker is near the legitimate user — i.e. when both experience the same channel conditions	The third-party authority is vulnerable to impersonation; a high level of trust must exist between the legitimate users and the third party	A key derived from the channel is a weak proposition, since it can be easily generated and acquired	Computationally complex; requires additional operations at both transmitter and receiver
Resource / communication cost	No additional cost and overhead	Verified through multiple rounds of communication (3 rounds) and simple operations such as XORing	Two steps before data transmission: (1) key extraction, (2) encryption/decryption	(1) key extraction, (2) tag generation, (3) appending tags to the transmitted messages
Complexity	Not computationally complex: performing a comparison	Not computationally complex: performing XOR operations	Computationally complex: encryption	Computationally complex: encryption

The structure of this table rewards careful reading. The two **channel-centric, computation-free** schemes (approaches 1 and 2) buy their low cost with real weaknesses: the pure comparison scheme collapses exactly in the nearby-attacker scenario already identified as PLS's Achilles' heel, while the third-party relay scheme introduces a trusted intermediary that itself becomes a target and a trust liability — though it uniquely gains **non-repudiation** in exchange. The two **key-centric, cryptographic** schemes (approaches 3 and 4) are stronger in spirit because they reintroduce an actual secret key, but they pay for it in computational complexity, and approach 3 carries the pointed caveat that a key scraped from the channel may be too weak to be worth much. Approach 4, the "tag" scheme, is the heaviest of all, appending cryptographic tags to every message at the price of extra work on both ends. The recurring lesson is the one from Section 4: there is no free authentication — every approach trades cost against the breadth of attacks it actually withstands.

IoT Authentication — Part 2: Physical Unclonable Functions and Fuzzy Extractors

6. Physically Unclonable Functions (PUFs)

Section 2 observed that every piece of silicon leaves the factory with tiny, uncontrollable manufacturing variations. The **Physically Unclonable Function (PUF)** is the deliberate engineering exploitation of that fact: a circuit designed so that its behaviour *depends on* these nano-scale physical variations, turning an unavoidable manufacturing nuisance into a hardware-rooted secret. The guiding ideas can be read directly off the four-panel logic that defines the concept.

First, **uncontrollable nano-scale process variations make integrated circuits unique.** No two chips, even from the same wafer and the same design mask, are physically identical at the scale of individual transistors; the exact doping, the precise line widths, the minute threshold-voltage differences are random and beyond the manufacturer's control. Second, these variations can be *read out* as a stable signature. A classic example is **start-up SRAM**: when an SRAM cell is powered on, each bit settles into a 0 or a 1 according to the microscopic mismatch between its two cross-coupled transistors, and the resulting pattern of start-up values is distinctive and repeatable for that specific chip. This power-on pattern establishes a **unique silicon fingerprint.** Third, that fingerprint is not used raw; it is processed — as Section 7 will detail — into a **strong secret cryptographic key.** Fourth, once a device has a key it never had to store, that key can be put to work: **user keys can be wrapped (encrypted) with the PUF-derived key**, an arrangement often described as a **red/black system**, in which sensitive "red" key material is protected by the device's intrinsic "black" PUF key.

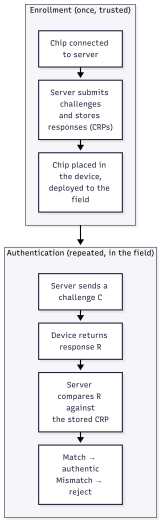
The decisive security property is in the name: a PUF is **unclonable**. Because its responses arise from physical randomness that the manufacturer could not control and cannot reproduce, not even the original maker can build a second chip with the same responses. The secret is not *written into* the device and therefore cannot be *read out of* it in the way a stored key can — it exists only as a physical behaviour, evoked on demand.

6.1 The challenge–response paradigm

A PUF is operated as a function that maps an input **challenge** to an output **response**: apply a specific stimulus, and the chip's physical randomness produces a specific, device-unique answer. A pairing of a particular challenge with the response it elicits is a **Challenge–Response Pair (CRP)**. Authentication is then organised in two phases.

Enrollment happens once, in a trusted setting, before the device is deployed. The chip is connected to the server; the server **submits challenges to the chip and records the responses**, building a database of CRPs it can later use to recognise this specific device; the enrolled chip is then **placed in the device** and sent into the field.

Authentication happens repeatedly thereafter. To verify the device, the server **sends a challenge and checks the response** against what it recorded at enrollment. A correct response proves the device possesses the exact physical fingerprint it had at enrollment — and because that fingerprint is unclonable, a correct response is convincing evidence of identity. No secret key ever had to be stored in non-volatile memory, so there is nothing for a hardware attacker to extract by probing memory.



7. From Noisy Responses to Keys: Fuzzy Extractors

There is a problem lurking in the paragraph above, and it is the same problem that afflicts channel-derived bits. **PUF responses are noisy.** The same challenge applied to the same chip does **not** produce exactly the same response every time: temperature drift, supply-voltage fluctuation, ageing, and plain measurement noise all cause a handful of bits to flip from one reading to the next. Channel bits behave identically — the same physical measurement gives a slightly different answer each time.

This is tolerable for the *comparison*-style authentication of Section 6.1, where the server can simply accept a response that is "close enough" to the stored one. But it is fatal for *key generation*, because a cryptographic key must be **exactly the same every single time** — a key that differs by even one bit decrypts nothing and verifies nothing. The device, which cannot store the key, must instead *regenerate* the identical key from a noisy physical reading on every boot. The mechanism that squares this circle is the **fuzzy extractor**.

7.1 The Gen / Rep interface

A fuzzy extractor exposes two procedures.

Generation, written `Gen(w) → (R, P)`, runs once at enrollment. It takes the enrollment response `w` — the "clean" reference reading of the PUF — and produces two outputs: a strong, uniformly random **key** `R`, and a piece of **public helper data** `P` (also called helper or assistance data). The key `R` is kept and used; the helper data `P` is stored openly, even transmitted in the clear, for later use.

Reproduction, written `Rep(w′, P) → R`, runs on every subsequent use. It takes a fresh, **noisy** response `w′` — which is *close* to the original `w` but not identical — together with the stored helper data `P`, and reconstructs the **exact same key** `R`. In short: `Gen` manufactures the key once and emits the public breadcrumb `P`; `Rep` follows that breadcrumb to rebuild the identical key from any sufficiently-close later reading.

7.2 The code-offset construction

The canonical way to build a fuzzy extractor is the **code-offset construction**, which borrows the error-correcting power of a linear code. Let `C(k)` denote the codeword encoding some random value `k`. At enrollment, the helper data is formed as

$$P = w \oplus C(k),$$

that is, the reference response XORed with a codeword. Notice that `P` is essentially the response masked by a random codeword, which is why publishing it leaks little. At reproduction time, the device reads a noisy `w′` and computes

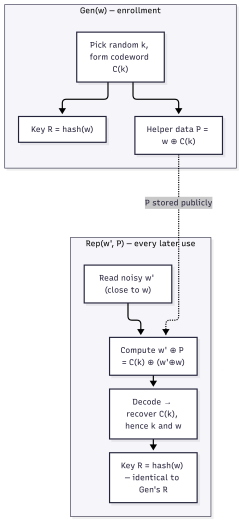
$$w' \oplus P = w' \oplus w \oplus C(k) = C(k) \oplus (w' \oplus w).$$

The term `(w′ ⊕ w)` is exactly the small pattern of flipped bits — the **noise** — between the two readings. So `w′ ⊕ P` is the codeword `C(k)` corrupted by a *small* number of bit errors, and that is precisely what an error-correcting decoder is built to repair: **decoding** `w′ ⊕ P` recovers the clean codeword and hence `k`, from which the original `w` is recovered, and finally the key is obtained as

$$R = \mathrm{hash}(w).$$

The noise is cancelled by the code; the hash in the last step performs the privacy-amplification role of compressing the recovered response into a clean, uniform key.

The reason the scheme is safe to deploy is that **`P` can be public: it reveals little about `R`.** Because `P` is the response XORed with a codeword chosen at random, an attacker who sees `P` learns almost nothing about the key `R`, which depends on the hash of the secret response `w`. The helper data can therefore be stored unprotected or sent over the open channel without compromising the key — the property that makes the whole construction practical for devices that have nowhere safe to keep anything.



7.3 Strong versus weak PUFs, and their attacks

PUFs are not all alike, and the distinction between **weak** and **strong** PUFs governs how each can be used and how each can be broken. The classification turns on **how many distinct challenge–response pairs** the device supports.

A **weak PUF** supports only a **small number of challenges** — in the limit, essentially one. The canonical example is the **SRAM PUF**, whose "challenge" is simply powering it on and whose "response" is the start-up pattern. With so few CRPs, a weak PUF cannot sustain a protocol that needs a fresh challenge each time; instead it is used as a **key store** — a way to derive one stable secret key (via a fuzzy extractor) that the device never has to save in memory.

A **strong PUF** supports an **enormous number of challenge–response pairs** — so many that an attacker cannot hope to record them all. The canonical example is the **arbiter PUF**, whose challenge selects among a vast space of signal-racing paths. This abundance of CRPs is what makes a strong PUF

suitable for **authentication**: the server can issue a *different* challenge on every session, never reusing a CRP, so that an eavesdropper who observed past exchanges cannot answer a future challenge.

That last defence, however, has a well-known crack: **modeling attacks.** Although the challenge space of a strong PUF is astronomically large, the *mapping* from challenge to response is produced by physical structure that is often mathematically regular. For the arbiter PUF in particular, **machine learning can learn an accurate model of the PUF from only a few thousand observed CRPs** — after which the attacker's trained model predicts the response to any new challenge without needing the physical device at all. Thus the very regularity that lets a strong PUF generate so many CRPs cheaply is also what lets an ML model generalise from a modest sample, and defending against modeling attacks (by adding non-linearity, obfuscation, or response hiding) is a central theme of PUF design.

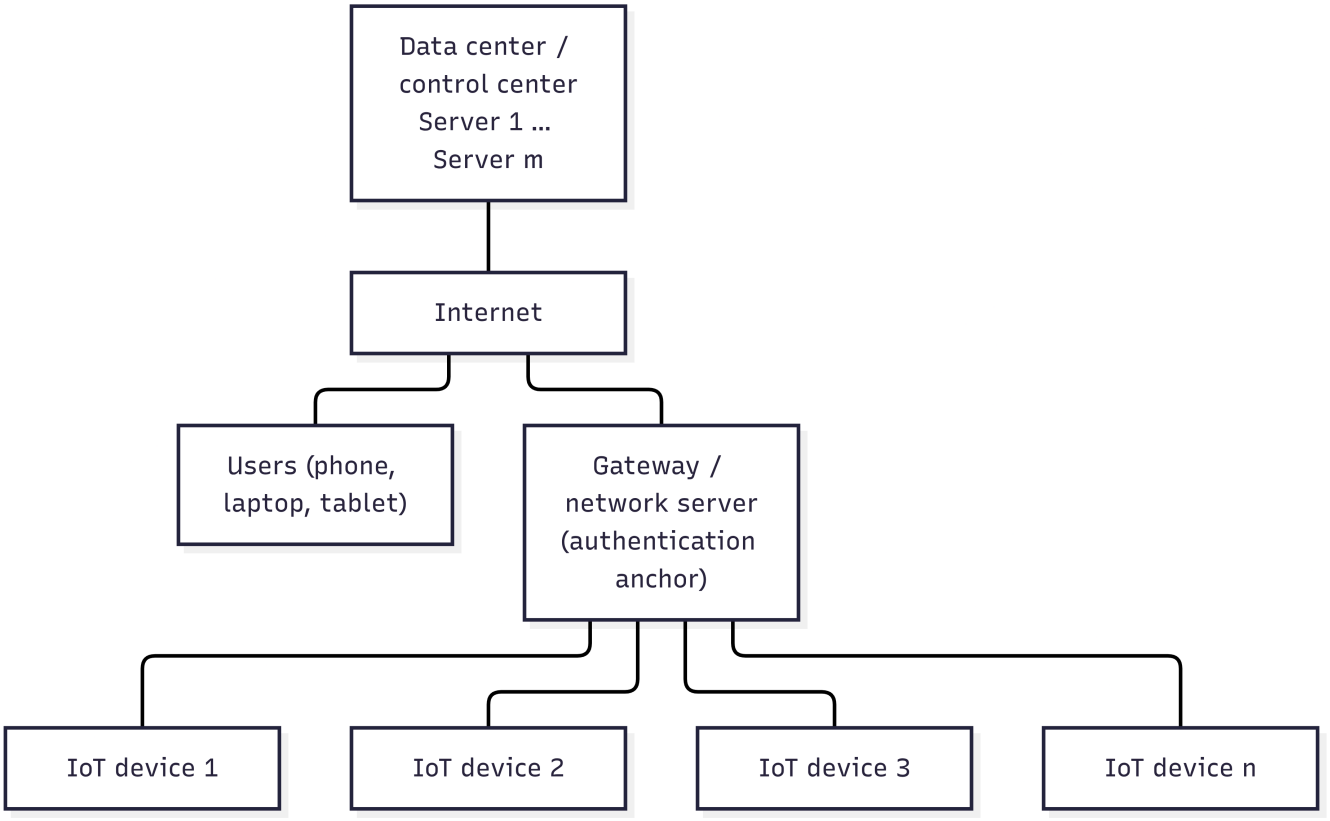
IoT Authentication — Part 3: Lightweight Multi-Factor Mutual Authentication

The previous sections assembled the raw ingredients — physical-layer secrets, PUFs, and fuzzy extractors. This section shows how they are combined into a complete, deployable protocol. The scheme studied here is a **lightweight multi-factor mutual authentication protocol for IoT devices**, designed around a PUF for hardware identity and a fuzzy extractor for stable key recovery, and built so that the heavy work falls on the powerful side of the link while the constrained device does almost nothing.

8. System Model

The deployment picture is a layered smart environment. At the top sits a **data center (control center)** containing one or more back-end servers (`Server 1 … Server m`). These servers reach the field through the public **Internet**, which also connects ordinary **users** on phones, laptops, and tablets. Between the Internet and the field devices sits the pivotal node: the **gateway (network server)**. Beneath the gateway hang the many field endpoints — `IoT device 1` , `IoT device 2` , … up to `IoT device n` — each communicating with the gateway over a wireless link.

The architectural point is that the **gateway is the authentication anchor**. It is the entity that enrolls each chip and later authenticates it; the distant data-center servers and the human users sit behind it. Everything in the protocol is a two-party exchange between **an IoT device (party A)** and **the gateway acting as authentication server (party B)**.



8.1 The building blocks: PUF and fuzzy system

The protocol stands on the two primitives from Parts 1 and 2, specialised to this setting.

The PUF. Each device carries a PUF, and **chip enrollment is performed by the gateway** to produce a secret **session identifier** `ID_S`. In other words, the gateway plays the role of the enrolling server from Section 6: it exercises the device's PUF during a trusted setup and derives from it the per-device secret `ID_S` that will anchor all later authentication. This `ID_S` is the long-term shared secret between that specific device and the gateway, rooted in unclonable hardware rather than in stored key material.

The fuzzy system. Because the PUF (and any physical reading `N`) is noisy, the protocol wraps it in a fuzzy extractor with exactly the `Gen` / `Rep` interface established in Section 7, here written over a nonce/response value `N`:

$$Gen(N) = (\tau, \sigma)$$

$$Rep(N', \tau) = \sigma$$

Reading these against the earlier notation: `Gen` takes a reference value `N` and outputs the **helper data** `τ` (the public assistance string, playing the role of `P`) together with the **secret** `σ` (the extracted stable secret, playing the role of `R`). The reproduction procedure `Rep` then takes a **noisy later reading** `N′` together with the public helper data `τ` and regenerates the identical secret `σ`. This is the same noise-cancelling machinery as before: `τ` is public, `σ` is secret and exactly reproducible, and the device can therefore rebuild `σ` on demand without storing it.

9. Assumptions and Threat Model

A protocol's guarantees are only meaningful relative to the assumptions it makes, so these must be stated precisely.

The field devices **communicate with the gateway directly over public wireless links**, with **LoRaWAN** named as the representative technology — long-range, low-power links of exactly the kind that carry constrained IoT traffic. This immediately implies that every message is exposed on an open medium.

The two parties are deliberately **asymmetric**. The **gateway has high computational power and large memory**; it **stores the PUF inputs for each device and the secret session identifiers** `ID_S`. The device, by contrast, is weak and memory-poor. This asymmetry is the whole design philosophy in miniature: the protocol must be arranged so that expensive storage and computation live on the gateway, while the device performs only cheap operations — hashes and XORs — because that is all it can afford.

On the adversarial side, the model is pessimistic in the standard way. The **endpoints are untrustworthy nodes**, and the **adversary is able to read, forge, manipulate, replay, delay, and delete messages** — the full Dolev–Yao capability over the public channel. The attacker completely controls the wire: it sees everything transmitted, can alter or inject anything, and can suppress or reorder traffic. Security must hold despite all of this.

10. The PLS Authentication Protocol

The heart of the scheme is a **three-message mutual authentication exchange** between the **IoT device (A)** and the **authentication server / gateway (B)**. By the end of it, each side has proven its identity to the other and both have agreed on a shared **session key** `SK`. The notation uses `h[·]` for a cryptographic hash, `⊕` for XOR, `||` for concatenation, `TS` for timestamps, `R` for random nonces, and `ΔT` for the allowed timestamp tolerance. Throughout, recall that `ID_S` is the PUF-derived secret shared from enrollment, and that `σ` is recovered through the fuzzy extractor.

10.1 Message 1 — device → server

The device begins by generating, via the fuzzy extractor's generation procedure, a fresh pair tied to its PUF reading `N_{0,A}`:

$$Gen(N_{0,A}) = (\sigma_i, \tau_i)$$

It then builds two masked values. The first binds the shared secret `ID_S` to a current timestamp `TS_A` and uses the result to hide a random nonce `R_A`; the second uses that same nonce to hide the helper/secret component `τ_i`:

$$M_1 = h[ID_S, ||, TS_A] \oplus R_A$$

$$M_2 = R_A \oplus \tau_i$$

The device transmits **Message 1: ⟨M₁, M₂, TS_A⟩**.

On receipt, the server performs the mirror-image computation, which doubles as its first set of verification checks. It first confirms **freshness**, requiring that the message's timestamp be within tolerance of its own clock:

$$|TS_A - TS_A^*| \leq \Delta T$$

If the timestamp is stale, the message is a replay or a delayed frame and is discarded. Otherwise the server, knowing `ID_S`, recomputes the hash and peels back the XOR masks in sequence:

$$M_3 = h[ID_S, ||, TS_A] \qquad R_A = M_3 \oplus M_1 \qquad \tau_i = M_2 \oplus R_A$$

Having recovered the device's helper data `τ_i`, and holding the device's enrolled PUF input `N_{0,B}`, the server regenerates the device's secret through the reproduction procedure:

$$\sigma_i' = Rep(N_{0,B}, \tau_i)$$

At this point the server has reconstructed everything the device used, which it will carry into Message 2.

10.2 Message 2 — server → device

The server now proves *its* knowledge of the shared secret and proposes the session key. It computes the **session key** as a hash binding the shared identifier, both timestamps, both nonces, and the reproduced secret:

$$SK = h[ID_S, ||, TS_A, ||, TS_B, ||, R_A, ||, R_B, ||, \sigma_i']$$

It generates its own nonce `R_B` and timestamp `TS_B`, then forms two values mirroring the structure of Message 1 — one to convey `R_B` under a hash of the shared secrets and timestamps, the other to bind the freshly computed `SK` to the two nonces:

$$M_4 = h[ID_S, ||, TS_B, ||, TS_A, ||, R_A] \oplus R_B$$

$$M_5 = h[SK \oplus R_A \oplus R_B]$$

The server sends **Message 2: ⟨M₄, M₅, TS_B⟩**.

The device verifies in symmetric fashion. It first checks freshness:

$$|TS_B - TS_B^*| \leq \Delta T$$

Then, recomputing the hash from values it already knows, it recovers `R_B` and independently assembles its own view of the session key:

$$M_6 = h[ID_S, ||, TS_B, ||, TS_A, ||, R_A] \qquad R_B = M_6 \oplus M_4$$

$$SK' = h[ID_S, ||, TS_A, ||, TS_B, ||, R_A, ||, R_B, ||, \sigma_i]$$

Finally it recomputes the server's authenticator and compares:

$$M_7 = h[SK' \oplus R_A \oplus R_B] \qquad M_7 \stackrel{?}{=} M_5$$

If the two agree, the device has confirmed that the server derived the **same session key** — which is only possible if the server truly knew `ID_S` and correctly reproduced `σ`. The device concludes `SK = SK'` and continues; a mismatch aborts the protocol. This is the point at which the **server is authenticated to the device**.

10.3 Message 3 — device → server

One message remains, to authenticate the **device to the server** and to confirm that the device also holds the agreed key. The device computes a final authenticator binding the session key to the shared identifier:

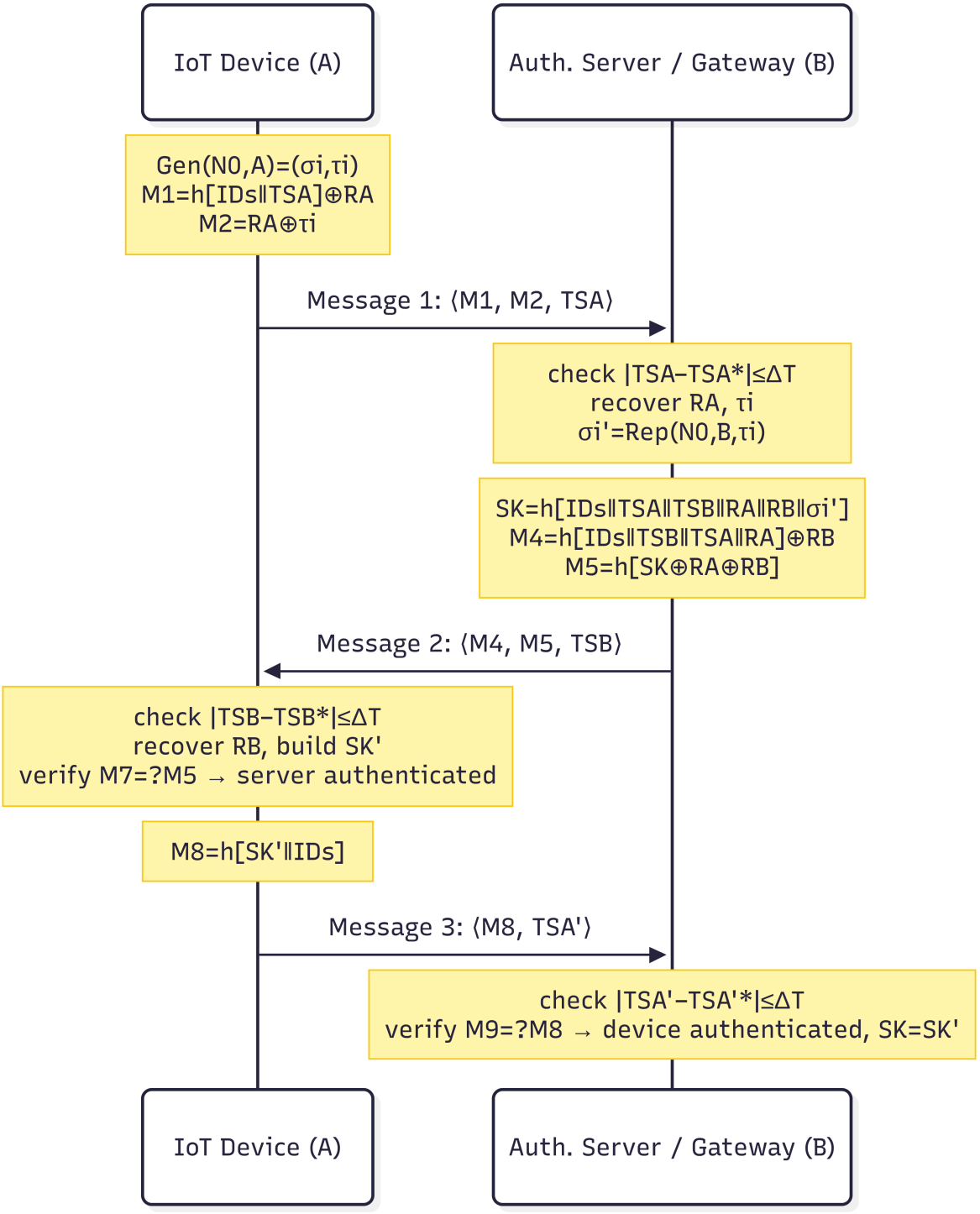
$$M_8 = h[SK', ||, ID_S]$$

and sends **Message 3: ⟨M₈, TS_{A'}⟩** with a fresh timestamp.

The server checks freshness one last time, recomputes the expected authenticator from its own `SK`, and compares:

$$|TS_{A'} - TS^*_{A'}| \leq \Delta T \qquad M_9 = h[SK, ||, ID_S] \qquad M_9 \stackrel{?}{=} M_8$$

If they match, **authentication is established and `SK = SK'` is confirmed on both sides**. Mutual authentication is now complete, and both parties possess an identical, freshly agreed session key that was never transmitted in the clear.



10.4 Reading the design

Three features of this exchange are worth drawing out explicitly, because they recur in every well-built lightweight protocol. First, **the device only ever hashes and XORs** — it runs no public-key operation, no modular exponentiation; the fuzzy-extractor reproduction `Rep` is the heaviest thing it does, and even the storage of enrolled PUF inputs lives on the gateway. This is the asymmetry of Section 9 realised in the arithmetic itself. Second, **every message carries a fresh timestamp and every party carries a fresh nonce**, which together make each run unique and bind the session key to this specific exchange. Third, **the secret `ID_S` never travels on the wire** — it appears only inside hash arguments, so an eavesdropper observing $M_1 \ldots M_8$ sees only hash outputs and XOR masks, never the underlying secret. These three properties are exactly what the security analysis will cash out.

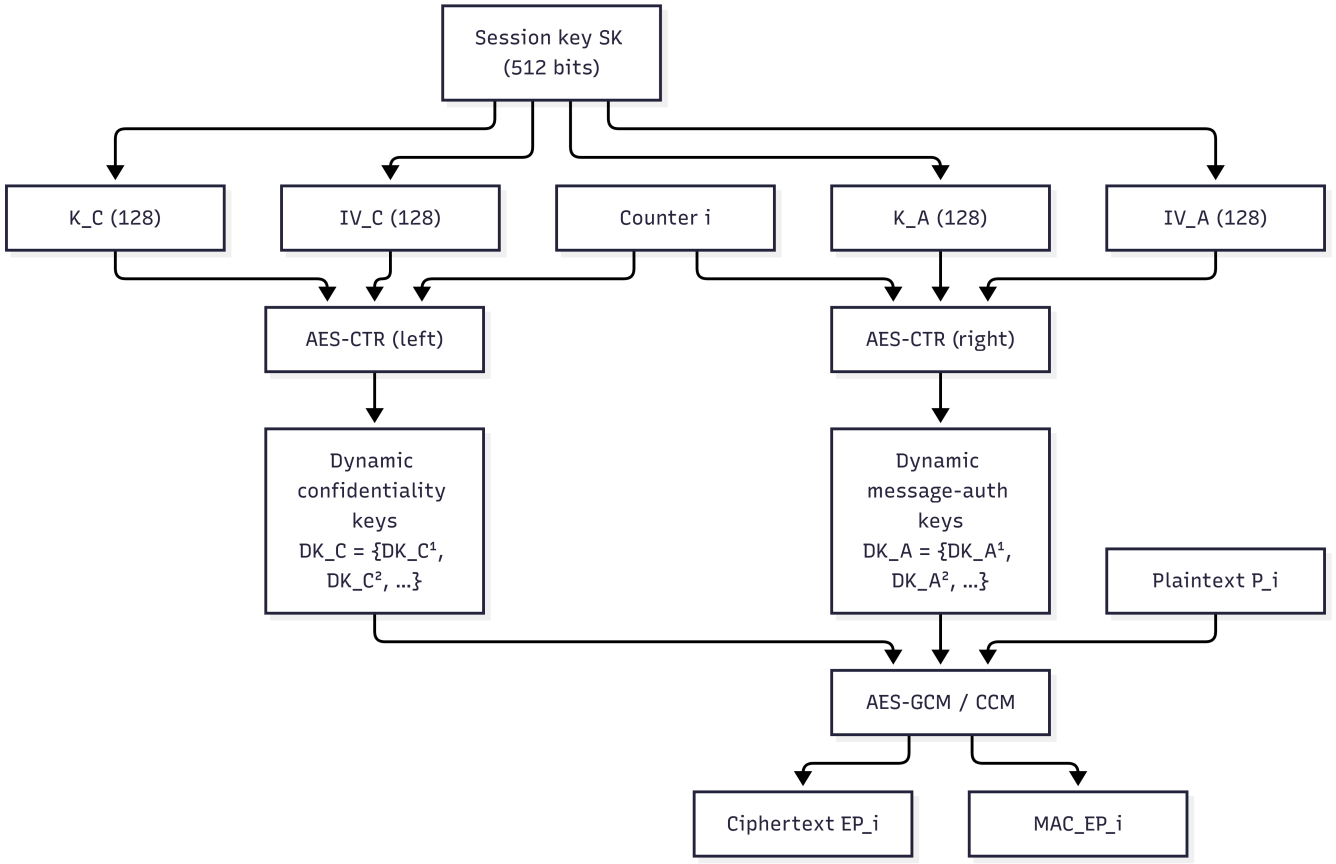
11. Dynamic Key Derivation Scheme

Authentication produces a single shared session key `SK`, but a real session encrypts and authenticates a long stream of messages, and reusing one key for everything is bad practice. The **dynamic key derivation scheme** expands the agreed `SK` into a continually refreshed supply of per-message keys for both confidentiality and integrity.

The construction takes the session key `SK`, which is **512 bits** wide, and splits it into **four 128-bit quantities**: a confidentiality key `K_C` and its initialization vector `IV_C`, and an authentication key `K_A` with its initialization vector `IV_A`. The two halves then drive two independent **AES in counter mode (AES-CTR)** generators. Each generator is clocked by a shared **counter `i`**, so that as the counter advances it emits a *sequence* of fresh keys rather than a single static one:

- The left generator produces the stream of **dynamic confidentiality keys** $DK_C = DK_C^1, DK_C^2, \ldots$.
- The right generator produces the stream of **dynamic message authentication keys** $DK_A = DK_A^1, DK_A^2, \ldots$.

For each message, the corresponding dynamic keys feed an **authenticated-encryption block (AES-GCM or AES-CCM)**, which takes the **plaintext `P_i`** and emits two outputs: the **ciphertext `EP_i`** (encrypted under the confidentiality key) and its **authentication tag `MAC_{EP_i}`** (computed under the authentication key). Because every message index `i` draws a *different* pair of dynamic keys, compromising the material protecting one message gives an attacker no leverage over any other message — the keys are continuously rotated out of the single authenticated `SK`.



12. Security Analysis

The protocol's resilience is argued against a standard checklist of properties and attacks. Each one traces back to a specific structural feature already visible in the message flow.

Indistinguishability. The secret `ID_S` cannot be distinguished or recovered from the transmitted messages. This is guaranteed jointly by the **one-way property of the hash function** — which prevents inverting the hashes that carry `ID_S` — and by the **randomly generated nonces and timestamps** (`R_A` , `R_B` , `TS_A` , `TS_B`), which make every message a fresh, unpredictable function of the secret rather than a fixed, learnable one.

Anonymity. The data remain anonymous during transmission across the public channel: because identifiers appear only inside hashes and under XOR masks, an eavesdropper on the open link cannot tie the traffic to a particular identity.

Man-in-the-middle attack. An adversary who interposes itself and attempts to participate with a **different ID or different random values** will fail the verification checks. The authenticators M_5 / M_7 and M_8 / M_9 only match when both sides genuinely share `ID_S` and the same nonces, so a MITM using wrong secrets produces mismatches and is rejected.

Replay attack. Replays are **easily prevented by the timestamps.** Every message carries a `TS` that the receiver checks against ΔT ; an old captured message fails the freshness test $|TS - TS^{*}| \le \Delta T$ and is discarded before any further processing.

Masquerading. Impersonation is **prevented by keeping the `ID` secret.** Since authentication ultimately rests on proving knowledge of `ID_S` , and `ID_S` never leaves the device or the gateway in the clear, an attacker has nothing to masquerade with.

Forward secrecy. Even if long-term material were later exposed, past session keys remain protected, **achieved by the one-way property of the hash function.** Because each `SK` is a hash of fresh per-

session nonces, recovering a current secret does not let an attacker roll the hash backward to reconstruct the keys of earlier sessions.

The through-line of the entire analysis is that **only two primitives are doing the work — the one-way hash and freshness (nonces plus timestamps)** — and yet between them they deliver the full suite of properties. That economy is exactly the point of a *lightweight* protocol: maximal security guarantees from minimal, device-affordable operations.

IoT Authentication — Part 4: Standardized Onboarding and Session Protocols

Parts 1–3 built authentication from physical primitives and a bespoke academic protocol. Real deployments, however, mostly run on *standards* — published, analysed, interoperable protocols that vendors implement in common. This part surveys the standardized landscape: how a smart-home device is commissioned, how a LoRaWAN node joins its network, how constrained devices run a modern handshake, how Bluetooth devices pair, how a device acquires a trustworthy identity in the first place, and finally how all of this divides into the two great classes of authentication — user-to-device and device-to-device.

13. Matter: Commissioning a Smart-Home Device

Matter is a common standard for smart-home devices, introduced by the Connectivity Standards Alliance (CSA) in 2022, designed to work uniformly across **Wi-Fi, Thread, and Ethernet**. Its significance for authentication is that it addresses the whole chain of trust — from the factory, through the human owner, to autonomous device-to-device operation — in one coherent commissioning flow built from three layered steps.

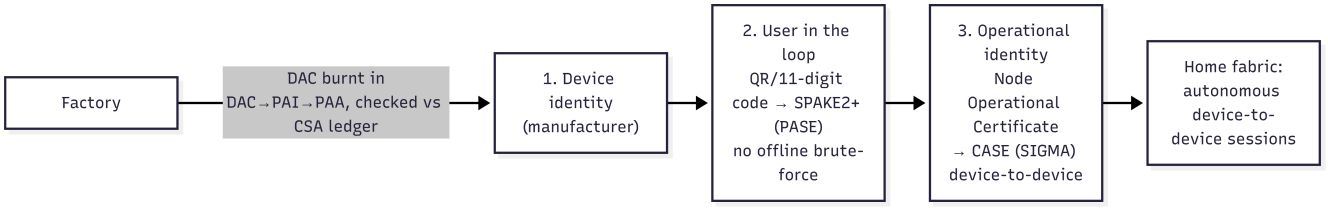
Step 1 — Device identity (the manufacturer level). Every Matter device is born with a **Device Attestation Certificate (DAC)** that is **burnt in at the factory**, cryptographically proving the device is a genuine product of its claimed manufacturer. The DAC does not stand alone; it sits at the bottom of a certificate chain: the **DAC → PAI (Product Attestation Intermediate, the vendor's intermediate) → PAA (Product Attestation Authority, the root)**. During commissioning this chain is validated **against the CSA ledger**, the authoritative registry of legitimate vendors and roots. A counterfeit device cannot produce a DAC that chains to a genuine, ledger-listed root, so attestation screens out fakes at the moment of onboarding.

Step 2 — User in the loop (the user level). Attestation proves the device is genuine, but *the owner must still prove physical possession and consent.* The user supplies a secret printed on the device — a **QR code or an 11-digit setup code encoding a passcode**. This passcode drives a **password-authenticated key exchange (PAKE)**, specifically **SPAKE2+**, which establishes a secure channel keyed on the passcode. The defining virtue of a PAKE is stated directly: **an eavesdropper cannot brute-force the passcode offline.** Even though the passcode is short and low-entropy, the PAKE construction ensures that each guess requires an *online* interaction, so a passive observer who records the exchange cannot grind through candidate passcodes on their own machine. Physical access to the printed code thus becomes a genuine authentication factor.

Step 3 — Operational identity (the autonomous-device level). Once trust is established, the **commissioner issues a Node Operational Certificate (NOC)** to the device — a locally meaningful identity for day-to-day operation within the user's home fabric. From then on, **all later device-to-device sessions use CASE (Certificate Authenticated Session Establishment), a SIGMA-**

based handshake, with no user involved. The human is needed only once, at commissioning; thereafter devices authenticate one another autonomously using their operational certificates.

The elegance of Matter is that these three steps **cover the three levels of trust in a smart environment — manufacturer, user, and autonomous device** — in a single pipeline, a structure that will reappear as a general principle in Section 17.



14. LoRaWAN: Joining the Network (OTAA)

LoRaWAN is the long-range, low-power wireless technology already named in Section 9 as a representative public IoT link. Its secure onboarding mechanism is **Over-The-Air Activation (OTAA)**, the procedure by which a device negotiates its working keys when it first joins a network.

The trust is rooted in a provisioned secret: **each device is provisioned with a root key — the `AppKey`, and in LoRaWAN 1.1 also a `NwkKey` — shared with the Join Server.** The join itself is a two-message handshake. The device sends a **Join Request** containing its device identifier `DevEUI`, the join-server/application identifier `JoinEUI`, and a nonce `DevNonce`, the whole message **authenticated with a MIC (Message Integrity Code) computed by AES-CMAC**. The network replies with a **Join Accept** carrying a `JoinNonce` and the network parameters, **encrypted and MIC-protected**. From the root key and the exchanged nonces, **both sides then derive the session keys.** In **1.0.x** these are the `NwkSKey` (for network integrity) and the `AppSKey` (for **end-to-end payload encryption**). In **1.1**, the design is refined into **separate network-integrity and encryption keys, with counters used as nonces** — a cleaner separation of duties.

LoRaWAN is pedagogically valuable as much for its **known weaknesses** as for its mechanism, because each flaw illustrates a general failure mode:

- **Shared root key across a product line.** If a manufacturer burns the *same* root key into every unit of a product line, then **one device dumped compromises all of them** — the extraction attack of Section 1 amplified across an entire fleet.
- **Random `DevNonce` in 1.0.** The use of a *random* (rather than monotonically counting) `DevNonce` in version 1.0 enables **replay of old Join Accepts** and a **denial-of-service by nonce exhaustion**, since the limited nonce space can be deliberately used up.
- **ABP devices with static keys.** Devices using **Activation By Personalization (ABP)** — which hard-codes static session keys instead of negotiating them — are exposed to **replay and eavesdropping** whenever their frame counters reset, because a reset lets old, valid-looking frames be reused.
- **No device-to-device authentication.** LoRaWAN authenticates a device to the network only; there is **no device-to-device authentication, and the gateway is merely a relay.** This stands in deliberate contrast to Matter's CASE, which authenticates devices directly to one another.

15. EDHOC + OSCORE: The IETF Answer for Constrained Devices

The standard internet-security handshake, TLS (or its datagram variant DTLS), is too heavy for the smallest nodes: **constrained nodes cannot afford a full TLS/DTLS handshake**, whose message sizes and computational load exceed their budgets. The IETF's answer is a pair of lightweight protocols designed to deliver equivalent guarantees within an IoT frame budget.

EDHOC (Ephemeral Diffie-Hellman Over COSE), standardized as **RFC 9528 in 2024**, is the handshake. It is a **three-message exchange providing mutual authentication and forward secrecy.** It is flexible about credentials: **authentication can use signatures or static Diffie-Hellman keys, and identities can be raw public keys or certificates.** Its defining property is compactness — **the handshake is only a few hundred bytes, small enough to fit inside LoRaWAN and 6TiSCH frames**, where a full TLS handshake would not.

OSCORE (Object Security for Constrained RESTful Environments), standardized as **RFC 8613**, is the protection layer that uses the key EDHOC produces. It **protects CoAP (the constrained-device web protocol) messages end-to-end with the EDHOC-derived key.** Its crucial advantage over DTLS is that, because it secures the *application-layer objects* rather than the transport, it **works through proxies and gateways** — intermediaries that would break a transport-layer tunnel like DTLS can forward OSCORE-protected messages without being able to read them.

The conceptual takeaway is that EDHOC + OSCORE pursue **the same goals as the academic protocols of Part 3 — mutual authentication, forward secrecy, lightweight operation — but with the backing of a formal analysis and an open standard.** Where the bespoke protocol of Section 10 was one research design, EDHOC is the standardized, scrutinized, interoperable realisation of the same ambitions.

16. Bluetooth Low Energy Pairing

Pairing is the process by which two Bluetooth Low Energy (BLE) devices establish a **long-term key**, and it is especially delicate because it **often involves devices without screens or keyboards** — a fitness band or a sensor has no obvious way for a human to confirm what it is talking to. The history of BLE pairing is a case study in how input/output limitations shape security.

Legacy pairing (BLE 4.0 / 4.1) derives the long-term key from a short **Temporary Key (TK)**. Its notorious failure mode is the **"Just Works" association model, which sets TK = 0.** With a known, fixed TK, a **passive eavesdropper can recover the resulting key** — the attack demonstrated by the tool **crackle in 2013**. Legacy Just Works pairing offers, in effect, no protection at all against a listener.

LE Secure Connections (BLE 4.2 and later) fixes the cryptographic core by replacing the TK derivation with a proper **Elliptic-Curve Diffie-Hellman exchange on the P-256 curve**, followed by an **authentication of that exchange**. The authentication step comes in four **association models**: **Just Works, Numeric Comparison, Passkey Entry, and Out-of-Band (OOB).** The ECDH makes passive eavesdropping infeasible — but the critical caveat remains: **Just Works still provides no protection against a man-in-the-middle**, because without any user confirmation step there is nothing to bind the exchange to the intended peer. The stronger models (Numeric Comparison, Passkey Entry, OOB) add exactly that binding, and which one a device *can* use is dictated by its I/O hardware.

Even a well-paired link can be attacked **beyond pairing**:

- **KNOB (Key Negotiation of Bluetooth).** The attacker interferes with the key-agreement negotiation to **force the key entropy down — to as little as 7 bytes in BLE** — after which the weakened key can be brute-forced.

- **BLESA (Bluetooth Low Energy Spoofing Attack).** The attacker **spoofs a device during reconnection**, exploiting the fact that on reconnection **authentication is optional** and may be skipped.

The overarching lesson is blunt: **the I/O capabilities of the device decide its security level.** A device with a display can run Numeric Comparison and resist MITM; a screenless, buttonless device is pushed toward Just Works and its weaknesses. Security here is not merely a protocol choice but a hardware constraint.

17. Device Identity and Secure Onboarding

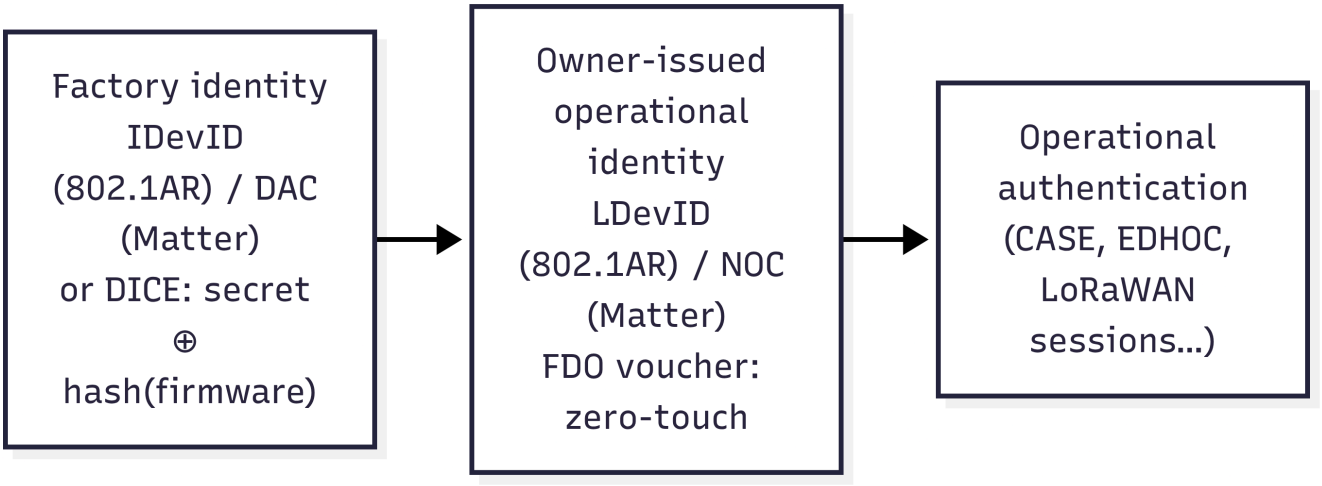
Every protocol so far has assumed the device already *has* some trustworthy identity to authenticate with. But that assumption must itself be established: **before any protocol can run, how does a device come to possess a trustworthy identity at all?** Several frameworks answer this.

IEEE 802.1AR defines two complementary identities. The **IDevID (Initial Device Identifier)** is a **factory-installed certificate** — the permanent, manufacturer-issued identity burnt in at production (the role Matter's DAC plays). The **LDevID (Locally significant Device Identifier)** is a **local identity issued by the owner** after the device is deployed into its operational environment (the role Matter's NOC plays). The pattern — factory identity first, owner identity second — is the recurring shape of secure onboarding.

DICE (Device Identifier Composition Engine), from the Trusted Computing Group, binds **identity to the code that actually runs. A unique device secret is mixed with the hash of the first mutable code** the device executes, and the device's cryptographic identity is derived from that combination. The consequence is powerful: **different firmware produces a different identity**, so **a compromised device — one whose firmware has been tampered with — cannot keep its keys.** Malicious code alters the hash, which alters the derived secret, which invalidates the identity. Identity is thereby made *conditional on integrity.*

FIDO Device Onboard (FDO) targets **zero-touch onboarding** — bringing a device online without manual configuration. The **device boots, contacts a rendezvous server, and the owner proves ownership by presenting a voucher** (a transferable credential created along the supply chain). This lets a device be shipped, powered on, and automatically bound to its rightful owner with no technician typing credentials.

Across all three frameworks runs a single **common principle: a factory identity first, then an owner-issued operational identity.** This is precisely the manufacturer → user → autonomous-device progression seen in Matter, now recognised as the universal structure of device onboarding.



18. Users versus Autonomous Devices

The whole subject divides cleanly into **two kinds of authentication**, distinguished by whether a human being is in the loop — and recognising which kind a given problem belongs to is the organising insight of the entire lecture.

A user authenticates to the smart environment. Here the problem is the familiar human-facing one, adapted to devices that often lack keyboards. Users authenticate **through an app or a cloud account using passwords, multi-factor authentication (MFA), or passkeys (FIDO2).** For **devices without a keyboard**, the **OAuth 2.0 Device Authorization Grant (RFC 8628)** lets a user authorise a screenless device by completing the login on a separate phone or computer. **Physical presence itself becomes a factor** — QR codes, physical buttons, and proximity (as in Matter and BLE). And increasingly, **continuous and behavioural authentication** — identifying a person by **voice, gait, or motion** patterns over time — supplements the one-time login.

A device authenticates to another device or service, with no human in the loop. This is the machine-to-machine case that dominates a smart environment numerically. It is handled by **symmetric keys (LoRaWAN), PUF-based protocols (Part 3), and certificates (Matter CASE, EDHOC).** Its defining demands are scale and resilience: the solution **must scale to thousands of devices and must survive key compromise**, so that cracking one node does not unravel the rest.

The sharpest observation closes the topic: **the weak points live at the boundary — in the delegation of authority from the user to the device.** The user-facing mechanisms are mature and the device-facing mechanisms are increasingly robust, but the *hand-off* between them — the moment a human grants a machine the authority to act autonomously on their behalf — is where trust is most fragile and where attacks most often succeed. Every commissioning flow in this part (Matter's three steps, FDO's voucher, the IDevID→LDevID transition) is ultimately an attempt to make that one delegation boundary secure.

CHAPTER 4: Secure Computation

Introduction: The Meaning and Limits of Privacy

Privacy is best understood as the **protection of an individual's personal information**. Expressed more formally, privacy concerns the rights and obligations of individuals and organizations with respect to the collection, use, retention, disclosure, and disposal of personal information. These two framings are complementary: the first states the goal (protecting the individual), while the second describes the

machinery of duties and permissions through which that goal is pursued across the entire lifecycle of data, from the moment it is gathered to the moment it is destroyed.

A useful intuition is that **privacy is about promising people freedom from harm**. If participating in some data-processing activity can be guaranteed not to expose a person to adverse consequences, then their privacy has, in a meaningful sense, been respected. Turning this intuition into a precise, usable definition, however, is surprisingly difficult, and the history of failed attempts is itself instructive.

A First Definition and Why It Fails

A natural first attempt defines privacy in absolute terms:

> *An analysis of a dataset D is private if the data analyst knows no more about Alice after the analysis than he knew about Alice before the analysis.*

This reads well, but it is impossible to satisfy in the presence of **auxiliary information**, that is, knowledge the analyst already possesses or can obtain from other sources. Consider an insurance company that already knows Alice is a smoker. Suppose an analysis of a medical dataset reveals the statistical fact that smoking and lung cancer are correlated. Armed with this newly learned correlation and its pre-existing knowledge of Alice's smoking habit, the company might decide to raise her insurance rates.

Was Alice's privacy violated? The crucial observation is that **this harm can occur even if Alice's data was never in the database at all**. The analysis did not reveal anything specific about Alice; it revealed a general truth about the world. Yet that general truth, combined with auxiliary information, produced a concrete consequence for her. This exposes a fundamental tension: the correlation between smoking and lung cancer is *exactly the kind of knowledge we build datasets to discover*. A definition of privacy that forbids learning it would forbid science itself. Any workable notion of privacy must therefore distinguish between learning about the population (legitimate and desirable) and learning about a specific individual because of their inclusion in the data.

A Refined Definition

The refinement that resolves this tension shifts the comparison from "before versus after" to "with Alice versus without Alice":

> *An analysis of a dataset D is private if the data analyst knows almost no more about Alice after the analysis than he would have known had he conducted the same analysis on an identical database with Alice's data removed.*

Two modifications carry all the weight. The word **"almost"** acknowledges that perfect indistinguishability is unattainable and introduces a tolerance that can be quantified. The phrase **"with Alice's data removed"** redefines the baseline: privacy is no longer about the analyst gaining no knowledge, but about Alice's *presence* in the dataset making no appreciable difference to what the analyst learns. Under this definition, discovering the smoking–cancer correlation is permissible, because that correlation would emerge whether or not Alice's record was present. What is protected is Alice's individual contribution, not the aggregate knowledge of the population. This is the conceptual seed from which the formal framework of differential privacy later grows.

A critical consequence, which should be internalized at the outset, is that **simply removing identifying information from a dataset is not sufficient** to guarantee privacy. The combination of

released data with external auxiliary information is where re-identification attacks live, as the case studies below demonstrate.

Areas of Privacy

The field organizes itself around several distinct areas, each addressing a different surface through which personal information can leak.

The first is **anonymity**, principally in the form of anonymous communication. The canonical example is the **TOR** software, which is designed to defend against traffic analysis so that an observer cannot easily link who is communicating with whom. Here the content may be protected by encryption, but the *metadata* of communication, who talks to whom and when, is itself the sensitive quantity being hidden.

The second is **web privacy**, which concerns the ability to understand and control what websites collect and maintain regarding personal data. The goal is transparency and control over the trail a user leaves while browsing.

The third is **mobile data privacy**, with **location privacy** as its most prominent instance. Mobile devices continuously generate and often transmit their position, producing a detailed record of a person's movements that is deeply revealing.

The fourth, which is the main focus of this chapter, is **privacy-preserving data usage**: the question of how data can be shared, published, and computed upon while still protecting the individuals it describes.

The Need to Share Data, and Its Dangers

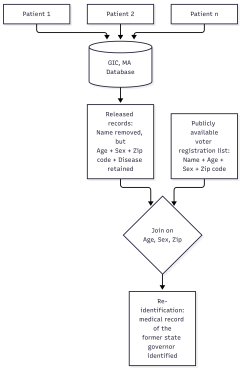
Modern society is thoroughly data-driven, and there is a genuine and recurring necessity to share data. Data is shared **for research purposes** across social, medical, and technological domains; it is shared because it is **mandated by laws and regulations**, as with the census; it is shared **for security and business decision-making**, for example when network flow data is pooled to enable Internet-scale alert correlation; and it is shared **for system testing before deployment**, among many other reasons.

The difficulty is that **publishing data may result in privacy violations**, even when the publisher has made a good-faith effort to anonymize it. The following three real-world episodes each illustrate the same underlying lesson: anonymization by removal of obvious identifiers fails against an adversary with auxiliary information, and **re-identification occurs**.

GIC Incidence (Sweeney 2002)

The Group Insurance Commission (GIC) of Massachusetts collected medical data for approximately 135,000 state employees and then gave this data to researchers and sold it to industry. The released records had names removed, retaining fields such as age, sex, zip code, and diagnosed disease, under the assumption that stripping names rendered the data anonymous.

The structure of the attack is straightforward once one notices that the remaining attributes, though individually innocuous, are *jointly* nearly unique. The following diagram captures the flow by which individual patients contribute to a central database that is then released:



By cross-referencing the "anonymized" medical records with a publicly available voter registration list that contained names alongside age, sex, and zip code, it was possible to match records uniquely. The medical record of the former state governor was identified in this way. The lesson is that the triple (age, sex, zip code) acts as a **quasi-identifier**: a combination of attributes that is not an identifier on its own but becomes one when linked against external data.

AOL Data Release (NYTimes 2006)

In August 2006, AOL released the search keywords of 650,000 users collected over a three-month period. In an attempt at anonymization, **user IDs were replaced by random numbers**. Recognizing the problem, AOL pulled the data from public access just three days later, but by then it had already been copied and analyzed.

The failure here is more subtle and more damning than in the GIC case, because the sensitive information was not a separate auxiliary dataset but was embedded *within the released data itself*. The sequence of queries made by a single pseudonymous user forms a revealing fingerprint. For instance, user number 4417749 had issued searches including "landscapers in Lilburn, GA," repeated queries on the last name "Arnold," "homes sold in shadow lake subdivision Gwinnett County, GA," and "dog that urinates on everything." Taken together, these queries pointed to a specific real person: Thelma Arnold, a 62-year-old widow living in Lilburn, GA, with three dogs, who frequently searched for her friends' medical ailments. The random replacement of the user ID did nothing to protect her, because the *content* of her searches was self-identifying. Re-identification occurred.

Netflix Movie Rating Data (Narayanan and Shmatikov 2009)

Netflix released anonymized movie rating data as part of the Netflix Challenge, a public competition to improve its recommendation algorithm. The released records included, for each anonymous user, the **date and value of movie ratings**.

The de-anonymization result was striking: knowing only **6 to 8 approximate movie ratings together with their approximate dates** is enough to uniquely identify a record **with over 90% probability**. The attack worked by correlating the sparse, high-dimensional rating data with auxiliary information. Correlating the Netflix data against a set of just 50 users drawn from the public reviews on imdb.com was sufficient to yield two confidently re-identified records. The practical consequence was severe enough that **Netflix cancelled the planned second phase of the challenge**. Once again, re-identification occurred, confirming that high-dimensional behavioral data is extraordinarily difficult to anonymize, because each individual's pattern of preferences is close to unique.

The Unifying Lesson

Across all three episodes the message is identical and worth stating plainly as the motivation for everything that follows. Removing names or replacing identifiers with random tokens does **not** achieve

privacy. Real individuals can be re-identified by linking released data against auxiliary information, whether that auxiliary information is an external public database (GIC, Netflix) or is latent in the content of the data itself (AOL). Because naive anonymization is insufficient, we are driven to seek stronger, cryptographically grounded tools that allow useful computation on data without exposing the underlying individual records. The first and most powerful such family of tools is **secure computation**.

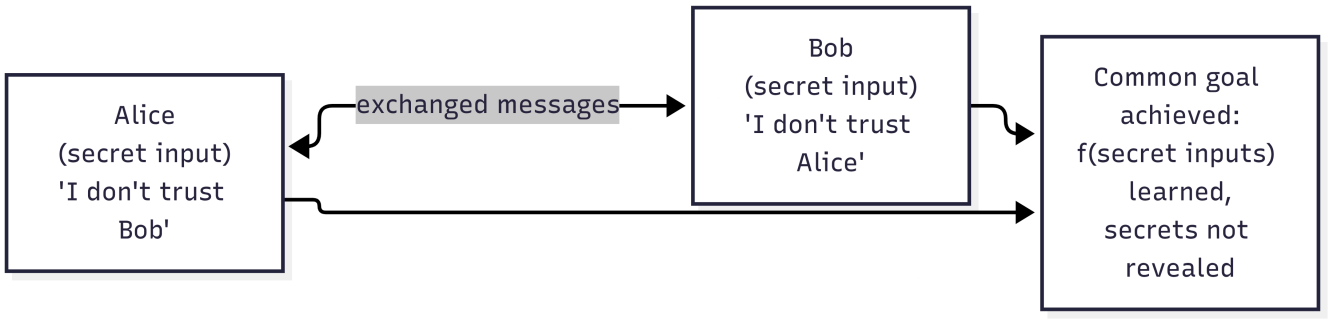
Secure Computation

Having established that the removal of identifiers cannot protect privacy, we turn to a far more ambitious idea: allowing several parties to jointly compute a result that depends on all of their private inputs, while revealing nothing about those inputs beyond what the result itself implies. This is the domain of **secure computation**, and its general form is known as **Multi-Party Computation**.

Multi-Party Computation

Multi-Party Computation (MPC) refers to protocols in which the participating players **do not trust each other**, yet nevertheless wish to **achieve a common goal**. The goal is typically expressed as the evaluation of a function computed over the parties' **secret inputs**. We denote by n the number of players taking part in the protocol.

The paradoxical character of MPC is captured by the simultaneous demands that each party makes: Alice does not trust Bob, and Bob does not trust Alice, and yet both agree to run a protocol whose common goal they both want achieved. The resolution of this apparent paradox is the entire subject. The parties exchange messages back and forth, and at the end the common goal is achieved, without either having had to reveal their secret to the other.



The foundational guarantee we are reaching for can be stated as a principle that will recur throughout: **at the end of the protocol we do not want to know more about the other party than what we knew before**, except for whatever is unavoidably implied by the output of the function. It is important to be honest about that last qualification. Because a party has chosen to participate in the evaluation, the function f *will* be computed, and the output of f may itself leak some information about the other party's input. That leakage is inherent to the task, not a flaw in the protocol; what a secure protocol promises is that *nothing beyond* the output leaks.

The Millionaires' Problem

The classic illustration, and historically the problem that launched the field, is the **Millionaires' Problem**. Two millionaires wish to determine who is richer **without revealing their respective wealths** to each other. If Alice holds the value x_1 and Bob holds x_2, the function to be computed is

$$f(x_1, x_2) = \{1 \quad \text{if } x_1 > x_2 \; 0 \quad \text{if } x_2 \geq x_1$$

Each party privately believes "I am richer," and the protocol must settle the dispute by outputting a single bit indicating who holds the larger value, while leaking nothing else. Here the inherent-leakage caveat is visible: once the result is known, the loser does learn one fact about the winner, namely that the winner's wealth exceeds their own. That single comparison bit is the unavoidable output; a secure protocol ensures nothing more than that bit is exposed.

A Concrete Protocol: It Is Not Black Magic

To show that this is achievable with ordinary cryptography rather than sleight of hand, consider a concrete instantiation. The **goal** is for two millionaires to learn who is richest without revealing their respective wealths, where each wealth is an integer in the range $[1, 10]$. Call the two parties US (Uncle Scrooge) and RD (Rockerduck). The setup requires that **US knows RD's public key**. Suppose for illustration that US has $X = 6$ million and RD has $Y = 4$ million; neither of these values is known to the other party.

US's view — preparing the message. US performs the following steps:

1. US picks a **random number** R (in the worked example, $R = 67959$).
2. US **encrypts** this random number under RD's public key, computing $C = E_{PK}[R]$ (in the example, $C = 92033493$).
3. US **subtracts its own wealth** X from the ciphertext, preparing the message $M = C - X = 92033493 - 6 = 92033487$.

US then sends $M = 92033487$ to RD. Notice that RD, upon receiving M, cannot recover either R or X: the value M is a single number formed from the encryption of a random quantity minus a secret, and RD cannot untangle the two.

RD's view — building the comparison table. RD now processes M across all possible wealth values. Let W range over every candidate wealth in $[1, 10]$. RD performs the following:

1. RD **adds each possible wealth** $W \in [1, 10]$ to M, forming the column $M + W$.
2. RD **decrypts each of these values** with its own secret key, obtaining $U = D_{SK}(M + W)$ for every row.
3. RD **adds $+1$ to every entry from the Y-th position onward** (that is, for all rows where $W \geq Y$, where $Y = 4$ is RD's own wealth), producing the array Z.
4. RD **computes** $Z \bmod p$ for a prime p (in the example p is such that the reduction yields values below 10000) and transmits the resulting column back to US.

The worked table makes the mechanism transparent. With $M = 92033487$:

W	$M + W$	$U = D_{SK}(M + W)$	add $+1$?	Z
1	92033488	333900		333900
2	92033489	627991		627991
3	92033490	901125		901125
4	92033491	2998	+1	2999
5	92033492	72013	+1	72014
6	**92033493**	**67959**	**+1**	**67960**
7	92033494	82663	+1	82664
8	92033495	102745	+1	102746

W	$M+W$	$U = D_{SK}(M+W)$	add $+1$?	Z
9	92033496	3789	$+1$	3790
10	92033497	897114	$+1$	897115

Two features of this table are the key to the protocol's correctness. First, for the rows where no $+1$ is added (the rows with $W < Y$), and for most rows generally, the decrypted values U appear **essentially random**, because decrypting $M + W = E_{PK}[R] - X + W$ produces a meaningful plaintext only at the single position where $W - X$ cancels correctly. Second, observe the row $W = 6$: here $M + W = 92033493 = C$ exactly, so $D_{SK}(C) = R = 67959$, the original random number US chose. This is the position $W = X = 6$, US's own wealth. Because this row satisfies $W = 6 \geq Y = 4$, it receives the $+1$, giving $Z = 67960$ at that position.

The deeper point is that **only US knows which plaintext is the "correct" one** — US alone knows that the meaningful value, the one tied to its random R, sits at position $W = X = 6$. RD, by contrast, has merely applied the mechanical $+1$ from position Y onward without knowing which row is significant.

US's view — reading off the answer. US receives the column $Z \bmod p$ from RD (in the example, the values $3900, 7991, 1125, 2999, 2014, 7960, 2664, 2746, 3790, 7115$). US then **checks the value at its own position** $X = 6$, which is $Z(X) = 7960$. US compares this against $R \bmod p = 67959 \bmod p = 7959$. The decision rule is:

- If the value at position X is **larger** than $R \bmod p$, then the $+1$ was applied at US's position, which happens precisely when $X \geq Y$, so **US is the richest** (or tied).
- If it equals $R \bmod p$ with no increment, then $X < Y$, and RD is richer.

In the example, $Z(6) = 7960 > 7959 = R \bmod p$, so US correctly concludes it is the richest, having learned only the single comparison bit, and RD has learned nothing at all about X. The reduction modulo p is what collapses the arbitrary random magnitudes into a bounded range where the presence or absence of the single $+1$ can be detected without revealing the full values.

Coin Tossing

A second foundational secure-computation task is **coin tossing**: two mutually distrustful parties wish to agree on a common random bit y, uniformly distributed, such that

$$y = \{1 \quad \text{with probability } 1/2 \; 0 \quad \text{with probability } 1/2$$

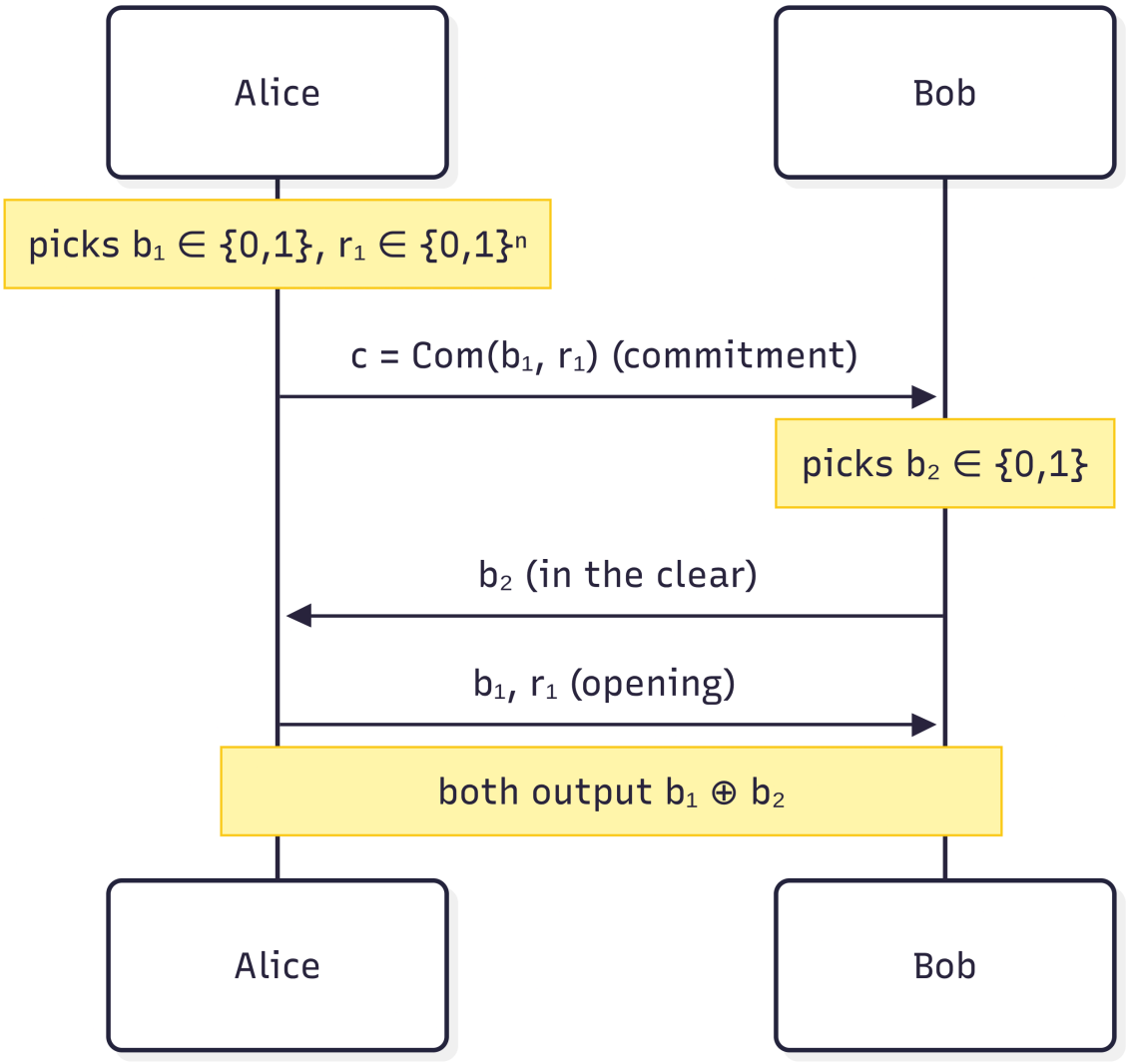
The challenge is that neither party trusts the other to generate the bit fairly. If Alice simply announces a bit, Bob has no assurance it was random; if they try to combine contributions, whoever speaks last can bias the outcome. The resolution is to force commitment before revelation.

Blum's Coin Flipping Protocol

Blum's protocol achieves a fair coin over a channel between two distrustful parties using a **commitment scheme**, parametrized by a security parameter 1^n. The protocol proceeds as follows, with Alice and Bob each contributing one bit.

1. Alice **selects a random bit** $b_1 \in_R 0, 1$ and a random string $r_1 \in_R 0, 1^n$. She computes a **commitment** $c = \text{Com}(b_1, r_1)$ and sends c to Bob. The commitment hides b_1 (Bob cannot learn it from c) while binding Alice to it (she cannot later change it).

2. Bob, upon receiving the commitment, **selects his own random bit** $b_2 \in_R 0,1$ and sends b_2 to Alice in the clear.
3. Alice **opens her commitment** by sending (b_1, r_1) to Bob, allowing him to verify that c was indeed a commitment to b_1.
4. Both parties **output** the final coin $b_1 \oplus b_2$.



The security rests on the two properties of the commitment and on well-defined handling of misbehavior. Because Bob chooses b_2 *after* seeing only the hiding commitment c, he cannot bias the result toward a value correlated with b_1; and because Alice is bound by c, she cannot change b_1 after seeing b_2. The exclusive-or of two bits, at least one of which is genuinely random and committed independently of the other, is itself uniformly random. The protocol also specifies the handling of **aborts and cheating**:

- If **Bob doesn't send his bit**, the protocol sets $b_2 = 0$ by default, so Alice can still produce an output.
- If **Alice doesn't open** her commitment, Bob outputs the failure symbol $\perp$.
- If **Alice's opening is invalid** (the revealed (b_1, r_1) does not match c), Bob again outputs $\perp$.

The $\perp$ outcome signals detected cheating; it is not a valid coin, but it prevents a dishonest party from forcing a biased result and passing it off as fair.

Secure Dating

A third illustrative task is **secure dating**, which is structurally the private computation of a logical AND. Each party holds a private bit indicating interest: Alice's bit x_1 is 1 if she likes Bob and 0 otherwise, and Bob's bit x_2 is 1 if he likes Alice. The function computed is

$$f(x_1, x_2) = \{1 \quad \text{if } x_1 \cdot x_2 = 1\ 0 \quad \text{if } x_1 \cdot x_2 = 0$$

The result is 1 only when **both** parties are interested, in which case a match is revealed; otherwise the output is 0. The privacy requirement here has real social force: a party who is interested but is rejected (for example, Alice likes Bob but Bob does not like Alice) should not have their interest exposed. The function reveals a positive match only when the interest is mutual, and in the non-matching cases it must not disclose which party was willing. This is exactly the behavior of the AND function, and it foreshadows why the AND operation will play a central role in general secure computation.

Quantifying Security: Adversary Models

To reason rigorously about whether a protocol is secure, we must specify *what the adversary is allowed to do*. The strength of a security guarantee is meaningful only relative to a threat model, and the field distinguishes three standard classes of adversary, in increasing order of power.

Semi-honest adversaries (also called "honest-but-curious") **follow the protocol** exactly as specified, but **are curious**: they attempt to extract as much information as they can about the other parties' inputs from the messages they legitimately receive and from the final result. They do not deviate, cheat, or send malformed messages; their only misbehavior is passive inference. A protocol secure against semi-honest adversaries guarantees that a party who runs the protocol faithfully learns nothing beyond the prescribed output, even while scrutinizing everything it sees.

Malicious adversaries are far stronger: **they are willing to do anything** to break the protocol. They may deviate arbitrarily from the specification, send malformed or maliciously crafted messages, abort at inconvenient moments, lie about their inputs, or otherwise manipulate the execution in any way that might help them learn secrets or corrupt the output. Security against malicious adversaries is a much more demanding standard and typically requires additional machinery (such as zero-knowledge proofs that each step was performed correctly).

Covert adversaries occupy an intermediate position: they **are willing to do anything, but they do not want to be discovered**. A covert adversary will cheat if and only if it believes it can get away with it undetected; it is deterred by a sufficiently high probability of being caught. This model is motivated by realistic settings, such as business or reputational contexts, where the consequences of being publicly exposed as a cheater outweigh the gains from a successful attack. Protocols secure against covert adversaries aim to guarantee that any deviation is detected with high probability, making cheating irrational even though it is not technically prevented.

These three models frame every subsequent construction: whenever a secure-computation protocol is proposed, the first question is always which class of adversary it withstands, since a protocol that is perfectly adequate against semi-honest curiosity may collapse entirely in the face of a malicious participant.

Homomorphic Encryption

We now arrive at the first of the cryptographic engines that make privacy-preserving computation practical. The guiding question is deceptively simple: can we perform meaningful operations on data

while it remains encrypted, so that a party who processes the data never sees the underlying values? The answer is yes, and the property that makes it possible is called **homomorphism**.

A Brief Point of Departure: The Algebraic Structure of RSA

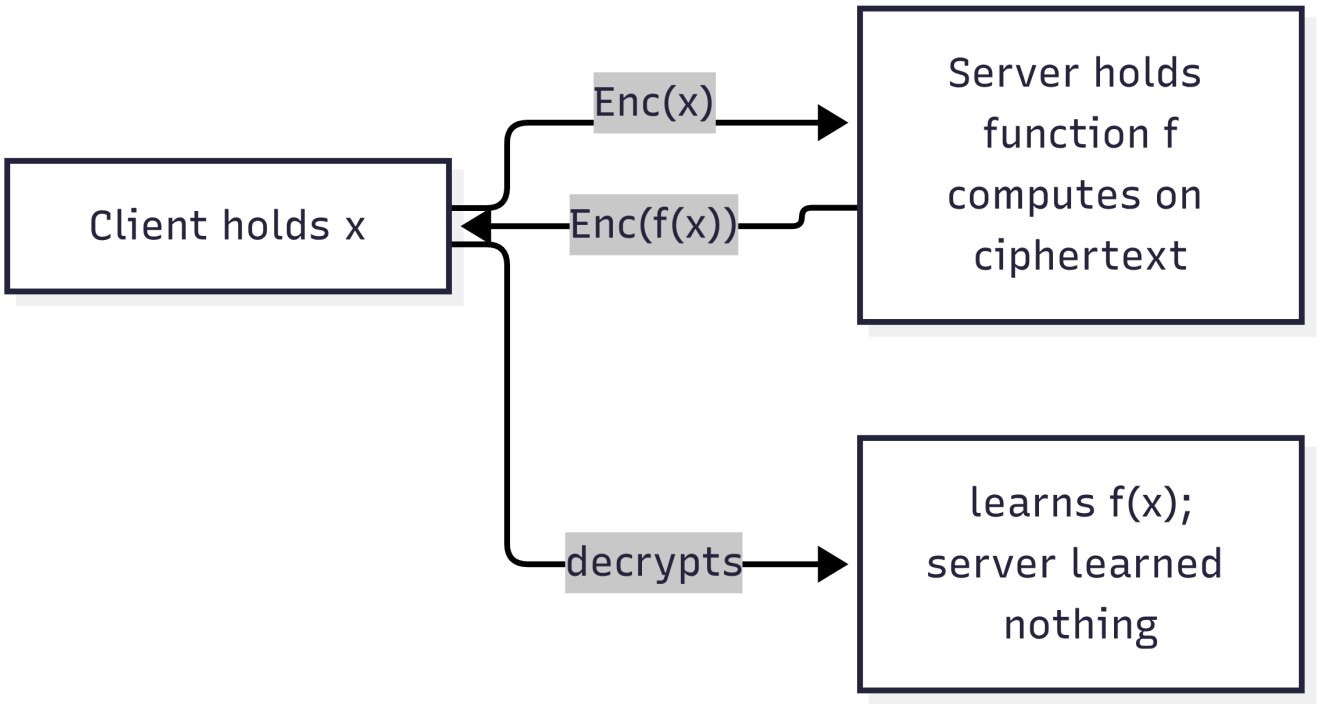
To understand homomorphic encryption, it helps to begin from the most widely known asymmetric scheme. **RSA** was the first and remains the most popular asymmetric (public-key) encryption. Its encryption and decryption are defined, for a public exponent e, a private exponent d, and a modulus n, by

$$E(m) = m^e \pmod{n}, \qquad D(c) = c^d \pmod{n}.$$

A message m is encrypted by raising it to the public exponent modulo n, and a ciphertext c is decrypted by raising it to the private exponent modulo n. For our purposes the interesting thing is not RSA's confidentiality but a structural accident hidden in its algebra.

Computing on Encrypted Data

The animating idea is to let one party compute on data it cannot read. Imagine a device that holds a value x and a remote server that holds a function f. The device sends $E(x)$, the server computes on the ciphertext, and the server returns $E(f(x))$, which only the device can decrypt. At no point does the server learn x or $f(x)$.



The question is what functions f a given encryption scheme permits us to evaluate in this blind fashion. This is entirely determined by the scheme's algebraic structure.

Multiplicative Homomorphism in RSA

Some observers noted the algebraic structure latent in RSA. Take two messages m_1 and m_2 with ciphertexts $E(m_1) = m_1^e$ and $E(m_2) = m_2^e$. Consider the product of the two ciphertexts:

$$E(m_1) \times E(m_2) = m_1^e \times m_2^e = (m_1 \times m_2)^e = E(m_1 \times m_2).$$

The derivation is purely a property of exponents: the product of two e-th powers is the e-th power of the product. The consequence is profound. Multiplying two ciphertexts yields the ciphertext of the product of the plaintexts, **without ever decrypting**:

$$\boxed{E(m_1) \times E(m_2) = E(m_1 \times m_2)}$$

This property is called **multiplicative homomorphism**. A party holding only ciphertexts can produce the encryption of the product of the hidden values. RSA, however, offers *only* this; it does not let us add plaintexts by manipulating ciphertexts.

Additive Homomorphism

Other encryption schemes were discovered to be **additively homomorphic** (but not multiplicatively homomorphic). For such a scheme, combining two ciphertexts yields the encryption of the *sum* of the plaintexts:

$$\boxed{E(m_1) + E(m_2) = E(m_1 + m_2)}$$

Here the notation "+" on the ciphertext side denotes whatever ciphertext operation the scheme uses to realize plaintext addition (often itself a modular multiplication of ciphertexts); the essential point is that one operation on ciphertexts maps to addition of plaintexts. Additive schemes and multiplicative schemes were, for a long time, two separate worlds: a scheme gave you one homomorphism or the other, never both.

Paillier's Cryptosystem

The most important additively homomorphic scheme is **Paillier's cryptosystem**, whose security rests on the **composite residuosity problem**. That problem is stated as follows:

> Given c, γ, and n, find m such that $c = \gamma^m, r^n \bmod n^2$ for some r.

Each symbol plays a defined role. The value γ is the **public key**, m is the **plain message**, r provides **randomization**, and the pair (p, q) with $n = pq$ is the **secret key**. The modulus of operation is n^2. For adequate **security**, both c and n^2 must be **at least 2048 bits**.

Two features of Paillier are essential to grasp. First, the presence of the random term r^n means Paillier is a **randomized (probabilistic) encryption**: encrypting the same message twice yields different ciphertexts, which is necessary for semantic security but also means the scheme carries randomness that must be managed. Second, and most important for our purposes:

- **Additive homomorphism follows from the properties of exponentials.** Because the message sits in the exponent as γ^m, multiplying two ciphertexts adds their exponents, which realizes addition of the underlying plaintexts, exactly the additive property described above.
- **Products are possible only with plain values.** Paillier does *not* allow multiplying two encrypted messages together to obtain the encryption of their product. What it does allow is multiplying an encrypted value by a *known plaintext* constant (equivalently, raising a ciphertext to a known plaintext power), which produces the encryption of the scaled value. In short: you may add two secrets, and you may multiply a secret by a public constant, but you may not multiply two secrets.

This asymmetry, additions between secrets but multiplications only by known constants, defines precisely the set of operations we can perform, and it turns out to be enough for a remarkably rich class of computations.

The Homomorphic Paradigm: Operators Built from Additive HE

With an additive homomorphic encryption (HE) scheme in hand, a number of interesting and practically important operators can be applied directly to encrypted data. Each of the following is a linear operation, and each reduces to additions of secrets and multiplications by known values, which is exactly what additive HE supports.

Component-wise encryption. A vector is encrypted element by element, so that encrypting a tuple produces the tuple of encryptions:

$$E[(a_1, a_2, \ldots, a_n)] = (E[a_1], E[a_2], \ldots, E[a_n]).$$

This is the foundation on which the vector operations below are built.

Scalar product with a known vector. Suppose $\mathbf{b}$ is a publicly known vector while $\mathbf{a}$ is encrypted. The scalar product $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^{n} a_i b_i$ can be computed under encryption as

$$E[\langle \mathbf{a}, \mathbf{b} \rangle] = \prod_{i=1}^{n} E[a_i]^{,b_i}.$$

Each encrypted component $E[a_i]$ is raised to the *known* scalar b_i (multiplication of a secret by a public constant), and the products are combined (addition of secrets). Both operations are within the additive scheme's power precisely because the b_i are known.

FIR filtering. A finite impulse response filter computes an output sample as a weighted sum of input samples, $a_n = \sum_{k=1}^{L} a_{n-k} h_k$, with publicly known filter coefficients h_k. Under encryption this becomes

$$E[a_n] = \prod_{k=1}^{L} E[a_{n-k}]^{,h_k},$$

which is again a product of encrypted samples raised to known coefficients. This means a signal can be filtered while it remains encrypted, as long as the filter taps are public.

Linear transforms. Any linear transform $X_k = \sum_{i=1}^{n} a_{k,i} x_i$ with a known transform matrix $(a_{k,i})$ can be evaluated homomorphically as

$$E[X_k] = \prod_{i=1}^{n} E[x_i]^{,a_{k,i}}.$$

This covers a broad swath of signal-processing and statistical operations, from projections to transforms, all performed blindly on encrypted inputs.

The common thread is that **additive HE handles any linear function of the secret data when the coefficients are public**. What it cannot do directly is anything *non-linear*, because non-linearity requires multiplying secrets by secrets, which the scheme forbids.

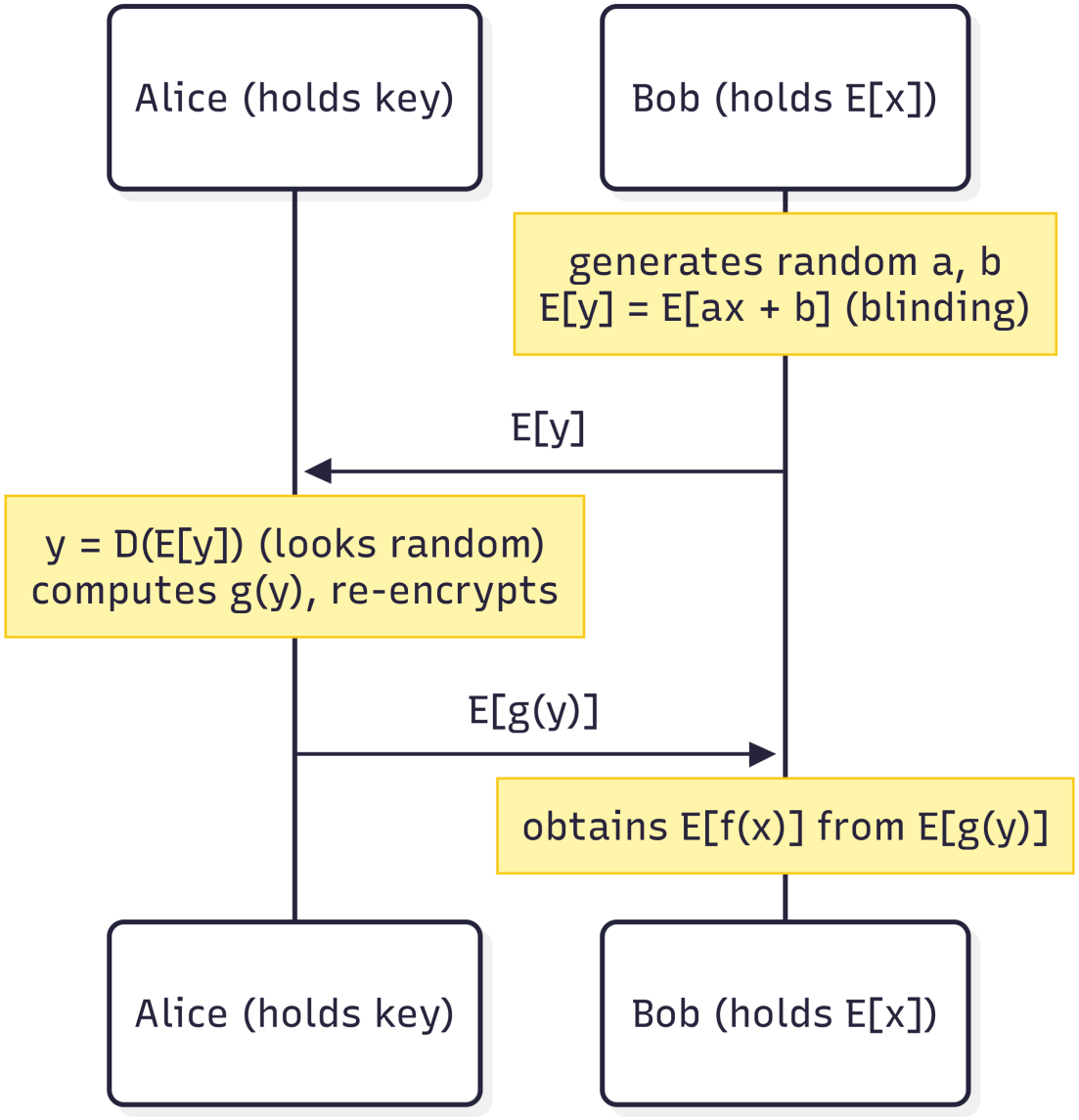
Non-Linear Functions Through Blinding

Since purely additive homomorphism cannot evaluate non-linear functions on its own, we need an interactive technique that lets two parties jointly compute a non-linear function while preserving privacy. The standard method is **blinding**, and it works by having one party temporarily transform the

encrypted value into a randomized form that can be safely decrypted by the other without revealing the secret.

The setting is as follows. Assume an additive cryptosystem is available. **Bob** possesses a value x in encrypted form, $E[x]$, and needs to apply a **non-linear function** $f(\cdot)$ to it. Bob cannot decrypt (he lacks the key), and additive HE alone cannot compute f. **Alice** holds the decryption key but must not learn x. The protocol proceeds:

1. **Bob blinds the value.** Given $E[x]$, Bob generates two random values a and b and forms a blinded ciphertext $E[y] = E[ax + b]$. Because the transformation $x \mapsto ax + b$ is linear (multiplication by the known-to-Bob constant a and addition of the known-to-Bob constant b), Bob can compute $E[y]$ from $E[x]$ using the additive HE operations. The random a and b mask the true value: y looks random to anyone who does not know a and b.
2. **Alice operates in the clear on the blinded value.** Bob sends $E[y]$ to Alice. Alice decrypts to obtain y (which tells her nothing about x, since $y = ax + b$ is a randomized affine image of x), applies a function g to it, obtains $g(y)$, re-encrypts, and sends $E[g(y)]$ back to Bob.
3. **Bob unblinds to recover the desired result.** From $E[g(y)]$, using his knowledge of a and b, Bob extracts $E[f(x)]$.



The technique **works if** the target function can be written in the form

$$f(x) = \alpha(a,b), g(y) + \beta(a,b), x + \gamma(a,b),$$

that is, if $f(x)$ can be expressed as a linear combination of $g(y)$, x, and a constant, where the coefficients α, β, γ depend only on the blinding parameters a and b (which Bob knows). The additional requirement is that it must be **difficult, ideally impossible, to recover x from y**, which is what guarantees that Alice learns nothing when she decrypts the blinded value. When these two conditions hold, the non-linear function is computed securely despite the underlying scheme being only additively homomorphic.

Example: Squaring an Encrypted Number

The squaring function $f(x) = x^2$ is the simplest non-trivial non-linear function, and it illustrates the blinding mechanism concretely. Bob holds $E[x]$ and wishes to obtain $E[x^2]$ without ever learning x in the clear and without Alice learning x either.

Step 1 — Bob blinds with an additive offset. Bob picks a random b and forms

$$E[y] = E[x + b] = E[x], E[b],$$

using the additive homomorphism (the product of ciphertexts encrypts the sum of plaintexts). Here the blinding is the special case $a = 1$, so $y = x + b$. Bob sends $E[y]$ to Alice.

Step 2 — Alice squares the blinded value. Alice decrypts to get $y = D[E[y]]$, which reveals nothing about x because b is random and unknown to her. She computes

$$g(y) = y^2 = (x + b)^2 = x^2 + b^2 + 2xb,$$

re-encrypts to obtain $E[g(y)]$, and returns it to Bob.

Step 3 — Bob unblinds to isolate x^2. Bob now removes the extra terms. Since $g(y) = x^2 + b^2 + 2bx$, it follows that $x^2 = g(y) - b^2 - 2bx$. Bob computes the encryption of this directly:

$$E[x^2] = E[, g(y) - b^2 - 2bx,] = E[g(y)]; E[-b^2]; E[x]^{-2b}.$$

Every factor on the right is something Bob can form: $E[g(y)]$ came from Alice; $E[-b^2]$ is an encryption of a constant Bob knows (he chose b); and $E[x]^{-2b}$ is the original encrypted value raised to the known exponent $-2b$, which is a multiplication-by-known-constant operation permitted by additive HE. The product of ciphertexts realizes the sum of the three plaintext terms, yielding exactly $E[x^2]$. Bob has obtained the encryption of the square of the secret, Alice never learned x, and the only non-linear step, the actual squaring, was performed by Alice on a value fully masked by the random b.

This example is a template. The same blind-operate-unblind pattern extends to other non-linear functions that admit the required affine decomposition, and it is the workhorse that lets additive homomorphic encryption reach beyond the strictly linear operations it supports natively. The remaining ambition, however, is grander still: to compute *arbitrary* functions, including unbounded multiplications of secrets by secrets, on encrypted data without any interaction. That ambition leads to fully homomorphic encryption, which is the subject of the next section.

Toward Fully Homomorphic Encryption

The additive schemes of the previous section let us compute many additions and, through interactive blinding, selected non-linear operations. But what people *really* wanted was the ability to do **arbitrary**

computing on encrypted data, non-interactively and without limit. This section traces the path from that wish to its realization in Craig Gentry's **fully homomorphic encryption (FHE)**, by way of a concrete integer-based scheme whose central obstacle, growing noise, motivates the key idea of bootstrapping.

Why Both Sums and Products Are Needed

Arbitrary computation on encrypted data requires the ability to compute **both sums and products** on the *same* encrypted dataset. A scheme that offers only one of the two, as RSA (multiplicative only) and Paillier (additive only) do, is fundamentally limited. The reason is a basic fact of Boolean logic.

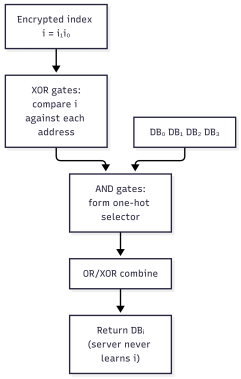
Every bit operation can be built from two gates, and these two gates correspond exactly to the two arithmetic operations modulo 2. **Addition modulo 2 is the XOR gate**, and **multiplication modulo 2 is the AND gate**:

XOR (SUM mod 2)		**AND (PRODUCT mod 2)**	
0 XOR 0	0	0 AND 0	0
1 XOR 0	1	1 AND 0	0
0 XOR 1	1	0 AND 1	0
1 XOR 1	0	1 AND 1	1

The XOR truth table is precisely addition of bits modulo 2 (note that $1 \oplus 1 = 0$, matching $1 + 1 = 2 \equiv 0 \pmod 2$), and the AND truth table is precisely multiplication of bits.

The decisive fact is that **XOR, AND is Turing-complete**: any Boolean function whatsoever can be expressed as a combination of XOR and AND gates. Consequently, a cryptosystem that can homomorphically evaluate *both* a bitwise sum (XOR) and a bitwise product (AND) can, by composing these gates, evaluate **any** function on encrypted bits. This is why the dual capability is the holy grail; it converts "compute on encrypted data" from a collection of special-case tricks into a universal capability.

As a concrete illustration that arbitrary tasks reduce to such gates, consider **indexing a database**. A small database holds entries DB_0, DB_1, DB_2, DB_3, and we wish to retrieve entry DB_i given a two-bit index $i = i_1 i_0$. The retrieval is realized by a circuit of XOR and AND gates: the index bits are compared (via XOR) against each address, the comparisons are combined (via AND) to produce a selector that is 1 for exactly the matching entry, and the selected entry is routed to the output (via a final OR/XOR combination). If all of this is expressed as XOR and AND gates and those gates can be evaluated homomorphically, then the entire database lookup can be performed on encrypted data, retrieving the right record without the server learning which record was requested.



Fully Homomorphic Encryption: Cryptography's Holy Grail

An encryption scheme that supports *both* unbounded addition and unbounded multiplication on ciphertexts is called **Fully Homomorphic Encryption**, and it was long regarded as **cryptography's holy grail**. People tried to construct it for years and years with no success. The breakthrough came in **October 2008**, when **Craig Gentry** devised the first fully homomorphic encryption scheme, a result dramatic enough to be reported widely in the press as a landmark in cryptography. The remainder of this section develops the ideas behind such a scheme, using a simple integer-based construction that exhibits all the essential phenomena.

A Concrete Somewhat-Homomorphic Scheme Over the Integers

The scheme operates on single bits and is built entirely from integer arithmetic, which makes its homomorphic properties easy to verify by hand.

Key generation. The **secret key is a large odd number** p.

Encryption of a bit b**.** To encrypt a single bit $b \in 0, 1$:

1. Pick a (random) **"large" multiple of** p, written $q \cdot p$.
2. Pick a (random) **"small" number** $2r + b$. Note that this quantity is **even if** $b = 0$ **and odd if** $b = 1$, because $2r$ is always even and adding b shifts the parity to match the message bit.
3. The **ciphertext** is

$$c = q \cdot p + 2r + b.$$

The term $2r + b$ is called the **noise**. The ciphertext is thus "a large multiple of the key, plus a small noisy term that secretly carries the bit in its parity."

Decryption of a ciphertext c**.** Decryption recovers the bit by first recovering the noise and then reading its parity. **Taking** $c \bmod p$ **recovers the noise**, because the $q \cdot p$ term vanishes under reduction modulo p, leaving exactly $2r + b$ (provided the noise is small enough, a crucial caveat developed below). One then takes this result modulo 2: since $2r + b \equiv b \pmod 2$, the parity of the recovered noise *is* the message bit. Geometrically, one can picture the integer line marked at multiples of p ($\ldots, -2p, -p, 0, p, 2p, 3p, \ldots$); a ciphertext sits near a multiple of p, and its small displacement from the nearest multiple is the noise $2r + b$ whose parity is read off as b.

Security: The Approximate GCD Assumption

Why is this secure, and why is the noise term indispensable? Consider first what would happen **if there were no noise** (take $r = 0$ and, for the argument, $b = 0$). Then a ciphertext would simply be $q \cdot p$, an exact multiple of the secret key. Given **two encryptions of 0**, namely $q_1 p$ and $q_2 p$, an attacker could **recover the secret key** p immediately by computing their greatest common divisor:

$$p = \gcd(q_1 p, ; q_2 p).$$

This is the fatal **GCD attack**, and it would break the scheme completely.

The noise is precisely what defeats this attack. **If there is noise**, each ciphertext is only *approximately* a multiple of p (it is qp plus a small perturbation), so the GCD attack **no longer works**, since the gcd of two near-multiples reveals nothing about p. It is believed that **no attack works** in this case. This hardness is codified as the **approximate GCD assumption**: recovering p from a collection of integers

that are each close to (but not exactly) a multiple of p is computationally infeasible. The security of the scheme rests on this assumption.

For the scheme to be secure and correct, the noise parameters must be chosen carefully. The noise r must be chosen so that the perturbed ciphertext stays nearer to its own multiple of p than to neighboring multiples: taking the base of representation $q = 2$ as a reference, the chosen noise is required to keep the value closer to $2p$ (its intended multiple, in the illustrative placement) than to p, which demands $r \ll p$. More precisely, the squared maximum noise must satisfy $r_{\max}^2 \ll \frac{p}{2}$, a condition whose significance becomes clear once we see how noise grows under multiplication.

Homomorphic Operations and the Growth of Noise

The reason this scheme is homomorphic is that ordinary integer addition and multiplication of ciphertexts correspond to XOR and AND of the underlying bits. We verify each in turn, tracking what happens to the noise.

XORing Two Encrypted Bits (Addition)

Let the two ciphertexts be

$$c_1 = q_1 p + (2r_1 + b_1), \qquad c_2 = q_2 p + (2r_2 + b_2).$$

Adding them gives

$$c_1 + c_2 = p, (q_1 + q_2) + 2, (r_1 + r_2) + (b_1 + b_2).$$

The highlighted part, $2(r_1 + r_2) + (b_1 + b_2)$, is the new noise, and its parity is what decryption reads. That parity is **odd if** $b_1 = 0, b_2 = 1$ **or** $b_1 = 1, b_2 = 0$, and **even if** $b_1 = 0, b_2 = 0$ **or** $b_1 = 1, b_2 = 1$. This is exactly the XOR truth table:

$$\text{lsb} = b_1 \oplus b_2.$$

So the sum of ciphertexts decrypts to the XOR of the plaintext bits. The effect on the noise is mild: the new noise is roughly $2 \times$ (initial noise), that is, the two noises simply **add**. Addition grows the noise additively, which is manageable.

ANDing Two Encrypted Bits (Multiplication)

Multiplying the same two ciphertexts, and regrouping all terms that are multiples of p into a single coefficient, gives

$$c_1 c_2 = p, (c_2 q_1 + c_1 q_2 - q_1 q_2) + 2, (r_1 r_2 + r_1 b_2 + r_2 b_1) + b_1 b_2.$$

The least significant bit of the noisy part is now $b_1 b_2$, which is exactly the AND truth table:

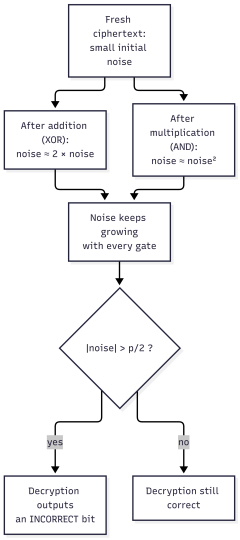
$$\text{lsb} = b_1 \wedge b_2.$$

So the product of ciphertexts decrypts to the AND of the plaintext bits. But the cost is severe. The new noise contains the term $r_1 r_2$ and its companions, so the noise is now on the order of (initial noise)^2. Multiplication **squares** the noise.

The Noise Grows — and Why It Matters

Putting the two together:

- For addition: $c_1 + c_2$ has **noise** $= 2 \times$ (initial noise).
- For multiplication: $c_1 c_2$ has **noise** $=$ (initial noise)^2.



So what is the problem with growing noise? Recall that decryption works by taking $c \bmod p$ to recover the noise, which is valid only while the noise stays within the "basin" of the correct multiple of p. **If $|\text{noise}| > p/2$, then decryption will output an incorrect bit**, because the noise has grown large enough to push the ciphertext past the midpoint toward a neighboring multiple of p, and the modular reduction locks onto the wrong one. Concretely, with marks at multiples of 17, a ciphertext intended to carry a small noise of 3 might, after too many operations, drift to an effective displacement of -14 relative to the next multiple; once the noise crosses the $p/2$ boundary, the recovered parity no longer corresponds to the true bit. This is exactly why the initial noise had to be chosen so small ($r_{\max}^2 \ll p/2$): a single multiplication squares the noise, and the budget $p/2$ must accommodate that growth.

What We Have So Far: Somewhat Homomorphic Encryption

Taking stock: we can do **lots of additions** and **some multiplications** before the noise exceeds $p/2$ and corrupts the result. A scheme with this bounded capability is called a **"somewhat homomorphic" encryption** scheme. This is already **enough to do many useful tasks**, for example database search and spam filtering, which require only a limited multiplicative depth. But full homomorphism promised *unbounded* computation of both sums and products, and the noise ceiling stands squarely in the way.

The Bootstrapping Method

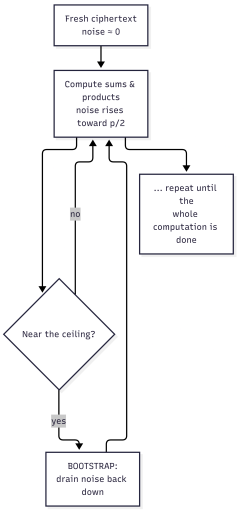
The route from "somewhat" to "fully" homomorphic rests on a striking principle:

> **If you can go a (large) part of the way, then you can go all the way.**

That is, a scheme that can evaluate a sufficiently rich but still bounded set of operations can be upgraded to evaluate *everything*. Starting from RSA-and-friends (many multiplications, zero additions, in the historical framing) we have reached a somewhat homomorphic scheme (many additions, some multiplications). The question is how to cross the remaining gap to fully homomorphic (many additions *and* many multiplications).

The obstacle is purely the noise. Picture the noise as the water level in a vessel whose rim is at noise $= p/2$ and whose bottom is at noise $= 0$. A fresh ciphertext starts with a little **initial noise** near the bottom. After **some sums and products**, the level rises toward the rim. Once it reaches $p/2$,

decryption fails. If only we could install a **"valve" at a fixed height** that drains the vessel back down before it overflows, we could keep computing indefinitely, draining whenever we approach the ceiling, and **repeat until done**.



How to Reduce the Noise

What operation could serve as this valve, something that **kills all the noise and recovers the message**? There is exactly one such operation: **decryption**. Decryption, by definition, strips away the noise and returns the clean message bit b, taking a noisy ciphertext $\text{Ctxt} = \text{Enc}(b)$ together with the **secret key** and producing the noiseless b.

The obvious objection is immediate: **"But I can't give the secret key out for free!"** If the party performing the computation on ciphertexts had the plaintext secret key, it could simply decrypt everything and there would be no privacy at all. So decryption cannot be applied in the clear.

The resolution is elegant: **I can release** $\text{Enc}(\text{secret key})$, an *encryption* of the secret key, which is safe to publish (it reveals nothing without the key to decrypt it). The **key observation** is that the input $\text{Enc}(b)$ and the output $\text{Enc}(b)$ of a homomorphic decryption have **different noise levels**. Concretely, we run the decryption procedure *homomorphically*: instead of feeding the plaintext secret key into the decryption circuit, we feed $\text{Enc}(\text{secret key})$ and evaluate the decryption circuit homomorphically on the noisy ciphertext. The output is a *fresh* encryption of the same bit b, but with the noise reset to the low level characteristic of the decryption circuit's own evaluation, rather than the high level the input had accumulated.

Bootstrapping and Gentry's Theorem

This procedure is **bootstrapping**: it **requires homomorphically evaluating the decryption circuit**. The valve is built by taking a dangerously noisy ciphertext and homomorphically "decrypting" it under the encrypted key, producing a new ciphertext of the same message whose noise has dropped back to a safe level, from which further computation can proceed.

This leads directly to **Gentry's "bootstrapping theorem"**:

> *If an encryption scheme can evaluate its own decryption circuit, then it can evaluate everything.*

The condition is self-referential and precise: the scheme need not be able to evaluate arbitrary circuits from the outset; it only needs enough homomorphic capacity (enough multiplicative depth before the noise ceiling) to run *its own decryption procedure plus a little more*. Once it can do that, it can refresh any ciphertext whenever the noise grows too large, and therefore it can evaluate circuits of unbounded

size, becoming fully homomorphic. This is the conceptual engine that turns a somewhat homomorphic scheme into a fully homomorphic one.

"I Can Do Everything!" — In Theory, and the Price

With bootstrapping, **in theory we can do everything**: any function on encrypted data, to any depth. The same construction can also be implemented **on integer numbers** rather than single bits, extending its reach to arithmetic workloads.

But the complexity? The practical cost is enormous, and it is important to appreciate its scale. The sizes of the keys and ciphertexts grow sharply with the security parameter λ and with the base of the plaintext space. For illustration, representative sizes (not counting the additional cost of publishing Enc(secret key) needed for bootstrapping) run roughly as follows:

λ	Base	Secret Key	Cipher	Public Key
10	2	10 B	0.1 KB	0.1 MB
10	2^{150}	1519 B	18.8 KB	21.7 MB
15	2^{150}	3.5 KB	1081 KB	1056 MB
20	2	45 B	66 KB	216 MB
20	2^{150}	6.7 KB	10 MB	32 GB

The trend is unmistakable: as the security parameter and plaintext base increase, ciphertexts swell from fractions of a kilobyte into megabytes, and public keys balloon from megabytes into tens of gigabytes, and this is *before* adding the encrypted secret key that bootstrapping requires. Fully homomorphic encryption is therefore a profound theoretical achievement, genuinely able to compute anything on encrypted data, while remaining, in its basic form, extraordinarily expensive in both storage and computation. Making FHE practical is the central engineering challenge that has driven the field since Gentry's breakthrough.

This completes the treatment of homomorphic approaches to secure computation. The next cryptographic technique for secure two-party computation takes an entirely different route, encoding the function as a Boolean circuit whose gates are individually encrypted: **garbled circuits**.

Garbled Circuits

Fully homomorphic encryption lets a single party compute any function on encrypted data, but at a steep cost and in a non-interactive, single-holder setting. A different and historically earlier approach to general secure *two-party* computation takes the circuit itself as the object to be encrypted. This is **Yao's Garbled Circuits**, and it realizes the vision raised earlier: evaluating any Boolean function on two parties' private inputs while revealing only the output.

Yao's Garbled Circuit Idea

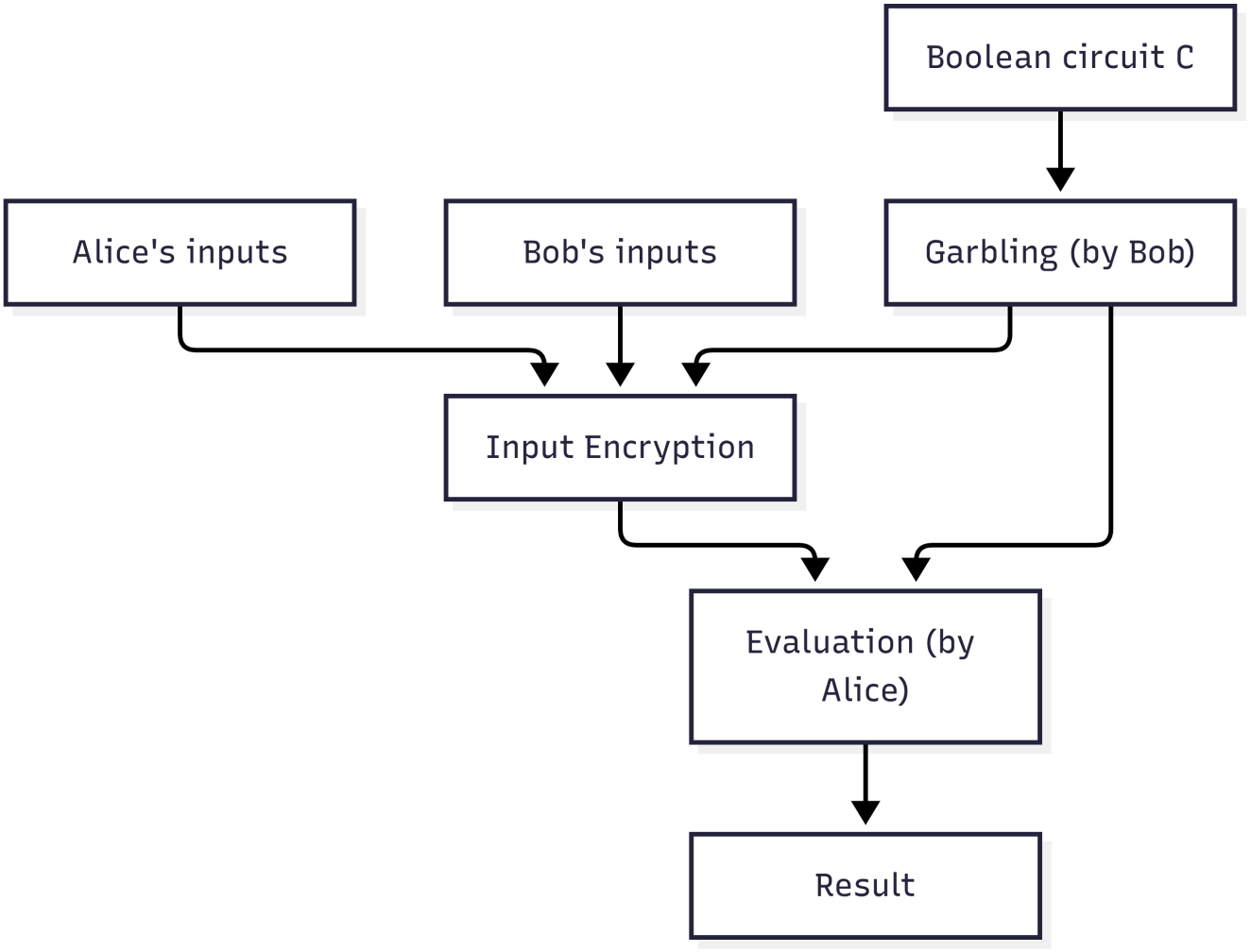
The starting point is to **represent a functionality f as a Boolean circuit C**. The circuit $C(x, y)$ is then **computed gate by gate, from the input wires to the output wires**, where x and y are inputs supplied by two different parties and must each be kept private from the other. To describe the construction precisely, we distinguish **four wire typologies**:

- **Circuit-input wires**, which carry the parties' private input bits into the circuit;

- **Circuit-output wires**, which carry the final result out;
- **Gate-input wires**, the wires entering an individual gate;
- **Gate-output wires**, the wires leaving an individual gate.

The goal Yao set is to *provide a method of computing a circuit so that the values obtained on all wires other than the circuit-output wires are never revealed.* If every intermediate value remains hidden and only the final output wires are opened, then neither party learns anything about the other's input beyond what the output itself discloses. The mechanism that achieves this hiding is the encryption, or "garbling," of wire values and gate behavior.

In the standard two-party arrangement, one party is the **garbler** (call him Bob) and the other is the **evaluator** (call her Alice). The garbler **encrypts, or "garbles," the nodes and transitions of the Boolean circuit** and sends the resulting **garbled circuit** to the evaluator. Both parties' **private inputs are mapped to garbled secrets** and provided to the evaluator. Crucially, the **evaluator follows only a single evaluation path**, the one determined by the circuit structure and the specific input attribute vector, and from this she **obtains the result**. If the garbler is also entitled to an output, that requires a simple additional step at the end.



Wire Encryption

The garbling begins at the level of individual wires. **For every wire in the circuit, two random values are specified, one representing the bit 0 and the other representing the bit 1.** Concretely, if w is the label of some wire, the garbler chooses two secret keys, k_w^0 and k_w^1, where k_w^0 encodes the value 0 on that wire and k_w^1 encodes the value 1. These are long random strings; for instance, the two secrets attached to a wire might be the values `573ac2a8f4c7d29a833b` (for 0) and

`9d283a6203bc82dd5e6` (for 1). The essential property is that the two strings are chosen independently at random and carry no visible mark of which bit they represent.

Wire value	Wire secret
0	`573ac2a8f4c7d29a833b`
1	`9d283a6203bc82dd5e6`

Wire Evaluation: Why the Hiding Works

The security of the scheme at the wire level is immediate once we see what the evaluator possesses. Suppose the evaluator obtains, for wire w, the key k_w^σ, which represents the bit σ. The evaluator **cannot determine whether $\sigma = 0$ or $\sigma = 1$**. The reason is that **both k_w^0 and k_w^1 are identically distributed**: they are drawn from the same random distribution, so a bare string such as `029f375a0298c452e928` carries no information about whether it is "related to 0 or 1." Holding one of the two keys tells the evaluator nothing about which logical bit it encodes, and, just as importantly, **it does not help her determine the other key $k_w^{1-\sigma}$**. She possesses exactly one random-looking string per wire and can neither interpret it nor derive its counterpart. This is the primitive that, composed across all the gates, keeps every intermediate value secret.

Gate Garbling

The challenge is to let the evaluator propagate these opaque keys through a gate, producing the correct output-wire key, without ever learning the bits involved. The technique is to **map random input values to random output values** by building, for each gate, an encrypted truth table.

Given two random secrets for each input- and output-gate wire, the garbler proceeds as follows for a gate g with input wires w_1, w_2 and output wire w_3:

1. For each of the four combinations of input bits, **encrypt the corresponding output secret with the corresponding input secrets** (a double encryption, under both input-wire keys).
2. **Place the four encrypted output secrets in a table whose rows are randomly permuted**, so that the position of a row reveals nothing about which input combination it corresponds to.

For an AND gate, whose output is 1 only when both inputs are 1, the garbled table consists of the four ciphertexts (before permutation):

$$E_{k_1^0}!\left(E_{k_2^0}(k_3^0)\right) \qquad E_{k_1^0}!\left(E_{k_2^1}(k_3^0)\right) \quad E_{k_1^1}!\left(E_{k_2^0}(k_3^0)\right) \qquad E_{k_1^1}!\left(E_{k_2^1}(k_3^1)\right)$$

Each row encrypts the output-wire key appropriate to that input combination: the first three rows encrypt k_3^0 (output 0), because AND yields 0 unless both inputs are 1, while the last row encrypts k_3^1 (output 1), the single case where both inputs are 1. The decisive design property is that **given the correct input secrets, only one row can be correctly decrypted.** A party holding input keys k_1^α and k_2^β can decrypt exactly the row encrypted under those two keys and obtain $k_3^{g(\alpha,\beta)}$, the output-wire key corresponding to the gate's output on those inputs; the other three rows remain indecipherable because she lacks the keys that lock them.

Gate Evaluation

Evaluation of a single garbled gate is the mirror image of garbling. A garbled AND row has the form

$$E_{k_1^\alpha}!\left(E_{k_2^\beta}!\left(k_3^{,g(\alpha,\beta)}\right)\right),$$

a double encryption of the output key under the two relevant input keys. The evaluator, holding k_1^α and k_2^β, obtains the gate's output key by peeling off the two layers of encryption:

$$k_3^{,g(\alpha,\beta)} = D_{k_2^\beta}!\left(D_{k_1^\alpha}(c)\right),$$

where c is a row of the garbled gate. The security guarantees are precisely those we require. **Given the correct input secrets, only one row decrypts correctly**, so from k_1^α and k_2^β the evaluator obtains $k_3^{g(\alpha,\beta)}$ and nothing else. She **obtains no information about the real values of α, β, or $g(\alpha,\beta)$**, because every key is an unlabeled random string. And she **cannot obtain the complementary keys** $k_1^{1-\alpha}$, $k_2^{1-\beta}$, or $k_3^{1-g(\alpha,\beta)}$: she is confined to the single evaluation path consistent with the actual inputs, unable to explore the counterfactual branches that would reveal the gate's full truth table. By repeating this gate-by-gate, the evaluator propagates one output key per wire all the way to the circuit-output wires, computing the entire circuit blindly.

Alice's Output Decryption

At the circuit-output wires, the evaluator holds output-wire *keys*, not the output *bits* themselves, so a final translation step is needed. The standard method is to **provide decryption tables to Alice, one decryption table for each circuit-output wire.** Such a table simply lists the correspondence between each output key and the bit it represents:

$$(0, k_w^0), \qquad (1, k_w^1).$$

With this table, Alice matches the output key she computed against the two entries and reads off the final bit.

A natural question is whether handing Alice this correspondence for the output wires compromises anything "upstream." It does not. Consider the garbled table of the final gate, whose rows are

$$E_{k_1^0}!\left(E_{k_2^0}(k_3^0)\right), \quad E_{k_1^0}!\left(E_{k_2^1}(k_3^0)\right), \qquad E_{k_1^1}!\left(E_{k_2^0}(k_3^0)\right), \quad E_{k_1^1}!\left(E_{k_2^1}(k_3^1)\right).$$

At the output decryption stage Alice may well know *all the output secrets* (the k_3 values and their bit meanings), but **it is impossible to retrieve the missing input secrets** from them. Knowing which output key means 0 and which means 1 does not let her invert the encryptions to recover the input-wire keys she never received, so the privacy of the intermediate and input wires is preserved.

Alternative Output Decryption

A simpler alternative dispenses with the separate decryption table for the output wires: the **output gates can directly encrypt the output value** itself rather than an output-wire key. For a final AND gate, the garbled table becomes

$$E_{k_1^0}!\left(E_{k_2^0}(0)\right), \quad E_{k_1^0}!\left(E_{k_2^1}(0)\right), \qquad E_{k_1^1}!\left(E_{k_2^0}(0)\right), \quad E_{k_1^1}!\left(E_{k_2^1}(1)\right).$$

Here the decryptable row yields the plaintext output bit (0 or 1) directly, so no further lookup is required.

Other Outputs

The treatment so far gives the output to Alice, the evaluator. Two further cases arise depending on who is entitled to learn the result.

For **Bob's output** (the garbler is to receive the result), **Alice sends the output secrets to Bob**. Because Bob generated all the wire secrets during garbling, he knows the bit meaning of each output key, so receiving the raw output keys lets him recover his output bits.

For **both Alice's and Bob's outputs**, **Alice decrypts the output secrets and sends the output bits to Bob**. She could instead send the secrets themselves, but **transmitting the bits is more efficient** than transmitting the full-length secrets. This step relies on the protocol's trust model: recall that in this setting **the parties are semi-honest and follow the protocol**, so Alice will faithfully forward Bob's correct output bits rather than fabricating them.

Summary of the Security Argument So Far

Pulling the pieces together yields the central claim of the construction. **Given the set of keys associated with the parties' inputs, it is possible to "compute the circuit" by opening the garbled gates one at a time**, propagating a single output key per wire from inputs to outputs. And **the computation of the circuit reveals absolutely no information beyond the output itself.** This follows from two facts established above: first, **the keys are not labeled, so it is impossible to know whether a given key is associated with 0 or with 1**, which hides every intermediate value; and second, this guarantee holds **under the assumption that the keys associated with the circuit-input wires are obtained in an "oblivious manner" that does not reveal their association with the parties' inputs.** This last assumption is not yet justified: it raises the question of how the evaluator can acquire the correct input keys for her own private input *without* the garbler learning that input, and without the garbler simply handing over both keys. Resolving that question is the role of oblivious transfer, which the next section develops.

Supplying Inputs and Optimizing the Garbled Circuit

The garbled-circuit machinery of the previous section left one question open: how does the evaluator obtain the garbled key corresponding to *her own* private input bit, without the garbler learning that bit and without receiving both keys? This section resolves that through **oblivious transfer**, assembles the complete construction, and then presents the classic efficiency improvements that make garbled circuits practical.

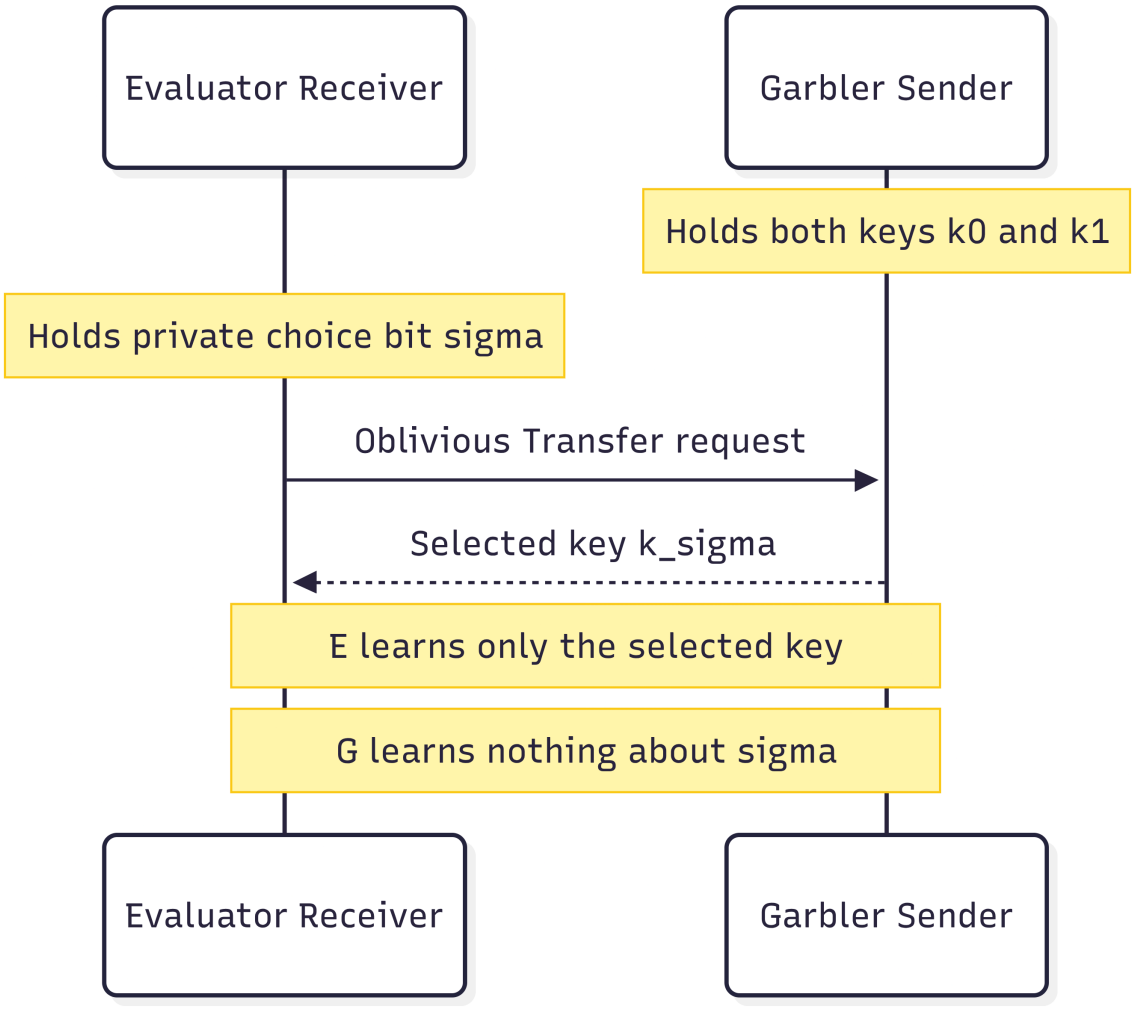
Circuit Input Wires

The input keys enter the circuit differently for the two parties, and the asymmetry is the crux.

For the **garbler's inputs**, the matter is simple. The garbler **sends the secrets associated with its own input bits** directly. If one of the garbler's input wires w takes the value σ, the garbler simply sends the single key k_w^σ to the evaluator. This leaks nothing about the garbler's input, because the evaluator cannot tell whether the random-looking key she received represents 0 or 1.

For the **evaluator's inputs**, a dilemma arises. The garbler **cannot send both k_w^0 and k_w^1**, because possessing both keys for an input wire would let the evaluator explore both branches of the circuit and break privacy. Yet the evaluator **cannot simply send her input bit to the garbler** to request the matching key k_w^σ, because that would reveal her private input. The evaluator needs to obtain *exactly one* of the two keys, the one matching her bit, while (i) the garbler does not learn *which* key she got, and (ii) the evaluator does not learn the other key. This is precisely the specification of **oblivious transfer**

(OT). The two parties therefore **engage in parallel Oblivious Transfers, one per evaluator-input wire, with the garbler acting as Sender and the evaluator as Receiver.**



Oblivious transfer is thus the bridge that supplies the evaluator's input keys in the "oblivious manner" that the security argument assumed.

How to Implement Oblivious Transfer

A concrete 1-out-of-2 OT can be built from an additively homomorphic encryption scheme, reusing the blinding ideas from homomorphic encryption. The receiver (Alice) holds a choice bit b and wishes to obtain k_b from the sender's pair (k_0, k_1) without revealing b and without learning k_{1-b}.

1. **Alice sends** $\mathrm{Enc}(b)$ to the sender, an encryption of her choice bit under her own key. The sender cannot read b.
2. **The sender computes** $\mathrm{Enc}(k_b)$ **homomorphically** and returns it. Using the homomorphic structure, the sender forms

$$\mathrm{Enc}(k_b) = \mathrm{Enc}(b)^{k_1} \cdot \left(\mathrm{Enc}(1), \mathrm{Enc}(b)^{-1}\right)^{k_0} = \mathrm{Enc!}\left(k_1 b + k_0(1-b)\right).$$

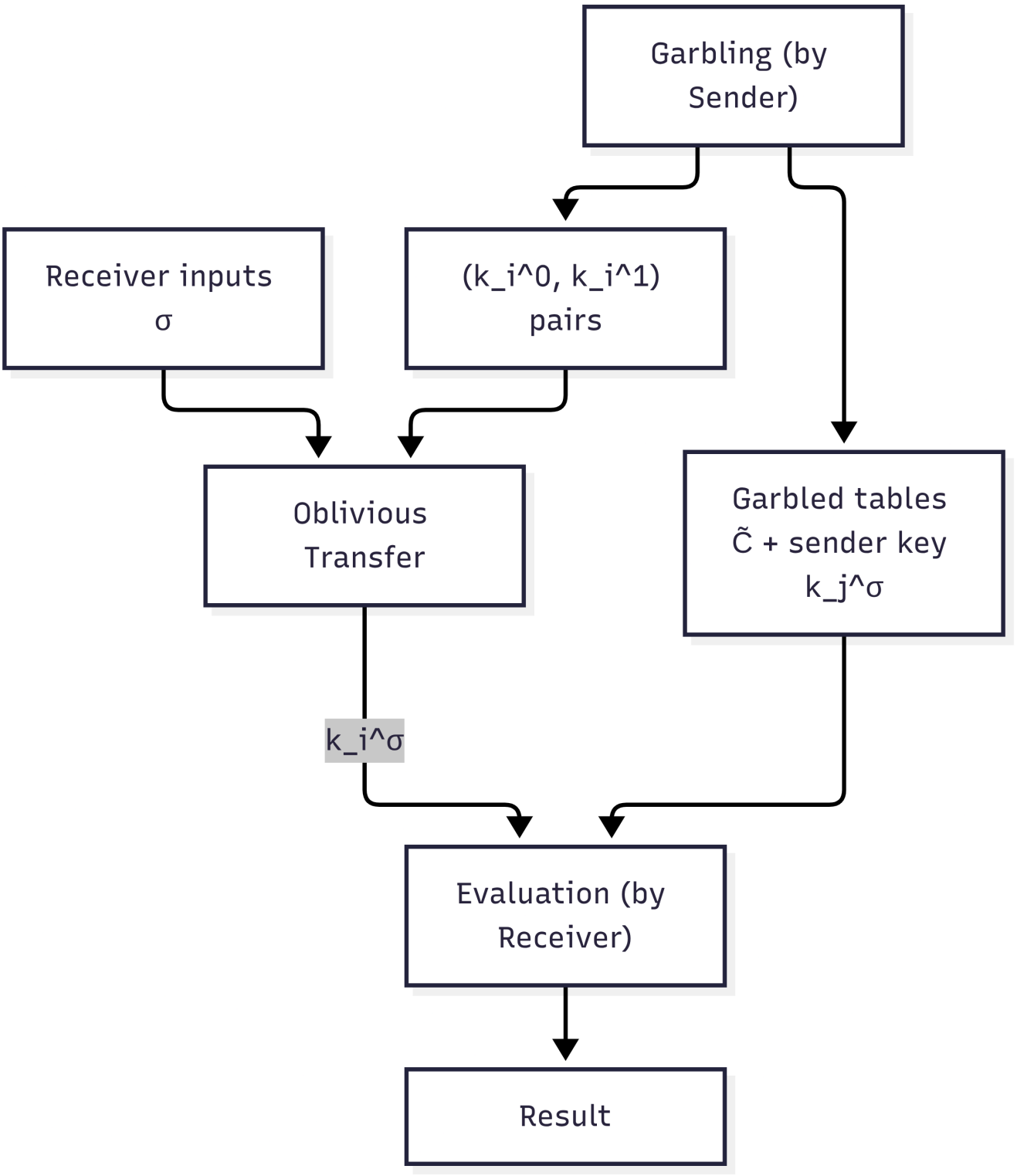
   The exponent $k_1 b + k_0(1-b)$ evaluates to k_1 when $b = 1$ and to k_0 when $b = 0$, so the returned ciphertext encrypts exactly the chosen key. The construction works because $\mathrm{Enc}(b)$ raised to k_1 contributes the $k_1 b$ term, while the factor $\mathrm{Enc}(1), \mathrm{Enc}(b)^{-1} = \mathrm{Enc}(1-b)$ raised to k_0 contributes the $k_0(1-b)$ term.
3. **Alice decrypts** $\mathrm{Enc}(k_b)$ to recover k_b, and only k_b.

The sender never learns b (he only ever saw $\mathrm{Enc}(b)$), and Alice never learns k_{1-b} (the ciphertext she received encrypts only the chosen key). This is a correct, if not the most efficient, OT; in practice **more efficient protocols exist**, and for many input bits one uses *OT extension* to amortize the cost.

Garbled Circuit Construction: The Complete Protocol

With OT in hand, the full garbled-circuit protocol between a **sender** (garbler) and a **receiver** (evaluator) assembles as follows. Given a circuit C, the sender generates **two garbled secrets for each wire** and **a permuted garbled table for each gate**. Then:

1. The sender **transmits the garbled tables** $\tilde{C}$ to the receiver.
2. The sender **transmits the garbled secrets associated with his own inputs**, k_j^σ, to the receiver (one key per sender-input wire).
3. The receiver **obtains the garbled secrets corresponding to its own inputs**, k_i^σ, **by batch oblivious transfer**, drawing the correct key from each pair (k_i^0, k_i^1) without revealing its input.
4. The receiver **evaluates the circuit** using the secrets received directly, the secrets obtained via OT, and the secrets obtained from the evaluation of previous garbled tables, propagating one key per wire to the circuit-output wires.



Improvement 1: Point-and-Permute

The first classic optimization, due to Naor, Pinkas, and Sumner, is **Point-and-Permute**. Its purpose is to let the evaluator immediately identify *which* row of a garbled gate to decrypt, rather than trying all four. For each wire, the garbler generates two keys randomly chosen in $0,1^t$ together with a **permutation bit** $\pi \in 0,1$. The garbler then associates to each wire's two keys a pair consisting of the key and a "pointer" bit derived from π. For wires w_1, w_2, w_3:

$$w_1 : \langle k_1^0, \pi_1 \rangle,\ \langle k_1^1, 1 \oplus \pi_1 \rangle; \qquad w_2 : \langle k_2^0, \pi_2 \rangle,\ \langle k_2^1, 1 \oplus \pi_2 \rangle; \qquad w_3 : \langle k_3^0, \pi_3 \rangle,\ \langle k_3^1, 1 \oplus \pi_3 \rangle.$$

The garbler creates the garbled tables as usual **but now also includes the permutation bits** in the encrypted payload, so that each row has the form (writing $\pi^0 = \pi$ and $\pi^1 = 1 \oplus \pi$):

$$c_{\pi_1^\alpha,\pi_2^\beta} = E_{k_1^\alpha}!\left(E_{k_2^\beta}!\left(k_3^{;g(\alpha,\beta)},|,\pi_3^{;g(\alpha,\beta)}\right)\right).$$

The four rows are **permuted according to the permutation bits** of the two input wires, and the **client obtains the input permutation bits together with the input keys and directly decrypts the correct row.** Instead of attempting all four rows, the evaluator uses the two pointer bits as an index to jump straight to the one decryptable row, concatenated with the output pointer bit so the process chains to the next gate.

Improvement 2: High-Speed Encryption

The second improvement, from Lindell, Pinkas, and Smart, is **High-Speed Encryption**, which **replaces the original double encryption** of each row with a single masking by a hash-based key derivation function. Let t be the bit length of a key and s a string associated with a gate. A garbled row becomes

$$c_{\pi_1^\alpha,\pi_2^\beta} = \left\langle k_3^{;g(\alpha\beta)},,\pi_3^{;g(\alpha\beta)}\right\rangle \oplus \mathrm{KDF}!\left(k_1^\alpha,|,\pi_1^\alpha,|,k_2^\beta,|,\pi_2^\beta,|,s,;t+1\right),$$

where **KDF is a hash function**. The output key and its pointer bit are simply XORed with a pseudorandom pad derived by hashing the two input keys (with their pointer bits and the gate identifier). Decryption re-derives the same pad and XORs it off. This replaces two symmetric decryptions per row with a single hash evaluation, with the following efficiency:

	Complexity
Garbler (encryptions)	$4,
Evaluator (encryptions)	$1,
Circuit size (bits)	$4(t+1),

Here $|C|$ is the number of gates: the garbler performs four hash operations per gate, the evaluator one, and the transmitted circuit occupies $4(t+1)$ bits per gate.

Improvement 3: Free XOR

The third and most striking improvement, due to Kolesnikov and Schneider, is **Free XOR**, which makes XOR gates cost *nothing* in communication or (essentially) computation. The garbler randomly chooses a single global offset $R \in 0,1^t$ **for the whole circuit**. For each wire i, the garbler randomly chooses the zero-key $k_i^0 \in 0,1^t$, and the other key is defined as

$$k_i^1 = k_i^0 \oplus R.$$

That is, the two keys of every wire differ by the same global R. For an **XOR gate**, the output-wire keys are then computed directly as a function of the input-wire keys, with no garbled table at all:

$$k_3^0 = k_1^0 \oplus k_2^0, \quad \pi_3^0 = \pi_1^0 \oplus \pi_2^0; \qquad k_3^1 = k_1^0 \oplus k_2^0 \oplus R, \quad \pi_3^1 = \pi_1^0 \oplus \pi_2^0 \oplus 1.$$

The evaluator simply computes $k_3 = k_1 \oplus k_2$ and $\pi_3 = \pi_1 \oplus \pi_2$ on whatever keys she holds; because both keys share the offset R, this XOR of keys automatically yields the correct output key for the XOR of the underlying bits. **Non-XOR gates are encrypted as usual.** The consequence is dramatic: for an XOR gate the **communication is 0 bits and the computation is approximately 0**, whereas a general

gate g still requires $4(t+1)$ bits and four hashes for the server, one for the client. Because many useful circuits are XOR-heavy, this optimization yields large savings.

	XOR gate	**general gate g**
Communication	0 bits	$4(t+1)$ bits
Computation	≈ 0	Server: 4 hashes, Client: 1 hash

Standard Building-Block Circuits

In practice, higher-level functionalities are assembled from a library of standard circuits built out of these gates, designed to minimize the number of costly non-XOR gates. The common building blocks include an **Addition circuit (ADD)** composed of chained full adders with carry propagation, and an **improved 1-bit adder** that computes the sum bit s_i and the carry c_{i+1} using a single AND together with XOR gates; a **Subtraction circuit (SUB)** with a corresponding **improved 1-bit subtractor** producing difference bits d_i; a **Comparison circuit (CMP)** built from chained 1-bit comparators yielding a single comparison bit z (directly solving the millionaires' problem); a **Multiplexer circuit (MUX)** that selects between inputs under a control bit; and more elaborate blocks such as a **Minimum circuit (MIN)**, which arranges pairwise `min` blocks in a tree to find the smallest of many values along with its index, where each `min` block combines a comparator with multiplexers. These are the standard ingredients from which comparisons, selections, arithmetic, and distance computations are composed in garbled-circuit protocols, with the design goal always being to realize as much of the circuit as possible with free XOR gates and as few AND gates as possible.

Moving Computation Offline

A final practical lever exploits the fact that much of the garbled-circuit protocol does not depend on the actual input values, so it can be performed **before it is strictly necessary**. This is possible **whenever we know in advance which functionality we will evaluate.** The key observation is that **wire garbling and circuit garbling do not depend on the real inputs**: the garbler can generate all wire secrets and construct all garbled tables ahead of time (**offline computation**), and can even transmit the garbled circuit in advance (**offline circuit transmission**). Only the **oblivious transfer and the circuit evaluation depend on the input**, and so must occur online once the inputs are known. By shifting the heavy, input-independent work into an offline precomputation phase, the online phase, the part that happens when the parties actually want their answer, becomes much faster.

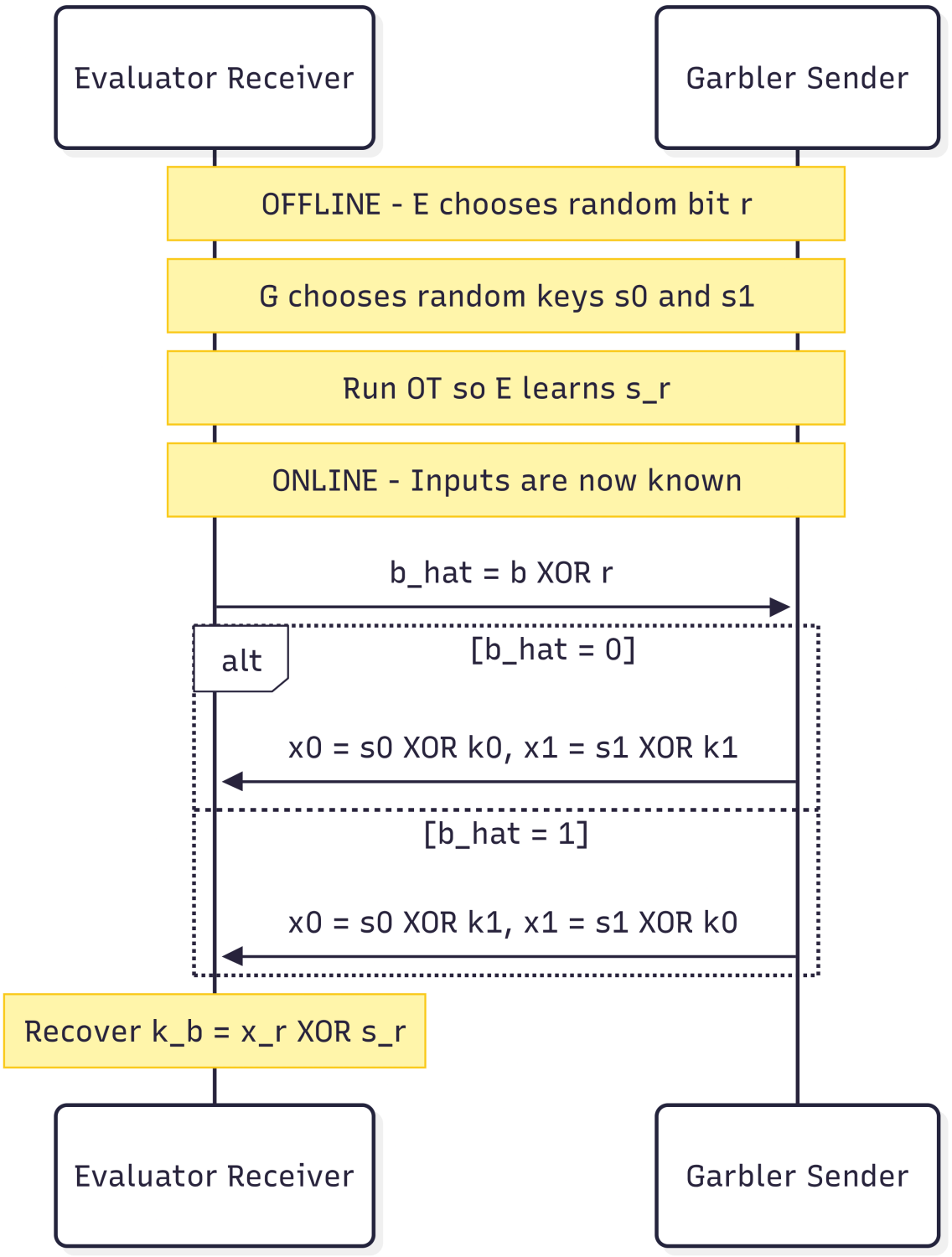
Can We Move OT Offline Too?

Even oblivious transfer, which seems inherently input-dependent, can be largely precomputed using a **random OT in the offline phase followed by a cheap correction in the online phase**. In the offline phase, before the real inputs are known, the receiver (evaluator) chooses a random bit $r \in 0,1$ and the sender chooses random strings $s_0, s_1 \in 0,1^t$; they run an OT so that the receiver learns s_r. Nothing here depends on the real input b or the real keys k_0, k_1.

In the online phase, once the receiver's true choice bit b and the sender's true pair (k_0, k_1) are fixed, only lightweight XOR corrections are exchanged:

1. The receiver **computes** $\hat{b} = b \oplus r$ and sends the single bit $\hat{b}$ to the sender.
2. The sender responds with a pair $\langle x_0, x_1 \rangle$ that re-aligns the precomputed random strings to the real keys: **if** $\hat{b} = 0$ then $\langle x_0, x_1 \rangle = \langle s_0 \oplus k_0, ; s_1 \oplus k_1 \rangle$, **else** (if $\hat{b} = 1$) $\langle x_0, x_1 \rangle = \langle s_0 \oplus k_1, ; s_1 \oplus k_0 \rangle$.

3. The receiver **computes** $k_b = x_r \oplus s_r$, recovering exactly the chosen key.



The correctness follows because the offline random OT already gave the receiver s_r, and the online messages are nothing more than one-time-pad shifts that carry the real keys under cover of the precomputed randomness. The expensive public-key operations of OT happen offline; the online OT reduces to sending one bit and a pair of masked strings and XORing. This completes the toolkit of garbled circuits and sets up a direct comparison with the homomorphic-encryption approach, which the next section undertakes before introducing a third paradigm, secret sharing.

Hybrid Protocols and Secret Sharing

We now have two complete engines for secure two-party computation, homomorphic encryption and garbled circuits, each with distinct strengths. This section compares them directly, shows how they can be combined into **hybrid protocols** that use the better tool for each sub-operation, and then introduces a third, structurally different paradigm: **secret sharing**.

Two-Party Cryptographic Primitives: HE versus GC

Homomorphic encryption and garbled circuits (here with the Free-XOR optimization) solve the same problem but with opposite performance profiles, and the choice between them depends on the operation at hand.

Recall their defining capabilities. **Homomorphic encryption** supports $[[a+b]] = [[a]]\,[[b]]$ and $[\![\,[\![c \cdot a]\!]\,]\!] = [\![\,[\![a]\!]\,]\!]^c$., that is, addition of two encrypted values and multiplication of an encrypted value by a known constant. **Garbled circuits** evaluate any function expressible as a Boolean circuit. The contrast across the key dimensions is as follows:

	Homomorphic Encryption	**Garbled Circuit (Free-XOR)**
Permits to compute	Linear operations (no interaction); products and square values (with interaction, via blinding)	Any function representable by a Boolean circuit (with interaction)
Data representation	Encryption of integer numbers (~1024 bits)	Encryption of each bit (~80 bits)
Dependence on data size	Quite independent	Highly related
Computation complexity	High	Small
Communication complexity	Small (only if interaction is required)	High
Suitable for	Sums, products, square values, filtering, linear transformations	Linear operations on data represented with few bits; any function that cannot be computed by HE

The trade-off is clear and complementary. Homomorphic encryption operates on whole integers, is largely independent of the bit-width of the data, and is **communication-cheap but computation-expensive**; it shines at linear algebra, filtering, and the occasional product or square. Garbled circuits operate bit by bit, so their cost is **highly sensitive to the data's bit length**, and they are **computation-cheap but communication-expensive**; they are indispensable for genuinely non-linear operations such as comparison, minimum selection, or division that HE cannot handle. A function-by-function comparison bears this out: operations such as sum/difference, product, scalar and matrix products, linear filtering, multiplexing, square value, and energy computation can be done efficiently with HE, whereas comparison, minimum selection, Euclidean and Hamming distance, counters, logarithm, and division are more naturally, or only, handled by GC. Several functions can be realized by either method, and the right choice depends on the input bit-length: for small bit-lengths GC bandwidth is competitive, while for larger bit-lengths HE's data-size independence wins.

Hybrid Protocols

Because the two primitives excel at different operations, a composite computation is often best served by **choosing HE or GC as a function of the single operation** being performed at each stage, linear stretches in HE, non-linear stages in GC. This requires stitching the two representations together, which creates the **necessity of an interface** to convert a value held in one encrypted form into the other.

The interface is built with **additive blinding**, the same masking technique seen earlier, and it must be evaluated along two axes: **efficiency** (the conversion should add little overhead) and **security** (the blinding must statistically hide the value being transferred). A typical pipeline alternates representations, HE, then an interface, then GC, then another interface, then HE, applying each primitive where it is strongest.



From HE to GC concretely, consider converting an encrypted value $[[x]]$ held by the client into garbled-circuit inputs for a subtraction block SUB^{ℓ}:

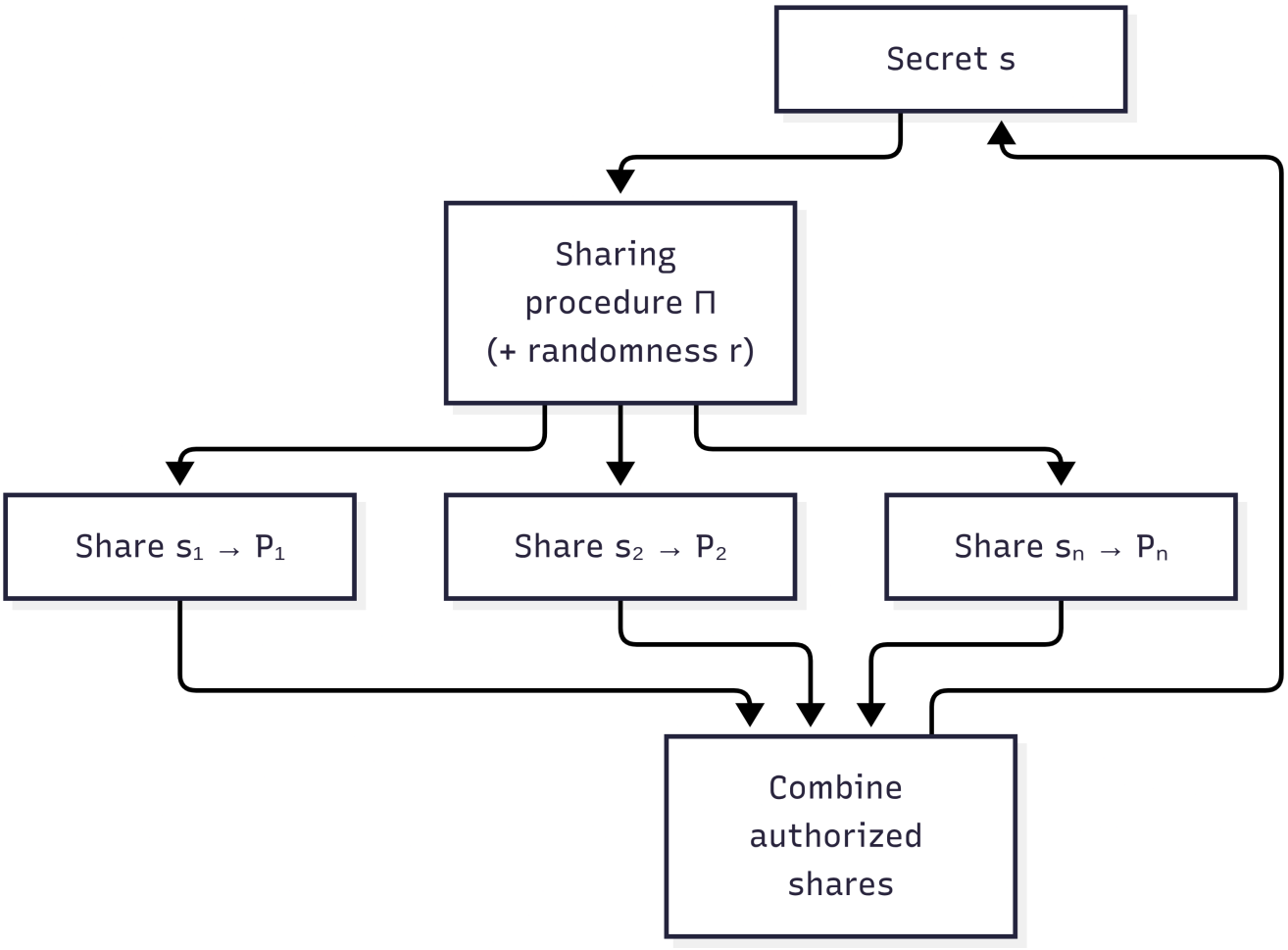
1. The **server chooses a random mask** $r \in_R 0,1^{\ell+\kappa}$ (where κ is a statistical security parameter ensuring the mask fully hides x) and computes the blinded ciphertext homomorphically: $[[z]] = [[x]]\,[[r]] = [[x+r]]$. The server sends $[[z]]$ to the client.
2. The **client decrypts** $[[z]]$ to obtain the masked value $z = x + r$, which reveals nothing about x because r is random. The client now holds the bits $z_{\ell-1}, \ldots, z_0$, and the server holds the mask bits $r_{\ell-1}, \ldots, r_0$.
3. The two parties feed these bit-vectors as inputs into a **garbled subtraction circuit** $\mathrm{GC}(\mathrm{SUB}^{\ell})$, which computes $z - r = x$ inside the circuit, producing the garbled wire values $\tilde{w}_{\ell-1}^{x_{\ell-1}}, \ldots, \tilde{w}_0^{x_0}$ that represent x in garbled form.

After this interface, the value x, originally an HE ciphertext, is now encoded as garbled inputs, ready for whatever non-linear GC computation follows. The subtraction inside the circuit removes the additive mask, so the garbled representation faithfully encodes the true value while neither party ever saw it in the clear. A symmetric interface converts garbled outputs back into HE ciphertexts, completing the ability to move freely between the two worlds.

Secret Sharing

The third paradigm abandons encryption of a single value in favor of *splitting* a value among many parties. **Secret sharing** (introduced independently by Shamir, Blakley, and Ito–Saito–Nishizeki) takes a secret and divides it into **shares**, one per participant, such that the shares individually look random but can be recombined to reconstruct the secret.

Pictorially, a holder of a secret bit-string (say `0001101001001`) produces several shares (such as `010010011011111`, `111110011011101`, `001010010101111`, and so on), one handed to each participant, with the property that combining them, in the simplest additive scheme by XORing all shares together, reconstructs the original secret. No single share, and in general no unauthorized subset of shares, reveals anything about the secret.



Initial Motivation

The original motivation is the problem of **safeguarding cryptographic keys against loss**. One might make several backup copies of a key, but this creates a dilemma: the **greater the number of copies, the greater the risk of exposure** (any one copy falling into the wrong hands compromises the key), while the **smaller the number of copies, the greater the risk of loss** (a single accident destroys the only copy). Simple replication cannot optimize both risks simultaneously.

Secret sharing resolves the dilemma by breaking the key into pieces such that **each piece reveals no information about the key**. But this raises its own question: what if a single piece is lost? Merely making copies of pieces is an inefficient and insecure fix (it reintroduces the exposure problem). The desired property is redundancy without full replication: given n pieces, we want the key to be **reconstructible if at least $t+1 \leq n$ pieces are available.** This is the notion of a **threshold** scheme, in which any $t+1$ shares suffice to recover the secret, while t or fewer reveal nothing, giving resilience against both loss (up to $n-(t+1)$ pieces may vanish) and exposure (any t pieces leak nothing).

Definition of Secret Sharing

Formally, a secret-sharing scheme is a procedure Π that takes a secret s together with fresh randomness r and produces shares $s_1, s_2, \ldots, s_n$ distributed to parties $P_1, P_2, \ldots, P_n$. Which subsets of parties are allowed to reconstruct the secret is specified by an **access structure** $P \subseteq 2^{P_1,\ldots,P_n}$, that is, a collection of "authorized" subsets of the participants. The scheme Π **realizes** the access structure P if it satisfies two properties:

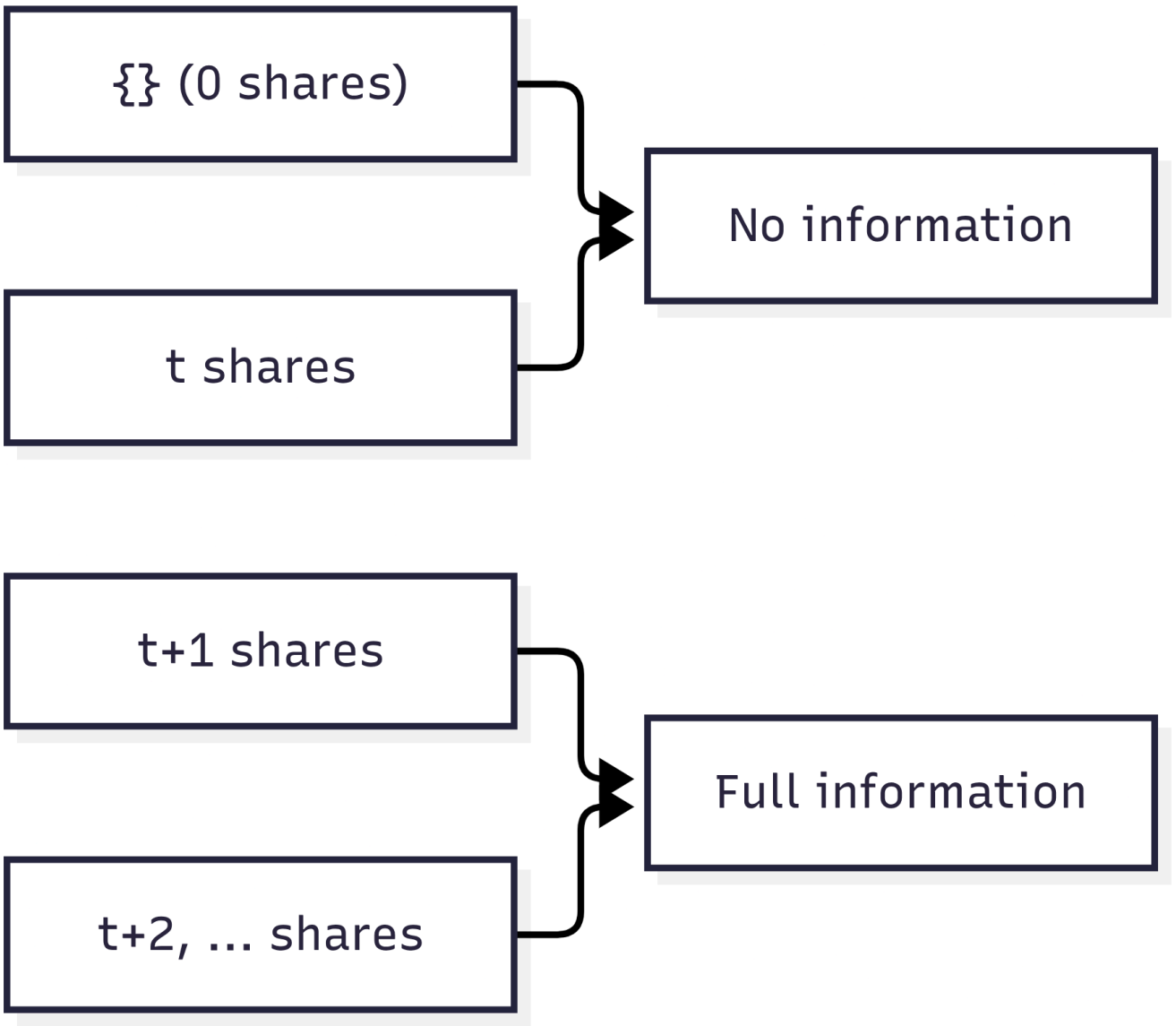
- **Correctness**: every authorized set $B \in P$ can always recover the secret s.
- **Privacy**: every unauthorized set $B \notin P$ cannot learn anything about the secret s.

The randomness r is essential: it is what makes the individual shares look random and ensures that unauthorized subsets gain no information. The **threshold scheme** described above is the special case where the access structure consists of all subsets of size at least $t + 1$.

t-Privacy: Privacy Is Not Incremental

A subtle and important property of threshold secret sharing is that **privacy is not incremental**. This means that acquiring shares one at a time, up to the threshold, yields *no* gradual leakage of the secret. Precisely: **if the adversary had no information before accessing t shares, then the adversary still has no information afterwards.** An adversary who collects the empty set, then one share, then two, and so on up to t shares, learns **"No information"** at every step; the transition to **"Full information"** happens only at the $(t + 1)$-th share, as an abrupt jump, not a gradual accumulation. There is no partial knowledge in between: possessing t shares is informationally identical to possessing none.

The dual, fault-tolerance side of the same threshold is equally sharp: **if the adversary somehow causes the loss (erasure) of t' shares, where $n - t' \geq t + 1$, then the secret can still be reconstructed uniquely from the $t + 1$ remaining shares.** So long as at least $t + 1$ shares survive, erasures are fully recoverable, which is precisely the robustness against loss that motivated the scheme.



Applications

Secret sharing is a foundational primitive with a broad range of applications beyond key backup:

- **Secure storage**, distributing data so that no single location holds the whole secret;
- **Secure multiparty computation**, where shares of inputs are computed upon directly (this is the primary application, as secret-sharing-based MPC lets parties jointly evaluate functions over shared inputs);
- **Threshold cryptography**, where a cryptographic operation (such as signing or decryption) requires the cooperation of a threshold of key-holders;
- **Byzantine agreement**, achieving consensus in the presence of faulty or malicious participants;
- **Access control**, granting a capability only to authorized coalitions;
- **Private information retrieval**, querying a database without revealing the query;
- **Attribute-based encryption**, where decryption is tied to satisfying an access policy expressed over attributes.

In each case the core idea is the same: distribute trust so that no individual, and no unauthorized coalition, can act alone, while authorized groups retain full capability. This makes secret sharing the natural third pillar of privacy-preserving computation, complementing the homomorphic-encryption and garbled-circuit approaches developed earlier.

Secret Sharing: Constructions and Thresholds

The previous treatment established *what* a secret-sharing scheme must guarantee, correctness for authorized sets and privacy for unauthorized ones, and introduced the threshold notion in which any $t+1$ shares reconstruct while t reveal nothing. We now turn to *how* such schemes are actually built, beginning with the two degenerate extremes and then developing the polynomial construction that realizes an arbitrary threshold.

Two Special Cases

Before the general construction, it is instructive to examine the boundary values of the threshold t, because they reveal both a trivial failure and a straightforward but inflexible success.

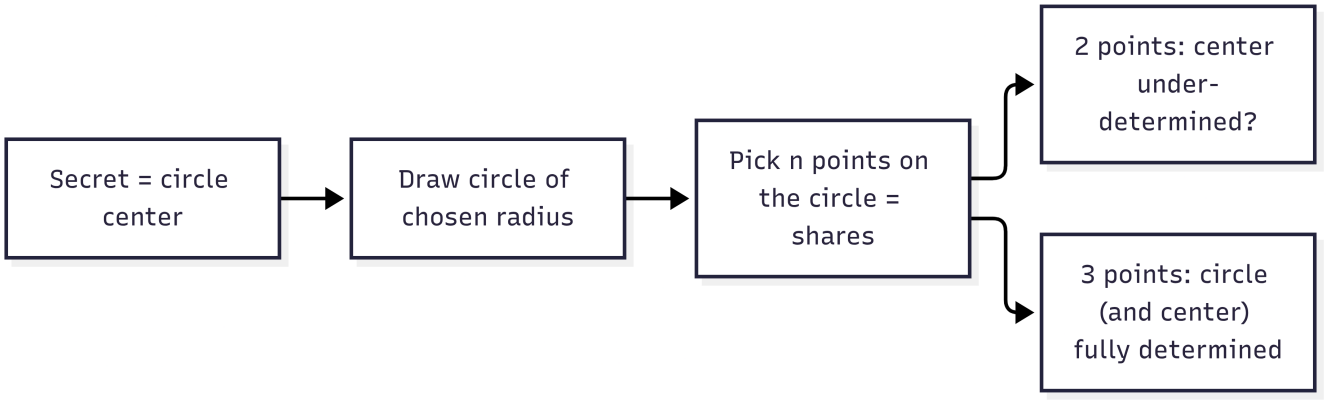
The case $t = 0$ is a *bad* secret sharing, in the sense that it provides no privacy at all. With $t = 0$, a single share suffices to reconstruct the secret, so possessing even one share is equivalent to possessing the secret itself. This can happen in the obvious way where the **share equals the secret**, or slightly more subtly where the **share is a function of the secret**, $\text{share} = f(\text{secret})$, from which the secret is recovered by inverting, $\text{secret} = f^{-1}(\text{share})$. In either form, any one participant can unilaterally recover the secret, which defeats the entire purpose of sharing; a 0-private scheme is really no protection.

The case $t = n - 1$ sits at the opposite extreme: here **all n created shares must be retrieved** to reconstruct the secret, and any subset of $n - 1$ or fewer reveals nothing. This maximal-threshold scheme is easy to build using any group operation that admits an inverse. The secret S can be decomposed as a **product** of shares modulo some modulus, $S = s_1 \cdot s_2 \cdots s_{t-1} \cdot s_t$, by randomly selecting $s_1, s_2, \ldots, s_{t-1}$ and then computing the last share as $s_t = S \cdot s_1^{-1} \cdot s_2^{-1} \cdots s_{t-1}^{-1}$ so that the product telescopes back to S. Equivalently, one can use **addition**, $S = s_1 + s_2 + \cdots + s_{t-1} + s_t$, by randomly selecting the first $t - 1$ shares and setting $s_t = S - (s_1 + s_2 + \cdots + s_{t-1})$; or the **XOR** operation, $S = s_1 \oplus s_2 \oplus \cdots \oplus s_{t-1} \oplus s_t$, with $s_t = S \oplus s_1 \oplus s_2 \oplus \cdots \oplus s_{t-1}$. In every variant, the first $n - 1$ shares are chosen uniformly at random, carrying no information about S, while the final share is computed to "close the equation." Only when all shares are combined does the randomness cancel and the secret emerge. This is secure and simple, but rigid: it demands *every* participant's cooperation and tolerates no loss whatsoever.

The interesting and practically useful regime lies strictly between these extremes: **how can we construct a secret sharing requiring fewer than n shares**, that is, a genuine threshold $0 < t < n$ where some, but not all, participants must combine their shares?

Building a Threshold Scheme from Geometry

A first, intuitive, but ultimately flawed attempt illustrates the right idea. Suppose the secret is represented as a **point in space**, taken to be the **center of a circle**. The dealer selects a radius, draws a circle centered at the secret, and then selects n points on that circle; the shares are the **coordinates of these points**. The reconstruction property seems promising: with only two shares (two points on the circle) one cannot determine the center, because infinitely many circles of differing radii pass through two given points. But with **three points one can reconstruct the circle exactly** and hence recover its center, the secret. This suggests a threshold behavior around $t = 2$.



The scheme has a **simple but fatal limitation**, however, which disqualifies it on privacy grounds. Working in the Cartesian plane, given two points one knows that the *perpendicular bisector* of the segment joining them passes through the center: the line connecting the two points yields, at its midpoint, a perpendicular line that must intersect the center of the circle. So two shares do *not* leave the center completely undetermined; they confine it to a line, which **gives the adversary some advantage**. A scheme in which two shares narrow down the secret is not 2-private, so the circle construction fails the strict privacy requirement even though its reconstruction behavior is appealing. The lesson is not that the geometric idea is wrong, but that the chosen geometric object (a circle) leaks partial information, and we need an object whose points reveal *nothing* about the hidden parameter until enough of them are collected.

The salvageable and powerful idea is to **construct secret sharing from interpolation codes**: use an algebraic object whose defining parameters are perfectly hidden by any insufficient set of evaluation points, and perfectly determined by a sufficient set. Polynomials, via Lagrange interpolation, are exactly such an object, and this is the basis of Shamir's scheme.

Secret Sharing from Interpolation Codes

The governing principle is simple to state: **if we know the interpolation function (its form and degree) and t shares together with their indices, we can reconstruct the secret.** The construction replaces the circle with a polynomial. A polynomial of degree t is uniquely determined by $t + 1$ of its evaluation points, and, crucially, by *fewer* than $t + 1$ points it is completely undetermined in a way that leaks nothing about any particular coefficient. One indexes the evaluation abscissas $\alpha_1, \alpha_2, \ldots$ on the x-axis; these indices are **distinct** and, importantly, are **public, not secret** (they must be known to perform interpolation). Describing a polynomial by its coefficient formula or by a sufficient set of m

points on it is entirely equivalent: having $f(X)$ is the same as having $f(\alpha_1), f(\alpha_2), \ldots$ for enough distinct α_i.

Lagrange's Interpolation Scheme

The formal backbone is Lagrange interpolation over a field K. Let $t \geq 0$ be an integer; then $K[X]$ denotes the K-vector space consisting of the polynomials $f(X) \in K[X]$ with $\deg(f) \leq m$ (the degree bound playing the role of the threshold parameter). The **Lagrange theorem** states: let $m \geq 1$ be an integer, and suppose $\alpha_1, \ldots, \alpha_m \in K$ are **pairwise distinct**. Then the evaluation map

$$\varepsilon : K[X] \to K^m, \qquad f(X) \mapsto \Big(f(\alpha_1), \ldots, f(\alpha_m)\Big),$$

is an **isomorphism of K-vector spaces**. The content of this statement is precisely what makes interpolation-based secret sharing work: a polynomial (of appropriate degree) and its vector of evaluations at m distinct points are two representations of the *same* object, and one can pass losslessly between them. Specifying the polynomial by its coefficients or by its m evaluation points are interchangeable.

The inverse map $\varepsilon^{-1} : K^m \to K[X]$, which recovers the polynomial from its evaluations, is given explicitly by the Lagrange interpolation formula. Define, for each index i, the **Lagrange basis polynomial**

$$\delta_i(X) = \prod_{1 \leq k \leq m, ; k \neq i} \frac{X - \alpha_k}{\alpha_i - \alpha_k}.$$

This $\delta_i(X)$ is constructed so that it equals 1 at $X = \alpha_i$ and 0 at every other α_k ($k \neq i$); computing it requires knowing the indices α_k, which is exactly why the indices must be public. The polynomial is then reconstructed as the weighted sum of these basis polynomials, each weighted by the corresponding evaluation (the share value):

$$f(X) = \sum_{i=1}^{m} y_i, \delta_i(X), \qquad \text{where } y_1 = f(\alpha_1), \ldots, y_m = f(\alpha_m).$$

To reconstruct f, one multiplies each δ_i by its share y_i and sums. The secret is then read off as a distinguished evaluation of the reconstructed polynomial, typically $f(0)$ (the constant term), which is the point on the y-axis where the polynomial crosses.

Shamir's Threshold Scheme (1979)

Shamir's 1979 scheme is the direct realization of this idea and is, for that reason, also called the **Lagrange interpolation scheme**. It is based on the Lagrange interpolation polynomial. One chooses a **large prime** p, and the secret S is represented as a number modulo p. The dealer constructs the polynomial

$$s(x) \equiv S + s_1 x + s_2 x^2 + \cdots + s_{t-1} x^{t-1} \pmod{p},$$

in which the **constant term is the secret** $S = s(0)$ and the remaining coefficients $s_1, \ldots, s_{t-1}$ are chosen at random. Each party is then provided a point on this polynomial: party i receives (x_i, y_i) where $y_i \equiv s(x_i) \pmod{p}$, for $i = 1, 2, \ldots, n$. Because the polynomial has degree $t - 1$, any t points determine it uniquely (recovering S as $s(0)$), while any $t - 1$ points leave the constant term completely undetermined, which is the privacy guarantee.

Shamir's Scheme: Precise Implementation

The implementation is cleanly stated over a finite field $\mathbb{F}_q$. Assume $q > n$. Let t and n be integers with $0 \leq t < n$, and let $\alpha_1, \ldots, \alpha_n \in \mathbb{F}_q$ be **pairwise distinct and non-zero** (which is possible precisely because $q > n$). It must be stressed that t**,** n**,** q**, and** $\alpha_1, \ldots, \alpha_n$ **are all public data**; only the secret and the random coefficients are hidden. The secret is $S \in \mathbb{F}_q$.

The dealer selects a polynomial $f(X) \in \mathbb{F}_q[X]$ **uniformly at random, conditioned on two requirements**: (1) $\deg(f) \leq t$, and (2) $f(0) = S$. These two conditions are equivalent to choosing the secrets in the interpolation code and deriving the function, and are easily implemented by randomly selecting the polynomial's coefficients: fix $a_0 = S$ and draw $(a_1, \ldots, a_t) \in \mathbb{F}_q^t$ uniformly at random. The polynomial is therefore

$$f(X) = a_0 + a_1 X + \cdots + a_t X^t,$$

and the n **shares of the secret** S are its evaluations at the public points: $s_1 = f(\alpha_1) \in \mathbb{F}_q, \ldots, s_n = f(\alpha_n) \in \mathbb{F}_q$.

This scheme **satisfies** t**-privacy and** $(t+1)$**-reconstruction, with** $t + 1 \leq n$. The secret S can be reconstructed from any $t + 1$ shares s_i. Concretely, for each set $B \subset 1, \ldots, n$ with $|B| = t + 1$, the secret is recovered by the Lagrange formula evaluated at zero:

$$S = \sum_{i \in B} \delta_{i,B}, s_i, \qquad \text{where } \delta_{i,B} = \prod_{i \in B, ; j \neq i} \frac{-\alpha_j}{\alpha_i - \alpha_j} \in \mathbb{F}_q.$$

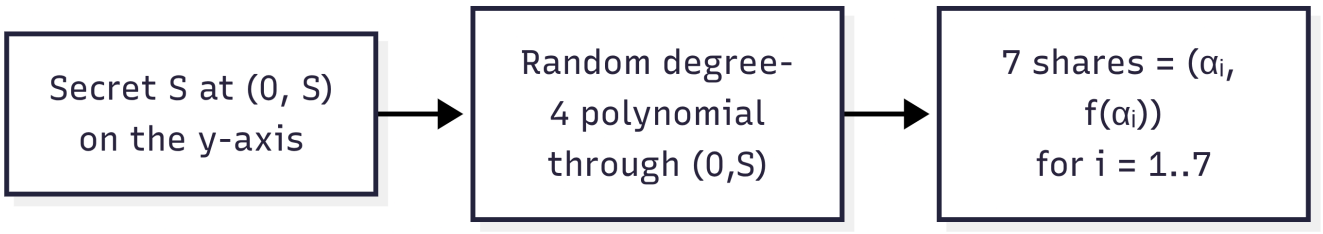
The coefficients $\delta_{i,B}$ are the Lagrange basis polynomials evaluated at $X = 0$ (hence the $-\alpha_j$ in the numerator), and they depend only on the public indices in the chosen reconstructing set B. Any authorized set of $t + 1$ participants can thus combine their shares with these publicly computable weights to recover S, while any t participants, lacking one evaluation of a degree-t polynomial, face a value of $f(0)$ that is uniformly distributed over $\mathbb{F}_q$ and therefore learn nothing. This is the precise, algebraic realization of the threshold secret sharing whose properties were asserted earlier.

Share Generation and the Formal Theory of Secret Sharing

With Shamir's construction in hand, it helps to visualize *why* it provides exactly the promised threshold behavior, and then to lift that intuition into a rigorous, information-theoretic definition that applies to any secret-sharing scheme, not just the polynomial one.

The Geometry of Share Generation

Consider a concrete $(5, 7)$ secret sharing: a scheme with $n = 7$ participants and a threshold such that $t + 1 = 5$ shares reconstruct the secret while any 4 reveal nothing (so $t = 4$). Geometrically, the dealer places the secret S as the point where the polynomial crosses the y-axis, that is, at $f(0) = S$. The seven public abscissas $\alpha_1, \alpha_2, \ldots, \alpha_7$ are marked along the x-axis. The dealer then draws a single random polynomial of degree $t = 4$ passing through the point $(0, S)$, and the seven shares are simply the heights of this curve above the seven abscissas: the points $(\alpha_i, f(\alpha_i))$.



The security intuition becomes vivid when we ask, for each number of available shares, how much the adversary can infer about S, which is the y-intercept.

With **0 shares available**, the adversary has no constraints at all: **any point on the y-axis can be the secret**, since a degree-4 polynomial can be drawn through $(0, S)$ for every conceivable value of S. The secret is completely undetermined.

With **1 share available**, the situation is unchanged from the adversary's standpoint. A single point $(\alpha_1, f(\alpha_1))$ is consistent with a degree-4 polynomial taking *any* value at $x = 0$: through one fixed point and an arbitrary y-intercept there still pass infinitely many degree-4 curves. **Any point on the y-axis can still be the secret**; one share constrains nothing about S.

With **4 shares available**, we are one short of the threshold, and this is the crucial case. Four points do not determine a degree-4 polynomial (which needs five), so for *every* candidate value of the y-intercept there remains exactly one degree-4 polynomial passing through the four known points and hitting that intercept. Thus **any point on the y-axis is still a possible secret**: the four shares, despite pinning down four evaluations, leave the secret uniformly undetermined. This is the geometric embodiment of t-privacy with $t = 4$: knowledge strictly below the threshold yields *no* information about S.

With **5 shares available**, the threshold is met. Five points uniquely determine the degree-4 polynomial, so the curve, and with it the single value $f(0) = S$, is completely fixed. The secret is recovered exactly. Moreover, **any other set of 5 shares** reconstructs the *same* secret: the particular five participants who cooperate do not matter, only that there are at least five of them. And with **7 shares available** (all of them), the polynomial is of course determined as well, with redundancy to spare. The transition from "no information" at four shares to "full information" at five is abrupt, with nothing in between, exactly the non-incremental privacy property established earlier.

Toward a Formal, Information-Theoretic Definition

The geometric picture captures Shamir's scheme, but secret sharing is a general concept, and its guarantees are most precisely expressed in the language of information theory, using entropy $H(\cdot)$ to measure uncertainty. We build up the formal definition from two primitive notions: reconstructing sets and privacy sets.

Let $\mathbf{X}$ be a vector of random variables, indexed by a set I. (Each index corresponds to a share, and one distinguished index will correspond to the secret.) We define two kinds of subsets.

A **reconstructing set** captures "enough shares to determine the secret." Let $B \subset I$ with $B \neq \emptyset$, and let $j \in I$. Then B is a reconstructing set for j if

$$H(X_j \mid \mathbf{X}_B) = 0,$$

that is, the collection of variables $\mathbf{X}_B$ **completely determines** X_j (there is zero residual uncertainty about X_j once $\mathbf{X}_B$ is known). Two immediate observations: trivially, if $j \in B$ then B is a reconstructing set for j (a variable determines itself); and if B is *not* a reconstructing set for j, then $0 < H(X_j \mid \mathbf{X}_B)$

$\leq H(X_j)$, meaning **there is some residual uncertainty** about X_j, though $\mathbf{X}_B$ may still reduce it partially.

A **privacy set** captures "shares that reveal nothing about the secret." Let $B \subset I$ and let $j \in I$. If $B \neq \emptyset$, then B is a privacy set for j if

$$H(X_j \mid \mathbf{X}_B) = H(X_j),$$

that is, $\mathbf{X}_B$ is **statistically independent** of X_j: conditioning on the shares in B does not reduce the uncertainty about X_j at all. By convention, if $B = \emptyset$, then B is a **privacy set by default** (knowing nothing reveals nothing). The contrast between the two notions is sharp: a reconstructing set drives the conditional entropy to zero, a privacy set leaves it at its maximum, and the gap between these is where "partial leakage" would live, which a good scheme forbids for all unauthorized sets.

The Formal Definition of Secret Sharing

With these primitives, a secret-sharing scheme is defined as a vector $\mathbf{S}$ of random variables, indexed by a set I with $0 \in I$ and $|I| > 1$, such that three properties hold.

Uniformity of the secret: $H(S_0) = \log_2 |S_0| \geq 1$. In other words, the random variable S_0, which will be the secret, is **uniformly distributed** over its space, which has size $|S_0| > 1$. The secret carries genuine entropy; it is not a constant.

Reconstruction: the set $I^* := I \setminus 0$ is a reconstructing set for 0. That is, the full collection of shares (all indices except the secret's own index 0) determines the secret.

Privacy: by default, the empty set is a privacy set for 0, so with no shares nothing is known about the secret.

The terminology then attaches as follows. The set I is the **player set**, and $n(\mathbf{S}) = |I|$ is the **number of players**. The variable S_0 is the **secret**. For each $j \in I^*$, the variable S_j is the **share for player** j, and the set S_j is the corresponding **share-space** (the set of values that share can take). This definition is representation-independent: it speaks only of random variables and their entropies, so it applies equally to Shamir's polynomial scheme, the XOR scheme, and any other.

Access Structures, Reconstruction, and Adversary Structures

Several derived notions organize *which* subsets of players can do what.

The **access structure** $\Gamma(\mathbf{S})$ consists of **all reconstructing sets** $B \subset I^*$, that is, every coalition of players able to recover the secret.

r-Reconstruction quantifies the threshold from the reconstruction side. Let r be a positive integer. The scheme $\mathbf{S}$ has r-reconstruction if for **each** subset $B \subset I^*$ with $|B| \geq r$ it holds that $B \in \Gamma(\mathbf{S})$, that is, every set of at least r players can reconstruct. The **smallest** positive integer r for which $\mathbf{S}$ has r-reconstruction is denoted $r(\mathbf{S})$. Note that $1 \leq r(\mathbf{S}) \leq n(\mathbf{S})$, and for all integers r with $r(\mathbf{S}) \leq r \leq n(\mathbf{S})$ there is r-reconstruction (once enough shares always suffice, more shares also always suffice).

The **adversary structure** $A(\mathbf{S})$ consists of **all privacy sets** $B \subset I^*$, that is, every coalition that an adversary could corrupt without learning anything about the secret. Crucially, the access and adversary structures are disjoint, $A(\mathbf{S}) \cap \Gamma(\mathbf{S}) = \emptyset$: no coalition can simultaneously be unable to learn anything and able to reconstruct, which would be a contradiction.

t-Privacy quantifies the threshold from the privacy side. Let t be a nonnegative integer. The scheme $\mathbf{S}$ has t-privacy if for each subset $B \subset I^*$ with $|B| \leq t$ it holds that $B \in A(\mathbf{S})$, that is, every set of at most t players is a privacy set and learns nothing. The **largest** nonnegative integer t for which $\mathbf{S}$ has t-privacy is denoted $t(\mathbf{S})$. For Shamir's scheme, $t(\mathbf{S}) = t$ and $r(\mathbf{S}) = t + 1$, so the privacy and reconstruction thresholds are adjacent, which is the optimal, gap-free situation.

Two Structural Results

Two further results sharpen the theory.

A **sufficient condition for privacy** gives a concrete test for when a coalition learns nothing. Let $B \subset I^*$ with $B \neq \emptyset$. If the distribution of $S_0 \times \mathbf{S}_B$ is the **uniform distribution** on $S_0 \times \text{supp}(\mathbf{S}_B)$, then B is a privacy set. Intuitively, if the secret and the shares held by B are jointly uniform (every combination of secret value and share values equally likely), then observing the shares tells the adversary nothing about which secret is in play, because all secrets remain equally consistent with what was seen.

A **theorem on the length of shares** establishes a fundamental lower bound on efficiency. Let $\mathbf{S}$ be a secret-sharing scheme indexed by a set I^*; then for each player j,

$$H(S_j) \geq H(S_0), \qquad \text{and in particular } |S_j| \geq |S_0|.$$

In words, **the share length must be at least the length of the secret**: every individual share must carry at least as much entropy (and hence be at least as large) as the secret itself. This is an unavoidable cost of information-theoretic secret sharing: one cannot compress shares below the size of the secret without sacrificing the perfect privacy guarantee. It explains why distributing an n-bit secret among many players requires each to store at least n bits, and it marks the boundary between information-theoretic schemes (which obey this bound) and computational relaxations that trade perfect privacy for smaller shares.

Secret Image Sharing and Secure Computation from Shares

The abstract theory of secret sharing becomes concrete and visually compelling in the problem of sharing an *image*, and it becomes powerful when the shares are not merely stored but *computed upon*, which leads directly to secure multiparty computation built on secret sharing.

Application: Secret Image Sharing

A (k, n)-threshold secret image sharing scheme adapts Shamir's idea to pixel data and must satisfy three requirements. First, the **secret image S is used to generate n shadow images** (the shares, now themselves images). Second, **any $\geq k$ shadow images can reconstruct the secret image.** Third, **any $< k$ shadow images cannot get sufficient information to reveal the secret image.** These are exactly the (k, n)-threshold guarantees, with k playing the role of the reconstruction threshold.

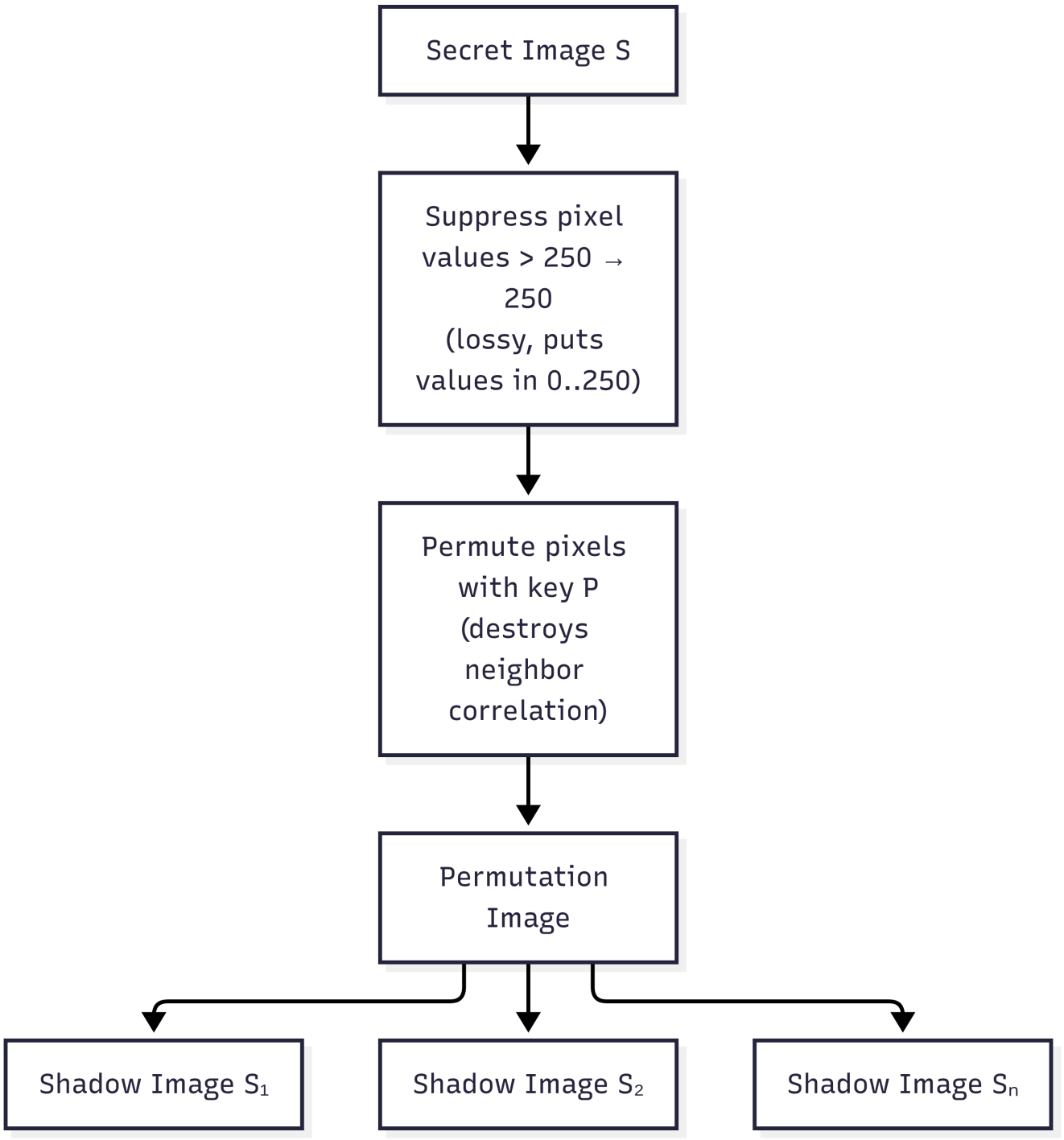
Scrambling the Secret Image: The Pre-Operation

Before any polynomial sharing occurs, a **pre-operation** prepares the image, and this step is essential for a reason that is subtle but important. The pre-operation has two parts, applied to an image whose gray levels lie in the range $[0, 255]$.

The first part addresses the choice of prime modulus. Shamir's scheme works modulo a prime p, and the natural choice near the pixel range is $p = 251$ (the largest prime not exceeding 255). Since pixel values

can be as large as 255 but the arithmetic is modulo 251, one must **suppress all values larger than 250 to 250**, forcing every pixel into the range $0 \sim 250$. This is a **lossy method**: the handful of pixels originally valued 251–255 are clamped to 250, so the reconstructed image differs slightly from the original. The loss is small and visually negligible, but it is real, and it is the price of using a prime modulus just below the pixel range.

The second part is a **permutation**. One selects a **key P to create a permutation matrix** that rearranges the pixels, producing a *permutation image*. The stated purpose is **to decrease the correlation between any neighboring pixels.** This is the crux of why permutation is necessary, and it deserves emphasis. The sharing polynomial is normally applied to each pixel individually to compute that pixel's contribution to each shadow. If the sharing were applied directly to the spatially structured original, the shadow images would retain the spatial correlations of the original, and an adversary holding fewer than k shadows could still perceive the outline and content of the secret image from these correlations, even without being able to reconstruct exact pixel values. In other words, the raw per-pixel sharing leaks *visual structure*. By first permuting the pixels so that neighboring pixels become uncorrelated, the structure is destroyed before sharing, and the shadows become noise-like. For the same reason one must **use a fresh or different permutation each time**, so that the permutation itself does not become a reconstruction shortcut or allow cross-correlation of multiple sharings of related images.



Generating and Acquiring the Shadow Images

Once the permutation image is prepared, the shares are generated by Shamir sharing applied across the pixels. The sharing polynomial is written

$$q(x) \equiv a_0 + a_1 x + a_2 x^2 + \cdots + a_{k-1} x^{k-1} \pmod{251},$$

a degree-$(k-1)$ polynomial whose coefficients are filled from the (permuted) pixel values. One selects n **distinct secret keys** $x_1, x_2, \ldots, x_n$ (the public abscissas, one per participant), and delivers the pair $(x_i, s(x_i))$ to the i-th participant. Concretely, successive groups of k pixels of the permutation image become the coefficients $a_0, \ldots, a_{k-1}$ of one polynomial, which is then evaluated at the n abscissas; the value $q(x_i)$ becomes the corresponding pixel of the i-th shadow image. Processing the whole permutation image this way yields n shadow images, each a fraction of the size of the original (since k pixels produce one pixel per shadow).

Reconstruction

Recovery reverses the process. To **get the permutation image**, one collects **any k shadow images**; because each pixel of the permutation image was shared with a degree-$(k-1)$ polynomial, any k evaluation points suffice. Without loss of generality one has the pairs $(x_1, s(x_1)), (x_2, s(x_2)), \ldots, (x_k, s(x_k))$, and one applies **Lagrange interpolation to reconstruct** each polynomial $q(x) \equiv a_0 + a_1 x + \cdots + a_{k-1} x^{k-1} \pmod{p}$, reading off its coefficients to recover the k permuted pixels. Having rebuilt the entire permutation image, one then applies the **inverse permutation key** P^{-1} to undo the scrambling and obtain the **recovered image**.



Why Permutation Matters, Demonstrated

The necessity of the permutation step is dramatically visible when it is omitted. **Without permutation**, applying the polynomial sharing directly to a structured image (for example, a photograph of a mandrill or a koala) produces shadow images in which the **outline and content of the original remain clearly perceptible**: the shadows look like noisy but recognizable versions of the secret, leaking the visual information that secret sharing is supposed to hide. **With the full scheme including permutation**, the result is correct. For instance, a $(2, 4)$-threshold applied to the 512×512 Lenna image first produces a **permutation image** that looks like pure random noise (its histogram flattened compared to the structured histogram of the original Lenna), and then four shadow images $(1, s(1)), (2, s(2)), (3, s(3)), (4, s(4))$ that are individually indistinguishable from random noise. Any two of the four reconstruct Lenna exactly (up to the small lossy clamping); any single one reveals nothing. The experimental comparison confirms that permutation is what converts a structure-leaking scheme into a genuinely private one.

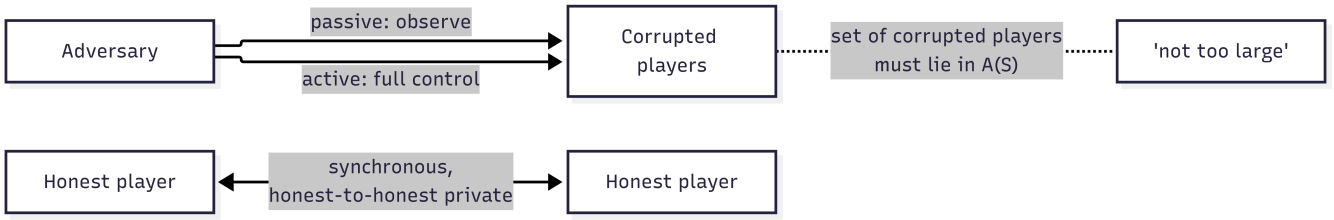
Summary: The Multiparty Setting and Its Adversary

Secret sharing is the gateway to a richer goal: not merely storing a secret among players, but having the players **compute together** on their private inputs. The general picture is a set of players, each holding a private input, who communicate in order to jointly evaluate some function and reconstruct a desired output, all while an adversary tries to subvert them.

The adversary's **corruption** of players comes in two strengths. A **passive** corruption means the adversary merely **observes the computation** of a corrupted player (reading its internal state and messages) in an attempt to learn secrets, without altering behavior. An **active** corruption means the adversary takes **full control** of a corrupted player, making it deviate arbitrarily, in order to either reconstruct the secret or **disrupt** the computation. These correspond, respectively, to the semi-honest and malicious adversary models seen earlier, now in the multiparty setting.

The standard focus is the **information-theoretic (I.T.) scenario**, in which there is **no information leaked on honest-to-honest communication**: messages exchanged between two honest players are assumed private from the adversary (for instance, over secure channels). The adversary may choose which players to corrupt either **statically** (fixed in advance) or **adaptively** (deciding as the protocol unfolds), but the set of corrupted players must be **"not too large"**, meaning it must lie within the **given adversary structure** $A(\mathbf{S})$, that is, within the sets that the secret-sharing scheme guarantees learn

nothing. Communication is taken to be **synchronous**. This is the setting in which secret-sharing-based secure multiparty computation is analyzed.

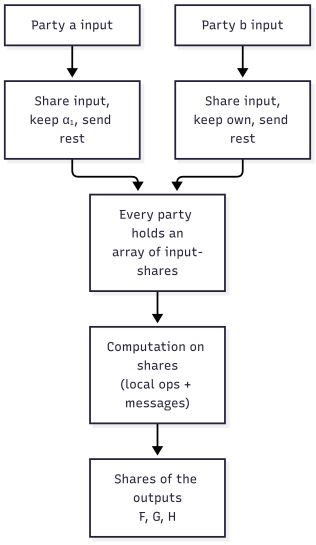


Secure Multiparty Computation from Secret Sharing

The central idea is to perform the entire computation *on shares*, so that the inputs and intermediate values never exist in the clear at any single location. **Each party has its private input**, together with a public index. The construction proceeds as follows.

A party that holds an input generates a secret sharing of that input: the main party (dealer for that input) generates the shares, **keeping one share for itself** (say the share at α_1) and **distributing the other shares to the other parties**. When every party has shared its input this way, **everyone ends up holding an array of shares**, one share of each party's input. No party holds any complete input except its own, and even its own input is now represented as a sharing distributed across the group.

The parties then run the computation directly on these shares. **From the shares of the inputs, the protocol arrives at shares of the outputs**: by exchanging messages and performing local operations on their share-arrays (addition of shares and, with more work, multiplication of shares), the players transform a sharing of the inputs into a sharing of the function's outputs, all without any party ever seeing a cleartext input or intermediate value. At the end, each party holds a share of each desired output.



Opening the Outputs

What distinguishes this from plain secret sharing is the flexibility of *who* learns *which* output, decided independently per output. Suppose the computation produces three output functionalities, F, G, and H. Each can be **opened** to a different audience by directing where the output-shares are sent:

- Suppose F **must be output only to one party** (say Homer). Then each party sends its share of F to Homer, and through all the collected shares Homer alone reconstructs the result F. No other party learns F.

- Suppose G **must be opened to everyone**. Then the parties form a **complete communication graph and broadcast their own shares of** G to all others, so that every party collects enough shares to reconstruct G. The result becomes common knowledge.
- Suppose H **must be given to someone outside the protocol** (for example, an external machine-learning model that will consume the result). Then the output-shares of H are sent to that external recipient, who reconstructs H while the computing parties need not learn it themselves.

This selective opening is the practical payoff of computing on shares: the same joint computation can deliver private outputs to individuals, public outputs to all, and outputs to external consumers, each without exposing the private inputs that produced them. It realizes secure multiparty computation, where mutually distrustful parties compute any agreed function over their secret inputs and reveal only the chosen outputs to the chosen recipients, built entirely on the threshold secret-sharing machinery developed in this chapter.

Statistical Approaches to Privacy: Anonymization

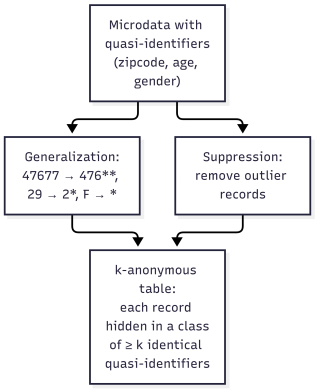
The cryptographic approaches developed so far, homomorphic encryption, garbled circuits, and secret sharing, protect data *during computation.* A complementary family of techniques, the **statistical approaches**, instead concerns itself with *publishing* a dataset in a form that preserves analytical utility while resisting re-identification. The earliest and most intuitive of these is anonymization through k-anonymity, which this section develops together with its refinements ℓ-diversity and t-closeness. These methods respond directly to the re-identification failures (GIC, AOL, Netflix) that motivated the whole subject.

k-Anonymity

The guiding definition of privacy here, due to Sweeney and Samarati, is "**protection from being brought to the attention of others.**" The idea is that an individual's privacy is preserved so long as their record cannot be singled out from a crowd of similar records. Formalizing this, a table satisfies **k-anonymity** if **each record is indistinguishable from at least $k-1$ other records when only the "quasi-identifiers" are considered.** A quasi-identifier is a combination of attributes (such as zip code, age, and gender) that is not an explicit identifier like a name, but which, in combination, could be linked against external data to re-identify an individual, exactly the vulnerability exploited in the GIC re-identification. The group of k records sharing identical quasi-identifier values is called an **equivalence class**. Within such a class, an adversary who knows a target's quasi-identifiers cannot determine which of the k records is the target's, so the target is hidden among k indistinguishable peers.

Two operations are used **to achieve k-anonymity**:

- **Generalization**: replace specific values with less-specific ones. For example, a zip code `47677` can be generalized to `476**`, an age of `29` to the range `2*` (meaning the twenties), and a specific gender to `*` (any). Generalization climbs a hierarchy from precise to coarse values, deliberately blurring quasi-identifiers until enough records collapse into the same equivalence class.
- **Suppression**: remove outliers entirely. Records whose quasi-identifiers are so unusual that they cannot be placed into a class of size k without excessive generalization are simply deleted, so that they do not force the entire table to be over-generalized.



Example of k-Anonymity and Generalization

Consider a microdata table whose quasi-identifier (QID) columns are zipcode, age, and gender, and whose sensitive attribute (SA) is a disease diagnosis:

Zipcode	Age	Gen	Disease
47677	29	F	Ovarian Cancer
47602	22	F	Ovarian Cancer
47678	27	M	Prostate Cancer
47905	43	M	Flu
47909	52	F	Heart Disease
47906	47	M	Heart Disease

Generalizing the quasi-identifiers produces a **3-anonymous table**, in which records fall into equivalence classes of size 3:

Zipcode	Age	Gen	Disease
476**	2*	*	Ovarian Cancer
476**	2*	*	Ovarian Cancer
476**	2*	*	Prostate Cancer
4790*	[43,52]	*	Flu
4790*	[43,52]	*	Heart Disease
4790*	[43,52]	*	Heart Disease

The protection this affords is concrete. Suppose the **adversary knows Alice's QI values** to be `(47677, 29, F)`. In the original microdata, this uniquely identifies her first record (and thus her diagnosis of ovarian cancer). In the 3-anonymous table, however, Alice's quasi-identifiers match the generalized class `(476**, 2*, *)`, which contains three records, and **the adversary does not know which one of the first three records corresponds to Alice's record.** She is hidden among three indistinguishable individuals, and her diagnosis is no longer pinned to her.

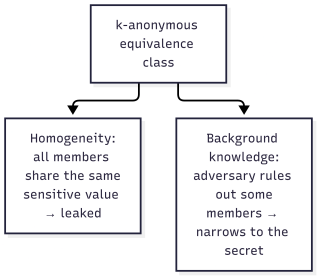
Attacks on k-Anonymity

The weakness of k-anonymity is that it protects the quasi-identifiers while ignoring the *sensitive attribute itself*. Thus **k-anonymity does not provide privacy if** either the **sensitive values lack diversity** or

the **attacker has background knowledge.** Two attacks make this precise.

The **Homogeneity Attack** exploits equivalence classes in which the sensitive value is identical across all members. Consider a 3-anonymous patient table, and an adversary who knows Bob's quasi-identifiers to be zipcode `47678`, age `27`. These match an equivalence class `(476**, 2*)` all three of whose records carry the diagnosis **Heart Disease**. Although Bob is "hidden" among three records in the k-anonymity sense, every one of those records has the same sensitive value, so the adversary learns with certainty that **Bob has heart disease.** The anonymity of the quasi-identifier is useless when the sensitive attribute is homogeneous within the class.

The **Background Knowledge Attack** exploits external information that lets the adversary eliminate possibilities. Suppose the adversary knows Carl's quasi-identifiers (zipcode `47673`, age `36`), which match an equivalence class containing the diseases Heart Disease, Cancer, Cancer. On its own this class is diverse. But if the adversary also knows, from outside information, that **Carl does not have heart disease**, then only the two Cancer records remain consistent with Carl, and the adversary concludes Carl has cancer. Background knowledge narrows the equivalence class from the outside, defeating the protection.



ℓ-Diversity

The response to the homogeneity attack is to require diversity *within* each equivalence class, which is the principle of **ℓ-diversity** (Machanavajjhala et al., 2006). The **principle** is that each equivalence class (equi-class) must contain at least ℓ **"well-represented" sensitive values.** By guaranteeing that every class offers at least ℓ genuinely different possibilities for the sensitive attribute, a homogeneity attack is prevented: knowing the quasi-identifiers never collapses the sensitive value to a single possibility.

The principle admits several **instantiations** of increasing strength. **Distinct ℓ-diversity** simply requires that each equi-class contain ℓ *distinct* sensitive values. This is the weakest form and, as we will see, can be undermined when the distinct values are badly skewed in frequency. **Entropy ℓ-diversity** strengthens the requirement using the Shannon entropy

$$H(X) = \mathbb{E}(I(X)) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i),$$

by demanding that the **entropy of each equi-class be at least** $\log_2(\ell)$. Because entropy is maximized when values are uniformly distributed, requiring entropy $\geq \log_2 \ell$ forces the sensitive values not merely to be distinct but to be reasonably *balanced* in frequency, which is a stronger and more meaningful notion of "well-represented."

Attacks on ℓ-Diversity

Even entropy ℓ-diversity has limitations, because it still ignores how the sensitive values within a class relate to the *overall distribution* and to each other. Two attacks reveal this.

The **Skewness Attack** exploits sensitive attributes with heavily skewed overall distributions. Suppose a sensitive attribute has two values: HIV positive (occurring in 1% of the population) and HIV negative (99%). The **highest diversity still carries serious privacy risk.** Consider an equi-class containing an equal number of positive and negative records, a 50/50 split, which maximizes diversity. Yet ℓ-diversity **does not differentiate** between two very different classes: Equi-class 1 with 49 positive + 1 negative, and Equi-class 2 with 1 positive + 49 negative. Both are "diverse," but membership in Equi-class 1 tells an adversary that the target is almost certainly HIV positive, a catastrophic leak, while membership in the balanced 50/50 class raises an individual's apparent HIV-positive probability from the population baseline of 1% to 50%. The root cause is that **ℓ-diversity does not consider the overall distribution of sensitive values**, so a class that looks diverse in isolation can still be wildly more revealing than the population as a whole.

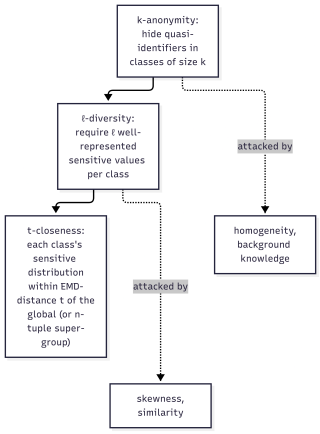
The **Similarity Attack** exploits the case where the distinct sensitive values within a class, though different, are *semantically similar*. Consider a 3-diverse patient table that also includes salary, and an adversary who knows Bob's quasi-identifiers (zip `47678`, age `27`), matching an equi-class with salaries `{20K, 30K, 40K}` and diseases `{Gastric Ulcer, Gastritis, Stomach Cancer}`. The class is technically 3-diverse (three distinct diseases, three distinct salaries). Yet the adversary draws two damaging **conclusions**: first, Bob's salary lies in `[20K, 40K]`, which is **relatively low**; and second, all three diseases are stomach-related, so **Bob has some stomach-related disease.** Diversity of *labels* provided no protection because the labels were *close in meaning*. The lesson is that **ℓ-diversity does not consider the semantic meanings of sensitive values**, so a class of distinct-but-similar values leaks the shared attribute they have in common.

t-Closeness

The response to the skewness and similarity attacks is to tie each equivalence class to the *global* distribution, which is the principle of **t-closeness** (N. Li, T. Li, Venkatasubramanian, 2007). The **principle** is that the **distribution of the sensitive attribute value in each equi-class should be close to that of the overall dataset**, with the distance bounded by a parameter t (distance $\leq t$). The underlying assumption is that **publishing a completely generalized table is always acceptable**, that is, revealing only the global distribution of the sensitive attribute leaks nothing beyond what the whole dataset already discloses; so requiring each class to resemble that global distribution ensures no class reveals more than the dataset as a whole.

Crucially, the distance between distributions is measured with the **Earth Mover Distance (EMD)**, which is chosen precisely because it **captures the semantic relationship among sensitive attribute values.** EMD models the cost of transforming one distribution into another as the minimum "work" of moving probability mass across the space of values, where moving mass between semantically distant values costs more than between similar ones. In the illustrative comparison, transforming one small distribution into a nearby target might cost 2, while transforming it into a more "spread" target costs 5, and the ordering of these costs reflects genuine semantic distance. By penalizing concentration of probability on similar or extreme values, EMD-based t-closeness defeats both the skewness attack (a class skewed away from the global distribution is far in EMD and thus disallowed) and the similarity attack (a class concentrated on semantically similar values is far from the spread-out global distribution).

A practical relaxation, **(n, t)-closeness**, loosens the requirement so that the **distribution of the sensitive attribute value in each equi-class need only be close to that of some natural super-group consisting of at least n tuples**, rather than to the entire dataset. This permits tighter, more useful generalizations when the data has natural subpopulations, while still bounding how much any class can deviate from a sufficiently large reference group.



The progression from k-anonymity through ℓ-diversity to t-closeness is a sequence of patches: each refinement closes an attack that the previous notion left open, moving from "hide the quasi-identifiers" to "diversify the sensitive values" to "match the global distribution semantically." Yet all three remain *syntactic* conditions on a released table, and each new attack suggests that this patch-and-respond cycle may never reach a definitive guarantee. This fundamental limitation of anonymization is exactly what motivates the shift to a rigorous, provable, and composition-friendly definition grounded in the behavior of the *release mechanism* rather than the shape of the table, which is the subject of **Differential Privacy**.

Differential Privacy

The anonymization methods of the previous section were a sequence of syntactic patches, each defeated by a new attack. Differential privacy breaks this cycle by shifting the guarantee from the *shape of the released table* to the *behavior of the release mechanism*, yielding a rigorous, provable, and attack-independent definition of privacy. It is the culmination of the entire privacy-preserving-computation subject, and it directly realizes the refined "with-Alice-versus-without-Alice" intuition introduced at the very beginning.

The Setting and the Neighboring-Datasets Definition

Fix the following objects. Let X be the data **universe**, the space of all possible records, and let $D \subset X$ be **the dataset**, containing one element per person. The central structural notion is that of **neighboring datasets**:

> **Definition:** Two datasets $D, D' \subset X$ are *neighbors* if they differ in the data of a single individual.

A neighbor D' is obtained from D by changing exactly one person's record (equivalently, by adding, removing, or modifying one individual). This neighbor relation is the formal embodiment of the "identical database with Alice's data removed" baseline: a privacy guarantee stated over all neighboring pairs is precisely a guarantee that any single person's presence or data makes almost no difference.

ϵ-Differential Privacy

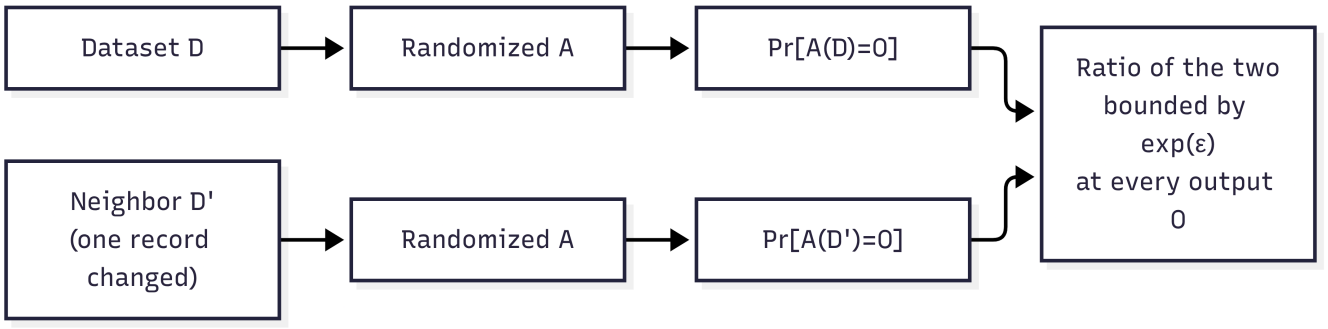
The definition quantifies "almost no difference" through the output distribution of a *randomized* algorithm. A first, linearized form states:

> **Definition:** An algorithm M is **ϵ-differentially private** if for all pairs of neighboring datasets D, D', and for all outputs x,
>
> $$\Pr[M(D) = x] \leq (1 + \epsilon), \Pr[M(D') = x].$$

The standard and more robust form (which composes cleanly and handles the full range of ϵ) replaces the factor $1+\epsilon$ with $\exp(\epsilon)$. Writing the randomized algorithm as A:

> A randomized algorithm A satisfies **ϵ-differential privacy** if and only if for any two neighboring datasets D and D' and for any output O of A,
>
> $$\Pr[A(D) = O] \leq \exp(\epsilon), \Pr[A(D') = O].$$

The **intuition** is that **it is acceptable to publish information that is insensitive to changes of any particular tuple.** If changing one person's record barely alters the probability of *any* output, then observing the output tells an adversary almost nothing about whether that person was in the dataset or what their record was. Graphically, for the two neighboring datasets the output distributions $\Pr[A(D) = O]$ and $\Pr[A(D') = O]$ must lie within a multiplicative factor $\exp(\epsilon)$ of each other at **every** output value: the two curves track each other closely, and their **ratio is bounded by** $\exp(\epsilon)$ everywhere.



The parameter ϵ is the **privacy budget**: **the value of ϵ decides the degree of privacy protection.** A small ϵ (near zero) forces the two distributions to be nearly identical, giving strong privacy but, as we will see, requiring more noise and thus less accuracy; a large ϵ permits the distributions to differ more, giving weaker privacy but higher accuracy. ϵ is the formal dial trading privacy against utility. A key virtue of this definition, in contrast to the anonymization notions, is that it is a property of the *mechanism* and holds against *any* adversary with *any* background knowledge, so it is immune to the homogeneity, background-knowledge, skewness, and similarity attacks by construction.

Why Releasing the Exact Answer Fails

To see why randomization is essential, consider a concrete task. The dataset is a set of patient records, and the objective is to release the number of patients with diabetes under ϵ-differential privacy, which requires satisfying:

$$\Pr[\mathcal{A}(D) = O] \leq \exp(\epsilon) \cdot \Pr[\mathcal{A}(D') = O]$$

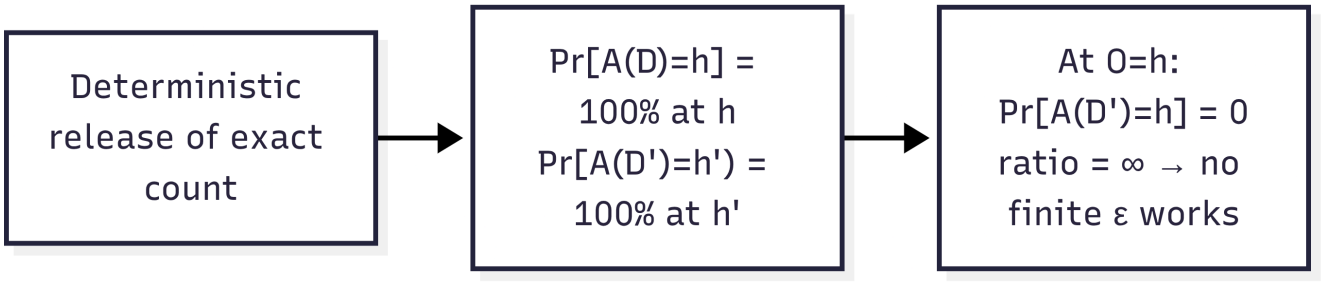
Releasing the count directly will not work. Let D be the original dataset with a true count of h diabetes patients, and let D' be a neighboring dataset obtained by modifying a single record, resulting in a true count of h' (where $h' \neq h$, differing by exactly 1).
A deterministic release mechanism places all its probability mass on the single true value:

$$\Pr[\mathcal{A}(D) = h] = 1 \quad \text{and} \quad \Pr[\mathcal{A}(D') = h'] = 1$$

with zero probability everywhere else. Evaluating the output at $O = h$, we have $\Pr[\mathcal{A}(D) = h] = 1$ but $\Pr[\mathcal{A}(D') = h] = 0$. Consequently, the ratio $\frac{\Pr[\mathcal{A}(D)=h]}{\Pr[\mathcal{A}(D')=h]}$ is infinite, as there is no finite ϵ for which:

$$1 \leq \exp(\epsilon) \cdot 0$$

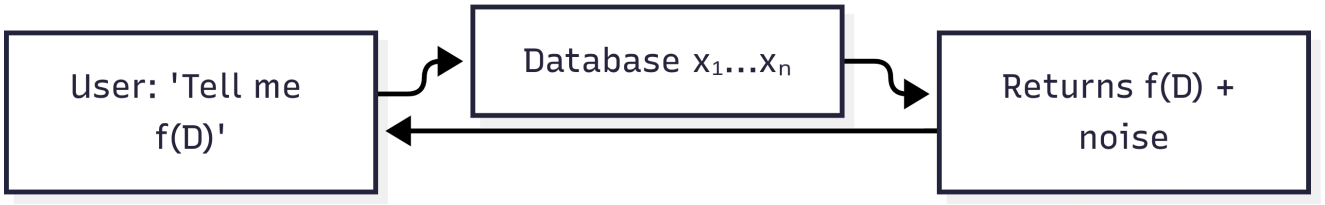
A deterministic, exact release can never be differentially private because changing a single record alters the output with certainty, rendering the two output distributions perfectly distinguishable.



The **idea** that rescues the task is to **perturb the number of diabetes patients to obtain a smooth distribution.** Rather than two disjoint spikes, the mechanism should spread probability so that both neighboring datasets assign nonzero, comparable probability to every output, bringing their ratio under control.

The Primitive: Add Noise to the Output

The workhorse mechanism of differential privacy is to **add noise to the output.** A user queries the database, "Tell me $f(D)$," and the database responds with $f(D) + \text{noise}$ rather than the exact value. The **intuition** is that $f(D)$ **can be released accurately when** f **is insensitive to individual entries** $x_1, \ldots, x_n$: the less a single record can swing the true answer, the less noise is needed to mask that record's influence.



This insensitivity is made precise by the **global sensitivity** of the query function f:

$$GS_f = \max_{\text{neighbors } D,D'} \|f(D) - f(D')\|_1,$$

the largest possible change in f's output across *any* pair of neighboring datasets, measured in the ℓ_1 norm. Global sensitivity measures the worst-case influence of one individual on the answer. For example, $GS_{\text{average}} = 1/n$ for the average of a set of n numbers each lying between 0 and 1, because changing one of the n values by at most 1 changes their average by at most $1/n$. A counting query (such as "number of diabetes patients") has sensitivity 1, since adding or removing one person changes the count by exactly 1.

The fundamental **theorem** ties sensitivity, the privacy budget, and the required noise together:

> $f(x) + \text{Lap}(GS_f/\epsilon)$ is ϵ-indistinguishable,

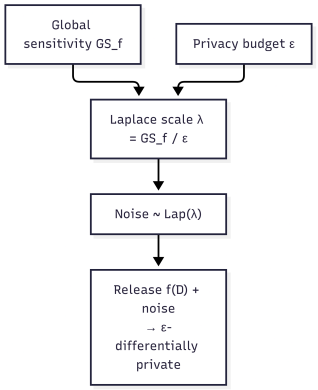
where the **noise is generated from the Laplace distribution** with scale GS_f/ϵ. In words, to answer a query f with ϵ-differential privacy, release the true answer plus Laplace noise whose scale is the function's global sensitivity divided by the privacy budget. Higher sensitivity demands more noise (a more influential individual must be hidden under a wider perturbation), and a smaller ϵ (stronger privacy) also demands more noise, which is the precise form of the privacy–accuracy trade-off.

The Laplace Distribution

The Laplace distribution is chosen because its multiplicative tails match the multiplicative guarantee of differential privacy exactly. Its probability density function is

$$pdf(x) = \frac{1}{2\lambda}\exp!\left(-\frac{|x|}{\lambda}\right),$$

a symmetric distribution sharply peaked at 0 with exponentially decaying tails. Its decisive property is its behavior under a shift: if x is **increased or decreased by** α, then $pdf(x)$ **changes by a factor of** $\exp!\left(-\frac{|\alpha|}{\lambda}\right)$**.** This is exactly the multiplicative, bounded-ratio behavior that ϵ-differential privacy requires, which is why Laplace noise yields a clean ϵ guarantee. The distribution has **variance** $2\lambda^2$, and λ is referred to as the **scale**: larger λ produces a flatter, wider distribution (more noise, more privacy, less accuracy), while smaller λ produces a sharper peak (less noise, less privacy, more accuracy). Plotting the density for $\lambda = 1, 2, 4$ shows the peak at $x = 0$ dropping from 0.5 to 0.25 to 0.125 as the scale widens.



Differential Privacy via Laplace Noise: The Mechanism in Action

Return to the diabetes example to see the mechanism achieve the guarantee. The **dataset** is a set of patients; the **objective** is to release the number of diabetes patients with ϵ-differential privacy, $\Pr[A(D) = O] \le \exp(\epsilon) \cdot \Pr[A(D') = O]$. The **method** is to release the number plus Laplace noise, so that the output distribution is

$$pdf(x) = \exp\left(-\frac{|x|}{\lambda}\right) \Big/ 2\lambda.$$

The **rationale** unfolds across the neighboring datasets. Let D be the original dataset with true count h, and let D' be a neighbor (modify one patient) with true count h'. Under the Laplace mechanism, the probability of releasing a particular value y is centered on the true count:

$$\Pr[A(D) = y] = pdf(y - h) = \exp!\left(-\frac{|y - h|}{\lambda}\right) \Big/ 2\lambda,$$

a Laplace curve peaked at h. For the neighbor,

$$\Pr[A(D') = y] = pdf(y - h') = \exp!\left(-\frac{|y - h'|}{\lambda}\right) \Big/ 2\lambda,$$

the same curve peaked at h'. (The same holds whether the neighbor is obtained by modifying a patient or by changing one individual's attribute; in every case the two peaks sit at h and h', one sensitivity-unit apart.) Because the two curves are identical except for a horizontal shift from h to h', and because shifting a Laplace density by $\alpha = h' - h$ scales it by at most $\exp(-|\alpha|/\lambda)$, their **ratio is bounded** at every output y:

$$\frac{\Pr[A(D)=y]}{\Pr[A(D')=y]} = \exp!\left(\frac{|y-h'|-|y-h|}{\lambda}\right) \le \exp!\left(\frac{|h-h'|}{\lambda}\right) = \exp!\left(\frac{GS_f}{\lambda}\right).$$

Setting the scale $\lambda = GS_f/\epsilon$ makes this bound exactly $\exp(\epsilon)$, delivering the required ϵ-differential privacy. The two overlapping Laplace curves, one centered at h and one at h', are everywhere within a factor $\exp(\epsilon)$ of each other, so no observer of the released value y can confidently tell whether the underlying dataset was D or D', that is, whether any particular patient's record was present or changed.

This is the complete realization of the privacy ideal that opened the subject. Where naive anonymization failed against auxiliary information and the k-anonymity family fell to successive attacks, the Laplace mechanism provides a mathematically provable guarantee: the released statistic is almost exactly as likely regardless of any single individual's data, so participation carries almost no incremental risk, while the aggregate signal, perturbed only by calibrated noise, remains useful for analysis. The parameters ϵ (privacy budget) and GS_f (sensitivity) together govern precisely how much noise this costs, making the privacy–utility trade-off explicit, tunable, and, unlike every approach before it, robust against adversaries of arbitrary background knowledge.

CHAPTER 5: Computing with private data

Data Processing in the Encrypted Domain

1. Motivation: Why Cryptography Alone Is Not Enough

Modern information systems routinely require one party to process another party's data. A client submits a fingerprint to be matched against a database; a patient sends a cardiac recording to a remote classifier; a sensor contributes a measurement to a distributed estimate. In all of these situations the data is sensitive, and the parties do not fully trust one another. The natural reflex is to reach for cryptography — but conventional cryptography protects data only **at rest** and **in transit**. It secures a message while it is stored on disk or travelling across a network, yet it says nothing about the moment the data is actually *used*. To compute on encrypted data with standard tools, one must first decrypt it, and at that instant the protection vanishes: whoever performs the computation sees the plaintext.

This is the gap that **data processing in the encrypted domain** — often called **signal processing in the encrypted domain (s.p.e.d.)** — is designed to close. The goal is to perform a meaningful computation (a comparison, a distance, a classification) directly on data that remains encrypted throughout, so that the party doing the work never learns the inputs, and often not even the output. The conceptual contrast is between two regimes. In the **plain domain**, a user hands over their data (for example a fingerprint) to a server, which computes an answer on the cleartext and returns it; the server sees everything. In the **encrypted domain**, the same answer is produced, but the server operates only on ciphertext and learns nothing about the biometric it just processed. The user obtains the correct result while the raw data is never exposed.

Despite its appeal, encrypted-domain computation still has **few real-world deployments**. The reason is not a lack of theoretical tools but their cost: generic secure computation is dramatically slower and more bandwidth-hungry than its plaintext equivalent. Making it practical is therefore not a matter of applying a cryptographic primitive off the shelf, but of **tailoring privacy-preserving protocols** to each specific application — redesigning the computation so that the expensive cryptographic machinery is invoked as little as possible. This discipline of **protocol optimization** is the central theme of everything that follows.

2. The Methodology of Privacy-Preserving Protocol Design

Designing a practical secure protocol is an engineering process with a recognizable life cycle. It is governed throughout by a three-way tension between **accuracy**, **privacy**, and **performance**: improving any one of these typically costs something in the other two, and the designer's job is to locate an acceptable trade-off rather than to maximize a single axis. Underpinning the entire process is a solid **knowledge of the available cryptographic tools**, because the feasibility of every design decision depends on what those tools can and cannot do cheaply.

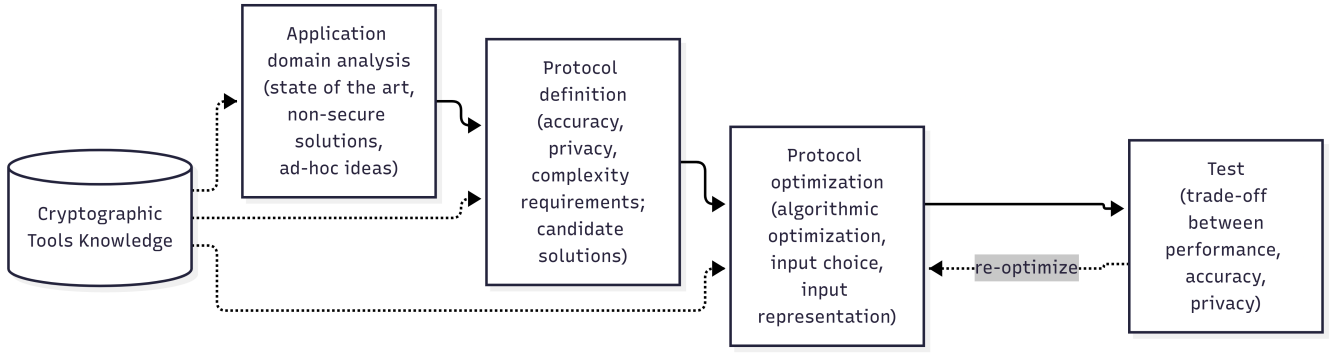
The methodology proceeds through four phases.

The first is **application-domain analysis**. Before any cryptography is written, the designer studies the problem itself: the state of the art, the existing non-secure (plaintext) solutions that already solve the task, and the opportunity for innovative, ad-hoc approaches that may be more amenable to secure implementation than the standard algorithm. The point is to understand the computation deeply enough to know which parts truly need protecting and which parts can be restructured.

The second phase is **protocol definition**. Here the requirements are made explicit — the required **accuracy**, the **privacy** guarantees (what each party is allowed to learn), and the acceptable **complexity** — and the candidate cryptographic solutions that could meet them are identified. This phase fixes the contract the protocol must satisfy.

The third and most creative phase is **protocol optimization**, which operates on three distinct levers. **Algorithmic optimization** reshapes the computation to avoid or approximate operations that are expensive in the encrypted domain. **Input choice** selects *which* features or quantities actually enter the secure computation, since fewer and simpler inputs mean a smaller protocol. **Input representation** decides *how* those quantities are encoded — the number of bits, the number format — because representation directly determines ciphertext sizes and circuit widths.

The fourth phase is **test**, where the optimized protocol is measured and the search for a workable trade-off between performance, accuracy, and privacy is validated empirically. The phases are not strictly linear: test results frequently send the designer back to re-optimize.



3. Privacy-Preserving Biometric Recognition

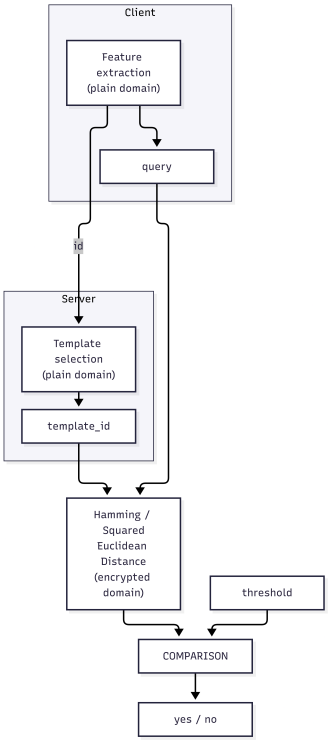
Biometric systems are the archetypal application of encrypted-domain processing, because a biometric template is the most irreplaceable kind of personal data — a leaked password can be changed, a leaked fingerprint cannot. Two tasks dominate the field, and they differ in structure in a way that matters for protocol design.

3.1 Biometric Authentication (one-to-one verification)

In **authentication** — also called verification — a user claims an identity and the system checks a single assertion: *are you who you say you are?* The architecture places the user's device in the role of **client**

and the biometric database holder in the role of **server**. On the client side, a biometric sample is captured and passed through **feature extraction in the plain domain**, producing a compact numerical descriptor called the **query**. Crucially, feature extraction happens locally on cleartext, because it is a signal-processing step on the user's own data; only the resulting feature vector enters the secure protocol.

The user also transmits a claimed identity, an **`id`**. The server uses this `id` to perform **template selection in the plain domain**, retrieving the single stored reference template, `template_id`, corresponding to the claimed identity. At this point the secure computation begins: the query and the selected template are fed into a **distance computation** — typically the **Hamming distance** (for binary templates) or the **squared Euclidean distance** (for real-valued feature vectors) — evaluated in the encrypted domain so that neither side learns the other's vector. The resulting distance is then passed to a **comparison** against a **threshold**: if the distance is small enough the biometrics are deemed to match. The protocol finally outputs a single bit, **yes / no**, revealing only whether verification succeeded.

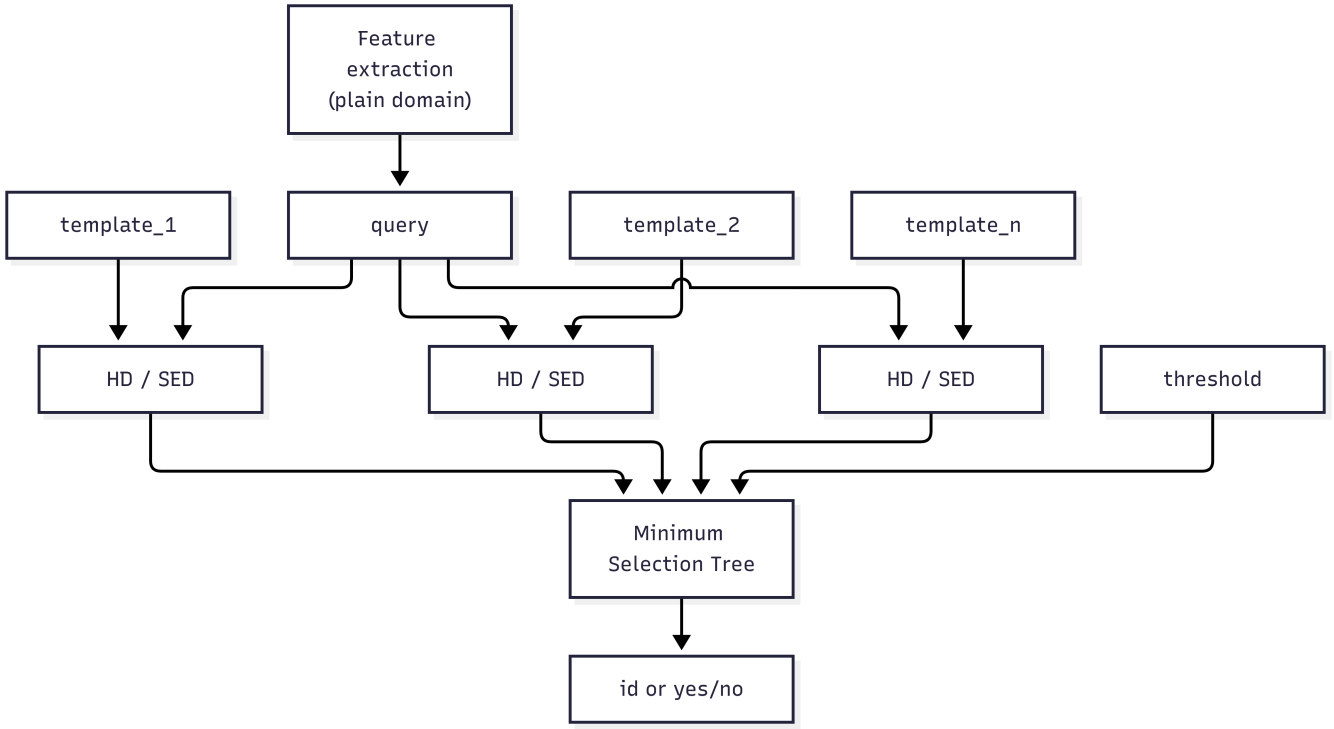


3.2 Biometric Identification (one-to-many search)

Identification answers a harder question: *who are you?* Instead of checking one claimed template, the system compares the query against **every** template in the database. The client-side feature extraction is identical, but now the single query is dispatched to a **bank of distance computations**, one **HD / SED** (Hamming Distance / Squared Euclidean Distance) module per enrolled template `template_1, template_2, …, template_n`. Each module produces the distance between the query and one stored identity.

These `n` distances must be reduced to a decision, and this is done by a **Minimum Selection Tree** — a structure that finds the smallest distance (the best-matching identity) across all candidates, comparing it where appropriate against a **threshold**. The output is either an **`id`** (the identity of the closest match) or, when the task is cast as a watch-list check, a **yes/no** indicating whether any enrolled identity matched. The essential structural difference from authentication is the fan-out: identification multiplies the costly secure distance computation by the size of the database and then adds a secure minimum-

finding stage on top, which makes it substantially more expensive and a prime target for optimization.



3.3 Input Representation Choice: Minutiae versus FingerCode

The optimization principle that **input representation drives protocol cost** is vividly illustrated by fingerprint recognition, where two competing feature representations lead to radically different secure implementations. The choice of which features represent the input is not merely a signal-processing decision; it **directly impacts the protocol complexity**.

The traditional representation is **minutiae-based**. Minutiae are the local ridge features of a fingerprint — ridge endings and bifurcations, each described by a position and an orientation. Matching two minutiae sets is a geometric alignment problem that is irregular, variable-length, and full of conditional logic. This representation delivers **higher accuracy**, but it is **difficult to implement in the encrypted domain (s.p.e.d.)** precisely because its matching algorithm does not reduce to a clean arithmetic expression.

The alternative is the **FingerCode** representation. A FingerCode is obtained by overlaying a circular tessellation on the fingerprint (centred on a reference point), filtering each sector, and summarizing it into a **fixed-length vector of real-valued features**. Two FingerCodes are compared simply by the **Euclidean distance** between their vectors. This representation yields **slightly worse accuracy** than minutiae, but it enables an **easy s.p.e.d. implementation**, because a fixed-length Euclidean distance is a sum of squared differences — exactly the kind of regular, data-independent arithmetic that homomorphic encryption and circuits handle efficiently. The lesson is general: a small, deliberate sacrifice in plaintext accuracy can unlock an enormous gain in secure-protocol feasibility, and choosing the representation that linearizes the matching function is often the single most important design decision.

3.4 Algorithmic-Level Optimization: Iris Identification and the Common Mask

Beyond choosing the representation, the **algorithm itself** can be redesigned to be cheaper in the encrypted domain. The two guiding tactics are to **avoid expensive operations** wherever possible and to **approximate expensive functionalities** when exact evaluation is not worth its cost.

Iris recognition provides a clean example. The iris is encoded as a **binary template** (the IrisCode), so two irises are compared by **Hamming distance** — the count of differing bits — which is naturally efficient to evaluate securely. However, real iris images are partly obscured by eyelids, eyelashes, and reflections, and the standard algorithm therefore pairs each template with an **individual hiding mask**: a per-user bitmask marking which bits are valid and which must be ignored. Incorporating this mask into the secure comparison is costly, because applying a *private* mask bit to each feature requires logic gates that cannot be evaluated for free in a garbled circuit — concretely, **two non-free gates per feature**.

The optimization is to replace every user's individual mask with a single **common mask** shared across the population. The decisive observation is that a common mask **is public**: because it is the same for everyone and carries no individual information, it need not be hidden inside the secure computation at all. Applying a public mask is free, which **significantly decreases the number of gates** in the circuit. The price is a small approximation error, since the common mask does not perfectly fit each individual iris — but the resulting **accuracy loss is low**, making the trade-off strongly favourable.

3.5 Accuracy Validation of the Common Mask

The common-mask approximation must be justified empirically, and it is validated by examining the **Hamming-distance distributions** it produces. For a biometric comparison these distributions come in two populations: the **"same"** distribution (distances between two samples of the *same* iris, which should be small) and the **"different"** distribution (distances between *different* irises, which should cluster around one half, as expected for unrelated binary strings). Good separation between these two curves is what makes reliable thresholding possible.

Comparing the distributions obtained with the original **real (individual) masks** against those obtained with the **common mask** shows that the two populations remain almost equally well separated — the shapes shift only slightly. Quantitatively, the **accuracy is reduced by less than 1%**, a negligible degradation. In exchange, the performance gains are large: the **online time for a single iris match falls to 65 milliseconds**, a **speed-up factor of up to 8.7**, while the **communication bandwidth drops to 133.7 KBytes**, a reduction by a factor of roughly **4.3**. A sub-one-percent accuracy cost bought nearly an order-of-magnitude improvement in both latency and bandwidth — a textbook demonstration of why algorithmic approximation is the heart of practical s.p.e.d.

3.6 Feature-Vector Representation and the Accuracy–Complexity Surface

The final representational lever is the joint choice of **feature-vector length** (how many features) and **feature bit-size** (how many bits per feature), both weighed against representation accuracy. The original plaintext FingerCode implementation used **640 features**. A privacy-preserving implementation cannot afford that width, so it was tested across **eight configurations** with different feature counts and bit-sizes, measured by the **Equal Error Rate (EER)** — the operating point at which the false-acceptance and false-rejection rates coincide, a standard single-number summary of biometric accuracy where lower is better.

The results expose the trade-off directly. A representative set of configurations is:

Number of features	Feature bit-size	EER	Bandwidth, 1 match (bits)	Runtime, 1 match (sec)
192	2	0.0715	16 916	0.4443
192	4	0.0673	19 940	0.5366

Number of features	Feature bit-size	EER	Bandwidth, 1 match (bits)	Runtime, 1 match (sec)
96	2	0.0758	16 100	0.3743
96	4	0.0732	19 124	0.4558

Reading the table confirms the expected relationships. Increasing the bit-size from 2 to 4 slightly **improves accuracy** (EER falls) but **increases both bandwidth and runtime**, because wider operands make the secure arithmetic heavier. Reducing the feature count from 192 to 96 **cuts cost** (lower runtime) at a modest **accuracy penalty**. None of these configurations is universally "best"; each occupies a different point on the accuracy–complexity surface, and the designer selects among them according to which resource — error rate, latency, or bandwidth — is most constrained in the target deployment. The overarching message of the whole biometric case study is that representation and algorithm choices, not the cryptographic primitive alone, determine whether a privacy-preserving system is practical.

4. The Problem: Remote Diagnosis Without Disclosure

Now we dive into Privacy-Preserving Medical Diagnosis.
Telemedicine creates a privacy dilemma in its purest form. A patient wants a remote service to answer a medical question — in the canonical example, *"Am I healthy?"* — by analyzing a physiological signal such as an electrocardiogram (ECG). To do this the patient must send the signal to the service, which runs a diagnostic classifier and returns a verdict. But an ECG is deeply sensitive: it reveals cardiac conditions, can be linked to identity, and could be abused by insurers or employers. Equally, the diagnostic service has its own asset to protect — the **classifier itself**, which embodies proprietary medical knowledge and commercial value that the provider will not simply give away.

The privacy-preserving solution lets the patient encrypt the signal, send ciphertext to the server, have the server classify it in the encrypted domain, and return an encrypted answer that only the patient can decrypt — arriving at *"I am healthy!"* without the server ever seeing the ECG and without the patient ever seeing the classifier. This is a **two-sided privacy** problem: both parties have secrets, and a correct protocol must protect both simultaneously.

5. The Underlying Plain-Domain Algorithm (Ge et al.)

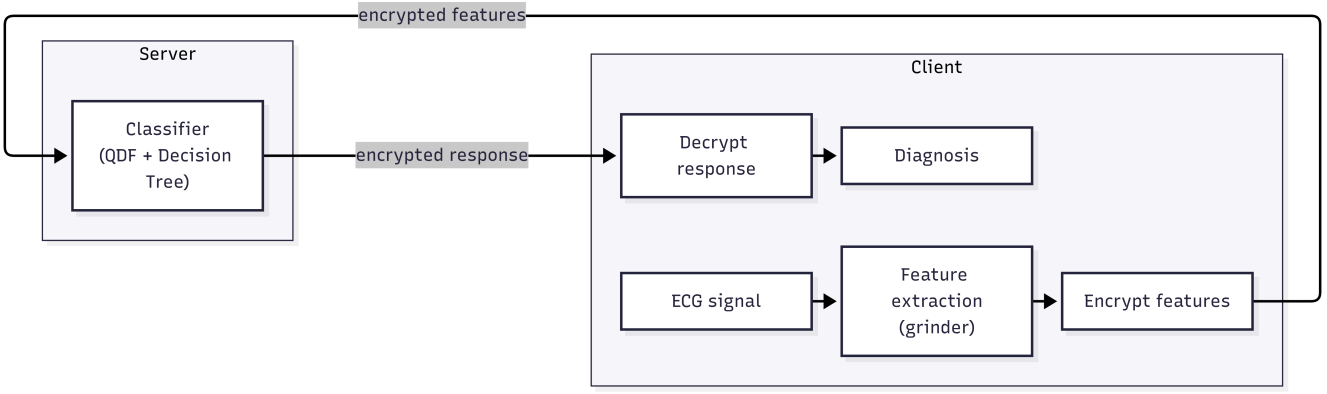
Before a secure protocol can be built, the plaintext computation must be understood exactly, because the secure version will mirror its structure. The chosen baseline is the **Ge et al. algorithm** for cardiac arrhythmia classification, which performs **single-beat classification**: it looks at one heartbeat at a time and assigns it to one of several cardiac states. The states it distinguishes are **NSR** (Normal Sinus Rhythm — a healthy beat), **APC** (Atrial Premature Contraction), **PVC** (Premature Ventricular Contraction), **VT** (Ventricular Tachycardia), **VF** (Ventricular Fibrillation), and **SVT** (Supraventricular Tachycardia). These are the six classes the system must output.

The algorithm runs in three stages. First comes **feature extraction**, carried out by fitting an **autoregressive (AR) model** to the beat; the AR coefficients become the feature vector that compactly describes the shape of the heartbeat. Second is a **quadratic discriminant function (QDF)**: the feature vector is **projected onto six different directions**, one associated with each class, producing a set of discriminant scores. Third is a **decision tree** that takes those scores and navigates a sequence of comparisons to reach the final class label. On plaintext data this pipeline achieves an accuracy of **88.57%**, which becomes the reference figure the secure implementation tries to approach.

6. The Secure ECG Diagnosis Architecture

Mapping the Ge et al. pipeline onto a client–server protocol requires deciding **which party performs which stage** and **where the privacy-preserving boundary falls**. The patient is the **client**; the diagnostic provider is the **server** holding the classifier.

On the client side, the ECG is acquired and then passed through **feature extraction** (the "grinder" that reduces the raw waveform to AR features). These features are encrypted and sent to the server as the protocol's protected input. The server holds the **classifier** — the QDF projection directions and the decision tree — and evaluates it on the encrypted features. The server's output, an encrypted **response**, travels back to the client, who decrypts it to obtain the diagnosis. The keys never leave the client, so the server computes blindly throughout.

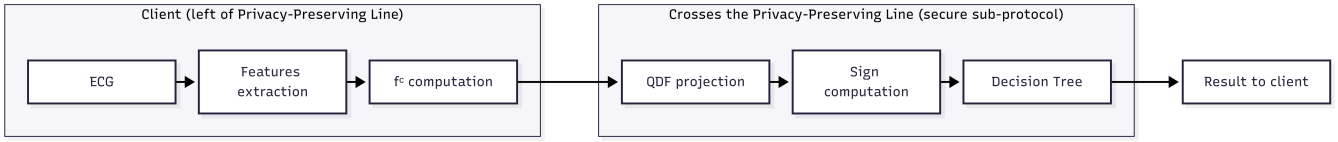


7. The Security Model

The protocol must state precisely **what each party protects**, **what adversary it defends against**, and **where the trust boundary lies**. This is captured by a **privacy-preserving line** that cuts through the computation: operations on the client's side of the line run on cleartext owned by the client, while operations crossing the line must be executed securely so that neither party learns the other's secret.

The division of the pipeline is as follows. The **client** performs the early, data-local stages on its own machine: the **ECG acquisition**, the **feature extraction**, and a preliminary **feature computation** (often written $\mathbf{f}^c$, the client's feature vector). These touch raw patient data and therefore stay inside the client's protected region. The stages that use the server's model — the **QDF projection**, the **sign computation** (determining the sign of each discriminant, which drives the branching), and the **decision-tree evaluation** — straddle the privacy-preserving line and are carried out by the secure sub-protocol, after which the client receives the final **result**.

The two parties' protection requirements are explicit and complementary. The **client wants to protect** its **ECG data**, the extracted **features**, and the final **classification result** — everything about the patient. The **server wants to protect** its **QDF coefficients**, the branching weights β_i, and the structure of its **decision tree** — everything about the model. The guarantee offered is security **in the semi-honest (honest-but-curious) setting**: each party faithfully follows the protocol steps but may try to infer the other's secret from whatever messages it legitimately receives. This is a weaker but far more efficient model than security against actively malicious parties, and it is the standard assumption for this class of signal-processing protocols.



The client's protected set is {ECG data, features, classification result}; the server's protected set is {QDF coefficients, β_i, Decision Tree}; the whole construction is **secure in a semi-honest setting**.

8. Linear Branching Programs (LBPs)

The decision stage of the classifier is formalized as a **Linear Branching Program (LBP)**, a model that generalizes the ordinary decision tree by allowing each decision to depend on a **linear combination** of the inputs rather than a single attribute. This abstraction is what lets the QDF-plus-tree pipeline be expressed as one clean object that can be evaluated securely.

Formally, an LBP evaluated on an input vector $\mathbf{x}$ is a triple

$$L = \langle P_1, \ldots, P_z, \text{ Left}, \text{ Right} \rangle,$$

where $P_1, \ldots, P_z$ are the **nodes** of a directed acyclic graph. The nodes split into two kinds. The first d nodes, $P_1, \ldots, P_d$, are **decision nodes**, each written as a pair

$$P_i = \langle \mathbf{a}_i, t_i \rangle,$$

in which $\mathbf{a}_i$ is the **linear-combination vector** used to weight the input $\mathbf{x}$, and t_i is a **threshold**. The remaining nodes, $P_{d+1}, \ldots, P_z$, are **classification nodes** — the leaves — each carrying a final label.

Evaluation starts at P_1 and, at each decision node P_i, computes the scalar product $\mathbf{a}_i \cdot \mathbf{x}$ and compares it to the threshold t_i. The two successor functions encode the branching: **Left(i)** gives the index of the next node when

$$\mathbf{a}_i \cdot \mathbf{x} < t_i,$$

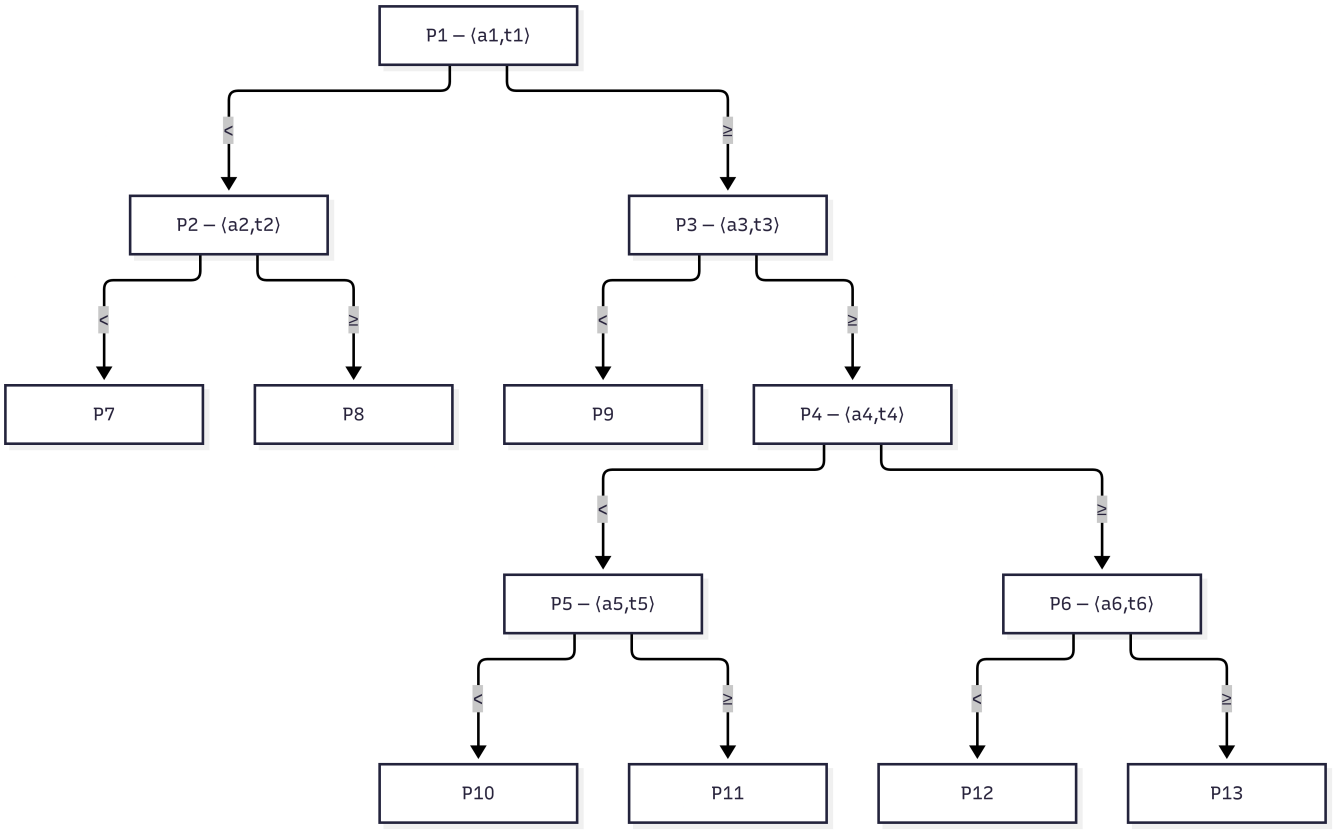
and **Right(i)** gives the index of the next node when

$$\mathbf{a}_i \cdot \mathbf{x} \geq t_i.$$

Traversal continues, following Left or Right at each decision node, until a classification node is reached; its label is the output.

For the ECG application the general LBP specializes neatly. The linear weights are the model's branching coefficients, $\mathbf{a}_i = \boldsymbol{\beta}_i$; every threshold is zero, $t_i = 0$; and the input vector is the client's feature vector, $\mathbf{x} = f^c$. Each decision therefore reduces to testing the **sign** of the scalar product $\boldsymbol{\beta}_i \cdot f^c$ — which is exactly why the architecture of Section 7 contained a dedicated **sign-computation** stage. Branching left or right according to whether a discriminant is negative or non-negative is the mechanism by which the QDF scores are turned into a class.

The concrete LBP used has a small, balanced structure: a root decision node P_1, two further decision nodes P_2 and P_3 beneath it, a fourth decision node P_4 below P_3, and then P_5 and P_6 beneath P_4, for **six decision nodes** in total. The leaves are the **classification nodes** P_7 through P_{13}, each corresponding to a possible diagnostic outcome, with every edge labelled by whether the test came out `<` (Left) or `≥` (Right).



In this tree the decision nodes $P_1, P_2, P_3, P_4, P_5, P_6$ each evaluate a linear test $\boldsymbol{\beta}_i \cdot \boldsymbol{f}^c$ against zero, while the twelve leaves $P_7, \ldots, P_{13}$ hold the classification results. Securely evaluating this LBP without revealing $\boldsymbol{f}^c$ to the server or the $\boldsymbol{\beta}_i$ and tree shape to the client is the problem the next sections solve.

Secure Evaluation of the Linear Branching Program

Having formalized the classifier as a Linear Branching Program, the question becomes *how* to evaluate it when the input vector $\boldsymbol{f}^c$ belongs to the client and the weights $\boldsymbol{\beta}_i$, thresholds, and tree topology belong to the server. Two architectures are possible, and comparing them exposes a recurring trade-off in secure computation: a single cryptographic tool applied uniformly is conceptually simple but inefficient, whereas combining complementary tools — each used where it is cheapest — yields a far more practical protocol.

9. Background: The Two Primitive Families

To follow both solutions, two cryptographic primitives must be kept in mind, because the whole design is a story about assigning each operation to whichever primitive handles it well.

Garbled Circuits (GC) express a computation as a Boolean circuit of logic gates, which is then "garbled" so that one party (the generator) encrypts the gate tables and the other (the evaluator) runs the circuit on encrypted wire values, learning only the final output. Garbled circuits evaluate *any* Boolean function and are especially efficient for **non-arithmetic** operations — comparisons, selections, and tree traversal — because these map to shallow Boolean logic. Their weakness is **arithmetic**: in particular, a multiplication of two ℓ-bit numbers requires a number of gates that grows **quadratically** in the bit length ℓ, so circuits full of multiplications become enormous.

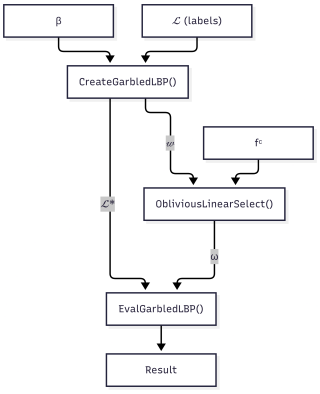
Homomorphic Encryption (HE) allows arithmetic to be performed directly on ciphertexts. In an *additively* homomorphic scheme (such as Paillier), the product of two ciphertexts decrypts to the *sum* of the plaintexts, and a ciphertext raised to a public exponent decrypts to the plaintext *multiplied* by that exponent. This makes HE ideal for **linear operations** — weighted sums and scalar products — but it is poor at comparisons and data-dependent branching, which are awkward or impossible to express as pure additions.

The dividing line is therefore clear: HE is cheap for the **linear** parts (the scalar products $\boldsymbol{\beta}_i \cdot \boldsymbol{f}^c$), and GC is cheap for the **non-linear** parts (the sign tests and the tree evaluation). A good protocol routes each operation to its natural home.

10. The Full-Garbled-Circuit Solution

The first, uniform approach evaluates the **entire** LBP inside one large garbled circuit. Everything the classifier does is compiled into Boolean logic: the **products** $\boldsymbol{\beta}_i \cdot \boldsymbol{f}^c$, the **additions** that accumulate each scalar product, the **comparisons** of each result against its threshold, and the **tree evaluation** that routes the computation from the root to a leaf. The circuit takes the client's features $\boldsymbol{f}^c$, the branching weights $\boldsymbol{\beta}$, and the leaf labels $\mathcal{L}$, and produces the result.

The workflow of this solution, often named **SecureEvalPrivateLBP**, is a pipeline of garbled-circuit sub-procedures. **CreateGarbledLBP()** builds the garbled representation of the branching program from $\boldsymbol{\beta}$ and the labels $\mathcal{L}$, producing a garbled program w together with the garbled leaf labels $\mathcal{L}^*$. **ObliviousLinearSelect()** then combines the client's input with the garbled program — obliviously, so that neither the selection nor the operands are revealed — yielding the garbled wire values ω needed to drive the evaluation. Finally **EvalGarbledLBP()** runs the garbled circuit on ω and $\boldsymbol{f}^c$ and outputs the classification result.



The fatal weakness of this design is its **high communication complexity**. Because the LBP contains genuine multiplications — each decision node forms a scalar product of multi-bit numbers — and because **multiplications in a garbled circuit have a gate count that depends quadratically on the bit length of the inputs**, the circuit becomes very large. Garbled-circuit communication is proportional to the number of (non-free) gates, so a quadratic blow-up in gates means a quadratic blow-up in the data that must be transmitted. The arithmetic that HE would handle almost for free is exactly what makes the all-GC circuit prohibitively expensive.

11. The Hybrid Protocol (HE + GC)

The hybrid protocol fixes this by moving the arithmetic out of the circuit and into **homomorphic encryption**, leaving the garbled circuit to do only what it is good at. The architecture splits the LBP evaluation at the natural seam between its linear and non-linear parts.

11.1 The Homomorphic Scalar Product

The expensive multiplications are the computation of each discriminant $y_i = \boldsymbol{\beta}_i \cdot \boldsymbol{f}^c$. These are evaluated under additively homomorphic encryption. The client encrypts its feature vector and sends the ciphertexts $\mathrm{Enc}[\boldsymbol{f}^c]$ across the privacy-preserving line to the server. The server, which knows the weights $\boldsymbol{\beta}_i$ in the clear, computes the encrypted scalar product using the homomorphic property:

$$\mathrm{Enc}[y_i] = \mathrm{Enc}!\left[\sum_{j=1}^{n} f_j^c \cdot \beta_i^j\right] = \prod_{j=1}^{n} \mathrm{Enc}[f_j^c]^{\beta_i^j}.$$

Reading this equation reveals exactly why HE is the right tool. A plaintext weighted sum of products $\sum_j f_j^c \beta_i^j$ becomes, in the encrypted domain, a **product of ciphertexts each raised to the corresponding public weight** β_i^j. Multiplying a feature by a known coefficient becomes an exponentiation, and summing the terms becomes multiplying the ciphertexts together — no interaction, no circuit, no quadratic gate explosion. The server obtains $\mathrm{Enc}[y]$, the encrypted vector of discriminant scores, purely by local computation on ciphertext. In the architecture this stage appears as the **Scalar product** block on the server, consuming $\mathrm{Enc}[f^c]$ and producing $\mathrm{Enc}[y]$.

11.2 The HE-to-GC Interface via Blinding

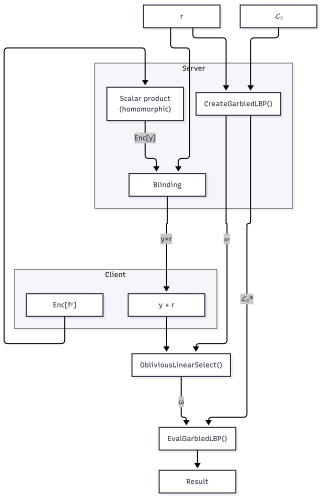
The discriminants y now exist as ciphertexts, but the next stage — comparing their signs and walking the tree — must happen inside a garbled circuit, which operates on Boolean wire values, not Paillier ciphertexts. A conversion is required, and it is achieved by **blinding**, the standard **HE-to-GC interface**.

The server draws a random value r and adds it homomorphically to the encrypted result, obtaining $\mathrm{Enc}[y + r]$, which it sends to the client. The client decrypts to recover $y + r$ — a value that is *masked*, since without knowing r it reveals nothing about the true discriminant y. Now the masked value $y + r$ is held by the client and the mask r is held by the server, so the secret y is **additively shared** between the two parties. This shared representation is precisely what a garbled circuit can consume: the circuit's first job is to subtract r from $y + r$ to reconstruct y internally, after which it proceeds with the comparison and tree logic without either party ever seeing y in the clear.

11.3 The Smaller Circuit

Because the multiplications have been offloaded to HE, the garbled circuit is now responsible only for the **comparison** (the sign tests on the reconstructed discriminants) and the **tree evaluation** (routing to the correct leaf). These are the operations at which GC excels and whose gate count does *not* blow up quadratically, so the resulting circuit is dramatically smaller than the full-GC version.

The hybrid workflow therefore interleaves the two worlds. After the homomorphic **scalar product** produces $\mathrm{Enc}[y]$ and **blinding** turns it into the shared $y + r$, the garbled part takes over: **CreateGarbledLBP()** builds the (now small) garbled program from r and the garbled labels $\mathcal{L}_2$, producing w and $\mathcal{L}_2^*$; **ObliviousLinearSelect()** feeds the client's masked value in to produce the garbled wires ω; and **EvalGarbledLBP()** evaluates the comparisons and tree to yield the result.



The hybrid protocol is the concrete embodiment of the methodology from Section 2: by analyzing the computation, splitting it along the linear / non-linear seam, and assigning each piece to the primitive that evaluates it cheaply, it achieves the same functionality as the full-GC solution at a fraction of the communication cost. The remaining design questions — how to represent the numbers so that the HE arithmetic stays correct and the circuit stays small — are what the data-representation analysis addresses next.

Making the ECG Protocol Practical: Representation and Complexity

12. Data Representation: The Theoretical (Worst-Case) Analysis

The cryptographic machinery of the previous section — homomorphic scalar products and garbled comparisons — operates exclusively on **integers**, because both Paillier encryption and Boolean circuits are defined over integer or bit domains. The plaintext Ge et al. algorithm, by contrast, works on **floating-point numbers (reals)**: the AR features and the QDF coefficients are continuous quantities. Bridging this mismatch is the role of **quantization**, the step that converts real-valued data into integers suitable for encrypted computation. The plain-domain algorithm thus works on reals; after quantization, the encrypted-domain algorithm works on integers.

Quantization multiplies each real by a large scaling factor and rounds to the nearest integer. For the two sensitive quantities this is written as

$$\beta_{q,j} = \rho_1 \beta_j + \varepsilon_{1,j} = \lfloor \rho_1 \beta_j \rceil,$$

$$f^c_{q,j} = \rho_2 f^c_j + \varepsilon_{2,j} = \lfloor \rho_2 f^c_j \rceil,$$

where ρ_1 and ρ_2 are the scaling factors, the symbol $\lfloor \cdot \rceil$ denotes rounding to the nearest integer, and $\varepsilon_{1,j}, \varepsilon_{2,j}$ are the **rounding errors** introduced by that rounding. Each error lies in the interval $[-\frac{1}{2}, \frac{1}{2}]$. The larger the scaling factor ρ, the finer the resolution and the smaller the *relative* error — but also the larger the resulting integers, and hence the more bits every encrypted value consumes. Representation design is therefore a direct instance of the accuracy-versus-complexity trade-off: scale up for accuracy, scale down for efficiency.

Two goals pull against each other: **minimize the data size** (the number of bits, which governs both ciphertext length and circuit width) while **preserving the result accuracy** (keeping the propagated rounding error below a tolerance). To guarantee correctness, the designer performs a **worst-case theoretical analysis**: assuming the least favourable combination of rounding errors, how many bits are needed so that the quantized computation still yields the same classification as the exact one?

Carrying this analysis through for a target error bound ε^* of 10^{-5} gives a scaling factor $\rho \approx 2.8 \times 10^{11}$, a **quantized input data bit-size of 56 bits**, and a **scalar-product result bit-size of 116 bits**. The scalar product result is roughly twice the input width, which is expected: multiplying two $\sim$56-bit operands and summing produces a value near 112–116 bits. These worst-case numbers are safe but pessimistic, and the next step asks whether they can be relaxed in practice.

ε^*	**Bit size of quantized input data**	ρ	**Bit size of scalar-product result**
10^{-5}	56	2.8×10^{11}	116

13. Experimental Analysis: How Many Bits Are Really Needed?

The worst-case bound of 56 bits assumes an adversarially unlucky alignment of rounding errors that essentially never occurs with real data. To discover the bit-length actually required, an **empirical study** is performed: a **plain-domain simulator** is built that reproduces the full classification pipeline but operates on **quantized values** of a chosen bit-length ℓ, allowing the **classification accuracy to be plotted as a function of the representation bit-length**.

The resulting curve has a characteristic shape. At very small bit-lengths (around 10 bits) accuracy sits near the floor — roughly 0.17, essentially random among the classes — because the quantization is so coarse that the features are destroyed. As ℓ grows the accuracy climbs, passing through an intermediate regime (near 0.58) and then rising steeply, until it **saturates at its maximum plateau around 44 bits**. Beyond 44 bits, adding more bits brings no further accuracy, and the curve for the full feature set is flat all the way to 90 bits. The practical conclusion is decisive: **44 bits suffice** to reach full classification accuracy, so the conservative 56-bit worst-case figure can be cut substantially without any measurable loss. The theoretical bound guarantees correctness; the experiment reveals how much slack that guarantee actually contains.

14. Speeding Up the Protocol by Feature Reduction

A second, independent lever for acceleration is **input choice** — reducing the *number* of features rather than their bit-width. Recall that each decision node of the LBP forms a scalar product over the feature vector, so every feature removed shrinks both the homomorphic computation and the circuit. The question is whether features can be dropped without wrecking accuracy.

The experiment shows that **discarding the fifth feature** lowers the plaintext accuracy only marginally — from **88.57% down to 86.30%** — a loss of about two percentage points. This is the explicit **trade-off with complexity**: a small, deliberate accuracy sacrifice buys a cheaper protocol. Re-running the bit-length study for the reduced representation reveals a further benefit. The full **21-feature** configuration needs about **44 bits** to saturate, whereas the reduced **15-feature** configuration saturates earlier, at roughly **24 bits**. Fewer features therefore reduce accuracy slightly *and* reach their accuracy ceiling at a shorter bit-length, compounding into a considerable overall saving. The 86.30% figure becomes the reference accuracy that the later neural-network solution will try to match, precisely because it represents the cheap, feature-reduced operating point.

15. Complexity of the Real Implementation

The two candidate architectures — the full **Garbled-Circuit (GC)** solution and the **Hybrid (HE + GC)** solution — are finally measured head-to-head on real implementations, across several test configurations and under two cryptographic **security levels**. Security level here fixes the key sizes: **short-term security** uses a homomorphic modulus of $T = 1248$ bits and a symmetric/statistical security parameter $t = 80$, while **medium-term security** uses the larger $T = 2432$ bits and $t = 112$. Larger parameters mean stronger long-term protection but heavier computation and communication.

The tests vary the number of **features**, the parameter n (feature-vector dimension), and the representation bit-length ℓ. Three main configurations are compared: Test 1 with 5 features, $n = 21$, $\ell = 54$; Test 2 with 5 features, $n = 21$, $\ell = 44$; and Test 3 with the simplified classifier of 4 features, $n = 15$, $\ell = 24$. The findings separate cleanly into three conclusions.

First, on **communication complexity** the hybrid protocol wins decisively. For Test 1 under short-term security the full GC transmits about **28.1 MByte** against the hybrid's **105 kByte**; under medium-term

security, **39.3 MByte** against **151 kByte**. The same pattern holds across every test (Test 2: 18.6 MByte vs 88 kByte, short-term; Test 3: 3.9 MByte vs 50 kByte). The **communication complexity of the GC approach is definitely higher** — by two to three orders of magnitude — which is exactly the quadratic-gate blow-up of in-circuit multiplication that the hybrid design was built to avoid.

Second, on **computational complexity** the picture is more nuanced and is read from the runtime table (client and server CPU and total times, in seconds). For **short-term security** the computation is **slightly in favour of the hybrid** implementation: in Test 1 the hybrid's total times (client 35.4 s, server 34.2 s) edge out the GC's (client 64.5 s, server 64.7 s), and in Test 2 the hybrid again leads (29.0 / 27.6 s vs 48.5 / 48.8 s). As the number of features decreases (Test 3) the two become **similar** — hybrid 18.7 / 16.2 s versus GC 17.5 / 19.2 s — the advantage narrowing as the circuit shrinks.

Third, and counter-intuitively, with **medium-term security** the GC solution can become **attractive**. In the starred configuration 3* (the simplified 4-feature, $n = 15$, $\ell = 24$ classifier under medium-term security), the GC implementation (client 20.4 s, server 20.8 s) actually **beats** the hybrid (client 40.5 s, server 30.9 s). The reason is that the hybrid protocol's cost is dominated by **homomorphic operations**, whose per-operation cost grows sharply with the modulus T; when T jumps from 1248 to 2432 bits the HE arithmetic becomes very expensive, whereas the garbled circuit — especially a small one on a reduced classifier — scales more gently. **With medium-term security and the simplified classifier, the GC implementation may become the better choice.**

The overall takeaway reinforces the methodology: there is no single winning primitive. The hybrid protocol is the right default because of its overwhelming bandwidth advantage, but as security parameters grow and the classifier shrinks, the balance of computational cost can tip back toward the pure garbled-circuit design, and the designer must re-measure rather than assume.

The Neural Network Solution for ECG Classification

16. Rationale and Design Goal

The Linear Branching Program is not the only classifier that can distinguish the six cardiac states. An alternative is a **neural network**, which replaces the explicit decision tree with a learned non-linear mapping from features to classes. Casting the diagnosis this way is attractive because a neural network is built almost entirely from the operations that secure computation handles well — weighted sums and a simple, repeated non-linearity — which makes its structure regular and predictable in the encrypted domain. The explicit **design goal** is to match the cheap operating point established earlier: **86.3% correct classification**, the accuracy of the feature-reduced LBP. The network is engineered to reach that target while remaining implementable with fixed-point integer arithmetic inside a garbled circuit.

17. Network Topology: Inputs, Outputs, and Hidden Layer

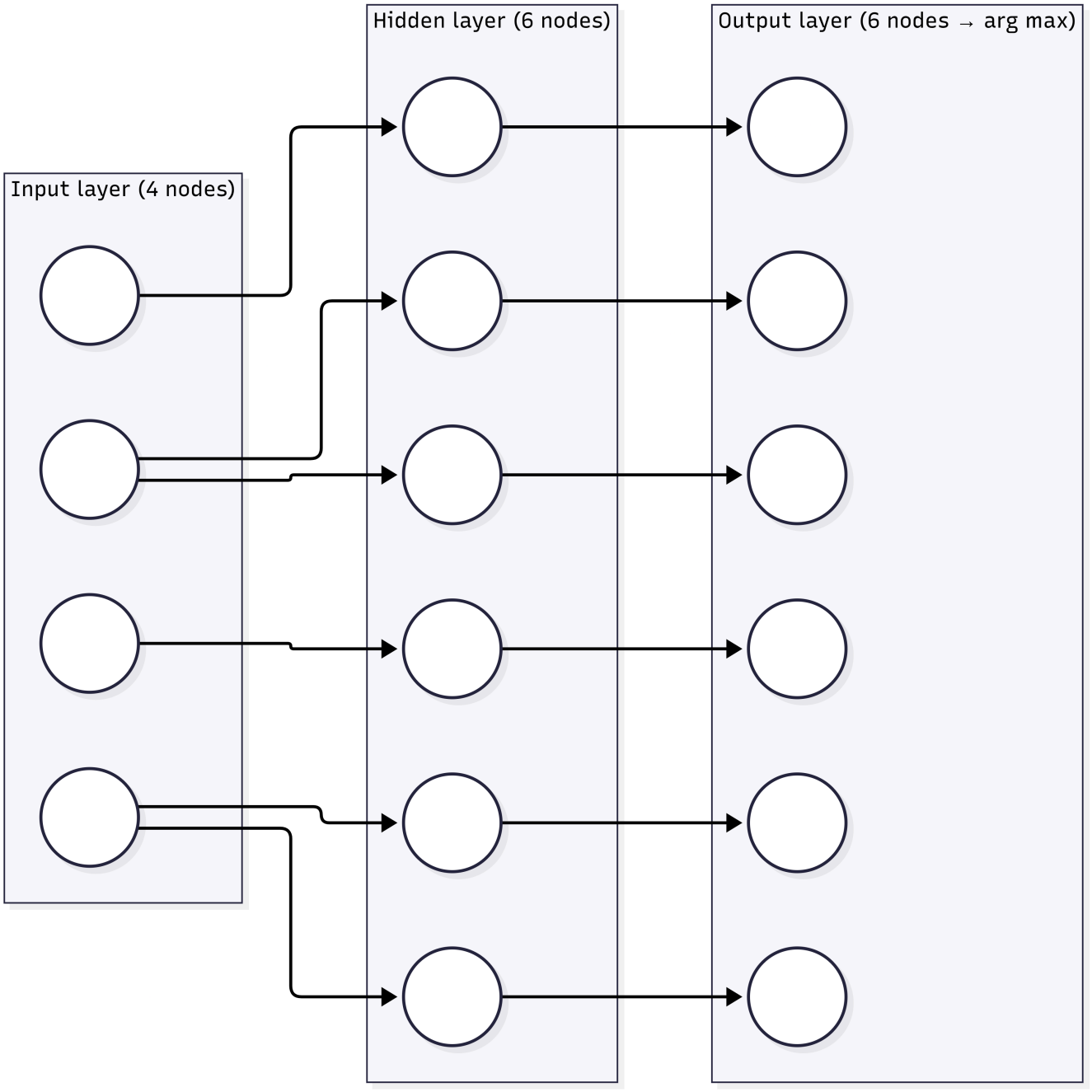
The network is a standard feed-forward multilayer perceptron, and each layer's size is fixed by the task.

The **inputs** are the **reduced feature vector** $f = [a_1, a_2, a_3, a_4]$ — the four autoregressive coefficients that survive feature reduction — so the network has **4 input nodes**. Using the already-reduced feature set keeps the first weight matrix small and aligns the neural solution with the same economical representation chosen for the LBP.

The **output** layer encodes the six cardiac classes directly, one node per class, giving **6 output nodes** arranged as an array of six elements. The encoding is a **one-vs-all target scheme**: during training each output element is driven to -1 when the ECG chunk does **not** belong to the corresponding class and to $+1$ when it **does** belong to that class. At test time the network does not output clean ± 1 values but a

spread of real numbers, and the class is decided by **arg max** — the output node carrying the **maximum value** identifies the predicted class. This max-selection is important for the secure implementation, because arg max is a sequence of comparisons, exactly the kind of operation a garbled circuit evaluates cheaply.

The size of the **hidden layer** is not dictated by the problem and must be chosen empirically. It is fixed by **accuracy tests**: the network is trained with different hidden-layer sizes and the configuration that first meets the goal is selected. Plotting accuracy against the number of hidden nodes shows the accuracy rising and then hovering around the **86.30% goal line**; the smallest size that reliably sits on or above that line is **6 nodes**. The final topology is therefore **4 inputs → 6 hidden → 6 outputs**, with arg max on the output array.



18. Implementation: Computational Topology and Bit-length Choice

Translated into arithmetic, the network is a fixed pipeline of matrix operations punctuated by a single non-linearity, and this is the form that is actually garbled. Starting from the input feature vector **f** of

shape $(n \times 1)$, the computation proceeds as follows: multiply by the **hidden weight matrix** $\mathbf{W}_h$ of shape $(n_h \times n)$, add the **hidden bias** $\mathbf{b}_h$ of shape $(n_h \times 1)$, and apply the activation function (**SATLIN**) to obtain the hidden activations $\mathbf{y}_h$ of shape $(n_h \times 1)$. These are then multiplied by the **output weight matrix** $\mathbf{W}_o$ of shape $(n_o \times n_h)$, the **output bias** $\mathbf{b}_o$ of shape $(n_o \times 1)$ is added to produce the output vector $\mathbf{y}_o$ of shape $(n_o \times 1)$, and finally **arg max** selects the winning class as the scalar output `out`. The quantities that must be represented as fixed-point integers and kept secret from the client are the four parameter blocks $\mathbf{W}_h$, $\mathbf{b}_h$, $\mathbf{W}_o$, $\mathbf{b}_o$ — the model's weights and biases.



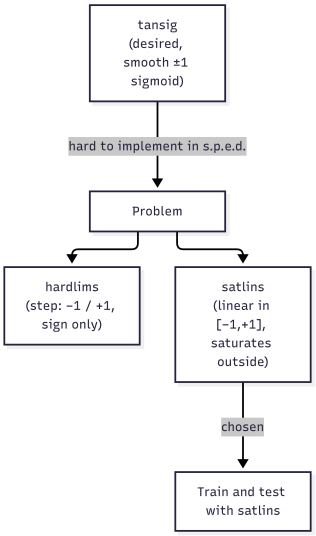
As with the LBP, the **bit-length of the fixed-point values** must be chosen to be as small as possible without losing accuracy, and again this is settled by sweeping ℓ and watching where accuracy saturates against the 86.30% goal. Two separate bit-lengths are tuned. The shared bit-length of the input, hidden, and intermediate representations — ℓ^i, ℓ^h, ℓ^o taken together — reaches the goal plateau at about **13 bits**: below roughly 8 bits accuracy collapses, it climbs steeply through the low teens, and flattens on the goal line by 13 bits. The bit-length of the **output stage** ℓ^o alone can be made even tighter, saturating at only **8 bits**, since the output values feed only an arg-max comparison and need far less precision than the internal arithmetic. Choosing 13 bits internally and 8 bits at the output minimizes the circuit while preserving the target accuracy.

19. Design of the Activation Function

The non-linearity is the one part of a neural network that is genuinely awkward for secure computation, and its choice is the crux of the design. The **desired** activation function, used in conventional training, is **`tansig`** — the hyperbolic-tangent sigmoid, a smooth S-shaped curve rising from -1 to $+1$. Its smoothness is exactly its problem: evaluating a transcendental function like tanh is **difficult to implement in s.p.e.d.**, because circuits and homomorphic schemes compute additions and multiplications, not exponentials, so `tansig` would have to be approximated by an expensive polynomial or lookup.

Two other functions bracket the design space. **`hardlims`** (the symmetric hard limiter) is a pure step: it outputs -1 for any negative input and $+1$ for any non-negative input, with an instantaneous jump at zero. It is trivial to compute — just a sign test — but its total flatness discards all magnitude information and degrades learning. **`satlins`** (the symmetric saturating linear function) is the compromise: it is **linear with unit slope between -1 and $+1$**, and **saturates** flat at -1 for inputs below -1 and at $+1$ for inputs above $+1$. Within the central band it behaves like a straight line — cheap, since it is just the identity — and outside it clips to the constants, requiring only two comparisons. It is, in effect, a piecewise-linear approximation of `tansig` that a garbled circuit can evaluate with a handful of gates.

Two experiments were run to validate this. In the **first test**, the network was **trained with `tansig`** (to get the best-quality weights) and then subjected to **integer tests using an approximation** of the activation; among the candidate approximations the **best result came from `satlins`**. In the **second test**, the network was **both trained and integer-tested entirely with `satlins`**, aligning training and deployment so that the model learns weights suited to the exact non-linearity it will use at inference. Training directly on the function that will be deployed removes the mismatch between a smooth training activation and a clipped evaluation activation, and `satlins` emerges as the activation of choice: accurate enough to hit the goal, cheap enough to garble.



20. Comparison: Hybrid LBP versus Full-GC Neural Network

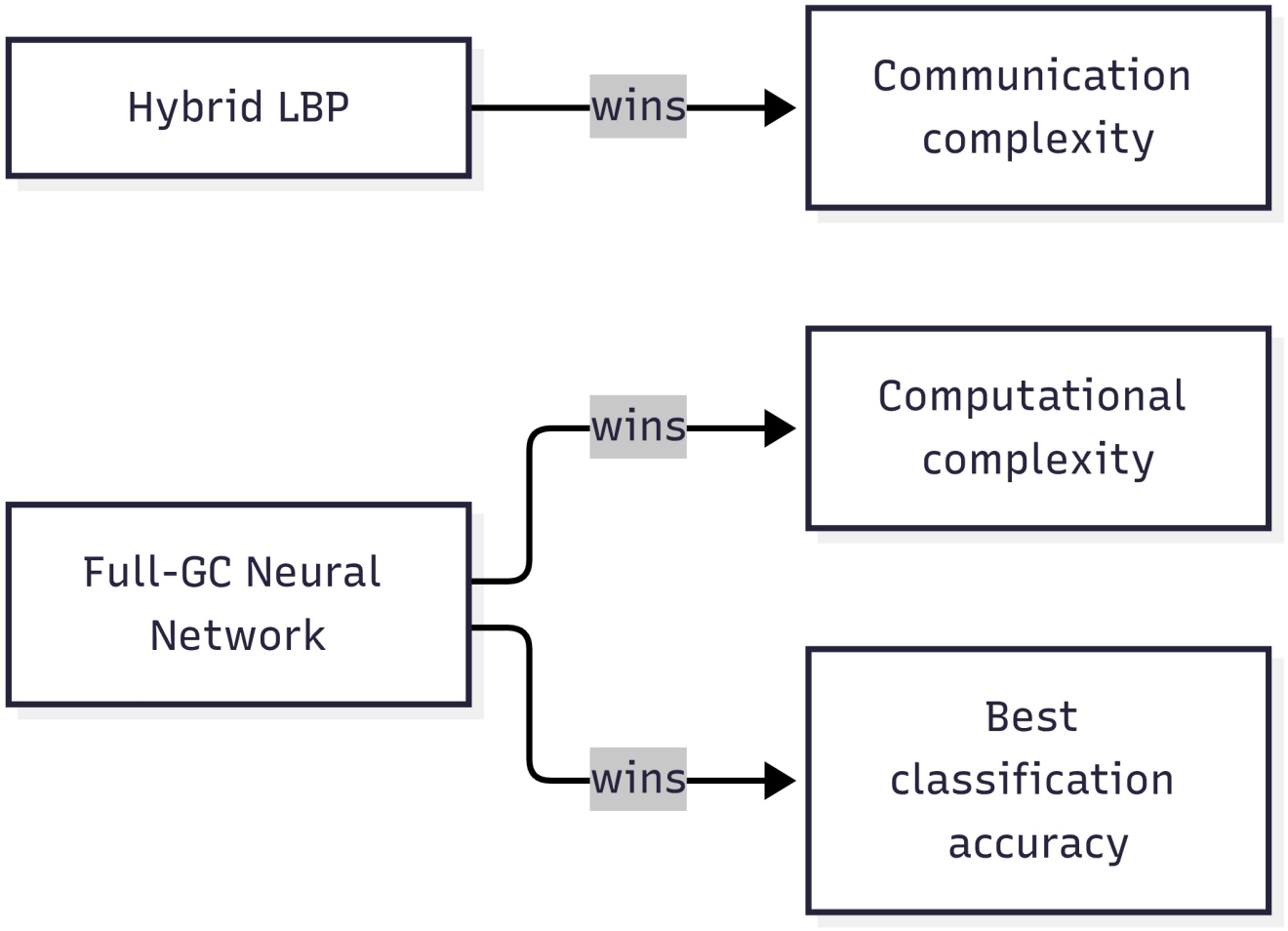
With both classifiers fully specified, the two privacy-preserving approaches to ECG diagnosis can be weighed against one another. The comparison is best organized by which solution wins on each of the three design axes.

On **communication complexity**, the **Hybrid LBP** is the winner. Because it offloads all multiplications to homomorphic encryption and garbles only the small comparison-and-tree stage, it transmits far less data than any full-circuit alternative — the same bandwidth advantage established in Section 15.

On **computational complexity**, the **Full-GC Neural Network** is preferable. The neural network is implemented entirely as one garbled circuit (weighted sums, `satlins`, arg max) with no homomorphic operations, so it avoids the expensive public-key arithmetic that dominates the hybrid protocol's CPU time — particularly valuable at larger security parameters where HE operations become costly.

On **best classification accuracy**, the **Neural Network** leads. Its learned non-linear mapping fits the data better than the piecewise-linear discriminants of the branching program, so for a given operating

point it tends to classify most accurately.



No single solution dominates on all three criteria, which is the recurring moral of the whole subject: the choice between the hybrid branching program and the garbled neural network depends on whether bandwidth, CPU time, or raw accuracy is the binding constraint in a given deployment.

21. Summary of the ECG Beat-Classification Study

Stepping back, the entire ECG case study is a complete pass through the design methodology of Section 2. The **application-domain analysis** began from the Ge et al. protocol and then broadened to machine-learning classifiers — the **decision tree** (realized as an LBP) and the **neural network** — as the two tools capable of the task. The **protocol definition** fixed the requirements: **security against semi-honest parties** at both **short-term and medium-term** levels, **accuracy close to the plain protocol**, and a practical **runtime on the order of one second** per classification. The optimization and testing phases then produced the quantization bounds, the reduced feature set, the bit-length choices, the hybrid HE+GC construction, and the garbled neural network — each a concrete instance of trading a little accuracy or a little privacy-neutral restructuring for a large gain in performance. The result is a pair of deployable protocols that classify a heartbeat into one of six cardiac states without the server ever seeing the ECG and without the client ever seeing the classifier.

ODIN: Obfuscation-Based Privacy-Preserving Consensus for Decentralized Information Fusion

22. The Problem: Information Fusion in the Internet of Things

The final application shifts from a two-party client–server setting to a genuinely **distributed, many-party** one. In the **Internet of Things (IoT)**, large numbers of sensing devices must combine their

individual measurements into a single collective conclusion — a task called **information fusion**, which is the keystone of countless IoT systems. The defining feature of this setting is that it is **decentralized**: there is no central server that collects all readings and computes the answer. Instead the devices reach a shared conclusion cooperatively, and the mechanism they use is a **consensus algorithm**, in which each node repeatedly exchanges its current estimate with its neighbours and updates toward a common value until the whole network agrees.

Security has long been studied for such systems — protecting the consensus against nodes that inject false data, for instance — but **information privacy has often been neglected**. In consensus, every node ordinarily reveals its state to its neighbours at every round, so a curious or compromised neighbour can read off another device's private measurement. What is needed is a **privacy-preserving consensus** that lets **non-trusting nodes cooperate securely**: they jointly compute the correct fused result while no node learns the individual measurements of the others. ODIN — **Obfuscation-based privacy-preserving consensus algorithm for Decentralized Information fusion in smart device Networks** — is such a protocol, designed to run on real constrained devices.

23. Security and Privacy in Consensus Networks

The consensus setting has two distinct threat dimensions, and it is worth separating them because ODIN addresses the second.

The first is **security against attacks**: an adversary corrupts measurements to steer the consensus to a wrong value. Defending against this is a detection-and-robustness problem — identifying and censoring corrupted data so the network still converges correctly despite malicious inputs — and it is addressed with game-theoretic methods in which the defender and the data-corrupting attacker are modelled as players optimizing opposing objectives.

The second dimension, and ODIN's focus, is **privacy-preserving implementation**: ensuring that the legitimate exchange of states required by consensus does not leak each node's private value. The lineage of ideas here runs from **secure multi-party consensus gossip algorithms**, through the combination of **consensus with obfuscation** for privacy-preserving learning, and culminates in **ODIN**, which fuses obfuscation with efficient cryptographic tools to make privacy-preserving consensus practical on smart devices.

Behind both dimensions sits the standard behaviour of a consensus run: the nodes of a connected graph begin with scattered initial **state values** and, iteration after iteration, exchange and average with neighbours so that all trajectories **converge toward a common value**. The randomness of *which* pair of neighbours interacts at each step (a gossip schedule) is visible in the way the state curves jitter early on and then settle together after roughly one to two hundred iterations. ODIN must preserve this convergence while hiding the individual states throughout.

24. The Implementation Strategy at a Glance

ODIN splits its work into two kinds of steps, each matched to a suitable cryptographic tool — the same divide-and-assign philosophy seen in the ECG protocols, now applied to a distributed algorithm.

The repeated **update steps**, in which neighbours exchange and combine their running estimates, are protected by two complementary mechanisms. **Modular additive blinding** masks a value by adding a random number modulo N, giving **high efficiency** because masking and unmasking are mere additions. **Proxy re-encryption** is used so that the complexity of an update **does not depend on the number**

of neighbours a node has — a crucial property in dense IoT graphs where a node may be connected to many others.

The single **final decision step**, taken once the running estimate is trusted to be close enough to the true fused value, is handled by **garbled circuits**, because that step is a *comparison* of the result against a threshold — again, the non-arithmetic operation at which circuits excel.

25. Proxy Re-Encryption

Proxy re-encryption (PRE) is the cryptographic primitive that makes ODIN's update step scale independently of connectivity, so it deserves a precise description. PRE allows a semi-trusted intermediary — the **proxy** — to transform a ciphertext encrypted under one party's public key into a ciphertext of the *same message* under a different party's public key, **without ever learning the message** and without either party's secret key.

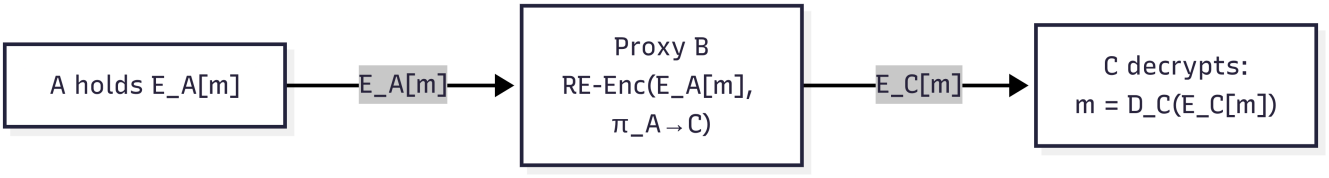
Concretely, suppose party **A** holds a ciphertext $E_A[m]$ — the message m encrypted under A's key. A **re-encryption key** $\pi_{A\to C}$ is generated that authorizes transformation from A to C. The proxy **B**, given $E_A[m]$ and $\pi_{A\to C}$, computes

$$E_C[m] = \text{RE-Enc}(E_A[m],\ \pi_{A\to C}),$$

producing a ciphertext of the very same m now readable by **C**. Finally C decrypts with its own secret key to recover

$$m = D_C(E_C[m]).$$

The security guarantee is that the proxy B performs this translation **blindly**: it sees only ciphertexts and the re-encryption key, never the plaintext m, and the re-encryption key alone does not let it decrypt. This lets a value encrypted for one node be handed to another node without round-trips back to the original encryptor, which is exactly what allows a consensus update to be routed between neighbours at a cost independent of how many neighbours there are.



26. Setup Phase

Before any consensus iterations run, the network performs a one-time **setup** that distributes the keys needed for blinding and proxy re-encryption. Each **agent** i carries out four actions. It **shares its own public key** with the network and, symmetrically, **receives the public keys of the other nodes**, so that any node can encrypt a message intended for any other. It then **generates its re-encryption keys** $\pi_{i\to j}$, one authorizing translation from itself to each other node j.

The distribution of these re-encryption keys follows a deliberately asymmetric rule that is the subtle heart of the setup. Agent i **shares with each adjacent node j all of its re-encryption keys *except*** $\pi_{i\to j}$ — the one key that would let j re-encrypt i's ciphertexts *to j itself*. Withholding $\pi_{i\to j}$ from j prevents j from unilaterally converting i's protected values into a form j can read, which is precisely the capability that would break i's privacy; by giving j every *other* re-encryption key, the protocol still lets j relay i's values onward to third parties as the consensus requires. Finally, the setup is not frozen: in

dynamic networks where nodes join, leave, or move, the **re-encryption keys can also be shared during the protocol**, not only at initialization, so the scheme adapts as the topology changes.

27. Data Representation

As in every encrypted-domain protocol, the continuous state values must be mapped to integers. ODIN does this with a shared quantization and then tracks each quantity as an exact fraction.

The **initial state values are quantized using a common multiplier** q. For node i the quantized starting value is

$$Y_i(0) = \lfloor q \times y_i(0) \rceil,$$

rounding the real initial estimate $y_i(0)$ to the nearest integer after scaling by the shared factor q (the common multiplier guarantees that every node's values live on the same integer scale, so they can be combined coherently).

Because the consensus update repeatedly averages values — an operation that produces fractions — **each value involved in the computation is represented as a rational number**, an explicit numerator over denominator:

$$Y_i(k) = n_i(k)/d_i(k).$$

Carrying numerator and denominator separately lets the protocol perform exact integer arithmetic (additions of fractions via a common denominator) and defer any division, which is essential because division is not something the encrypted-domain tools can do directly. In the **first step** the representation is trivial: the numerator is the quantized initial value and the denominator is one,

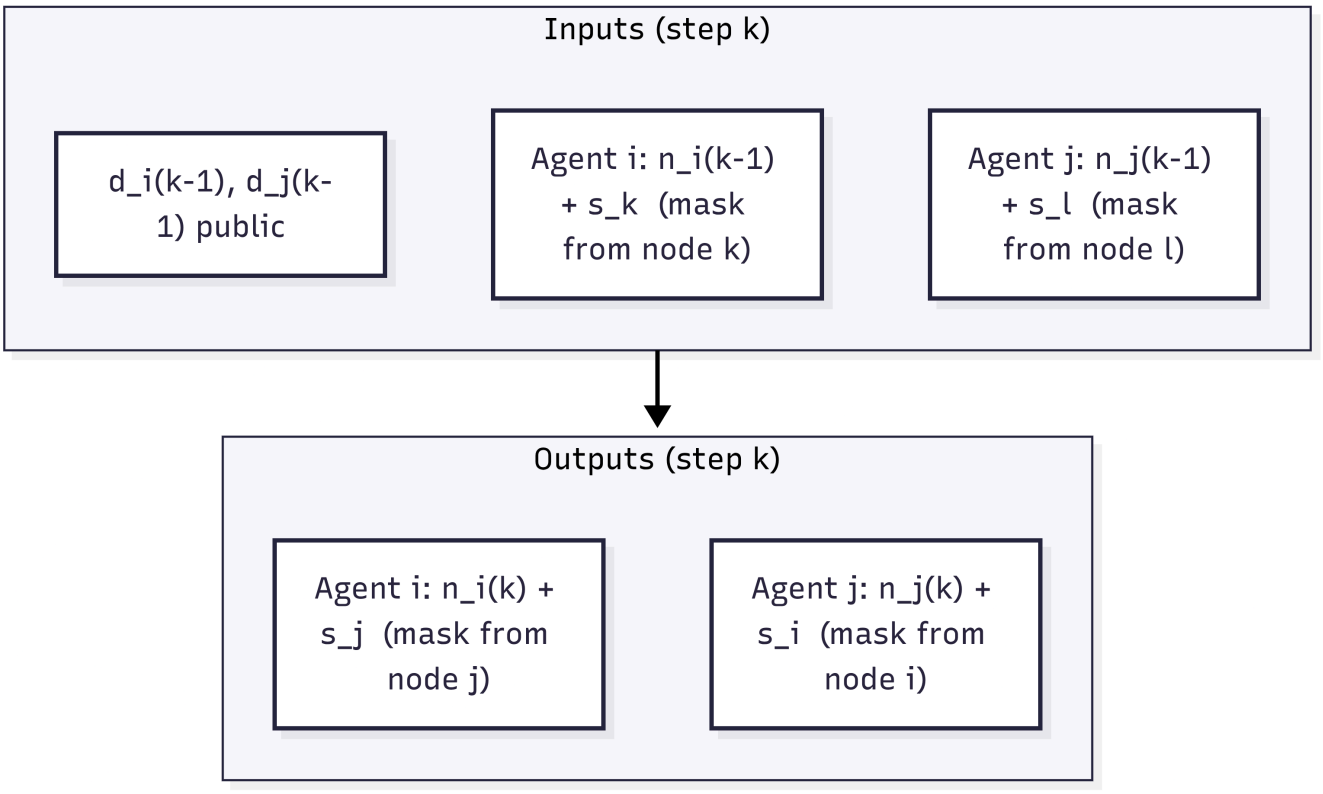
$$n_i(0) = Y_i(0), \qquad d_i(0) = 1.$$

28. The Privacy-Preserving Update Step (Step k)

Each consensus iteration pairs an **agent** i with one **neighbour** j, and the two update their running estimates jointly without revealing their numerators to each other. The design achieves this by keeping every node's secret numerator **blinded with a random value chosen by some *other* node**, so that the party holding the value cannot read it and the party who chose the mask does not hold the value. Understanding the input/output invariant is the key to the whole step.

Inputs. Agent i owns its numerator $n_i(k-1)$ **blinded with a random value chosen by another node** k — that is, i physically holds $n_i(k-1) + s_k(k-1)$, where the mask s_k was selected by node k, not by i, so i cannot recover its own cleartext numerator. Symmetrically, agent j owns $n_j(k-1)$ **blinded with a random value chosen by another node** l. In the very **first update step** this bootstrapping condition does not yet apply, so $n_i(0)$ and $n_j(0)$ are simply **known by the two parties**. The denominators $d_i(k-1)$ and $d_j(k-1)$ are **public**, since they carry no private information (they are just bookkeeping of how much averaging has occurred).

Outputs. After the step the invariant is re-established but with the masks rotated to the two participants: agent i owns its new numerator $n_i(k)$ **blinded with a random value chosen by node** j, and agent j owns $n_j(k)$ **blinded with a random value chosen by node** i. Each party now holds its updated value masked by the *other participant's* fresh randomness — neither can read its own value, and the privacy invariant is preserved for the next round.



29. Update Step — Detailed Implementation

The messages exchanged in step k (using τ for the iteration index) realize the input/output invariant above through a careful choreography of fresh randomness, proxy re-encryption, and homomorphic/additive masking. The following reconstructs the full flow, filling in the parts that the printed equation overlay obscures on the slide.

Fresh randomness. Each party draws two fresh random values uniformly from $\mathbb{Z}_N$: agent i draws $r_i(\tau), s_i(\tau) \leftarrow_R \mathbb{Z}_N$, and agent j draws $r_j(\tau), s_j(\tau) \leftarrow_R \mathbb{Z}_N$. Here r serves as a transient additive mask protecting a value in transit, while s is the mask each party contributes to the *other's* output (fulfilling the output invariant).

First message — exchanging the masked incoming numerators. Agent i starts from the value it holds, $n_i(\tau-1) + s_k(\tau-1)$ (masked by absent node k), and **re-masks it with its own fresh** $r_i(\tau)$, computing $n_i(\tau-1) + s_k(\tau-1) + r_i(\tau)$. It also takes the encrypted mask it is holding on k's behalf, $[![s_k(\tau-1)]!]_k$, and **proxy-re-encrypts it toward** j using the key $\pi_{k\to j}$, producing $[![s_k(\tau-1)]!]_j$ — a ciphertext of k's mask now readable by j. Agent i sends to j the triple

$$\Big(, n_i(\tau-1) + s_k(\tau-1) + r_i(\tau),\ \ [![s_k(\tau-1)]!]_j,\ \ d_i(\tau-1), \Big).$$

Agent j performs the mirror-image computation, re-masking its own held value with $r_j(\tau)$ and re-encrypting the mask it holds on l's behalf toward i with $\pi_{l\to i}$, sending

$$\Big(, n_j(\tau-1) + s_l(\tau-1) + r_j(\tau),\ \ [![s_l(\tau-1)]!]_i,\ \ d_j(\tau-1), \Big).$$

Unmasking the stale masks. Each party can now decrypt the re-encrypted mask it just received with its own secret key: j computes $s_k(\tau-1) \leftarrow \mathrm{Dec}([![s_k(\tau-1)]!]_j, pk_j)$ and i computes $s_l(\tau-1)$ $\leftarrow \mathrm{Dec}([![s_l(\tau-1)]!]_i, pk_i)$. Possessing $s_k(\tau-1)$ lets j strip that stale mask out of the value it received from i, and symmetrically for i. This is the point where proxy re-encryption pays off: the mask placed

long ago by an absent third node k can be removed by j *now*, without contacting k, because i was able to hand j a re-encrypted copy of it.

Denominator update. The denominators are advanced publicly via the least common multiple needed to add the two fractions. Each party computes $\mathrm{lcm}(\tau-1) \leftarrow \max(d_i(\tau-1), d_j(\tau-1))$ and sets the new denominator $d_i(\tau) \leftarrow 2, \mathrm{lcm}(\tau-1)$ (the factor of two reflecting that the update averages the two incoming values). Because denominators are public, both parties compute this identically.

Forming the new numerator (Eq. 6). Each party assembles the updated, masked numerator for the *other*. Writing the construction for the value j will end up holding, the quantity $n_j(\tau) + obf_j(\tau) + s_i(\tau)$ is formed as

$$
\begin{aligned}
& n_j(\tau) + obf_j(\tau) + s_i(\tau) \\
&\quad = \left(n_i(\tau-1) + s_k(\tau-1)\right) \frac{\mathrm{lcm}(\tau-1)}{d_i(\tau-1)} \\
&\qquad + \left(n_j(\tau-1) + r_j(\tau)\right) \frac{\mathrm{lcm}(\tau-1)}{d_j(\tau-1)} + s_i(\tau) \pmod{n} \\
&\quad = \underbrace{n_i(\tau-1)\frac{\mathrm{lcm}(\tau-1)}{d_i(\tau-1)} + n_j(\tau-1)\frac{\mathrm{lcm}(\tau-1)}{d_j(\tau-1)}}_{nj(\tau)\text{ (the true updated numerator)}} \\
&\qquad + \underbrace{s_k(\tau-1)\frac{\mathrm{lcm}(\tau-1)}{d_i(\tau-1)} + r_j(\tau)\frac{\mathrm{lcm}(\tau-1)}{d_j(\tau-1)}}_{obf_j(\tau)\text{ (the obfuscation term)}} + s_i(\tau) \pmod{n}.
\end{aligned}
$$

Reading this, the raw sum rescales both incoming numerators to the common denominator $\mathrm{lcm}(\tau-1)$ and adds them — which is exactly the true new numerator $n_j(\tau)$ — but it drags along the old masks (s_k, r_j) rescaled by the same factors, collected into the **obfuscation term** $obf_j(\tau)$, plus the fresh output mask $s_i(\tau)$. The obfuscation term is what keeps the value hidden during the exchange; it will be removed in the next phase.

Second message and de-obfuscation. The two parties exchange these assembled values together with the encryption of the fresh output mask for the recipient: i sends $\left(n_j(\tau) + obf_j(\tau) + s_i(\tau),\ [![s_i(\tau)]!]_i\right)$ to j, and j sends $\left(n_i(\tau) + obf_i(\tau) + s_j(\tau),\ [![s_j(\tau)]!]_j\right)$ to i, where the output masks are freshly encrypted as $[![s_i(\tau)]!]_i \leftarrow \mathrm{Enc}_2(s_i(\tau), pk_i)$ and correspondingly for s_j. Each party then **computes its own obfuscation term** $obf(\tau)$ — which it can do because it knows all the randomness and public denominators that went into the *other side's* obfuscation — and subtracts it, so that i finishes holding $n_i(\tau) + s_j(\tau)$ and j finishes holding $n_j(\tau) + s_i(\tau)$. This is precisely the output invariant of Section 28: each party now holds its updated numerator masked solely by the other participant's fresh s, ready for the next round.

The net effect of the whole dance is that two neighbours have correctly averaged their rational estimates while, at no point, did either hold its own value in a form it could read or learn the other's private numerator — and, thanks to proxy re-encryption, the masks contributed by absent nodes in previous rounds were cleaned up without any communication with those absent nodes.

ODIN: Decision Phase, Analyses, and Conclusions

30. The Decision Phase

The update steps of Section 29 only refine each node's running estimate; at some point the network must actually **decide** something from the fused value. After an update step, a pair of agents may choose to **engage the final step**, in which they compare their shared estimate against a threshold and each learns a decision bit.

The goal of the decision is to evaluate the inequality

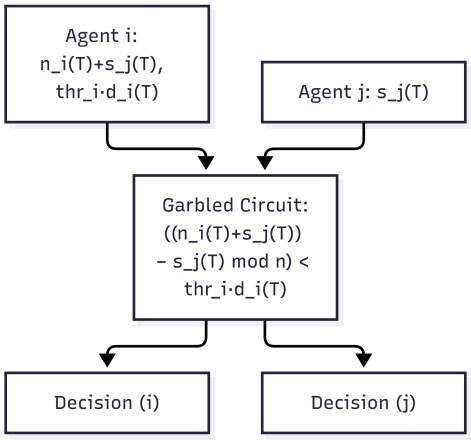
$$\frac{n_i(K)}{d_i(K)} \gtrless \tau,$$

that is, to test whether the current rational estimate is above or below a decision threshold τ. This division-and-comparison is impossible to perform directly on the additively-masked, rational representation, so it is pushed into a **garbled circuit**, which is exactly the tool for a secure comparison. The protocol is evaluated **in parallel for agent j with the roles reversed**, so that both participants obtain their decision symmetrically in one execution.

The circuit's inputs are arranged so that the masks cancel inside it. Agent i brings in $n_i(T) + s_j(T)$ (its masked numerator) together with $thr_i * d_i(T)$ — the threshold **pre-multiplied by its denominator**, which clears the division by turning the comparison $n/d \gtrless \tau$ into the multiplication-free form $n \gtrless \tau \cdot d$. Agent j brings in the mask $s_j(T)$ it placed on i's value. Inside the **garbled circuit** the mask is removed and the comparison performed:

$$\Big((n_i(T) + s_j(T)) - s_j(T) \mod n\Big) < thr_i * d_i(T),$$

after which each party receives its **decision** bit. Subtracting $s_j(T)$ reconstructs the true numerator $n_i(T)$ only transiently inside the circuit — neither party ever sees it in the clear — and the clipped comparison against $thr_i \cdot d_i(T)$ yields the thresholded result.



Two refinements make the decision flexible and cheap. First, **different thresholds for each agent can be used in the circuit**, since thr_i is supplied per-agent — useful when nodes have individually calibrated decision criteria. Second, the **previous update step can be modified to avoid the modular operations inside the garbled circuit**: by arranging the masking so that no reduction modulo n is needed at decision time, the circuit becomes a plain subtraction-and-compare, giving a **more efficient circuit**. Modular reduction is one of the costlier operations to garble, so removing it from the critical final step is a worthwhile optimization.

31. Bit-Size Analysis

Because each update adds rescaled quantities, the numerators grow over time, and the protocol must ensure they never overflow the modulus. The analysis bounds this growth. The initial numerator $n_i(0)$ is **represented with ℓ bits**. After k update steps, the numerator $n_i(k)$ has a bit-size **lower than $\ell + k$**

bits — it grows by at most one bit per step, because each step roughly doubles the denominator (the factor of two from averaging) and correspondingly the numerator.

This slow, predictable growth means that over a long run the numerators could eventually approach the bit-size of the modulus n. To prevent overflow, the protocol monitors the bit-size, and **if it is about to exceed the modulus n's bit-size, a modified step algorithm is performed** that renormalizes the representation (the precise mechanism being detailed in the full paper). The practical point is that the linear $\ell + k$ growth is gentle enough that this renormalization is rare.

32. Convergence Analysis

Privacy must not come at the cost of correctness: the obfuscated consensus still has to converge to the true average. The analysis gives a probabilistic convergence guarantee. The **consensus is ε away from the true average with probability** $1 - \varepsilon$ after a number of iterations

$$T = \mathcal{O}!\left(\frac{\log \varepsilon^{-1}}{,1 - \lambda_2(\mathbb{E}\mathbf{W}),}\right),$$

where $\mathbb{E}$ is the **expected-value operator**, $\mathbf{W}$ is the network's **connection matrix** (the weighted adjacency structure describing which nodes interact and how), and $\lambda_2(\mathbb{E}\mathbf{W})$ is the **second-largest eigenvalue** of the expected connection matrix.

Reading the formula explains how topology governs speed. The numerator $\log \varepsilon^{-1}$ shows that driving the error down is cheap — the required iterations grow only *logarithmically* as the tolerance ε tightens. The denominator $1 - \lambda_2(\mathbb{E}\mathbf{W})$ is the **spectral gap** of the network: the farther λ_2 sits below one, the larger the gap and the faster convergence; a poorly connected network with λ_2 near one converges slowly. The obfuscation layer does not change this classical consensus bound — it rides on top of it — so ODIN converges as fast as the underlying non-private gossip algorithm.

33. Complexity Analysis

The per-step cost of ODIN is tabulated for all the nodes involved and compared against the earlier secure-consensus baseline, separating computational cost (counts of cryptographic operations) from communication cost (bits transmitted and interaction rounds).

For ODIN, each step's complexity is as follows:

Step	Modular Expo	Modular Prod	Bilinear Map	Hash	Communication (bits)	Rounds
τ (update)	6	4	2	0	$< 4 \times 4\lceil \log_2 q \rceil + 6\lceil \log_2 n \rceil$	2
Final (decision)	0	0	0	$12(\ell + \lceil \log_2 d(T) \rceil)$	$15(\ell + \lceil \log_2 d(T) \rceil), t$	2

The structure of the table reflects the tool assignment. The **update step** τ uses public-key operations — six **modular exponentiations**, four **modular products**, and two **bilinear-map** evaluations (the pairings underpinning proxy re-encryption) — and no hashing; its communication is a small constant number of ciphertext-sized terms, governed by the bit-lengths of the quantization factor q and the modulus n, over just **two rounds**. The **final (decision) step** flips the profile entirely: it uses **no** public-key arithmetic but many **hash** operations (the symmetric cost of evaluating a garbled circuit, scaling

with the operand bit-length $\ell + \lceil \log_2 d(T) \rceil$), and its communication scales with the same bit-length times the security parameter t, again in **two rounds**.

The decisive property is in the update row: ODIN's per-update cost is a **small constant** — 6 exponentiations, 4 products, 2 pairings — that **does not depend on the number of neighbours**. By contrast, the baseline secure-consensus protocol has a per-step cost of $2(|\mathcal{N}_i| + |\mathcal{N}_j|) + 8$ exponentiations, which **grows linearly with the neighbourhood sizes** $|\mathcal{N}_i|, |\mathcal{N}_j|$. This is exactly the advantage promised by proxy re-encryption in Section 24: in a dense IoT network, where nodes may have many neighbours, ODIN's constant per-update cost is a large structural win over connectivity-dependent schemes.

34. Prototype Implementation and Evaluation

Theory is confirmed by a real prototype running on resource-constrained hardware — two generations of Raspberry Pi — measuring both the **runtime** and the **energy consumption** of each step, since energy is a first-class concern for battery-powered IoT devices.

Platform	Runtime, update step	Energy, update step	Runtime, final step	Energy, final step
Raspberry Pi 1	111.67 ms	241.20 mJ	25.80 ms	55.73 mJ
Raspberry Pi 3	41.71 ms	88.42 mJ	19.05 ms	40.39 mJ

The measurements show the protocol is genuinely lightweight. On the older **Raspberry Pi 1**, an update step takes about **112 ms** and consumes **241 mJ**, while the one-off final decision step is cheaper at **26 ms** and **56 mJ**. On the more capable **Raspberry Pi 3**, both costs fall sharply — the update step to roughly **42 ms / 88 mJ** and the final step to about **19 ms / 40 mJ** — reflecting the faster CPU. The communication footprint is correspondingly tiny: in a single update step, nodes exchange messages of only **140 B and 72 B**. Update steps are more expensive than the final step because they carry the public-key operations (exponentiations and pairings), whereas the final step is a symmetric-primitive garbled circuit; but both are small enough in time, energy, and bandwidth to run comfortably on commodity IoT hardware.

35. Conclusions

Across all three case studies — biometric recognition, medical diagnosis, and distributed consensus — a consistent picture emerges. **Secure computation is feasible**, but it remains **computationally expensive** relative to plaintext processing, and the gap is closed not by cryptography alone but by careful, application-specific protocol engineering.

Several **open problems** remain. Most of the protocols presented assume **semi-honest** adversaries, and **there is still a great deal to do with malicious adversaries** — parties that deviate arbitrarily from the protocol — whose protection is far costlier. On the primitive side, the **improvement of fully homomorphic encryption (FHE) schemes** is needed before arbitrary computation on ciphertext becomes cheap enough for routine use. And the broad practical challenge is simply to **make it real**: to move these techniques from prototypes into deployed systems.

The **trending directions** point toward machine learning: the **private training and evaluation of ML models**, so that a model can be learned from or applied to data without exposing either, and

private distributed learning, in which many parties collaboratively train a model while keeping their datasets confidential. These are the natural next frontier for data processing in the encrypted domain.

36. Future Works

The research programme going forward spans four thematic areas, each pairing a security-and-privacy domain with concrete target applications.

The first is **Security and Privacy in Healthcare**, aimed at building a privacy-preserving **infrastructure for smart-health** systems. The second is **Security and Privacy in IoT networks**, concentrating on **privacy-preserving drone coordination** and on **swarm attestation** — verifying the integrity of large groups of cooperating devices. The third is **Biometric Security and Privacy in Mobile devices**, extending the biometric work toward **multi-biometrics** (combining several biometric traits) and **behavioural biometrics** (identifying users by how they act rather than by a static trait). The fourth is **Threat intelligence**, developing **privacy-preserving protocols for critical-infrastructure cooperation**, so that organizations can share threat information without exposing their own sensitive data.

Together these directions carry forward the central thesis of the entire subject: that with the right combination of cryptographic tools and application-aware optimization, useful computation can be performed on data that never leaves the encrypted domain.

CHAPTER 6: ML security

Introduction: Why Machine Learning Belongs in Cybersecurity

The modern defensive posture of any networked system has shifted decisively away from purely deterministic detection and toward statistical inference. The reason for this shift is rooted in the nature of the data that security systems must process. Machine learning is a branch of artificial intelligence whose defining characteristic is the ability to automatically discover useful information from massive datasets, without a human engineer having to hand-specify every rule in advance. This property is precisely what contemporary security operations require, because the volume of network events, logs, and traffic flows produced by even a modest enterprise vastly exceeds what manual analysis can handle.

A second decisive advantage is generalizability. A well-constructed machine learning model does not merely memorize the specific attacks it was shown during training; it learns the underlying statistical structure that distinguishes malicious behavior from benign behavior. As a consequence, such models are able to detect both known attacks, whose patterns appear in the training data, and unknown attacks, whose patterns are novel but statistically similar to malicious activity the model has already internalized. This stands in sharp contrast to older techniques, which could only recognize what they had been explicitly told to look for.

The third advantage is operational speed. Security analysts are perpetually overwhelmed by the sheer noise of daily alerts, the overwhelming majority of which are benign. Machine learning models provide rapid insights that cut through this noise, surfacing the events that genuinely merit human attention and thereby drastically reducing response times. The practical effect is that a human analyst's scarce attention is redirected from triage toward genuine investigation.

The Historical Transition: From Signatures to Probabilities

To appreciate why this matters, it is worth understanding what machine learning replaced. The classical approach to attack detection relied on **signatures**. A signature is an exact, predefined pattern — a

specific byte sequence, a known malicious hash, a particular string in a payload — and detection consisted of checking whether an observed activity matched one of these stored patterns exactly. If a correspondence was found, the activity was flagged; if not, it was passed as clean.

The fundamental limitation of this paradigm is its exactness. A signature-based system is binary and literal: it can only catch what it has an exact description of, and even a trivial modification by an attacker is enough to evade it. Machine learning models operate on an entirely different principle. They are **probabilistic algorithms**: rather than declaring with certainty that an activity either is or is not an attack, they determine that an activity is an attack with a certain degree of belief. This probabilistic framing is what grants them their flexibility and their capacity to generalize, at the cost of the perfect certainty that signatures nominally offered.

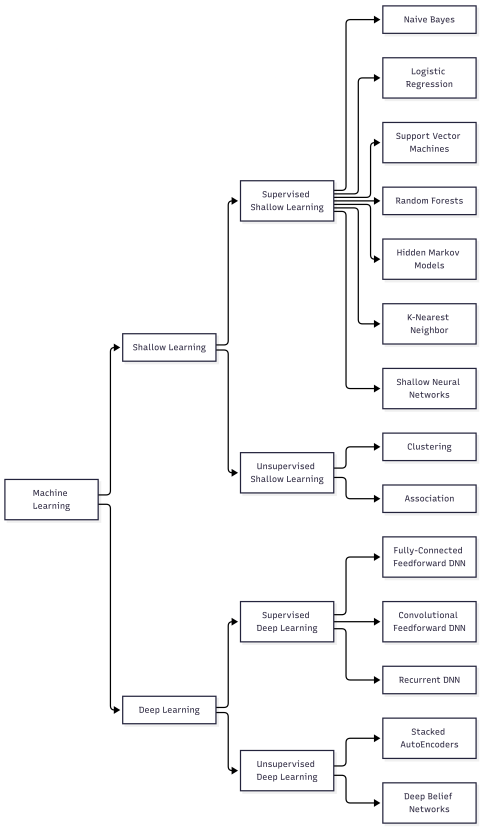
Where Machine Learning Is Applied in Practice

The reach of machine learning across the security landscape is broad, and it is useful to survey the principal application areas before descending into the algorithms themselves. Machine learning is used to find threats on a network, identifying anomalous or malicious traffic among legitimate communications. It is used to keep people safe when browsing, screening web content for danger before it reaches the user. It provides endpoint malware protection, analyzing files and process behavior on individual devices, and it extends this protection to data in the cloud. A particularly sophisticated application is the detection of malware in encrypted traffic, where the payload cannot be inspected directly and the model must instead reason from metadata and behavioral patterns. Machine learning also underpins modern authentication systems, where it can distinguish a legitimate user from an impostor based on behavioral characteristics, and it is deployed in fake news detection, where the task is to assess the credibility and provenance of information itself.

The Taxonomy of Machine Learning Algorithms

Machine learning algorithms can be organized into a coherent hierarchy, and understanding this hierarchy is essential because every specific technique discussed later inherits its basic properties from its position within it. The taxonomy branches along two independent axes. The first axis distinguishes **shallow learning** from **deep learning**, and the second axis distinguishes **supervised learning** from **unsupervised learning**. Any given algorithm occupies one cell in the resulting grid.

The following diagram captures the full structure of this taxonomy.



Shallow Learning versus Deep Learning

Shallow learning encompasses the family of comparatively simple models. The defining operational characteristic of shallow learning is that it requires domain expertise. Because these models have limited representational capacity, a human expert must perform **feature engineering** — the careful selection and construction of the specific measurable quantities that the model will receive as input. The model itself does not discover what is important about the data; the human tells it what to look at, and the model learns how to weigh those predefined inputs.

The great practical virtue of shallow learning is that its simplicity makes it deployable almost anywhere. These models can run on simple machines, and this is the decisive reason they remain in active use despite the rise of more powerful alternatives. In **constrained devices**, most notably the resource-limited processors of the Internet of Things, deep models are simply too heavy to run, and shallow learning is the only feasible option. The continued relevance of shallow learning is therefore not nostalgia but engineering necessity.

Deep learning encompasses the family of complex models. Its defining characteristic is that it autonomously selects features. Rather than depending on a human expert to engineer inputs, a deep model learns the relevant representations directly from raw data, discovering hierarchical structure through its many layers. This autonomy is purchased at a high price in computational resources and training data, which is exactly why deep models cannot be deployed on the constrained devices where shallow learning thrives.

Supervised Learning versus Unsupervised Learning

The second axis concerns the nature of the training data. In **supervised learning**, we possess labeled samples. Every example in the training set carries a correct answer — its true class — and this fully annotated training set is provided to the algorithm so that it can learn the mapping from inputs to the correct labels. Once trained, the resulting predictive model is applied to a separate test set of previously unseen data, and it assigns each new element to one of the classes it learned during training.

The intuition is clarified by a concrete example. Imagine a training set composed of images of fruit — apples, bananas, oranges, lemons, watermelons — where each image is tagged with a number identifying which fruit it is. The machine learning algorithm processes these labeled feature vectors and builds a predictive model. When the test set, containing new unlabeled fruit images, is presented, the model classifies each one by assigning it the appropriate learned label. The essential point is that the categories themselves were defined in advance by the labels.

In **unsupervised learning**, by contrast, we do not have labels and do not provide any formal categorization to the algorithm. The task is not to assign elements to predefined classes but to recognize that certain elements belong together — to discover structure in the data on its own terms. Returning to the fruit example, an unsupervised algorithm receives the same images but with no tags whatsoever. It processes the feature vectors and, in its results, groups the fruits into clusters of similar items — watermelons together, apples and oranges and lemons in their own respective groupings — without ever having been told what any of these groups is called. The algorithm has discovered the natural partitioning of the data rather than reproducing a partitioning it was given.

Supervised Shallow Learning Algorithms

With the taxonomy in place, we now examine the principal algorithms in each category in detail, beginning with the supervised shallow learners.

The Naive Bayes Classifier

The Naive Bayes classifier is one of the simplest supervised classifiers, and it is built directly upon Bayes' Law of probability. It possesses two attractive practical properties: it is scalable, and it does not require huge training datasets. These qualities make it a natural first choice when data or computational resources are limited.

The classifier's purpose is to classify an element by computing the probability that this element belongs to each possible class and then selecting the most probable class. The formal machinery is Bayes' theorem, which in this setting is written as follows. Let H denote a hypothesis, meaning a candidate class, and let E denote the evidence, meaning the observed data. The theorem states:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

Each term in this expression carries a precise interpretation. The quantity on the left, $P(H|E)$, is the **posterior probability** of the hypothesis given that the evidence is true — this is what we ultimately want to know, namely how probable a given class is once we have seen the data. The term $P(E|H)$ is the **likelihood**, the probability of observing the evidence under the assumption that the hypothesis is true. The term $P(H)$ is the **prior probability** of the hypothesis, representing what we believed about the class before observing any data; this quantity is constant and known a priori. Finally, $P(E)$ is the prior probability of the evidence itself, which depends on the observations we are taking.

The reason this classifier is called **naive** lies in a critical simplifying assumption. The decomposition it relies upon treats the input features as independent of one another. Concretely, the likelihood of the complete evidence is computed as the product of the individual feature likelihoods:

$$P(E|H) = P(E|h_1) \cdot P(E|h_2) \cdots P(E|h_n)$$

and similarly the prior is treated as a product of component priors, $P(H) = P(h_1) \cdots P(h_n)$, which remains constant. This factorization into a product is mathematically correct only in the moment that all

events are independent. In the real world, however, features are correlated with one another a great deal of the time, and when such correlation exists the independence assumption no longer holds and the formula is no longer strictly valid. The classifier is "naive" precisely because it proceeds as if this independence held regardless, and remarkably, it often performs well in practice despite the violated assumption.

The actual classification rule follows directly. Since the denominator $P(E)$ is the same for every class and therefore does not affect which class is largest, it can be dropped from the comparison. The predicted class is the one that maximizes the product of likelihood and prior:

$$\text{Classification} = \arg\max_{H} P(E|H), P(H)$$

Logistic Regression

Logistic regression is another simple supervised classifier, and it shares the same assumptions as the Naive Bayes classifier — in particular the treatment of the inputs under an assumed probabilistic structure. What distinguishes it is the explicit assumption that the data are distributed according to a specific functional form. This function could be linear, quadratic, a sigmoid, or another chosen shape, and the model's job is to fit the parameters of that assumed distribution to the observed data.

In its canonical binary form, logistic regression employs the **sigmoid function**:

$$f(x) = \frac{1}{1 + e^{-k(x - x_0)}}$$

Here x_0 sets the location of the function's midpoint and k controls the steepness of its transition. The behavior of this function is what makes it suitable for classification. A plain linear regression model, if used for classification, would produce an unbounded straight line that does not respect the fact that probabilities must lie between zero and one; its output simply grows without limit as the input increases. The sigmoid, by contrast, is bounded: it saturates toward 0 for strongly negative inputs and toward 1 for strongly positive inputs, with a sharp transition in between. This produces a clean separation in which data points are driven toward one of the two extremes, and the crossing of the 0.5 threshold marks the decision boundary between the two classes. The result is a model that outputs something interpretable as a class probability rather than an arbitrary real number.

Support Vector Machines

The support vector machine, or SVM, is a supervised shallow classifier that approaches the problem geometrically. Each sample is regarded as a point in a multidimensional space, where the coordinates of the point are the feature values of the sample. The goal of the algorithm is to find a line — or, in higher dimensions, a hyperplane — that divides this space into two parts, with the points of one class falling on one side and the points of the other class falling on the other.

What makes the method distinctive is how this dividing line is chosen. The boundary is not determined by taking into account all of the data points equally. Instead, it is determined by a particular subset of points, the **support vectors**, which are the points lying closest to the boundary and which are hardest to classify. These are the only points that matter for defining the separation; points lying comfortably within their own class territory have no influence on where the line is drawn.

The geometry can be made precise. The central decision boundary is the set of points satisfying $w \cdot x - b = 0$, where w is the weight vector perpendicular to the boundary and b is an offset. Running parallel to this central line are two margin boundaries, defined by $w \cdot x - b = 1$ on one side and $w \cdot x - b = -1$ on

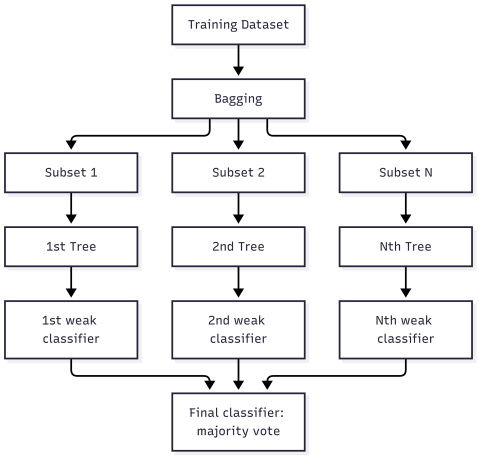
the other. The region between these two parallel boundaries is the **margin**, an uncertain band containing no training points, and its width is governed by the magnitude of w. The support vectors are precisely the points that lie exactly on these margin boundaries. The training procedure identifies these support vectors, fixes the dividing line that maximizes the margin between the classes, and stores the result. That stored result can take the form of the parameter w characterizing the width of the uncertain area, or equivalently the support vectors themselves, which can be substituted directly into the classification formula to label new points.

A crucial extension addresses the situation where the data cannot be separated by a straight line at all. When the two classes are tangled together in a way that no linear boundary can resolve, the SVM applies a **kernel**. A kernel is a transformation, denoted ϕ, that maps the data from its original space into a higher-dimensional space where a linear separation becomes possible. In the original low-dimensional view the boundary between the classes may be a hopelessly curved and convoluted shape; after the kernel transformation, that same boundary becomes a simple flat plane. Geometrically, one can picture a set of intermingled points on a flat plane being lifted into three dimensions by the kernel, so that one class rises above the other and a flat **decision surface** can then be slid between them. This kernel trick is what allows the fundamentally linear SVM to handle richly nonlinear classification problems.

Random Forest

The random forest is a supervised shallow classifier belonging to the family of **ensemble methods**, meaning it combines many simple models into one stronger aggregate. The building block of a random forest is the **decision tree**: a structure in which the training data, consisting of many points, is repeatedly partitioned by a sequence of decisions, descending from a root down through branching nodes to leaves that assign a classification. A single such tree constitutes a **weak classifier** — individually imperfect and prone to error.

The power of the method comes from building not one tree but many, and the mechanism that makes them usefully different from one another is **bagging** (bootstrap aggregating). From the full training dataset, numerous distinct subsets are drawn, and each subset is used to construct its own decision tree. Because each tree sees a different sample of the data, the trees develop different structures and make different mistakes. The forest therefore consists of a first tree yielding a first weak classifier, a second tree yielding a second weak classifier, and so on up to an N-th tree yielding an N-th weak classifier.



The final prediction is produced by combining the individual verdicts according to the **majority rule**: the class selected by the greatest number of trees becomes the forest's output. By pooling many independent weak classifiers in this way, the aggregate corrects the idiosyncratic errors of any single tree and achieves accuracy far beyond what one tree could reach. A further practical strength of the approach

is its versatility with respect to input: the data fed to a random forest can be of essentially any kind, including text, which broadens its applicability considerably.

K-Nearest Neighbour

The K-nearest neighbour classifier, abbreviated KNN, rests on an exceptionally intuitive principle: an element is likely to belong to the same class as the elements it most closely resembles. During training, the algorithm does little more than retain the labeled data points in the feature space, where points of each class occupy characteristic regions. When a new, unclassified point arrives, the algorithm examines the labeled points nearest to it and looks at their classes.

The decision follows from a vote among these neighbours. The new point is assigned to the class that holds the majority among its closest neighbours. The parameter K specifies how many neighbours are consulted. This choice is consequential, because the effective radius it implies determines which points count as neighbours, and the balance of classes within that radius can change as the radius grows. A new point might, for instance, be surrounded immediately by members of one class but find itself within a larger neighbourhood dominated by another. Concretely, consider a point to be classified that has three of one class and one of another among its three nearest neighbours when $K = 3$; expanding to $K = 7$ draws in additional points and may tip the majority toward a different class entirely. The value of K is therefore not a mere technicality but a genuine control over the classifier's behavior, and it must be chosen with care.

Shallow Neural Networks

A neural network is a supervised model inspired by the organization of biological neurons, and in its shallow form it provides a bridge toward the deep architectures examined later. Its structure is organized into layers: an **input layer**, one or more **hidden layers**, and an **output layer**. The governing convention that ties the network to the classification problem is simple and strict — there is one input node for each feature, and one output node for each class. Thus a problem with four features feeding a network that must distinguish three classes would present a four-node input layer and a three-node output layer, with hidden layers of chosen sizes in between. Each layer is connected to the next by a matrix of weights, so that information flows forward from inputs, through the hidden transformations, to the outputs.

The essential computation occurs at the level of the individual neuron, and understanding it clarifies the whole. A single neuron receives a set of inputs $x_1, x_2, \ldots, x_m$. Each input is multiplied by an associated weight $\omega_1, \omega_2, \ldots, \omega_m$, and these weighted inputs are summed together. To this sum is added a further term called the **bias**, denoted b, which shifts the neuron's response independently of the inputs. The resulting quantity is then passed through an **activation function** $\varphi(\cdot)$, which transforms it into the neuron's output y. In compact form, the neuron computes

$$y = \varphi!\left(\sum_{i=1}^{m} \omega_i x_i + b\right)$$

The activation function is the source of the network's nonlinearity and hence of its representational power; without it, the entire network would collapse into a single linear transformation no matter how many layers it contained. At the output layer, once each class node has produced its value, the network selects the class whose node yields the maximum output, an operation captured by the **argmax**. This per-neuron pattern — multiply, sum, add bias, activate — repeated across many neurons and layers, is the fundamental engine of all neural computation, shallow and deep alike.

Unsupervised Shallow Learning Algorithms

Having surveyed the supervised shallow learners, we turn to the two principal categories of unsupervised shallow learning, where the data arrive without labels and the algorithm must find structure unaided.

Clustering

Clustering is the unsupervised task of taking data and subdividing it into groups, called **clusters**, such that the members of each cluster are similar to one another and dissimilar to the members of other clusters. No labels guide this process; the grouping emerges entirely from the geometry of the data itself. Presented with a cloud of unclustered points that to the naked eye forms one undifferentiated mass, a clustering algorithm partitions it into coherent regions, revealing structure that was latent in the raw distribution.

The canonical representative of this family is **k-means clustering**. Here the number of desired clusters, k, is specified in advance, and the algorithm partitions the data into exactly k groups, assigning every point to the nearest cluster so as to produce compact, well-separated groupings. The result transforms a formless scatter of points into a set of clearly delineated clusters, each identifiable as a distinct population within the data. One useful property worth noting is how a cluster summarizes its members: the representative value of a group can be taken as the average of the values of its constituent points. If two nodes within a cluster carry the values 3 and 5, for example, the representative average value of that pairing is 4 — a small illustration of how clustering compresses many individual observations into a single characteristic summary.

Association

Association is the second major category of unsupervised shallow learning, and its aim is to uncover relationships between items rather than to group the items themselves. **Association rules** are made by searching the data for frequent if-then patterns and by applying criteria based on two quantities, support and confidence, to determine which of the many discoverable relationships are the most important. The familiar motivating scenario is retail market-basket analysis: discovering that customers who buy one item also tend to buy another, the pattern that underlies the "customers who bought this also bought" recommendations.

The two governing metrics quantify the strength of a candidate rule. The **support** of a pair of items A and B measures how common their joint occurrence is across all transactions:

$$\text{Support}(A, B) = \frac{\text{number of transactions containing } A \text{ and } B}{\text{total number of transactions}}$$

Support therefore answers the question of how frequently the combination appears at all; a rule involving items that almost never co-occur is unlikely to be useful regardless of any other property. The **confidence** of a rule $A \Rightarrow B$ measures how reliably the presence of A predicts the presence of B:

$$\text{Confidence}(A \Rightarrow B) = \frac{\text{number of transactions containing } A \text{ and } B}{\text{number of transactions containing } A}$$

Confidence thus answers a conditional question: given that a transaction contains A, how often does it also contain B? A rule is considered important and worth retaining only when both quantities clear chosen thresholds — the combination must be frequent enough to matter (sufficient support) and reliable enough to trust (sufficient confidence).

Deep Learning Architectures

Deep learning encompasses the complex models capable of autonomous feature selection. Where shallow learning leans on a human expert to specify what is worth measuring, deep architectures learn their own internal representations by stacking many layers of transformation. We examine the supervised deep architectures first, then the unsupervised ones.

Deep Neural Networks

The deep neural network is the most direct generalization of the shallow neural network examined earlier. Structurally it preserves the same organization — an input layer, an output layer, and hidden layers between them — but the defining difference is depth: rather than one or two hidden layers, a deep network stacks many hidden layers in sequence, and these layers may be large and of varying width.

A representative network illustrates the pattern. An input layer accepting six features feeds a first hidden layer, which passes its output to successively wider and then narrower hidden layers — for instance one hundred neurons, then five hundred, then two hundred, then fifty — before arriving at an output layer producing the final class scores. The same per-neuron computation described for shallow networks operates at every node throughout: each neuron multiplies its inputs by weights, sums them, adds a bias, and applies an activation function. What the added depth buys is representational hierarchy. Early layers learn simple, local patterns in the raw input; later layers compose these into progressively more abstract and discriminative features. This layered abstraction is exactly what frees the deep network from hand-engineered features, because the intermediate layers construct the needed representations on their own.

Convolutional Neural Network

The convolutional neural network, or CNN, is a supervised deep architecture specialized for data with spatial structure, most prominently images. Its design introduces two new kinds of layers — convolutional layers and pooling layers — arranged in sequence before the network concludes with conventional fully-connected layers.

The processing pipeline can be traced end to end on the canonical example of classifying a handwritten digit from a small grayscale image. The input is a single-channel image of modest dimensions. The first stage is a **convolution** layer, which slides a small learnable filter, or kernel, across the image. Each position of the kernel computes a weighted combination of the pixels beneath it, producing a feature map that responds to a particular local pattern such as an edge or a corner. Applying several such kernels yields several channels of feature maps. Because the kernel is applied only where it fully fits within the image — the so-called **valid padding** — the spatial dimensions shrink slightly at each convolution.

The convolution is followed by **max-pooling**, a downsampling operation that partitions the feature map into small regions and keeps only the maximum value from each. Pooling compresses the spatial dimensions, typically halving them, and in doing so grants the representation a degree of invariance to the exact position of a feature; what matters becomes whether a pattern is present in a region, not precisely where. A typical CNN alternates these stages: a first convolution expands the single input channel into several feature channels while slightly reducing spatial size, a first pooling halves the dimensions, a second convolution produces still more channels, and a second pooling halves the dimensions again. Through this repetition the representation grows deeper in the channel dimension while contracting in the spatial dimensions.

Once the convolutional and pooling stages have distilled the image into a compact stack of feature maps, the data is **flattened** — the multidimensional feature maps are unrolled into a single long vector — and handed to one or more **fully-connected** layers. These final layers, often employing a ReLU activation and sometimes regularized with dropout to prevent overfitting, perform the actual classification, mapping

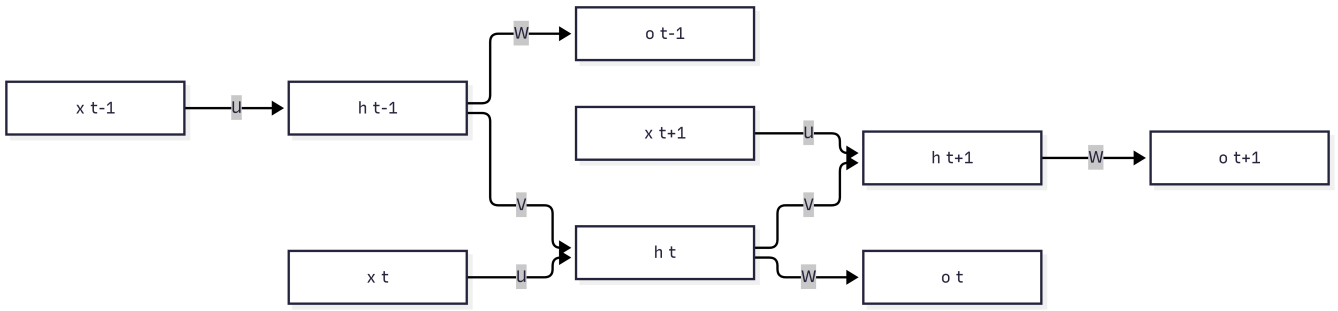
the learned features onto the output classes. In the digit example, the ten output nodes correspond to the digits zero through nine, and the node with the greatest activation names the network's answer.



Recurrent Neural Network

The recurrent neural network, or RNN, is the supervised deep architecture designed for **sequential data**, where the order of inputs carries meaning — text, time series, network traffic over time. Its defining innovation is a feedback connection: a hidden unit's output is fed back as an input to itself at the next step, giving the network a form of memory that lets earlier inputs influence the processing of later ones.

The mechanism is clearest when the recurrent loop is **unfolded** across time. In its compact form, the network consists of a hidden state h that receives an input x through a weight matrix U, produces an output o through a weight matrix W, and feeds back into itself through a weight matrix V. Unfolding this loop over successive time steps reveals a chain: at each step t there is an input x_t, a hidden state h_t, and an output o_t. Crucially, the hidden state h_t depends not only on the current input x_t (via U) but also on the previous hidden state h_{t-1} (via the recurrent connection V), while producing the output o_t via W. The same three weight matrices U, V, and W are reused at every step — the network applies one consistent rule across the entire sequence.



This structure is what allows the RNN to model dependencies that stretch across a sequence: because information from step $t-1$ is carried forward into step t and onward, the network can let context accumulated earlier shape its interpretation of what comes later.

Unsupervised Deep Learning Architectures

The two unsupervised deep architectures learn structure from unlabeled data, typically for the purpose of building compact internal representations that can later serve other tasks.

Deep Belief Networks

A deep belief network, or DBN, is an unsupervised deep architecture assembled by stacking simpler probabilistic building blocks called **restricted Boltzmann machines**, or RBMs. A single RBM consists of two layers — a **visible layer** holding units $v_1, v_2, \ldots, v_m$ and a **hidden layer** holding units $h_1, h_2, \ldots, h_n$ — fully connected between layers but with no connections among units within the same layer, which is the restriction that gives the machine its name. Each RBM learns, without supervision, to capture the statistical regularities of the data presented to its visible layer.

The deep belief network stacks several such RBMs so that the hidden layer of one becomes the visible layer of the next — a first RBM, then a second built atop it, then a third. Training proceeds **greedily, layer by layer**: the lowest RBM learns from the raw input, its learned representation becomes the input

for the next RBM, and so on up the stack, each layer discovering progressively more abstract features. Once this unsupervised pre-training has shaped all the layers, a final supervised component — a logistic regression layer at the top — can be attached to map the learned representation onto output classes, and the whole network is refined through a **fine-tuning** pass that adjusts the weights across all layers together. The architecture thus marries unsupervised feature discovery in its lower reaches with a supervised objective at its summit.

Stacked Autoencoders

The stacked autoencoder is the second unsupervised deep architecture, and it is organized around the principle of reconstruction. An **autoencoder** is a network trained to reproduce its own input at its output, which seems trivial until one notices the constraint that makes it useful: the network is forced to pass the information through a narrow central bottleneck, the **code**, which has far fewer units than the input. The architecture is symmetric and hourglass-shaped. The **encoder** is the first half, compressing the input through progressively smaller layers down to the compact code; the **decoder** is the second half, expanding the code back out through progressively larger layers to reconstruct the original input.

Because all information must be squeezed through the bottleneck, the network cannot simply copy its input verbatim. It is compelled to learn an efficient, compressed encoding that preserves the input's essential content while discarding redundancy — a learned form of dimensionality reduction. The illustrative case is an image of a handwritten digit: the encoder compresses the image into a short code, a dense pattern of values far smaller than the original pixel grid, and the decoder reconstructs from that code an image closely resembling the original. The fidelity of the reconstruction demonstrates that the code has captured the meaningful structure of the input. Stacking several such encoding stages yields the **stacked** autoencoder, which learns a hierarchy of increasingly abstract compressed representations, each useful as a feature set for downstream tasks.

Machine Learning Evaluation in Cybersecurity

Having assembled the catalogue of algorithms, we now confront a separate and equally important question: how is the quality of a security classifier actually judged? A model is only as valuable as the evidence that it works, and in the security domain the choice of evaluation metric is not a bookkeeping detail but a decision with direct operational consequences.

The Confusion Matrix and Its Four Outcomes

Every evaluation of a binary classifier begins with the **confusion matrix**, which cross-tabulates the model's predictions against the ground truth. The ground truth — what the manual, authoritative counting says is actually correct — forms one axis, and the machine learning model's verdict forms the other. The intersection of these two produces four possible outcomes for any given sample.

A **true positive** (TP) occurs when the model labels a sample as positive and it genuinely is positive — in security terms, an attack correctly flagged as an attack. A **true negative** (TN) occurs when the model labels a sample as negative and it genuinely is negative — benign traffic correctly recognized as benign. These two cases are the model's successes. The two failures are the **false positive** (FP), in which the model raises an alarm on something that is actually benign — a false alarm — and the **false negative** (FN), in which the model passes a genuine attack as harmless — a missed detection. These four quantities are the raw material from which all the standard performance metrics are constructed.

The Four Core Metrics

From the four cells of the confusion matrix, four summary metrics are derived, each answering a different question about the classifier's behavior.

Accuracy measures the overall fraction of correct verdicts across all samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FN + FP}$$

Accuracy is the most intuitive metric, but it is also the most treacherous in security settings, for reasons that become clear when class imbalance is discussed below. **Precision** asks, among all the samples the model flagged as positive, what fraction truly were positive:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision is thus the metric that penalizes false alarms; a model that cries wolf frequently will have many false positives and correspondingly low precision. **Recall** asks the complementary question — among all the samples that truly were positive, what fraction did the model manage to catch:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall is the metric that penalizes missed detections; a model that lets real attacks slip through accumulates false negatives and suffers low recall. Precision and recall stand in natural tension, since a classifier can trivially achieve perfect recall by flagging everything (at ruinous cost to precision) or high precision by flagging only the most certain cases (at the cost of recall). The **F1-score** resolves this tension by combining the two into a single number, their harmonic mean:

$$\text{F1-score} = \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The harmonic mean has the useful property of being dominated by the smaller of its two arguments, so that a high F1-score can be achieved only when precision and recall are both high; excelling at one while neglecting the other is not enough. For this reason the F1-score is the workhorse metric for imbalanced security problems.

Applications of Machine Learning to Cybersecurity

Before examining the comparative performance of these models, it is worth situating the metrics within the concrete tasks they evaluate. Machine learning is applied across several major security problem domains. The first is **intrusion detection**, itself a broad family that includes the detection of **botnets** — networks of compromised machines under an attacker's control — and the detection of **domain generation algorithms**, the technique by which malware algorithmically produces large numbers of candidate domain names to locate its command-and-control server, among many other intrusion types. A second domain is **malware analysis**, the identification and characterization of malicious software. A third is **spam and phishing detection**, the filtering of fraudulent and unwanted communications.

Shallow versus Deep Learning: An Empirical Comparison

A natural assumption is that deep learning, being more powerful and modern, must outperform shallow learning on security tasks. This assumption turns out to be unreliable, and a concrete network intrusion detection experiment demonstrates why. Consider a dataset consisting of five thousand malicious flows set against five hundred thousand benign flows — a ratio of one hundred benign flows for every malicious

one. This severe **class imbalance** is not an artifact of the experiment but a faithful reflection of reality, where genuine attacks are rare events drowning in a sea of ordinary traffic.

Two classifiers are compared on this data. A random forest, representing shallow learning, achieves an F1-score of 0.7985, with precision 0.8727 and recall 0.7360. A fully-connected feedforward deep neural network, representing deep learning, achieves an F1-score of only 0.6085, with precision 0.7708 and recall 0.5027. The shallow model decisively outperforms the deep one on every metric. The lesson is twofold. First, greater model complexity does not guarantee better results; the appropriateness of a model to the structure and scale of the data matters more than its raw sophistication. Second, and more subtly, this is exactly a case where accuracy would mislead: a trivial classifier that labeled every single flow as benign would achieve an accuracy above ninety-nine percent on this dataset while catching no attacks whatsoever, which is why F1-score, precision, and recall — not accuracy — are the metrics reported here.

General versus Specific Detectors

A further strategic question is whether one should build a single general-purpose detector trained to recognize all attacks at once, or a collection of specialized detectors each tuned to one specific attack type. The evidence strongly favors specialization. When random forest detectors are each trained and evaluated on a specific attack category, their performance is remarkable: a DOS (denial-of-service) attempt detector reaches an F1-score of 0.9953, an overflow attempt detector 0.9939, an SSH brute-force login detector 0.9916, a suspicious DNS query detector 0.9753, a cache poisoning attempt detector 0.9676, and a possible malware infection detector 0.9587. Every one of these specialized detectors far exceeds the general approach, whose baseline F1-score on the same underlying problem is only 0.7985.

The interpretation is that each attack type possesses its own distinctive signature in the feature space, and a detector that can devote its entire capacity to learning one such signature will characterize it far more sharply than a generalist forced to accommodate every attack simultaneously. The practical implication for system design is that a layered arsenal of narrow, expert detectors tends to outperform a single model asked to do everything.

The Difficulty of Selecting a Machine Learning Algorithm

These comparisons must, however, be read with considerable caution, because comparing published machine learning methods fairly is genuinely hard. When different methods are reported in the literature, they are often trained on the same training dataset and tested on the same test dataset, which would seem to make them comparable. Yet beneath this surface they differ in ways that undermine direct comparison. They frequently consider different features, each method engineering its own inputs, so that the methods are not really operating on the same information. They may employ a different split of the data during training, dividing the available samples into training and testing portions in different proportions or by different procedures. And most fundamentally, there is no guarantee that a method which performs best on one particular test dataset will retain its superiority on a different dataset. Performance measured in one setting does not transfer automatically to another, and a ranking established on one benchmark can be overturned on the next. The selection of an algorithm is therefore not a matter of consulting a single leaderboard but requires judgment about the specific data, features, and conditions of the deployment at hand.

The Operational Cost of False Positives and False Negatives

The abstract confusion-matrix categories acquire real weight once their operational consequences are considered, and the two error types are costly in entirely different ways. **False positives are time-consuming.** In a large enterprise, the massive volume of events generated every day means that even a

small per-event false-positive rate translates into hundreds or thousands of false alarms daily. Each of these demands analyst attention to investigate and dismiss, and the cumulative burden exhausts the finite capacity of a security team, burying genuine threats beneath a mountain of fruitless investigations. The damage done by false positives is therefore not a single catastrophic event but a slow, steady drain on the organization's defensive resources.

False negatives, by contrast, can compromise organizations outright. A false negative is a real attack that the system waved through as benign, and a single such miss can be sufficient for an adversary to establish a foothold, exfiltrate data, or cause lasting harm. Where the cost of a false positive is measured in wasted hours, the cost of a false negative is measured in breaches. The two error types thus pull system tuning in opposite directions — lowering the detection threshold catches more attacks but raises false alarms, while raising it quiets the alarms but lets more attacks through — and the art of operating a detector lies in balancing these competing costs according to the organization's risk tolerance.

The interplay is vividly illustrated by domain-generation-algorithm detectors built on random forests. When such a detector is trained only on a well-known reference set of domains, it classifies 431 domains, or 2.16% of a test population, as DGA-generated. When the training set is enriched with recent domains as well, the detector classifies 397 domains, or 1.99%, as DGA. The composition of the training data directly shapes how many alarms the detector raises, and therefore directly shapes the false-positive burden the analysts will shoulder.

Re-training Issues

This dependence on training data exposes one of the deepest practical weaknesses of machine learning in security: the models decay. The threat landscape is not static — new malware families appear, attackers alter their techniques, and the statistical patterns that defined malicious behavior last year may no longer hold this year. A model trained on yesterday's data gradually loses accuracy as the world drifts away from the conditions it learned. Consequently, security models have a standing need for frequent and continuous updates, and every such update requires labelling training datasets — the laborious, expertise-intensive task of correctly tagging large volumes of fresh samples as malicious or benign so the model can be retrained on them.

The stakes of keeping training data current are demonstrated starkly by the DGA detectors. A first classifier trained only on well-known domains achieves an F1-score of merely 0.3306, with precision 0.1984 and recall 0.9913. The pattern of these numbers is telling: recall is very high while precision is disastrously low, meaning the detector catches nearly every real DGA domain but does so by flagging an enormous number of legitimate domains as well, flooding analysts with false positives. A second classifier, trained on well-known domains supplemented with recent ones, achieves an F1-score of 0.8999, with precision 0.9126 and recall 0.8875 — a transformation from an almost useless detector into a strong one. The difference between the two is nothing but the currency of the training data, which is the clearest possible demonstration of why continuous re-training, for all its labor, is indispensable.

Machine Learning–based Intrusion Detection Systems

The final major theme is the intrusion detection system, or IDS, which is the concrete setting in which many of these algorithms are deployed. An IDS monitors activity and raises an alert when it observes something it judges to be an intrusion. Research on machine-learning-based IDSs can be organized according to the kind of raw material the system examines, and four principal data granularities define the field: **packet-based**, **flow-based**, **session-based**, and **log-based** attack detection. Each represents a different vantage point on network activity, trading off detail against context, and each carries its own characteristic advantages, drawbacks, and preferred processing techniques.

Packet-based Attack Detection

The **packet** is the basic unit of network communication, the atomic parcel into which all network traffic is divided. Packet-based detection operates at this finest level of granularity. A complication inherent to the approach is that packets consist of binary data, which is incomprehensible unless it is first parsed — the raw bytes must be decoded according to protocol structure before any meaning can be extracted. The internal layout of a packet is highly structured: its header contains, in a fixed arrangement of bit-fields, elements such as the version, header length, type of service, total length, identification, flags and fragmentation offset, time to live, protocol, header checksum, source address, destination address, and options, after which the actual data payload begins.

The advantages of working at the packet level follow from its richness and immediacy. Packets contain the actual communication contents, the substance of what is being transmitted. They contain IP addresses and timestamps, which locate the communication in network space and time. And they can be processed instantly without caching, since each packet is self-contained and can be examined the moment it arrives. The disadvantages are the mirror image of this atomicity. A single packet does not reflect the full communication state, the larger conversation of which it is one fragment, and it does not reflect the contextual information surrounding each packet — what came before and after. In processing terms, two main strategies are used: header parsing followed by clustering, which groups packets by the structured fields of their headers, and payload parsing followed by deep learning, which applies deep models to the decoded content of the packets.

Flow-based Attack Detection

A **flow** raises the level of abstraction: it contains packets grouped together over a period of time, aggregating many individual packets into a summary of a communication episode. This shift in granularity changes the profile of strengths and weaknesses entirely. The advantages are that a flow represents the whole network environment rather than a single fragment, capturing the shape of communication at a larger scale, and that it can be used without packet parsing or session restructuring, requiring only simple preprocessing — one does not need to decode every byte to characterize a flow. The disadvantages are that the flow ignores the content of the packets, discarding the actual payload in favor of aggregate statistics, and that the strong heterogeneity of flows — their wide and irregular variation — may cause poor detection effects, since a model may struggle to find consistent patterns across such diverse aggregates.

Three processing pipelines are characteristic of flow-based detection. The first applies feature engineering followed by shallow models such as KNN and clustering, extracting hand-designed statistical features from each flow and feeding them to a shallow learner. The second converts each flow into an image and applies a convolutional neural network for attack classification, exploiting the CNN's strength on spatially structured data by rendering flow statistics in visual form. The third uses unsupervised deep learning for feature extraction followed by XGBoost, a powerful gradient-boosted tree ensemble, as the final classifier, combining learned representations with a strong shallow classifier at the output.

Session-based Attack Detection

A **session** represents the interaction process between two terminal applications — the complete back-and-forth exchange between two communicating endpoints, a yet higher level of aggregation than the flow. Its advantages stem from this completeness of interaction. A session is well suited for detecting an attack between specific IP addresses, such as tunnel and Trojan attacks, where the malicious behavior is defined by the pattern of a sustained exchange between two particular parties. It also contains information that helps localize the attack source, since the full interaction identifies who is

communicating with whom. The principal disadvantage is one of scale: a session's duration can be long, and a long session results in huge volumes of data that must be stored and processed.

Session-based detection divides along the kind of features extracted. When **static-based features** are used — aggregate summary statistics of the whole session — they are fed into detection trees that classify the session as normal versus abnormal. When **sequence-based features** are used — preserving the temporal order of events within the session — they are fed into a recurrent neural network, whose memory of earlier events makes it the natural model for sequential structure.

Log-based Attack Detection

The fourth granularity departs from raw network traffic entirely. **Logs** are the activity records produced by operating systems and application programs, and log-based detection examines these records rather than the packets on the wire. Logs include system calls, alert logs, and access records, among other forms of recorded activity. Their advantages lie in the richness and intelligibility of their content. Logs carry detailed content information suitable for detecting attacks that leave their mark at the application level, such as SQL injection and the privilege-escalation categories U2R (user-to-root) and R2L (remote-to-local). They carry information about users and timestamps, attributing actions to accounts and placing them in time, and they can record the complete intrusion process, the full narrative of an attack as it unfolds across a system.

The disadvantages are equally characteristic. Effective use of logs depends on cyber security knowledge, because interpreting them requires understanding what the recorded events mean and which patterns signify danger. And different programs produce logs in different formats, so there is no single uniform structure to parse; each log source may demand its own handling. Three processing approaches address these challenges. **Rule and machine-learning-based hybrid methods** combine expert-written rules with learned models. **Log feature extraction-based detection** segments the log stream using a sliding window and applies a convolutional neural network to the resulting features. And **text analysis-based detection** treats log entries as text, extracting n-gram features and classifying them with a support vector machine or an isolation forest.

The Research Landscape in Summary

Taken together, the body of research on machine-learning-based intrusion detection maps neatly onto this four-way division of data sources, with each granularity having attracted its own cluster of methods, algorithms, and benchmark datasets. Packet-parsing work has paired SVM and K-means, or fuzzy C-means clustering, against private and DARPA datasets. Payload-analysis work has applied CNNs, LSTM networks, autoencoders, and generative adversarial networks to datasets such as ISCX 2012 and CTU-UNB. Flow-based research spans statistical-feature approaches using SVM, decision trees, Naive Bayes and KNN on the classic KDD99 benchmark; deep-learning approaches using CNNs, autoencoders with XGBoost, and GANs on NSL-KDD and UNSW-NB15; and traffic-grouping work using SVM and deep neural networks. Session-based research covers statistical-feature methods using decision trees and K-means on CICIDS 2017, and sequence-feature methods using CNN-LSTM combinations and LSTMs on ISCX and DARPA data. Log-based research encompasses rule-based methods using KNN and DNNs, sliding-window feature extraction using CNNs, DNN-RNN combinations and clustering with DBSCAN, and text-analysis methods using SVMs and isolation forests on datasets such as CSIC 2010. The consistent thread is that the choice of data granularity shapes every subsequent decision — which features are available, which algorithms are appropriate, and which datasets can validate the result.

Other Cybersecurity Domains

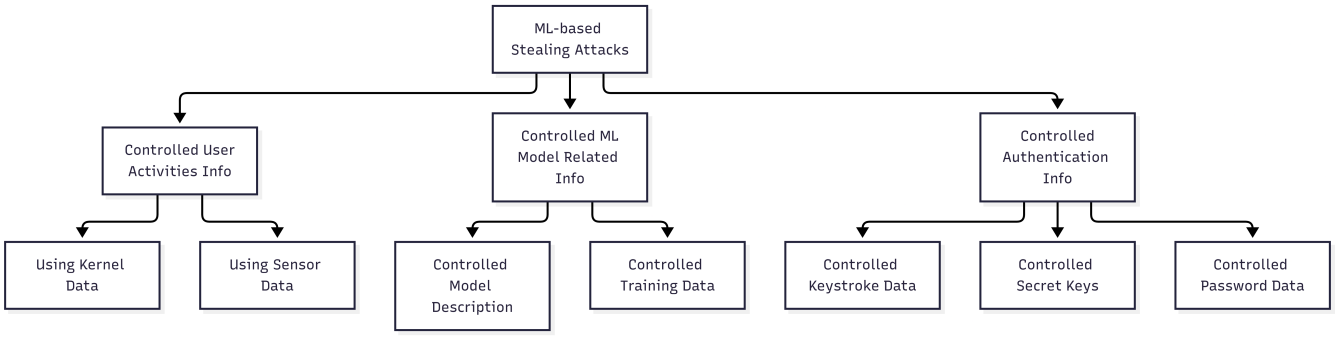
Beyond intrusion detection, machine learning extends into further security domains that illustrate the breadth of the field. **Continuous authentication** aims to verify a user's identity not once at login but continuously throughout a session, by modeling behavioral biometrics. It captures and analyzes keyboard timing patterns — the intervals of individual key presses, the flight times between keys, and the rhythm of longer typing sequences — alongside touch-motion and mouse-motion dynamics, fusing these behavioral streams into a continuously updated confidence score that a legitimate user, rather than an impostor, is at the controls. **Steganalysis** addresses the detection of hidden information: where steganography conceals a secret message imperceptibly within an innocuous carrier such as an image, steganalysis uses machine learning to detect the subtle statistical traces that such concealment leaves behind, distinguishing a clean image from one harboring embedded data.

Machine Learning–based Attacks

A final turn in the subject reverses the perspective. Machine learning is not only a tool for the defender; it is increasingly a weapon for the attacker, and the number of such ML-based attacks is increasing. Because modern systems rely so heavily on advanced analytics solutions — which is to say, on the use of machine learning — these very systems and the data feeding them become attractive targets. In this adversarial context, **information leakage** becomes a growing concern. Information leakage is defined as the violation of the confidentiality of the methods, mechanisms, or frameworks that store or have access to information; it is the escape of sensitive knowledge from the systems meant to contain it, and machine learning gives attackers powerful new means to induce and exploit such leakage.

Stealing Attack Targets

The objects that ML-based stealing attacks pursue can be organized into three broad families, according to the type of controlled information the attacker seeks to extract.



The first family targets **controlled user activities information** — knowledge of what a user is doing. Such attacks operate either by using kernel data, drawn from the low-level operating-system activity of the device, or by using sensor data, drawn from the device's physical sensors, to infer a user's private behavior. The second family targets **controlled machine-learning model-related information**, aiming to steal the intellectual and operational substance of a model itself. This divides into attacks on the controlled model description — the architecture and parameters that define how the model works — and attacks on the controlled training data, seeking to recover the sensitive examples the model was trained upon. The third family targets **controlled authentication information**, the secrets that guard access to systems. Here the targets are controlled keystroke data, from which typed secrets can be reconstructed; controlled secret keys, the cryptographic material underpinning security; and controlled password data, the credentials that authenticate users. Together these three families map out the attack surface that the pervasiveness of machine learning has opened, and they close the subject by making plain that the same statistical power which strengthens defense can, turned around, be made to serve the adversary.

CHAPTER 7: ML for APT

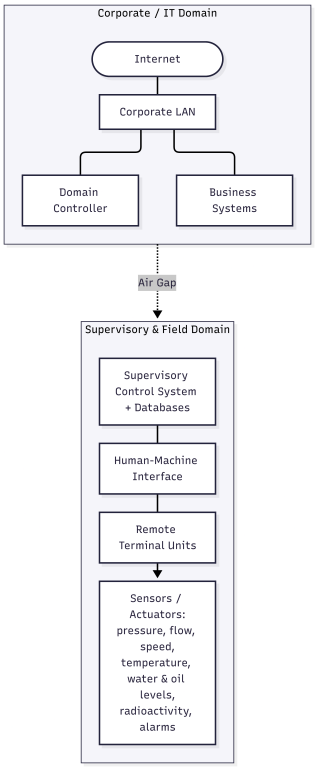
Critical Infrastructures and Their Security Context

Critical infrastructures are the assets and systems whose continued operation is vital to the functioning of society and the daily life of citizens. They encompass domains such as water supply, energy generation and distribution, heat, telecommunications, finance, healthcare, food, and justice. Because the services they deliver are indispensable, the governing operational requirement for a critical infrastructure is **continuous availability**: these systems should remain operational at all times, and any interruption can translate directly into harm to the population that depends on them.

A defining property of critical infrastructures is that they are **morphologically different from classical computer networks**. A conventional enterprise network is built chiefly around general-purpose computing devices exchanging data, whereas a critical infrastructure is a heterogeneous assembly composed of many distinct classes of devices, ranging from ordinary IT equipment to specialized industrial control components. This heterogeneity has direct consequences for security, because defensive techniques designed for standard networks cannot be transplanted wholesale onto infrastructures whose composition, communication patterns, and operational constraints are fundamentally different.

The internal structure of a typical critical infrastructure is organized into layers that separate ordinary business computing from the industrial process it controls. At the top sits the **corporate domain**, comprising the connection to the Internet, the corporate LAN, workstations, the domain controller, and general business systems. Below this lies the **supervisory and control domain**, which includes the Supervisory Control System and its associated databases, together with the Human–Machine Interface (HMI) through which operators observe and command the process. The lowest layer is the **field domain**, where Remote Terminal Units (RTUs) interface with the physical world through sensors and actuators. These field devices monitor and act upon concrete physical quantities such as pressure, pump and fan speed, flow rate, noise level, oil levels and maintenance alarms, radioactivity levels, water levels, and temperature.

A crucial architectural feature separating the business side from the industrial side is the **air gap**, a deliberate isolation intended to prevent threats originating in the corporate, Internet-facing environment from reaching the sensitive control and field systems. The air gap expresses the security philosophy that the process-control portion of the infrastructure should be insulated from the more exposed and more frequently attacked IT portion.



Organized Attacks and the Threat Landscape

Threats against infrastructures and organizations can be positioned along two intersecting dimensions: their **sophistication** and their **prevalence**. These two axes stand in inverse relation. The most prevalent threats are the least sophisticated, while the most sophisticated threats are comparatively rare. Each category of attacker can be characterized by its **motivation** (the goal it pursues) and its **target** (the type of victim it selects).

At the base of this hierarchy, where prevalence is highest and sophistication lowest, sit **commodity threats**. These are financially motivated and indiscriminately target all end-users; they represent the mass-market, opportunistic form of cybercrime. Above them are **targeted attacks**, again financially or otherwise materially motivated, but aimed specifically at corporations. Higher still are **hacktivists**, whose motivation is reputational or social rather than financial, and who direct their activity against corporations, governments, entire industries, and high-profile individuals. Near the top lies **industrial espionage**, motivated by the theft of trade secrets and insider information and aimed at corporations and government contractors. At the apex of sophistication, and the lowest in prevalence, are **nation-state sponsored** operations, whose motivation is espionage and political influence and whose targets are government agencies, think tanks, and corporations.

This ordering matters because defensive priority should track sophistication: the rarest and most capable adversaries, the nation-state sponsored actors, are precisely the ones whose activities correspond to **Advanced Persistent Threats**, the central subject of the discipline that follows.

Advanced Persistent Threats

An **Advanced Persistent Threat (APT)** is formally defined, following the standard established by the National Institute of Standards and Technology (NIST), as *an adversary that possesses sophisticated levels of expertise and significant resources which allow it to create opportunities to achieve its objectives by using multiple attack vectors.* Three elements of this definition deserve emphasis: the adversary is **expert**, it is **well-resourced**, and it operates through **several simultaneous or alternative avenues of intrusion** rather than relying on a single technique.

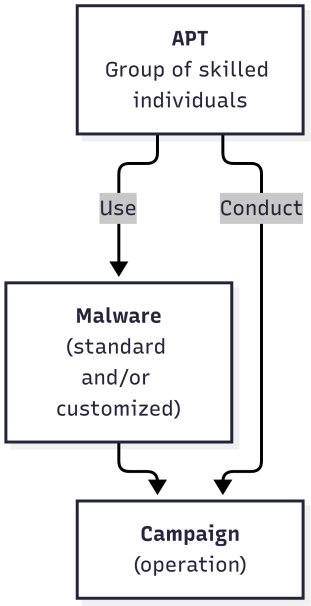
The three words forming the acronym each capture a distinct characteristic of the phenomenon. **Advanced** signifies the use of advanced cyber-intrusion techniques, which may include custom tooling, exploitation of previously unknown vulnerabilities, and carefully engineered evasion. **Persistent** indicates that, once inside, the adversary remains within the target systems for long periods, maintaining access covertly over months or years rather than striking and withdrawing. **Threat** underscores that the adversary is organized, well-financed, and driven by a specific objective, as opposed to the opportunistic character of commodity crime.

The objectives pursued by these actors fall into three principal categories: **stealing information**, **spying** on the target over time, and, where their mission requires it, **sabotaging** the target's operations. Because such actors concentrate on high-value victims and inflict disproportionately serious damage, it is **essential to assign higher analytical priority to malware associated with APTs** than to ordinary malware. When analytical resources are limited, the samples connected to these advanced actors are the ones that must be examined first.

The distribution of APT activity across economic sectors illustrates where these actors concentrate their effort. Observed APT activity is dominated by the **government** sector, which accounts for roughly a quarter of all observed activity (26%), followed by **defence contractors** (14%) and **media** (12%). The remaining activity is distributed across financial institutions (8%), NGOs, legal firms, and higher education (around 7% each), technology and engineering/manufacturing and energy (around 5% each), and smaller shares against social media and hospitality. The concentration against government, defence, and media confirms that these adversaries are predominantly oriented toward espionage and influence rather than indiscriminate financial gain.

The Structure of an APT Operation

It is important to distinguish clearly between the actor, the tooling, and the activity, because the three terms are often conflated. An **APT**, in the strict sense, is a **group of skilled individuals**. This group plays two distinct roles. On one hand it **uses malware**, which may be either standard off-the-shelf malicious software or code specifically customized for its purposes. On the other hand it **conducts campaigns**, where a campaign (also called an operation) is a coordinated sequence of malicious actions directed at achieving a particular objective. The malware is the instrument employed within the campaign; the campaign is the orchestrated activity that the group carries out.



This conceptual separation is the foundation of the analytical problem that follows. Since a given group tends to reuse code, techniques, and infrastructure across its campaigns, the malware it employs carries a kind of signature of its author. If that signature can be recovered from a malicious sample, then the sample can be **attributed** to the group that produced it, which is the essence of APT identification.

An Architecture for Semi-Automatic Collaborative Malware Analysis

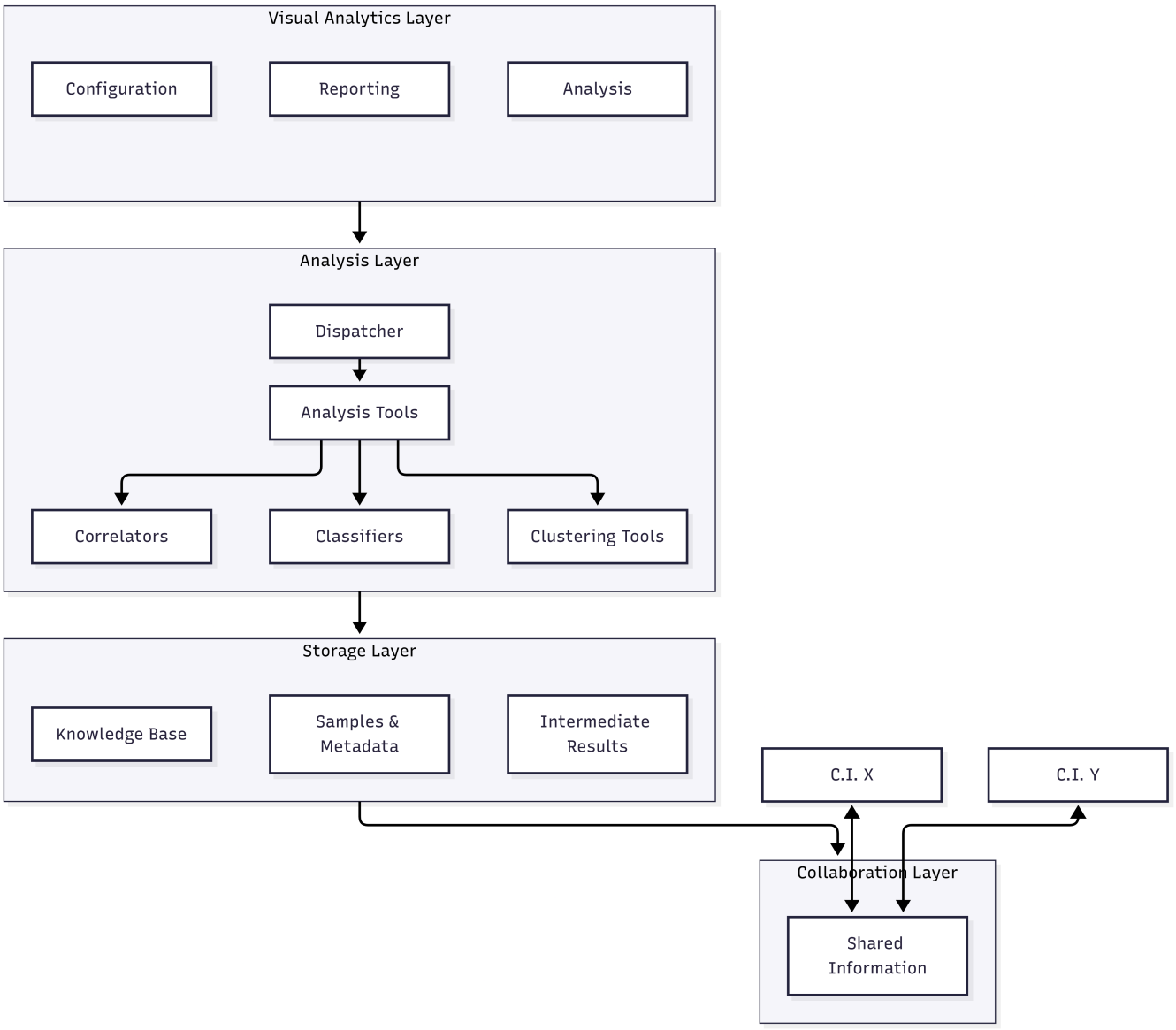
To analyze malware in the specific context of critical infrastructures, a layered architecture has been proposed that supports **semi-automatic** analysis (combining automated tools with human expertise) and **collaboration** across distinct infrastructures. The architecture is organized into four stacked layers, each serving a specific category of user or function, and it is designed so that multiple independent critical infrastructures can share analytical results among themselves.

The topmost layer is the **Visual Analytics Layer**, the interface through which the human users interact with the system. It serves three groups of stakeholders, each with a dedicated function. **Executives** (such as CEOs and managers) consume the **Reporting** function, which presents high-level summaries. **Malware analysts** operate the **Analysis** function, through which detailed examination is performed. **Security administrators** use the **Configuration** function to set up and tune the system.

Beneath it lies the **Analysis Layer**, which contains the computational machinery of the system. A **Dispatcher** routes samples to a collection of **Analysis Tools**, and the outputs of these tools feed into three families of automated engines: **Classifiers**, which assign samples to categories; **Clustering Tools**, which group similar samples together; and **Correlators**, which relate samples and events to one another. This is the layer in which machine learning methods are embedded.

The third layer is the **Storage Layer**, which persists three distinct kinds of data: the **Samples and metadata** (the raw malicious files together with descriptive information), the **Intermediate analysis results** (the partial outputs produced during examination), and the **Knowledge Base** (the accumulated, consolidated knowledge about threats).

The lowest layer is the **Collaboration Layer**, which enables different critical infrastructures, denoted for example C.I. X and C.I. Y, to exchange **Shared Information**. Through this layer, knowledge gained from analyzing an attack against one infrastructure can be propagated to others, so that a threat observed once can be recognized elsewhere without each organization having to rediscover it independently.



Within this broader architecture, the components of greatest interest for the remainder of this chapter are the **classifiers** in the Analysis Layer, since it is through classification that malicious samples are both detected as APT-related and attributed to specific APT groups.

The Triage Approach to APT-Related Malware Identification

The analysis of malware in a critical-infrastructure setting operates under a severe resource constraint: the number of samples requiring examination vastly exceeds the capacity of the human experts available to examine them. This situation is strongly analogous to **medical triage**, and the analogy is more than rhetorical, since it dictates the design of the entire identification pipeline.

In an emergency medical setting, when casualties overwhelm the available doctors, patients are sorted by urgency so that scarce medical attention is directed first to those who can benefit most. The standard medical triage procedure assesses each casualty through a rapid sequence of checks, whether the person is able to walk, whether they are breathing spontaneously, their respiratory rate, their perfusion as measured by radial pulse or capillary refill, and their mental status, and assigns one of a small set of priority categories. These categories are conventionally color-coded: **Minor** (green, for the walking wounded whose condition is unlikely to deteriorate), **Delayed** (yellow, for serious but not immediately life-threatening injuries), **Immediate** (red, for casualties who can be saved by prompt intervention and whose airway, breathing, or circulation is compromised), and **Expectant** (black, for those unlikely to survive given the severity of injury and the care available, who receive palliative attention). The purpose

of this rapid sorting is to allocate a limited resource (doctors) to the cases where it yields the greatest return.

The identification of APT-related malware is governed by the same logic. The **analysts are the doctors**, and like doctors they are a **limited resource**. Because they cannot examine every sample in depth, the system must perform **prioritization**, pushing to the front of the queue the samples most likely to be connected to advanced actors. A distinctive feature of this triage is the emphasis on **reducing false positives**. A false positive, here, is a benign or non-APT sample mistakenly flagged as APT-related; each such error wastes an analyst's scarce time on a sample that did not merit it, and therefore minimizing false positives is central to using the analytical workforce efficiently.

A second consequence of the triage philosophy is that the automated screening preceding human analysis must be **fast**. Triage is only useful if the sorting itself is quick; a slow classifier would defeat its own purpose. For this reason the approach relies on **basic static analysis tools**, that is, tools that examine a sample without executing it and extract readily available features, rather than on expensive dynamic analysis that runs the sample and observes its behavior over time. The means by which samples are actually sorted is a set of **classifiers**, machine-learning models that, given the statically extracted features of a sample, assign it to a category and thereby perform the role of identification.

Feature Extraction

For a classifier to operate on a malicious executable, that executable must first be reduced to a numerical representation, a vector of **features** summarizing its measurable properties. In this approach the features are drawn predominantly from the structure of the Windows Portable Executable file and from a static inspection of its code, and they are grouped into categories of differing size.

A substantial portion of the feature set comes from the **headers of the executable**. The **Optional Header** contributes 30 features, the **MS-DOS Header** contributes 17, and the **File Header** contributes 18. These header-derived features describe how the file is organized, how it is meant to be loaded, and the various structural parameters recorded at compile time. In addition, **String Statistics** contribute 3 features, capturing aggregate properties of the textual strings embedded in the binary, and the **Mutex** category contributes 7 features, reflecting the named synchronization objects a program may use (which are often characteristic of particular malware families). The **Packer** category contributes 64 features, indicating whether and how the executable has been compressed or obfuscated by a packer, a common malware technique for hindering analysis.

By far the largest category is the **Imported API**, which contributes **3917 features**. These correspond to the Windows API functions that the executable declares it will use; the set of functions a program imports is highly informative about its behavior and intentions, and because there are thousands of possible API functions, this category dominates the feature space. Finally, the **Buckets** category contributes 98 features. Summing across all categories yields a high-dimensional feature vector in which the API-usage information is overwhelmingly the richest signal.

Category	Count
Optional Header	30
MS-DOS Header	17
File Header	18
String Statistics	3
Mutex	7

Category	Count
Packer	64
Imported API	3917
Buckets	98

Two concrete tools perform the extraction. **PEFrame** is used to obtain the header information, parsing the structural metadata of the Portable Executable file. **radare2**, a reverse-engineering framework, is used to obtain information about the functions present in the binary. Together they furnish the static features from which the feature vector is assembled, and because both operate without running the sample, they satisfy the speed requirement imposed by the triage philosophy.

The Not-Belonging Class Problem

Having reduced each sample to a feature vector, one must train classifiers to recognize APT authorship. Here a fundamental difficulty of the machine-learning formulation emerges, which may be called the **not-belonging class problem**.

A conventional classifier learns to distinguish among classes by being shown labeled examples of each. To identify which APT group produced a given sample, one would naturally train the classifier on **samples from each APT** (for example, samples attributed to APT28, to the Winnti Group, to the Sandworm Team, and so on). But detecting whether a sample is APT-related *at all* requires, in addition, examples of the opposite case, namely **normal (non-APT) malware**, so that the model can learn the boundary between the two.

The problem is that these two bodies of data are wildly different in size. APT-related samples are rare, because advanced actors are few and their campaigns comparatively infrequent, whereas ordinary malware exists in enormous quantity. Training a conventional classifier on both therefore produces a **heavily unbalanced training set**, in which the "everything else" category dwarfs the APT categories. The difficulty is conceptual as well as quantitative: the negative class is not a coherent, well-defined category with its own internal structure, but rather an open-ended, essentially unbounded collection of "everything that is not one of our known APTs." It is impossible to collect a sample that faithfully represents the whole of this residual class, and any attempt to do so distorts the learning problem.

The proposed remedy is to **avoid modeling the not-belonging class explicitly**. Instead of training on both APT and non-APT data, the system is trained **only on APT-related malware**. The intuition is that the model should learn what each APT's output *looks like* from positive examples alone, and treat anything that fails to match any known APT profile as not belonging, rather than trying to learn a positive description of the unbounded "normal malware" category. This reframing, from a standard multi-class problem that requires a representative negative class into one that learns each target class from its own positive examples, is what motivates the specific classifier designs introduced next.

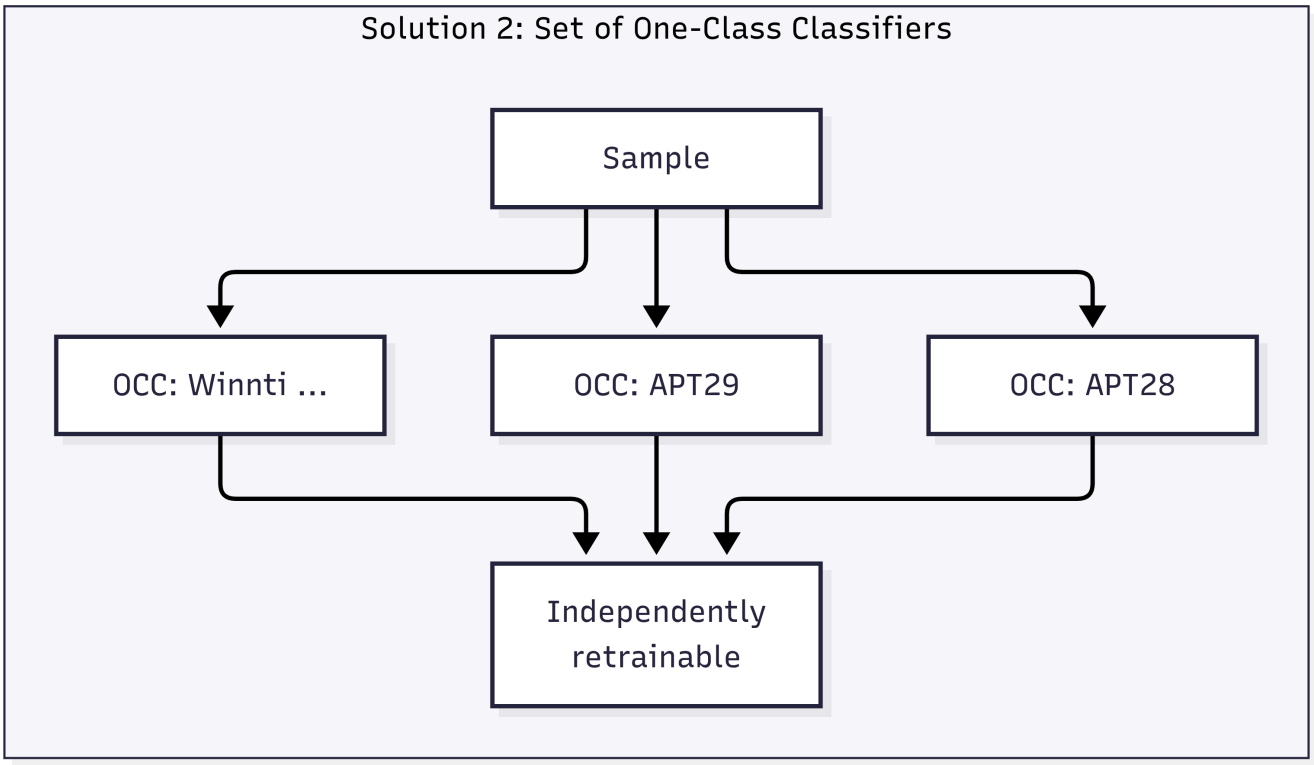
The Two Solutions

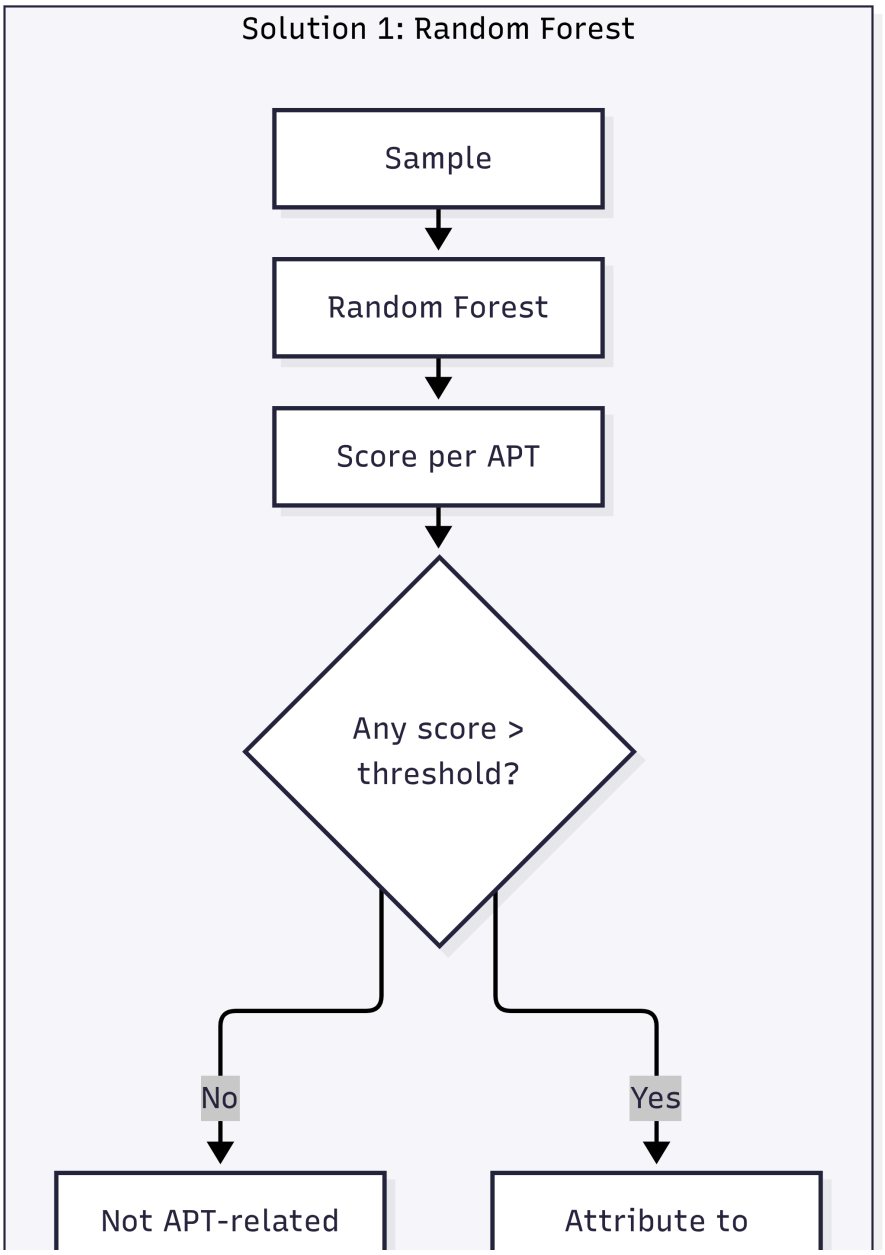
Two concrete classifier designs were developed in response to the not-belonging class problem, the second representing a refinement of the first.

The **first approach** uses a single **Random Forest** to compute a **score of authorship**. A Random Forest is an ensemble of decision trees whose collective vote yields, for a given sample, a score for **each possible APT**, expressing how strongly the sample resembles the output of that group. The decision rule for detection is then expressed through a **threshold**: if none of the per-APT scores exceeds the

threshold, the sample is judged **not APT-related**; if some score does exceed it, the sample is both detected as APT-related and attributed to the highest-scoring group. The principal **disadvantage** of this design is its **high retraining time**. Because the single forest jointly encodes all APTs, any change, such as the addition of a newly discovered group, requires retraining the entire model, which is costly.

The **second approach** is a **modular improvement** that replaces the single joint model with a **set of One-Class Classifiers, one for each APT**. A one-class classifier is trained exclusively on the positive examples of a single class and learns to decide whether a new sample belongs to that class or not, which aligns perfectly with the not-belonging formulation. In this design **each classifier identifies only its own APT**. The decisive practical advantage is modularity in retraining: because the classifiers are independent, one can **retrain only a single classifier of the set at a time**, for instance to incorporate new samples of one group or to add an entirely new group, without disturbing the others. This directly resolves the high-retraining-time weakness of the first approach.


Solution 2: Set of One-Class Classifiers
Sample
OCC: Winnti ...
OCC: APT29
OCC: APT28
Independently retrainable


Solution 1: Random Forest
Sample
Random Forest
Score per APT
Any score > threshold?
No
Yes
Not APT-related
Attribute to



Accuracy Evaluation: The Reference Dataset

The evaluation of these methods rests on a purpose-built reference dataset named **dAPTaset**, a comprehensive mapping of APT-related data. It comprises **2086 samples** distributed across **15 APT groups**. The per-group counts reveal a pronounced imbalance even *within* the APT population itself: some groups are represented by hundreds of samples while others barely reach a couple dozen.

Class	Count
APT28	68
APT29	205
APT30	101
Carbanak	105
Desert Falcon	45
Hurricane Panda	315
Lazarus Group	58
Mirage	54
Patchwork	559
Sandworm	44
Shiqiang	31
Transparent Tribe	267
Violin Panda	23

The largest classes, **Patchwork** (559 samples) and **Hurricane Panda** (315), together account for a large fraction of the dataset, while the smallest, **Violin Panda** (23) and **Shiqiang** (31), are represented only sparsely. This internal imbalance is itself a challenge for the classifiers, since a group with very few samples offers little data from which to learn its characteristic profile, a point to keep in mind when interpreting the experimental results that follow.

Experimental Results of the First Two Solutions

The two designs just described, the single Random Forest and the set of One-Class Classifiers, were evaluated on the dAPTaset reference dataset across two distinct tasks, **detection** and **identification**, and along a third practical dimension, **execution time**. Before interpreting the numbers it is worth distinguishing the two tasks precisely. **Detection** is the binary question of whether a sample is APT-related at all, separating the advanced-actor malware from everything else. **Identification** (equivalently, attribution) is the finer question of *which* specific APT group produced a sample already judged to be APT-related. The four standard metrics used throughout are **accuracy** (the overall proportion of correct decisions), **precision** (of the samples flagged as belonging to a class, the proportion that truly do, a direct measure of false-positive control), **recall** (of the samples that truly belong to a class, the proportion that were correctly caught, a measure of false-negative control), and the **F1 score** (the harmonic mean of precision and recall, summarizing the balance between the two).

On the **detection** task, both methods perform strongly, with every metric sitting close to the top of its range, near 0.97 and above. The set of One-Class Classifiers edges out the Random Forest slightly on all four measures (accuracy, F1, precision, and recall), with the one-class approach approaching 0.99 against the forest's roughly 0.97. The practical conclusion is that both designs detect APT-related malware very reliably, and the modular one-class design does so marginally better.

On the **identification** task, the results are even more striking: both methods achieve metrics essentially at the ceiling, with accuracy, F1, precision, and recall all effectively equal to 1. In other words, once a sample has been recognized as APT-related, both approaches attribute it to the correct group with near-perfect reliability on this dataset. The two designs are, for identification purposes, indistinguishable in quality.

The decisive difference between the two designs appears not in accuracy but in **execution time**, which is measured on a logarithmic scale because the differences span orders of magnitude, and which is broken into a **training phase** and a **test phase**. In the **training phase** the contrast is dramatic: the Random Forest requires on the order of **a thousand seconds or more**, whereas the One-Class Classifier set trains in only a **few seconds**, a difference of roughly three orders of magnitude. This quantifies concretely the "high retraining time" drawback of the Random Forest noted earlier, and confirms the central advantage of the modular design. In the **test phase** the two are far closer, both completing in the single-digit-seconds range, with the one-class set again slightly faster. Since the dominant operational cost is retraining (which must happen whenever new groups or samples are incorporated), the enormous training-time advantage of the one-class approach is the deciding factor in its favor.

The overall lesson of these experiments is that the two methods are of comparable and excellent *quality*, so the choice between them turns on *operational efficiency*, where the modular set of one-class classifiers is decisively superior because it can be retrained quickly and incrementally.

Limitations of the Single-Phase Approach

Despite these favorable results, a careful reading of the evaluation exposes a structural weakness. The methods deliver **very good results** in **accuracy**, **precision**, and **required time**, the dimensions on which they were praised above. However, they deliver **worse results** on one specific metric: **false negatives**. A false negative here is an APT-related sample that the system fails to recognize as such, allowing a genuinely dangerous sample to slip through unnoticed. In the triage context this is a serious failure mode, arguably more dangerous than a false positive, because a missed APT sample means an advanced threat goes unexamined.

The root cause is identified as a defect of **process design** rather than of the classifiers themselves. In both the Random Forest and the one-class approaches, **detection and identification are performed in a single phase**: the system attempts to decide simultaneously whether a sample is APT-related and which group it belongs to. In this arrangement **detection is merely a side effect of identification**. A sample is treated as APT-related only because it happened to match some APT profile strongly enough; detection is not pursued as a goal in its own right but falls out incidentally from the attribution machinery. Consequently, a sample that is genuinely APT-related but does not closely resemble any *specific* known group may fail to clear any group's threshold and thus be wrongly discarded as non-APT, producing a false negative.

The remedy that this diagnosis suggests is to **stop treating detection as a by-product** and instead give it its own dedicated stage, which is precisely the motivation for the multi-step architecture introduced next.

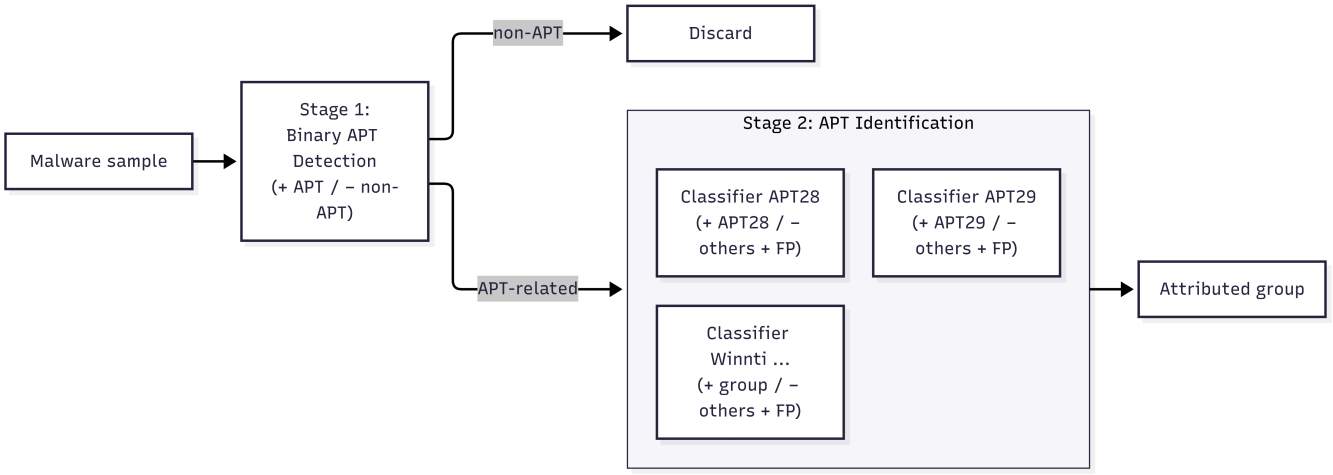
The Proposed Multi-Step Solution

The proposed solution **combines classifiers into a multi-step approach**, explicitly separating the two tasks so that each is optimized on its own terms. The pipeline consists of two sequential stages, each built from its own classifier or set of classifiers.

The first stage is a single **binary classifier for APT detection**. Being binary, it is trained on exactly two classes: **APT-related malware as the positive class**, and **non-APT malware as the negative class**. Its sole responsibility is the detection question, is this sample APT-related or not, and because it is trained directly and exclusively on that distinction, detection is now a first-class objective rather than an accident of attribution.

The second stage is a **set of binary classifiers for APT identification**, one classifier per APT group, activated only on the samples that the first stage has already judged to be APT-related. Each of these per-group classifiers is binary and is trained with a carefully constructed pair of classes. Its **positive class** consists of samples of the **current APT**, the single group that this particular classifier is responsible for recognizing. Its **negative class** is composed of two kinds of material: the samples of **all the other APTs**, and, crucially, the **false positives produced by the detection classifier of the first stage**.

The inclusion of the detection stage's false positives in the negative class of the identification classifiers is the subtle and important design choice. It means that the second stage is deliberately trained to recognize and reject exactly the kinds of samples that the first stage tends to mistake for APT malware. The two stages are thereby coupled: the identification stage learns to clean up after the characteristic errors of the detection stage, so that a non-APT sample which slips through detection is caught and rejected during identification rather than being falsely attributed to some group. This cooperative, error-aware coupling between the two stages is what the single-phase designs lacked, and it is intended to drive down the false-negative rate that limited the earlier approaches while preserving their strong accuracy, precision, and speed.



Methodology Details of the Two-Phase Pipeline

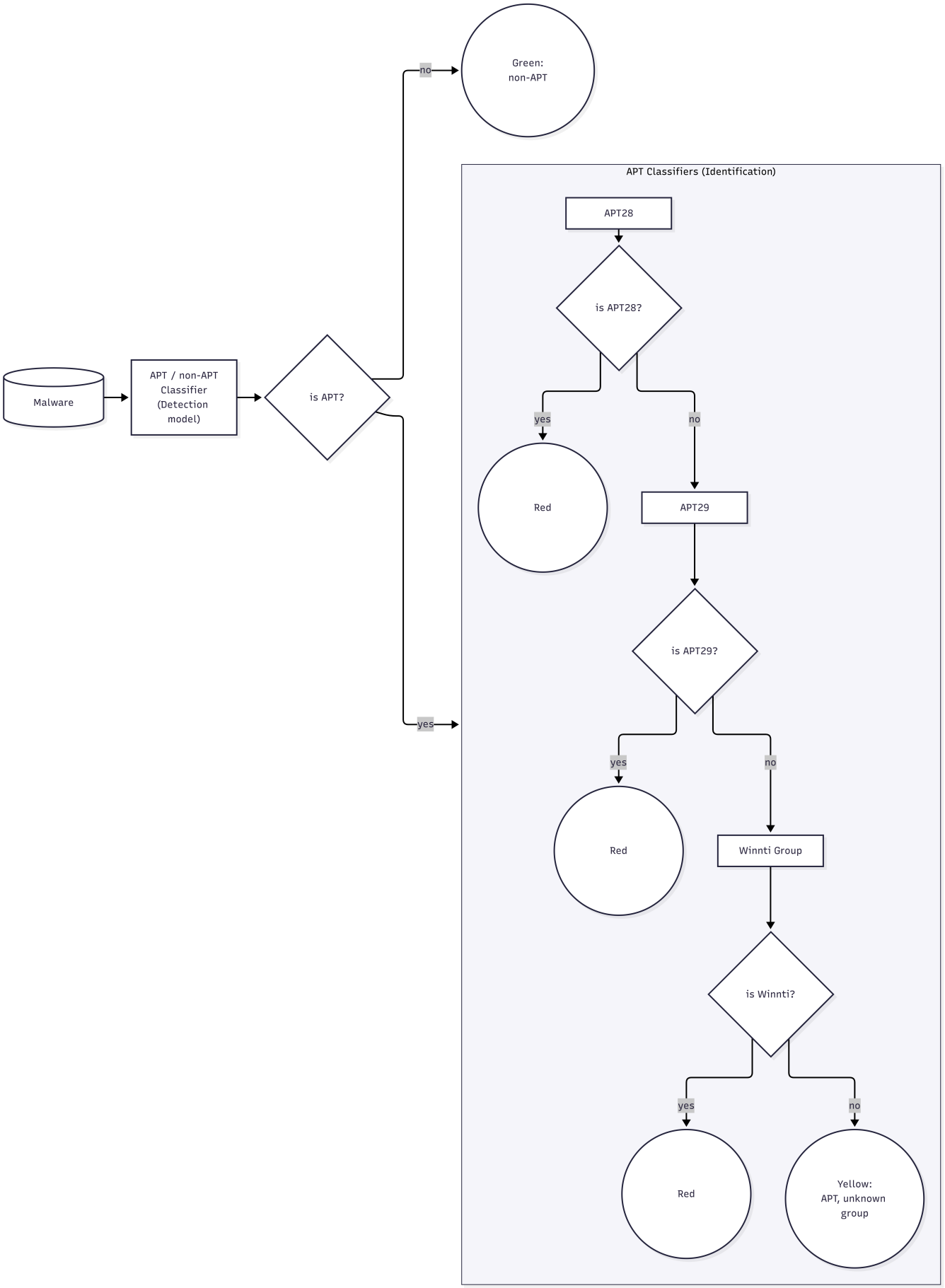
The multi-step solution can now be described as a concrete operational pipeline through which a malicious sample flows. The process is divided into two sequential stages, the **Detection Phase** and the **Identification Phase**, each driven by a trained model, and the flow of a sample through them produces one of three possible verdicts.

A sample drawn from the malware collection first enters the **Detection Phase**. Here it is processed by the **APT/non-APT classifier**, which consults a trained **detection model** and poses the single binary

question, *is this sample APT-related?* If the answer is **no**, the sample is immediately disposed of as non-APT malware and the pipeline terminates for that sample; in the color convention used to depict the pipeline this terminal verdict is marked **green**, signifying a sample screened out as not of interest. If the answer is **yes**, the sample is promoted to the second stage.

Having been judged APT-related, the sample enters the **Identification Phase**, where it is presented to the **set of APT classifiers** governed by the **APT identification model**. This model is not a single classifier but a collection of per-group binary classifiers, one trained for APT28, one for APT29, and so on through every group including the Winnti Group. The sample is tested against these classifiers in turn. Each classifier asks a yes/no question specific to its group, *is this APT28?*, *is this APT29?*, *is this the Winnti Group?*, and so forth.

The resolution of the Identification Phase follows a cascade. If a classifier answers **yes**, the sample is attributed to that classifier's group and the process concludes with a positive attribution, depicted as a **red** terminal state, signifying a confirmed, identified APT sample. If a classifier answers **no**, the sample passes on to be tested by the next group's classifier. Should a sample be confirmed as APT-related by the detection stage yet fail to be claimed by *any* of the per-group classifiers, it reaches a distinct outcome, depicted as **yellow**, signifying a sample that is APT-related but could not be attributed to a known group. This yellow outcome is significant: it is exactly the category that the single-phase approaches could not represent, because in those designs a sample that matched no specific group was simply lost as a false negative. The two-phase design, by contrast, preserves such a sample as a recognized-but-unattributed threat, which an analyst can then examine as a candidate for a new or previously uncatalogued group.



This three-outcome structure, **green** for screened-out non-APT, **red** for identified APT, and **yellow** for detected-but-unidentified APT, is the operational expression of having separated detection from identification. Detection alone decides green versus everything-else, while identification decides red versus yellow among the samples detection has retained.

Experimental Results of the Multi-Step Pipeline

The two-phase pipeline was evaluated using three different machine-learning algorithms as the underlying classifiers, so that their relative merits could be compared: **XGBoost**, a gradient-boosting ensemble of decision trees; **SVM** (Support Vector Machine), a margin-based classifier; and **Random Forest**, the bagging-based tree ensemble encountered earlier. As before, performance was assessed on the detection and identification tasks using accuracy, F1 score, precision, and recall, and separately on execution time.

On the **detection** task, all three algorithms perform well, but with a visible ordering. **Random Forest** attains the best results, with all four metrics reaching essentially the top of the scale (close to 1). **XGBoost** follows closely, with its metrics clustered around 0.95. **SVM** trails slightly behind XGBoost, with metrics in the vicinity of 0.93 to 0.94. The margins separating them are modest, but the consistent pattern across all four metrics is that Random Forest ≥ XGBoost ≥ SVM for detection quality.

On the **identification** task, the same ordering holds and the overall level rises. **Random Forest** again leads, with its metrics very near 1 (approximately 0.99). **XGBoost** is close behind at roughly 0.97 to 0.98. **SVM** again occupies the lowest position of the three, around 0.95 to 0.96. Thus for both the detection and identification tasks, Random Forest gives the highest-quality classification, XGBoost is a close second, and SVM, while still strong in absolute terms, is consistently the weakest of the three.

The **execution time** comparison, again plotted on a logarithmic scale and divided into a **training stage** and a **prediction stage**, reveals the trade-off that complicates the simple "Random Forest is best" conclusion. In the **training stage**, the three algorithms differ enormously. **SVM** is by far the most expensive to train, requiring on the order of several hundred seconds (approaching 1000). **XGBoost** is substantially cheaper at roughly 20 seconds. **Random Forest** is the cheapest of all to train, at around a second. In the **prediction stage**, the picture changes character: here **SVM** is again the slow one, taking on the order of ten-plus seconds to produce predictions, whereas both **XGBoost** and **Random Forest** predict almost instantaneously, in well under a second.

Taken together, these timing results make SVM unattractive on both counts, since it is simultaneously the weakest in quality and the slowest in both training and prediction. The real contest is between XGBoost and Random Forest, and here Random Forest emerges as the preferred choice: it matches or slightly exceeds XGBoost in classification quality while being faster both to train and to predict. The conclusion of this experimental campaign is therefore that, within the multi-step architecture, **Random Forest offers the best overall combination of accuracy and speed**, confirming its suitability as the engine for a triage system that must be both reliable and fast.

ApiScout and ApiVector: The Background

A distinct line of work on APT identification builds upon the observation that the *way a malicious program uses the Windows API* is a powerful fingerprint of its authorship. Two foundational tools, introduced by Plohmann and colleagues, underpin this line of work.

The first is **ApiScout**, a **dynamic analysis technique for accurately reconstructing Windows API usage information from memory dumps**. When a program runs, the operating system resolves the API functions it uses and records the addresses of those functions in memory. ApiScout works by examining a **memory dump**, a snapshot of the program's memory taken during or after execution, and reconstructing from it which Windows API functions the program actually invoked. Because it operates on the memory image produced by a running program, ApiScout is fundamentally a **dynamic** technique: it requires the sample to have been executed and its memory captured.

The second tool is **ApiVector**, which is not an analysis technique but a **methodology for representing and comparing API usage**. It serves two purposes: **efficiently storing data about a malware's usage of Windows APIs**, and **comparing that API usage across samples for the purpose of malware family classification**. The core idea is to encode the set of API functions a sample uses as a compact structure, essentially a bit vector, and then to measure the similarity between two such structures to judge whether two samples are likely to belong to the same family.

The connection between the two tools can be traced concretely. ApiScout, examining the decoded bytes of a memory dump, recovers that the sample uses, for example, functions from `kernel32.dll` such as `GetCurrentThreadId`, `CreateFileW`, and `WriteConsoleW`. This recovered list of API functions is then handed to the ApiVector methodology, which stores it compactly and enables it to be compared against the API-usage profiles of other samples. In this way, raw memory becomes a list of APIs, and the list of APIs becomes a comparable signature.

APTScout: Motivation and Contributions

The tool named **APTScout** is motivated by a critical practical weakness in the ApiScout pipeline just described. Although ApiScout accurately recovers API usage, it does so by **dynamic analysis**, and this dependence carries several serious drawbacks that make it ill-suited to the fast, scalable triage setting that APT analysis demands.

The **drawbacks of ApiScout** all stem from its reliance on **memory dump extraction**. Obtaining the memory dump is **usually a manually performed task**, and automating it is not easy, since it typically requires setting up and driving an execution environment for each sample. As a consequence, the approach **does not scale up**: a procedure that needs manual intervention per sample cannot keep pace with large volumes of malware. Finally, producing a memory dump **takes time**, on the order of **minutes** per sample, which is far too slow for a triage system whose entire justification is rapid prioritization.

In direct response to these shortcomings, APTScout offers two **contributions**. The first is a **new fast and automated static analysis technique for recovering the Windows API functions used by an executable**. The decisive words are *static* (no execution and therefore no memory dump is required) and *automated* (no manual intervention per sample), which together eliminate the scalability and speed problems of ApiScout. The second contribution is a **preliminary analysis showing how Windows API usage data and ApiVectors can be used also for APT malware identification**, extending the family-classification idea of the original ApiVector work into the specific domain of attributing malware to APT groups.

In essence, APTScout keeps the valuable representational machinery of ApiVector, the compact encoding and similarity comparison of API usage, but replaces the slow, manual, dynamic front-end of ApiScout with a fast, automatic, static one.

APTScout: API Recovery — The Core Idea

The central idea of APTScout's recovery stage is to exploit the **analysis and decompilation capabilities of Ghidra** to recover the Windows API functions that an executable uses, without ever running it.

Ghidra is an **open-source, cross-platform software reverse-engineering framework developed by the United States National Security Agency (NSA)**. It provides a suite of tools for **disassembling** code (translating machine code back into assembly instructions), **decompiling** code (reconstructing a higher-level, C-like representation from the machine code), and **analyzing** code that

was compiled on a variety of platforms, including Windows, Linux, and macOS. It **supports a wide variety of binary formats and architectures**, and, decisively for APTScout's goal of automation, it **can be automated and extended through its Java and Python scripting API**. This scriptability is what allows APTScout to drive Ghidra programmatically over many samples without human intervention.

The **main features** that this choice confers on APTScout follow directly from using a static reverse-engineering framework. The method is a **static analysis** method, requiring **no execution of the malware**. It is **completely automated**. It is **fast**, running in a **few seconds on average**, in stark contrast to the minutes required by the dynamic ApiScout. And it **accurately recovers the most common dynamic import code patterns**, a capability examined in detail below, since recovering dynamically imported functions is the hard part of static API recovery.

APTScout: Recovering Static Imports

The functions an executable uses fall into two categories, which must be recovered by different means. The simpler category is **static imports**.

Static imports are functions **provided by statically loaded shared libraries**, that is, by **DLL files that are loaded into memory before the actual execution of the program begins**. When a program is written to depend on such a library at compile time, the operating system's loader brings the library in automatically at launch. Crucially, **Windows executable files store information about these imports, including the names of the DLLs and the corresponding imported functions, in a dedicated structure called the Import Table**. Because this information is explicitly recorded inside the file itself, recovering static imports is straightforward: one need only read the Import Table.

APTScout accomplishes this by using the **scripting API of Ghidra to retrieve the data in the Import Table**. Ghidra parses the executable and exposes the symbols it imports; in its interface this appears as a symbol table listing, for each imported function, its name, its symbol type (External Function), its data type, its namespace (the DLL it comes from, such as `KERNEL32.DLL`), and its source (marked *Imported*). For example, functions such as `GetCurrentProcess`, `GetCurrentProcessId`, `GetCurrentThreadId`, `GetLastError`, `GetModuleFileNameA`, and `GetModuleHandleA` all appear as external functions imported from `KERNEL32.DLL`. APTScout's script simply reads these entries directly, obtaining the complete set of statically imported APIs with no ambiguity.

APTScout: Recovering Dynamic Imports

The harder category is **dynamic imports**. These are functions **provided by libraries that the program loads at run-time**, rather than at launch. Because the decision to load such a library and to resolve a function within it is made while the program executes, this information **cannot be found in any static data structure of the executable**; in particular, dynamically imported functions do not appear in the Import Table. This is precisely the mechanism malware authors exploit to hide their true behavior from static inspection: by resolving sensitive APIs dynamically, they keep those APIs out of the Import Table where a casual analyst would look.

APTScout's **solution** is an **automated search, within Ghidra's decompiled code, for the typical code patterns used to import functions dynamically**. The reasoning is that although the imported names are not stored in a table, the *code that performs the dynamic import* must still be present in the executable, and that code follows recognizable patterns that can be detected in the decompiled output.

To make this concrete, consider how dynamic importing looks in source code and in Ghidra's reconstruction of it. In the **original source code**, a programmer who wishes to call `MessageBoxA` dynamically first declares a function-pointer type for it, then calls `LoadLibraryA("user32.dll")` to load the library, checks that the handle is valid, calls `GetProcAddress` with that handle and the string `"MessageBoxA"` to obtain the function's address, and finally invokes the function through the resulting pointer before calling `FreeLibrary` to release the library. When Ghidra **decompiles** a compiled binary containing this logic, it produces functionally equivalent C-like code: a module handle is obtained from `LoadLibraryA("user32.dll")`, its validity is checked against a null handle, `GetProcAddress(hModule, "MessageBoxA")` retrieves the function pointer, and the pointer is then called. The names of the library and the function survive as **string literals** in this decompiled code, which is exactly what makes recovery possible.

However, this approach faces **three possible issues**. First, real malware may use **more complex patterns** than the textbook example, with the loading and resolution logic spread out, wrapped, or disguised. Second, **"noise" is introduced by the compiler's code optimizations**, which can rearrange, inline, or transform the code so that the pattern no longer appears in its canonical shape. Third, **bugs in the decompiler may produce wrong code**, so that the reconstructed output does not faithfully reflect the original logic. Each of these can cause a dynamic import to be missed or misread.

The Bottom-Up Recovery Strategy

To extract the dynamic imports reliably from the decompiled code, APTScout exploits an invariant of the dynamic-import mechanism: **dynamically importing a function always requires calling two specific functions**, namely **`LoadLibrary`** (or any function similar to it) to load the containing library, and **`GetProcAddress`** to resolve the desired function within that library. Every dynamic import, however disguised, must ultimately pass through this pair.

Building on this invariant, APTScout follows a **"bottom-up" approach**. It **begins by searching for every call to `GetProcAddress`**, since that is the function whose job is specifically to resolve an individual API by name, and it therefore sits closest to the information being sought. From each discovered `GetProcAddress` call, the analysis then **works backward to recover two things**: the **corresponding `LoadLibrary` call**, from which the **name of the DLL** is obtained, and the **string containing the API name**, which is the argument handed to `GetProcAddress`. Recovering both yields a complete dynamic import, the pair (DLL name, function name).

A practical fallback handles the case where recovery is only partial. **If the analysis finds the API name but cannot determine the DLL** (for instance because the `LoadLibrary` call could not be linked back to the `GetProcAddress` call through the noise), **an API database can be consulted to look up which DLL provides that function**. Since the mapping from well-known API names to their home DLLs is stable and publicly documented, a missing DLL name can usually be recovered from the function name alone, making the overall recovery robust even when the two halves of the pattern cannot be directly connected.

Storing and Comparing API Usage Information

Once APTScout has recovered the set of Windows API functions a sample uses, whether statically or dynamically imported, that set must be stored compactly and made comparable to the profiles of other samples. This is the role of the **ApiVector** representation.

An **ApiVector** is a **mapping between Windows API functions and the entries of a bit vector**. Each position in the vector is associated with a particular API function, and the value stored at that position, a single bit, **specifies whether that function was found in the sample or not**. A sample that imports a given API has the corresponding bit set to one; a sample that does not has it set to zero. The entire API-usage profile of a sample is thereby reduced to a string of bits, which is extremely efficient to store and to manipulate.

Comparison of ApiVectors: the Weighted Jaccard Metric

To decide how similar two samples are, one compares their ApiVectors, and the measure chosen for this is the **weighted Jaccard similarity metric**. The ordinary Jaccard similarity between two sets is the size of their intersection divided by the size of their union; it answers the question "of all the APIs used by either sample, what fraction is used by both?" The weighted version generalizes this by allowing each API position to carry a different importance.

Formally, the weighted Jaccard similarity between two ApiVectors A and B is

$$J_W(A,B) = \frac{\sum_i W_i, (A_i \wedge B_i)}{\sum_i W_i, (A_i \vee B_i)}$$

where A and B are the two ApiVectors being compared, W_i is a **weight vector** assigning an importance to each position, and i is the **offset** indexing the positions of the vectors. In the numerator, the logical AND $A_i \wedge B_i$ is one exactly when **both** samples use the API at offset i; the numerator thus accumulates the weights of the APIs the two samples share. In the denominator, the logical OR $A_i \vee B_i$ is one when **either** sample uses the API at offset i; the denominator accumulates the weights of all APIs used by at least one of them. The ratio therefore expresses the weighted proportion of shared API usage relative to total API usage, yielding a value between zero (no shared usage) and one (identical usage).

The introduction of the weights W_i is what makes the metric expressive, because not all API functions are equally informative. A function used by almost every program carries little discriminative value, whereas a rarely used one is highly telling. To exploit this, three possibilities for the weights were proposed: **equal weights**, in which every API counts the same and the metric reduces to the ordinary Jaccard similarity; **linear weights**, in which importance scales linearly according to some ordering of the APIs; and **non-linear weights**, in which the importance assigned to APIs grows or decays in a non-linear fashion. The choice of weighting scheme tunes how strongly rare, discriminative APIs dominate the similarity judgment.

APTScout: APT Identification

The purpose of recovering and comparing API usage is ultimately **APT identification**, and the underlying hypothesis is a natural one: **malware coming from the same APT group can be expected to share a common source-code base**, because a group reuses its own code across campaigns, **and hence to exhibit a similar usage of Windows API functions**. If this holds, then two samples with highly similar ApiVectors are likely to originate from the same group, and attribution reduces to a similarity search.

To test this hypothesis, a dataset of **2081 APT-related malware** (drawn from dAPTaset) was analyzed with Ghidra and APTScout's static API-recovery tool, and the recovered API usage of each sample was turned into an ApiVector.

Dataset Vectorization

Several choices govern how the samples are vectorized. The vector entries are **mapped to the first n most frequent Windows APIs**, meaning that rather than reserving a position for every conceivable API, only the n most commonly occurring ones are given positions, which keeps the vectors compact while retaining the most informative dimensions. The experiments used **different parameters**, varying both the **size** n of the vector and the **weight function** W, so that the effect of these choices on identification quality could be studied.

Beyond the equal, linear, and non-linear weightings already mentioned, a further weighting scheme based on the **entropy of information** was considered. For an API at offset i whose observed probability of occurrence across the dataset is p_i, the entropy-based weight is

$$H(p_i) = p_i \log_2(p_i) + (1 - p_i) \log_2(1 - p_i)$$

where p_i is the **observed occurrence probability of the API function at offset** i. This expression is the (negated) binary entropy of the API's presence: it reaches its extreme for APIs whose occurrence is maximally uncertain (present in about half the samples) and vanishes for APIs that are either almost always or almost never present. Weighting by this quantity emphasizes the APIs that most sharply divide the sample population, since those are the ones that carry the greatest discriminative information for telling groups apart.

The Classification Algorithm

Attribution itself is performed by a **K-Nearest Neighbor** procedure operating on ApiVector similarity. Given an input sample, the algorithm **searches for the sample or samples with the highest similarity score** to it, where similarity is measured by the weighted Jaccard metric. The label assigned to the input is then decided by a three-case rule applied to these nearest neighbors.

In the first case, **if all of the highest-scoring samples belong to the same APT**, that group is used as the label for the input sample; the neighbors agree, and their common group is the answer. In the second case, **if the highest-scoring samples do not all agree**, the input is assigned the label of the **APT group whose samples appear most frequently among the highest-scoring ones**; a majority vote among the neighbors breaks the disagreement. In the third case, covering **other situations** where no clear decision can be reached, the sample is labeled **"Unknown."** This Unknown category plays the same protective role as the yellow outcome of the two-phase pipeline: a sample that cannot be confidently attributed is explicitly set aside rather than being forced into a wrong group.

Identification Results

The effectiveness of this scheme is summarized in a **normalized confusion matrix**, in which each row corresponds to the **true** APT group of the samples and each column to the group **predicted** by the algorithm (with an extra "Unknown" column for unattributed samples). Each cell gives the fraction of a true group's samples that received a given prediction, so a perfect classifier would place all of its mass on the diagonal, where predicted equals true.

The results are strong: the overwhelming majority of each group's samples lie on the diagonal, with most groups correctly identified **85% to 100%** of the time. Several groups are recovered almost perfectly, **APT30** is identified correctly in essentially 100% of cases, and **Patchwork** reaches about 98%, with **APT28**, **Hurricane Panda**, **Transparent Tribe**, **Volatile Cedar**, and **Winnti Group** all around 96–97%. The weakest performers are groups such as **Violin Panda** (around 83%) and **Mirage** (around 85%), whose samples are occasionally scattered into neighboring groups, a result consistent with the small sample counts these groups have in the dataset, which gives the similarity search fewer reference

points to match against. The off-diagonal mass is small and diffuse, indicating that confusions are infrequent and not systematically concentrated between any particular pair of groups. Overall, the confusion matrix confirms the founding hypothesis: samples from the same APT really do share enough API-usage structure for a purely static, similarity-based method to attribute them correctly in the large majority of cases.

Malware Image-Based Classification

A third and conceptually different family of techniques approaches malware analysis not through headers, API tables, or decompiled code, but by **transforming a malicious binary into an image** and then applying image-processing and computer-vision methods to it. The appeal of this idea is that it converts the problem of recognizing malware families into the well-studied problem of recognizing visual patterns.

The transformation works as follows. A **malware binary is read as a vector of 8-bit unsigned integers**, since every byte of the file is naturally a number in the range 0 to 255. This one-dimensional vector of bytes is then **organized into a two-dimensional array**, and that array **can be visualized as a gray-scale image**, where each byte value maps to a pixel intensity **in the range [0, 255]**, with **0 rendered as black and 255 as white**. The file, in other words, is laid out row by row as a picture, and its byte values become shades of gray.

The property that makes this useful is empirical and striking: **images belonging to the same malware family are very similar in layout and texture**. Because members of a family share code, their byte-level content produces visually recurring bands, textures, and structures, so that two samples of the same family, viewed as images, look alike, while samples of different families look different. This visual family resemblance is the signal that image-based classifiers exploit.



The Basic Idea of the APT Malware Detector Framework

The guiding question behind this work is: **how can a malware be efficiently represented and classified for automatic analysis?** The proposed answer is a **new method** with two components. The first is to **represent the opcodes of an executable as grayscale images**, refining the raw-byte idea by focusing on the executable's operation codes rather than on all its bytes indiscriminately. The second is to **classify a malicious executable as either APT-related or non-APT-related malware using image-based techniques**, applying visual classification to the specific binary task of detecting APT involvement.

Accordingly, the **APT Malware Detector framework is divided into two main tasks**: **image-based malware representation**, which produces the visual encoding of each sample, and **malware classification**, which decides from that encoding whether the sample is APT-related. The first task is the subject of the following sections; the second is addressed in the evaluation.

Opcode Extraction

The representation is built not from arbitrary file bytes but from **opcodes**. An **opcode (operation code)** is the **part of an instruction in machine language that defines the behavior of the instruction itself**, that is, the component of a machine instruction specifying which operation the processor is to perform (add, move, jump, and so on), as distinct from the operands on which it acts.

The decision to focus on opcodes is motivated by the way new malware is created. When authors produce new malware, there is substantial **code re-use**, and in the course of adapting old code they often employ **equivalent opcodes**, instruction sequences that accomplish the same effect through slightly different machine encodings. Concentrating on opcodes therefore captures the behavioral essence of the code while being more robust to superficial variation than raw bytes would be.

The extraction procedure is a precise sequence of steps. The analysis is restricted to **just the `.text` section**, which is the section of a Portable Executable file that contains the executable code, so that data and resources do not dilute the representation. The procedure then **finds the entry point of the PE header**, specifically the field named **`AddressOfEntryPoint`**, which records where execution begins. Starting from that entry point, it **begins to disassemble** the code, translating the machine bytes into instructions. As it disassembles, it **saves the opcodes into an array**, building up the ordered sequence of operation codes that the program would execute.

This extraction sits within a two-phase workflow. In the **Data Extraction Phase**, malware samples are run through an isolated environment (an Oracle VirtualBox virtual machine) to obtain the binary file, from which the `.exe` is taken and its opcodes extracted into an 8-bit vector. In the subsequent **Data Mapping Phase**, that **8-bit vector is converted to a grayscale image**, completing the transition from executable to picture.

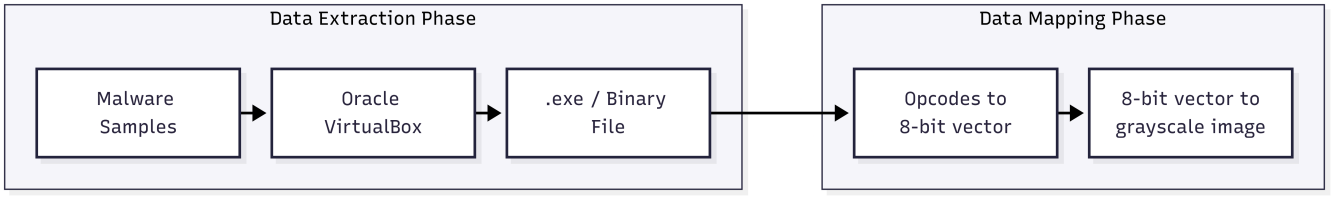


Image-Based Representation of Malware Binaries

With the opcode array in hand, it is turned into a gray-scale image through a specific set of conventions. The array is **read byte per byte as 8-bit unsigned integer vectors**, so that each opcode byte becomes a value from 0 to 255.

The width and height of the resulting image are chosen by different rules. The **line width of the array is set to a constant value**, but which constant depends on the size of the file: **if the file size is greater than 4 MB, a width of 4096 pixels is used; otherwise a width of 2048 pixels is used**. Fixing the width this way ensures that files of comparable size are laid out with comparable horizontal structure, which helps preserve the visual family resemblances. The **height, by contrast, changes with the size of the executable**: since width is fixed, a larger file simply produces a taller image, the number of rows growing in proportion to how much code there is.

The final representation is thus a **two-dimensional array in which the value range of each element is [0, 255]**, again with **0 corresponding to black and 255 to white**. When the grayscale images of **APT-related malware** are placed beside those of **non-APT malware** across different families, the earlier claim is visually confirmed: the images exhibit distinctive horizontal banding and texture patterns, with samples of a given family sharing a recognizable appearance and APT and non-APT populations differing in their overall visual character. It is these differences in layout and texture that a downstream classifier will learn to tell apart.

Image Processing: Bicubic Interpolation

The grayscale images produced by the previous stage have a serious inconvenience for machine learning: they are **not all the same size**. Because the image height grows with the size of the executable, two samples of different lengths yield images with different numbers of rows, and therefore different total numbers of pixels. A classifier, however, generally requires its inputs to be **feature vectors of a fixed, uniform size**. Some normalization of image dimensions is therefore indispensable, and the technique used for it is **bicubic interpolation**.

Bicubic interpolation is a **two-dimensional function that uses a polynomial technique for improving and resizing digital images**. When an image is resized, the pixel values of the new image must be computed from the pixels of the original, and bicubic interpolation does this by fitting smooth polynomial surfaces through the neighborhood of surrounding pixels (considering a wider neighborhood than simpler methods such as nearest-neighbor or bilinear interpolation), which yields visually smoother and more faithful resized images.

In this framework, **bicubic interpolation is applied to scale the converted grayscale images so that feature vectors of a fixed size can be extracted from them**. Regardless of the original executable's length, every sample's image is resized to common dimensions, after which all samples are described by vectors of equal length and can be fed uniformly to the classifier. Visually, when APT-related and non-APT malware images from different families are resized by bicubic interpolation, they are brought to a common shape while retaining the characteristic banding and texture that distinguish families, confirming that the resizing preserves the discriminative visual information rather than washing it out.

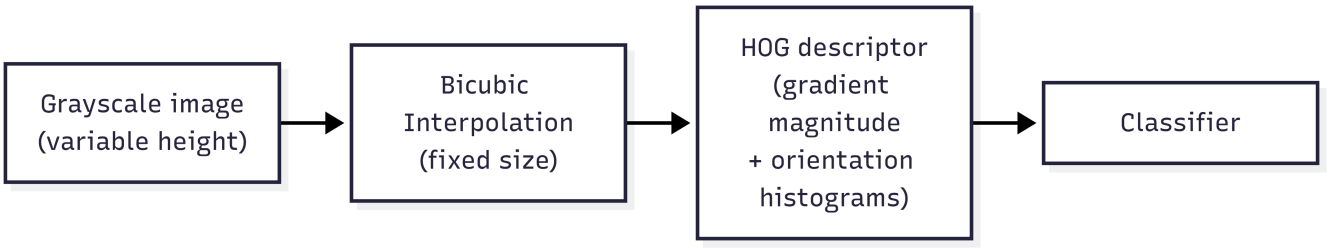
Image Processing: Histogram of Oriented Gradients

After the images have been normalized to a common size, a further processing step extracts the actual visual features that the classifier will use, and for this the framework employs the **Histogram of Oriented Gradients (HOG)**.

HOG is a feature descriptor for object detection. Its operating principle is to **count the occurrences of gradient orientation in localized portions of an image**. A gradient, at each pixel, points in the direction of greatest change in intensity and has a magnitude reflecting how sharp that change is; gradients are therefore strong along edges and boundaries, where intensity changes rapidly, and weak in flat regions. HOG exploits this by dividing the image into small regions and, **for each region, generating a histogram built from the magnitude and the orientation of the gradients** within it. Each histogram records how much gradient energy points in each range of directions, so that the local structure of edges and textures in that region is summarized compactly.

The reason HOG is appropriate here is that the distinguishing characteristics of these malware images are exactly **edges and textures**, the horizontal bands and striations that give each family its visual signature. HOG converts those textural patterns into a numerical descriptor that a classifier can process. When HOG is applied to the resized images of APT-related and non-APT malware across different families, it extracts for each the pattern of oriented gradients that encodes its texture, and these HOG descriptors become the feature vectors on which classification is performed. The full representation pipeline is thus: executable → opcode array → grayscale image → bicubically resized image → HOG

feature vector.



Evaluation: Datasets

The image-based framework is evaluated on **two different databases of malware**: one of **non-APT-related malware** and one of **APT-related malware**. The two differ enormously in size. The non-APT dataset counts **11,684 samples**, whereas the APT-related dataset has only **1,959**. In other words, **APT-related malware is far rarer than non-APT malware**, an echo of exactly the same scarcity that drove the not-belonging-class problem encountered earlier in the chapter.

This disparity makes the combined data an **unbalanced dataset**, and training a classifier directly on such data would bias it toward the abundant non-APT class. The standard remedy is **resampling**, which adjusts the class proportions before training, and it comes in two complementary forms. **Undersampling** reduces the majority class by discarding some of its samples until the classes are balanced, at the risk of throwing away useful information. **Oversampling** increases the minority class, either by duplicating its samples or by synthesizing new ones resembling them, until balance is achieved, at the risk of overfitting to the few genuine minority examples. Both strategies, as well as combinations of them, are tested in the evaluation.

Evaluation: Metrics

The quality of the proposed framework is measured with a set of metrics. To evaluate overall performance the framework uses the **AUC-ROC curve** (the Area Under the Receiver Operating Characteristic curve), which summarizes a classifier's ability to separate the two classes across all possible decision thresholds; a value near 1 indicates excellent separation, while 0.5 indicates no discriminative ability. In addition, four further metrics quantify quality: the **accuracy rate**, the **precision (performance) rate**, the **recall rate**, and the **F1-score**, the same quartet of complementary measures used throughout the chapter, capturing respectively overall correctness, false-positive control, false-negative control, and the balance between the latter two.

Evaluation: Tests

The framework was tested with two classifiers, a **Random Forest Classifier** and a **Multi-Layer Perceptron (MLP) Classifier** (a feed-forward neural network), under each resampling strategy. The results are organized by the type of resampling applied.

Undersampling Techniques

Under undersampling, several specific methods were compared. For the **Random Forest Classifier**, the simplest method, **RandomUnderSampler**, gave the weakest results (85% accuracy, precision 0.86, recall 0.84, F1 0.82), while the more selective methods performed markedly better: **EditedNearestNeighbours** reached 93.6% accuracy with precision, recall, and F1 all at 0.94; **AllKNN** reached 92.1% with all three rates at 0.92; **OneSidedSelection** was the best overall at **94.1% accuracy** (precision 0.96, recall 0.94, F1 0.95); and **NeighbourhoodCleaningRule** reached 93.8% (precision 0.95, recall 0.94, F1 0.94). For the **Multi-Layer Perceptron Classifier**, the pattern was similar but slightly

lower overall: **RandomUnderSampler** gave 86% (precision 0.86, recall 0.86, F1 0.85), **EditedNearestNeighbours** 90.8%, **AllKNN** 89.7%, **OneSidedSelection** the best at **93.1%** (all rates 0.93), and **NeighbourhoodCleaningRule** 90.9%. The consistent lesson is that intelligent, selective undersampling methods, which remove majority samples near the class boundary rather than at random, substantially outperform naive random removal, and that **OneSidedSelection** is the strongest undersampling choice for both classifiers.

Undersampling — Random Forest	Acc	P	R	F1
RandomUnderSampler	85%	0.86	0.84	0.82
EditedNearestNeighbours	93.6%	0.94	0.94	0.94
AllKNN	92.1%	0.92	0.92	0.92
OneSidedSelection	94.1%	0.96	0.94	0.95
NeighbourhoodCleaningRule	93.8%	0.95	0.94	0.94

Undersampling — MLP	Acc	P	R	F1
RandomUnderSampler	86%	0.86	0.86	0.85
EditedNearestNeighbours	90.8%	0.90	0.91	0.90
AllKNN	89.7%	0.89	0.90	0.89
OneSidedSelection	93.1%	0.93	0.93	0.93
NeighbourhoodCleaningRule	90.9%	0.90	0.91	0.90

Oversampling Techniques

Under oversampling, where the minority (APT) class is enlarged rather than the majority trimmed, the results are generally strong and tightly clustered. For the **Random Forest Classifier**, **ROS (Random OverSampling)** achieved the best result at **94.3% accuracy** (precision 0.95, recall 0.94, F1 0.95), closely followed by **BSMOTE1** at 94.2%, **SVMSMOTE** at 94%, **SMOTE** and **ADASYN** both at 93.8%, and **BSMOTE2** at 93%. For the **Multi-Layer Perceptron Classifier**, results were slightly lower, led by **ROS** at 92% and **BSMOTE1** at 92.6%, with **SVMSMOTE** at 91.9%, **BSMOTE2** at 91.6%, **SMOTE** at 91.4%, and **ADASYN** at 91.1%. Here too the Random Forest outperforms the MLP, and the various SMOTE-family synthesis methods perform comparably to one another, with simple random oversampling holding up surprisingly well.

Oversampling — Random Forest	Acc	P	R	F1
ROS	94.3%	0.95	0.94	0.95
SMOTE	93.8%	0.94	0.94	0.94
ADASYN	93.8%	0.94	0.94	0.94
BSMOTE1	94.2%	0.94	0.94	0.94
BSMOTE2	93%	0.93	0.93	0.93
SVMSMOTE	94%	0.94	0.94	0.94

Oversampling — MLP	Acc	P	R	F1
ROS	92%	0.92	0.92	0.92

Oversampling — MLP	Acc	P	R	F1
SMOTE	91.4%	0.91	0.91	0.91
ADASYN	91.1%	0.91	0.91	0.91
BSMOTE1	92.6%	0.92	0.93	0.93
BSMOTE2	91.6%	0.91	0.92	0.91
SVMSMOTE	91.9%	0.92	0.92	0.92

Combining Techniques and the Baseline Without Resampling

Two hybrid methods that **combine** oversampling with undersampling were also tested. For the **Random Forest Classifier**, **SMOTETomek** performed excellently at **94.5% accuracy** (precision 0.95, recall 0.94, F1 0.95), the highest single accuracy figure in the whole evaluation, whereas **SMOTEENN** performed notably poorly at only 79% (precision 0.83, recall 0.79, F1 0.76). For the **Multi-Layer Perceptron**, **SMOTETomek** again did well at 92.3% while **SMOTEENN** reached 83%. The clear takeaway is that SMOTETomek is an excellent combined strategy, while SMOTEENN, which aggressively cleans samples via edited-nearest-neighbours after synthesis, removes too much useful data and degrades performance.

Finally, a **baseline without any resampling** provides the essential point of comparison. Trained directly on the unbalanced data, the **Random Forest Classifier** achieved **94.3% accuracy** (precision 0.96, recall 0.94, F1 0.94) and the **Multi-Layer Perceptron** achieved 93.6% (precision, recall, and F1 all 0.93).

Combining — Random Forest	Acc	P	R	F1
SMOTEENN	79%	0.83	0.79	0.76
SMOTETomek	94.5%	0.95	0.94	0.95

Combining — MLP	Acc	P	R	F1
SMOTEENN	83%	0.84	0.83	0.84
SMOTETomek	92.3%	0.92	0.92	0.92

Without Resampling	Acc	P	R	F1
Random Forest Classifier	94.3%	0.96	0.94	0.94
Multi-Layer Perceptron Classifier	93.6%	0.93	0.93	0.93

The comparison with this baseline carries an important and somewhat sobering conclusion. The best resampling results (OneSidedSelection at 94.1%, ROS at 94.3%, SMOTETomek at 94.5%) are only marginally better than, or essentially equal to, the no-resampling baseline at 94.3%, while some resampling choices (RandomUnderSampler at 85%, SMOTEENN at 79%) are considerably *worse*. The practical implication is that for this image-based framework the Random Forest is robust enough to perform very well even on the unbalanced data as given, that only the most carefully chosen resampling methods yield any improvement at all, and that a poorly chosen resampling strategy can do real harm. Across every experimental condition, moreover, the **Random Forest Classifier consistently outperforms the Multi-Layer Perceptron**, reaffirming the pattern seen throughout the chapter that tree-ensemble methods are exceptionally well suited to this family of static malware-classification tasks.